

第一章 目录

第 1 章	龙芯 2K500 芯片	8
1.1	龙芯架构概述	8
1.1.1	龙芯架构发展介绍	8
1.1.2	龙芯架构技术简介	8
1.1.3	Loongson 与 LoongArch 的关系	9
1.1.4	Loongson 发展及应用领域	10
1.2	LA64 架构概述	10
1.2.1	指令编码概述	10
1.2.2	基础整数指令概述	11
1.2.3	基础浮点数指令概述	15
1.2.4	特权资源架构概述	21
1.2.5	存储管理概述	21
1.2.6	例外与中断概述	21
1.3	2K500 技术规格与特性	22
1.3.1	CPU 技术规格	22
1.3.2	内存及存储技术规格	22
1.3.3	系统软件支持	22
1.3.4	工作参数	22
第 2 章	龙芯 2K500 试验箱	23
2.1	龙芯 2K500 试验箱概述	23
2.1.1	试验箱设计理念与用途	23
2.1.2	试验箱组成结构	23
2.2	试验箱硬件接口	25
2.2.1	开发板参数	25
2.2.2	开发板引脚定义	26
2.2.3	开发板整体架构	29
2.2.4	主板接口及模块	31
2.2.5	拓展版接口及模块	37
2.3	试验箱软件环境	39
2.3.1	U-boot	39

2.3.2 系统内核	40
2.3.3 文件系统	41
第 3 章 熟悉开发环境	42
3.1 实验目的	42
3.2 实验步骤	42
3.2.1 使用终端工具连接到实验箱	42
3.2.2 升级内核、文件系统	43
第 4 章 交叉编译与程序运行	48
4.1 实验目的	48
4.2 实验步骤	48
4.2.1 编写一个简单程序	48
4.2.2 使用交叉编译工具链编译	48
第 5 章 七段数码管实验	52
5.1 实验目的	52
5.2 实验步骤	52
5.2.1 I2C 通用写函数及 CH422 通信	52
5.2.2 内核模块代码编写	55
1.1.1. 5.2.3 模块代码编译	62
1.1.2. 5.2.4 测试程序编写指南	63
1.1.3. 5.2.5 模块加载、运行测试和模块卸载	66
5.3 练习：数码管实现时钟功能	67
第 6 章 点阵控制实验	68
6.1 实验目的	68
6.2 实验步骤	68
6.2.1 实验箱点阵模块简介	68
6.2.2 内核模块代码编写	68
6.2.3 测试代码编写指南	73
6.3 练习：点阵实现任意文字显示	75
第 7 章 RS-485 通讯实验	76
7.1 实验目的	76
7.2 RS-485 通讯简介	76
7.3 实验步骤	76

7.3.1 物理层分析	76
7.3.2 转换器接线	77
7.3.3 试验箱收发程序	79
7.3.4 与上位机通讯——主机	82
7.3.5 与上位机通讯——虚拟机	84
7.4 练习：RS485 通信实现设备操控	86
第 8 章 LED 实验	87
8.1 实验目的	87
8.2 实验步骤	87
8.2.1 LED 模块简介	87
8.2.2 LED 模块内核驱动编写	87
8.2.3 LED 模块测试代码编写	93
8.3 练习：LED 基本实验和设备状态反馈	95
第 9 章 高速电机实验	96
9.1 实验目的	96
9.2 TC118S 马达驱动器	96
9.3 实验步骤	96
9.3.1 内核模块代码编写	96
9.3.2 测试代码编写指南	103
9.4 练习：上位机控制高速电机	104
第 10 章 蜂鸣器实验	105
10.1 实验目的	105
10.2 试验箱蜂鸣器简介	105
10.3 实验步骤	105
10.3.1 内核模块代码编写	105
10.3.2 测试代码编写指南	112
10.4 练习：蜂鸣器实现闹钟功能	113
第 11 章 陀螺仪实验	114
11.1 实验目的	114
11.2 STK8BA58 简介	114
11.3 实验步骤	114
11.3.1 I2C 通用操作函数	114

11.3.2 内核模块代码编写	116
11.3.3 测试代码编写指南	128
11.4 练习：陀螺仪姿态控制和判定	130
第 12 章 CAN 总线通信实验	131
12.1 实验目的	131
12.2 CAN 总线介绍	131
12.3 实验代码编写	131
12.3.1 服务端	131
12.3.2 客户端	134
12.4 练习：CAN 总线基本实验	137
第 13 章 音频实验	138
13.1 实验目的	138
13.2 实验步骤	138
13.2.1 拾音器与耳机接口	138
13.2.2 系统声卡配置	138
13.2.3 播放和录音测试	143
13.3 练习：音频设备的简易使用	144
第 14 章 摄像头采集实验	145
14.1 实验目的	145
14.2 实验步骤	145
14.2.1 实验代码编写	145
14.2.2 编译	157
14.2.3 摄像头采集测试	158
14.3 练习：摄像头实现拍照功能	159
第 15 章 NFC 模块实验	160
15.1 实验目的	160
15.2 RFID 简介	160
15.3 实验步骤	162
15.3.1 基本 API	162
15.3.2 内核模块代码编写	180
15.3.3 测试程序	193
15.4 练习：NFC 的基本读写	196

第 16 章	湿度传感器模块实验	197
16.1	实验目的	197
16.2	AHT20 简介	197
16.3	实验步骤	197
16.3.1	内核模块代码编写	197
16.3.2	测试代码编写指南	204
16.4	练习：温湿度传感器的基本应用	205
第 17 章	WIFI 无线网卡使用实验	207
17.1	实验目的	207
17.2	WIFI 模块简介	207
17.3	实验步骤	208
17.3.1	内核配置与编译	208
17.3.2	配置网卡并连接到网络	212
17.3.3	TCP 程序测试	216
17.4	练习：设备控制和多设备通信	220
第 18 章	矩阵键盘模块实验	221
18.1	实验目的	221
18.2	实验步骤	221
18.2.1	矩阵键盘模块简介	221
18.2.2	内核驱动代码编写	222
18.2.3	设备测试程序编写	231
18.3	练习：设备控制和复杂输入	234
第 19 章	红外遥控实验	235
19.1	实验目的	235
19.2	红外通信协议 NEC 简介	235
19.3	实验步骤	235
19.3.1	编写设备驱动函数	235
19.3.2	编写设备驱动测试代码	251
19.4	练习：红外设备控制	255
第 20 章	LibTorch 构建编译实验	256
20.1	实验目的	256
20.2	LibTorch for LoongArch64 简介	256

20.3 实验步骤.....	256
20.3.1 安装 Miniconda.....	256
20.3.2 获取 PyTorch 源码	257
20.3.3 配置交叉编译工具链.....	260
20.3.4 创建 Python 虚拟环境	261
20.3.5 编写 CMake 工具链文件.....	262
20.3.6 预编译 protoc 和 sleef 库.....	263
20.3.7 源码适配.....	265
20.3.7.1 应用 SLEEF 库补丁.....	265
20.3.7.2 应用 cpuinfo 库补丁（非必做）	266
20.3.7.3 gcc 13.3 源码适配（若使用 gcc 13.3 工具链）	267
20.3.8 设置环境变量	267
20.3.9 交叉编译 LibTorch	268
20.3.10 替换 TorchConfig.cmake	272
第 21 章 龙芯神经网络训练实验	274
21.1 实验目的.....	274
21.2 实验步骤.....	274
21.2.1 准备代码环境	274
21.2.2 交叉编译.....	277
21.2.3 部署到实验平台	280
21.3 练习：优化训练过程.....	281
第 22 章 龙芯教学平台的综合实验	282
22.1 实验目的.....	282
22.2 驱动模块的统一编译.....	282
22.2.1 I2C 模块	282
22.2.2 I2C 设备驱动修改.....	294
22.2.3 其余驱动的修改.....	298
22.3 创建工程项目	301
22.4 设备 API.....	303
22.4.1 温湿度传感器	303
22.4.2 数码管	307
22.4.3 蜂鸣器	313

22.4.4 电机.....	313
22.4.5 重力传感器.....	314
22.4.6 智能卡	315
22.4.7 16*16 点阵.....	317
22.4.8 4*4 键盘.....	318
22.4.9 LED	319
22.4.10 红外遥控.....	320
22.5 测试 API 功能.....	321
22.5.1 依赖头文件.....	321
22.5.2 全局声明.....	322
22.5.3 线程函数.....	324
22.5.4 主线程.....	327
22.6 练习：综合应用.....	336

第1章 龙芯 2K500 芯片

1.1 龙芯架构概述

1.1.1 龙芯架构发展介绍

2020 年，龙芯中科基于二十年的 CPU 研制和生态建设积累推出了龙架构（LoongArch™），包括基础架构部分和向量指令、虚拟化、二进制翻译等扩展部分，近 2000 条指令。龙架构从整个架构的顶层规划，到各部分的功能定义，再到细节上每条指令的编码、名称、含义，在架构上进行了自主重新设计，具有充分的自主性。龙架构摒弃了传统指令系统中部分不适当当前软硬件设计技术发展趋势的陈旧内容，吸纳了近年来指令系统设计领域诸多先进的技术发展成果。与原有兼容指令系统相比，不仅在硬件方面更易于高性能低功耗设计，而且在软件方面更易于编译优化和操作系统、虚拟机的开发。龙架构在设计时充分考虑兼容生态需求，融合了各国际主流指令系统的主要功能特性，同时依托龙芯团队在二进制翻译方面十余年的技术积累创新，能够实现多种国际主流指令系统的高效二进制翻译。龙芯中科从 2020 年起新研的 CPU 均支持 LoongArch™。

1.1.2 龙芯架构技术简介

龙芯架构具有 RISC 指令架构的典型特征。它的指令长度固定且编码格式规整，绝大多数指令只有两个源操作数和一个目的操作数，采用 load/store 架构，即仅有 load/store 访存指令可以访问内存，其它指令的操作对象均是处理器核内部的寄存器或指令码中的立即数。龙芯架构分为 32 位和 64 位两个版本，分别称为 LA32 架构和 LA64 架构。龙芯架构定义了四个特权级（PLV0~PLV3），其中 PLV0 是最高特权级，用于内核，而 PLV3 是最低特权级，用于应用程序。

LA64 架构应用级向下二进制兼容 LA32 架构。所谓“应用级向下二进制兼容”一方面是指采用 LA32 架构的应用软件的二进制可以直接运行在兼容 LA64 架构的机器上并获得相同的运行结果，另一方面是指这种向下二进制兼容仅限于应用软件，架构规范并不保证在兼容 LA32 架构的机器上运行的系统软件（如操作系统内核）的二进制直接在兼容 LA64 架构的机器上运行时总是获得相同的运行结果。

龙芯架构采用基础部分（Longson Base）加扩展部分的组织形式（如图 1-1 所示）。其中扩展部分包括：二进制翻译扩展（Longson Binary Translation, 简称 LBT）、虚拟化扩展（Longson Virtualization, 简称 LVZ）、向量扩展（Longson SIMD Extension, 简称 LSX）和高级向量扩展（Longson Advanced SIMD Extension, 简称 LASX）。

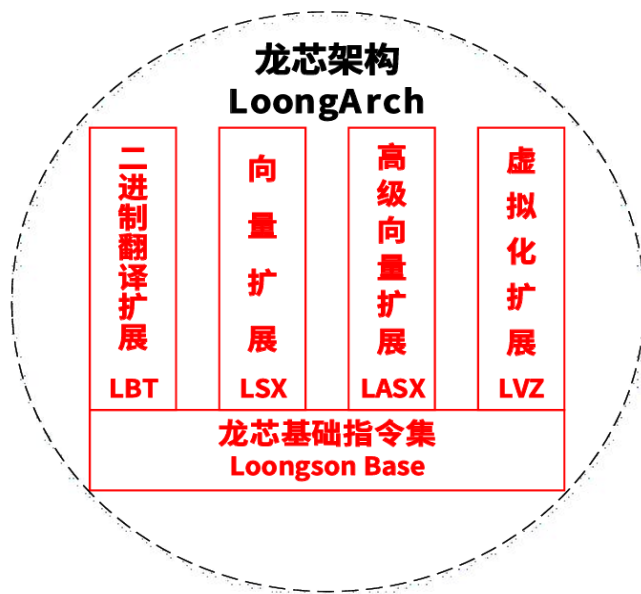


图 1-1 龙芯架构

龙芯架构的基础部分（Longson Base）包含非特权指令集和特权指令集两个部分，其中非特权指令集部分定义了常用的整数和浮点数指令，能够充分支持现有各主流编译系统生成高效的目标代码。

龙芯架构的虚拟化扩展（Longson Virtualization，简称 LVZ）部分用于为操作系统虚拟化提供硬件加速以提升性能。这部分涉及的基本上都是特权资源，包括一些特权指令和控制状态寄存器，以及在例外和中断、存储管理等方面添加新的功能。

龙芯架构的二进制翻译扩展（Longson Binary Translation，简称 LBT）部分用于提升跨指令系统二进制翻译在龙芯架构平台上的执行效率。其在基础部分之上进行扩展，同样包含非特权指令集和特权指令集两个部分。

龙芯向量扩展（Longson SIMD Extension，简称 LSX）和高级向量扩展（Longson Advanced SIMD Extension，简称 LASX）两部分均是采用 SIMD 指令来加速计算密集型应用。两个扩展部分在指令功能上基本一致，区别在于向量扩展操作的向量位宽是 128 位而高级向量扩展操作的向量位宽是 256 位。

对于一个兼容龙芯架构的实现，架构中的基础部分必须实现，扩展部分可以选择实现。各扩展部分可以灵活选择，不过，当选择实现 LASX 时必须同时实现 LSX。除了将一整个扩展部分作为可选项，在基础部分和各扩展部分中还进一步包含了一些可选实现的功能子集。所有这些可选实现的扩展部分或是各部分中可选实现功能子集的具体实现情况，软件可以用 CPUCFG 指令读取的配置信息字来动态予以识别。

1.1.3 Loongson 与 LoongArch 的关系

LoongArch 是一种 RISC 指令集架构（ISA），不同于现存的任何一种 ISA，而 Loongson（即龙芯）是一个处理器家族。龙芯包括三个系列：Loongson-1（龙芯 1 号）是 32 位处理器系列，Loongson-2（龙芯 2 号）是低端 64 位处理器系列，而 Loongson-3（龙芯 3 号）是高端 64 位处理器系列。旧的龙芯处理器基于 MIPS 架构，而新的龙芯处理器基于 LoongArch 架构。以龙芯 3 号为例：龙芯 3A1000/3B1500/3A2000/3A3000/3A4000 都是兼容 MIPS 的，而龙芯 3A5000（以及

将来的型号) 都是基于 LoongArch 的。

1.1.4 Loongson 发展及应用领域

2020 年后, 龙芯进入爆发阶段, 推出的四核龙芯 3A5000 处理器性能接近 Intel Skylake 系列。2021 年, 龙芯中科推出了 LoongArch 自主指令集, 发布了具有高端商用服务器竞争力的龙芯 3C5000L。2022 年, 龙芯发布面向大数据中心的龙芯 3C5000 服务器处理器, 性能与 ARM 64 核心处理器相当。

龙芯处理器在信息化和工控领域广泛应用, 信息化类芯片主要应用于桌面和服务器, 广泛用于电子政务、金融、电信和教育等行业; 工控类芯片则用于嵌入式设备、工业控制、物联网等领域, 如能源、交通和智能制造。龙芯 1 号系列处理器适用于低功耗嵌入式应用, 2 号系列处理器面向工业控制, 3 号系列处理器则在高性能通用处理方面具有优势。胡伟武和核心团队多为中科院计算所背景, 技术实力雄厚, 员工持股超过三成, 确保公司发展稳定。

龙芯中科采用 Fabless 经营模式, 通过直销为主、少量经销的模式, 与多个知名企业建立战略合作关系。2018 至 2021 年, 公司营收增长迅速, 主要来自处理器及配套芯片销售, 工控类和信息化类芯片销售均有所提升。公司在华北和华东地区销售占比最大, 并逐步向全国扩展, 产品销售呈现出上半年较低、下半年较高的季节性特点。龙芯中科凭借技术优势, 保持了较高的毛利率水平, 未来有望继续保持稳定增长。

1.2 LA64 架构概述

1.2.1 指令编码概述

龙芯架构中的所有指令均采用 32 位固定长度, 且指令的地址都要求 4 字节边界对齐。当指令地址不对齐时将触发地址错例外。指令编码的风格是所有寄存器操作数域都从第 0 比特开始从低到高依次摆放。操作码都是从第 31 比特开始从高到低依次摆放。其包含 9 种典型的指令编码格式(如图 1-2 所示), 即 3 种不含立即数的编码格式: 2R、3R、4R, 以及 6 种含立即数的编码格式: 2RI8、2RI12、2RI14、2RI16、1RI21、I26。

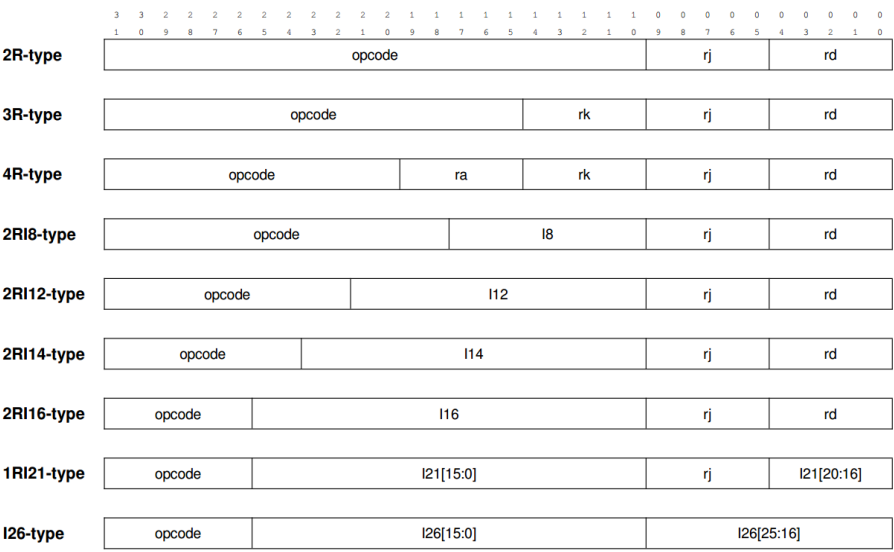


图 1-2 龙芯指令编码格式

指令汇编助记格式主要包括指令名和操作数两部分。龙芯架构对指令名和操作数的前、后缀进行了统一考虑，以方便汇编人员和编译器开发人员的使用。

首先，通过指令名的前缀字母来区分非向量指令和向量指令、整数和浮点数指令。所有 128 位向量指令的指令名以字母“V”开头；所有 256 位向量指令的指令名以字母“XV”开头。所有非向量浮点数指令的指令名以字母“F”开头；所有 128 位向量浮点指令的指令名以“VF”开头；所有 256 位向量浮点指令的指令名以“XVF”开头。

其次，绝大多数指令通过指令名中“.X”形式的后缀来指示指令的操作对象，且这种形式的后缀仅用来表征指令操作对象的类型。对于操作对象是整数类型的，指令名后缀为.B、.H、.W、.D、.BU、.HU、.WU、.DU 分别表示该指令操作的数据类型是有符号字节、有符号半字、有符号字、有符号双字、无符号字节、无符号半字、无符号字、无符号双字。

不过这里有一种特殊情况，当操作数是有符号数还是无符号数不影响运算结果时，指令名中携带的后缀均不带 U，但此时并不限制操作对象只能是有符号数。对于操作对象是浮点数类型的，或者更具体来说是那些指令名以“F”、“VF”和“XVF”开头的指令，其指令名后缀为.H、.S、.D、.W、.L、.WU、.LU 分别表示该指令操作的数据类型是半精度浮点数、单精度浮点数、双精度浮点数、有符号字、有符号双字、无符号字、无符号双字。此外在涉及向量操作的指令中，指令名后缀.V 表示该指令是将整个向量数据作为一个整体进行操作。需要指出的是，并不是所有指令都用“.XX”形式的后缀来指示指令的操作对象。当指令操作对象的数据位宽由所执行处理器是 32 位实现还是 64 位决定的，如 SLT 和 SLTU 指令，这种指令是不加后缀的。

1.2.2 基础整数指令概述

龙芯架构基础部分的非特权指令集按照软件运行时上下文内容的差异可划分为基础整数指令和基础浮点数指令两个部分。本节将对其中的整数指令部分进行概述。基础整数指令部分是非特权指令子集中最基础的一部分。

1.2.2.1 数据类型

基础整数指令操作的数据类型有 5 种，分别是：比特（bit，简记 b）、字节（Byte，简记 B，长度 8b）、半字（Halfword，简记 H，长度 16b）、字（Word，简记 W，长度 32b）、双字（Doubleword，简记 D，长度 64b）。字节、半字、字和双字数据类型均采用二进制补码的编码方式。

1.2.2.2 寄存器

基础整数指令涉及的寄存器包括通用寄存器（General-purpose Register，简称 GR）和程序计数器（Program Counter，简称 PC），如图 1-3 所示。

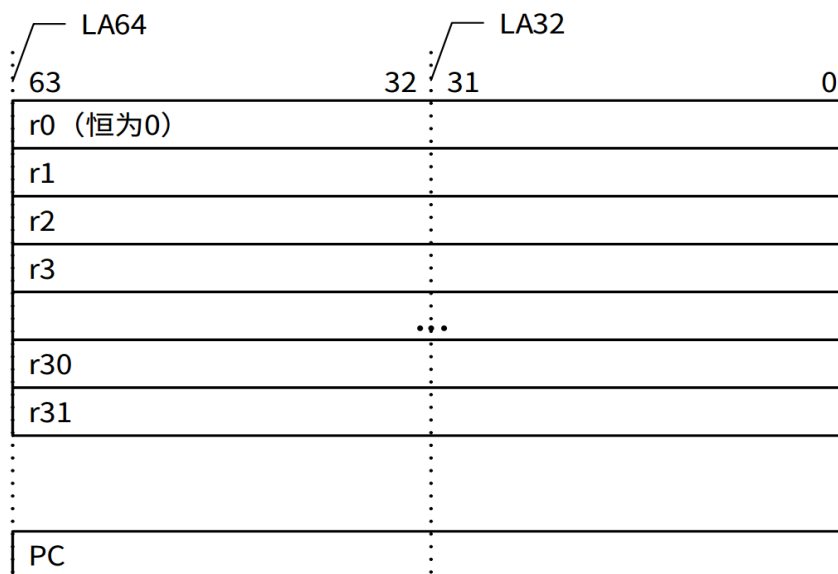


图 1-3 寄存器

通用寄存器 GR 有 32 个，记为 $r0 \sim r31$ ，其中第 0 号寄存器 $r0$ 的值恒为 0。GR 的位宽记作 $GRLEN$ 。LA64 架构下 GR 的位宽是 64 比特。基础整数指令与通用寄存器存在正交关系。即从架构角度而言，这些指令中任一个寄存器操作数都可以采用 32 个 GR 中的任一个。唯一的例外是 BL 指令中隐含的目的寄存器一定是第 1 号寄存器 $r1$ 。在标准的龙芯架构应用程序二进制接口（Application Binary Interface，简称 ABI）中， $r1$ 固定作为存放函数调用返回地址的寄存器。

PC 只有 1 个，记录着当前指令的地址。PC 寄存器不能被指令直接修改，它只能被转移指令、例外陷入和例外返回指令间接修改。不过，PC 寄存器可以作为一些非转移类指令的源操作数而被直接读取。PC 的宽度总是与 GR 的宽度一致。

1.2.2.3 指令概览

这一节将对 LA64 架构中应用级基础整数指令的功能进行概述。如图 1-4 所示，表中给出了 LA64 架构中基础整数指令的子集概览。

算术运算类指令	ADD.W, SUB.W, ADDI.W, ALSL.W, LU12L.W, SLT, SLTU, SLTI, SLTUI, PCADDI, PCADDUI2I, PCALAU12I, AND, OR, NOR, XOR, ANDN, ORN, ANDI, ORI, XORI, MUL.W, MULH.W, MULH.WU, DIV.W, MOD.W, DIV.WU, MOD.WU
移位运算类指令	SLL.W, SRL.W, SRA.W, ROTR.W, SLLI.W, SRLI.W, SRAI.W, ROTRI.W
位操作指令	EXT.W.B, EXT.W.H, CLO.W, CLZ.W, CTO.W, CTZ.W, BYTEPICK.W, REVB.2H, BITREV.4B, BITREV.W, BSTRINS.W, BSTRPICK.W, MASKEQZ, MASKNEZ
转移指令	BEQ, BNE, BLT, BCE, BLTU, BCEU, BEQZ, BNEZ, B, BL, JIRL
访存指令	LD.B, LD.H, LD.W, LD.BU, LD.HU, ST.B, ST.H, ST.W, PRELD
原子访存指令	LL.W, SC.W
栅障指令	DBAR, IBAR
其它杂项指令	SYSCALL, BREAK, RDTIMEL.W, RDTIMEH.W, CPUCFG

图 1-4 LA64 架构中基础整数指令

在 LA64 架构基础证书指令中，指令被分为算术运算指令、移位运算指令、位操作指令、转移指令、访存指令、原子方寸指令、栅障指令以及其他杂项指令这

八类指令。本节将以其中 ADD.W、SLI.W、EXT.W.B、BEQ、LD.B、LL.W、DBAR、SYCALL 这八条指令为例子介绍基础证书指令。

1.2.2.4 算数运算指令概述

算术运算指令用于执行基本的数学运算，如加法、减法、乘法和除法等。

以 ADD.W 指令为例，其指令格式为 ADD.W Rd, Rj, Rk。ADD.W 将通用寄存器 rj 中的[31:0]位数据加上通用寄存器 rk 中的[31:0]位数据，所得结果的[31:0]位符号扩展后写入通用寄存器 rd 中。

其过程如以下表达式所示，首先从寄存器 GR 的 rj 位置取出 32 位数据，然后从寄存器 GR 的 rk 位置也取出 32 位数据，并将这两部分数据进行拼接。接下来，从拼接后的 64 位数据中取前 32 位，对其进行符号扩展以填充到 GR 寄存器的完整长度，最后将结果保存到 GR 寄存器的 rd 位置。

```
tmp = GR[rj][31:0] + GR[rk][31:0]
GR[rd] = SignExtend(tmp[31:0], GRLEN)
```

1.2.2.5 移位运算指令概述

移位运算指令主要用于对二进制数进行左移或右移操作。这些指令在许多算法和程序设计中非常有用，特别是在低级编程、优化代码性能以及实现特定的算法时。

以 SLI.W 指令为例，其指令格式为 SLI.W Rd, Rj, Rk。SLL.W 将通用寄存器 Rj 中[31:0]位数据逻辑左移，移位结果符号扩展写入通用寄存器 Rd 中。

其过程如以下表达式所示，首先将寄存器 rk 中的值的低 5 位用于确定偏移量，然后根据这个偏移量，将寄存器 rj 中的值左移相应的位数，并将左移后的结果存储在临时变量 tmp 中。接着，将 tmp 中的值进行符号扩展，使其符合指定的寄存器长度 GRLEN，最后将扩展后的结果存储到目标寄存器 rd 中。

```
tmp = SL(GR[rj][31:0], GR[rk][4:0])
GR[rd] = SignExtend(tmp[31:0], GRLEN)
```

1.2.2.6 位操作指令概述

位运算指令是 LA64 指令集中用于执行二进制位级别操作的一类指令。这些指令通常用于快速实现位级操作，例如位移、按位逻辑运算等，广泛应用于各种计算任务中，特别是在低级编程和系统软件开发中。

以 EXT.W.B 指令为例，其指令格式为 SLI.W Rd, Rj。EXT.WB 将通用寄存器 rj 中[7:0]位数据符号扩展后写入通用寄存器 rd 中。

其过程如以下表达式所示，首先提取通用寄存器 rj 的低 8 位值，然后对这个 8 位值进行符号扩展到寄存器长度 GRLEN，最后将扩展后的值存储到目标寄存器 rd 中。

```
GR[rd] = SignExtend(GR[rj][7:0], GRLEN)
```

1.2.2.7 转移指令概述

转移指令是 LA64 指令集中用于改变程序执行流程的一类指令。这类指令能够控制 CPU 跳转到内存中的另一个位置继续执行指令，从而实现程序分支、循环和其他控制结构。

以 BEQ 指令为例，其指令格式为 BEQ Rd, Rj, offs16。BEQ 将通用寄存器

rj 和通用寄存器 rd 的值进行比较，如果两者相等则跳转到目标地址，否则不跳转。

其过程如以下表达式所示，首先检查通用寄存器 rj 和 rd 的值是否相等，如果相等，则更新程序计数器（PC）。具体来说，将一个 16 位偏移量 ofs16 左移 2 位后进行符号扩展到寄存器长度，然后将这个扩展值加到当前的 PC 上，从而实现跳转到新的指令地址。

```
if GR[rj]=GR[rd] :  
    PC = PC + SignExtend({ofs16, 2'b0}, GRLEN)
```

1.2.2.8 访存指令概述

访存指令主要用于在处理器和内存之间传输数据。这些指令允许 CPU 从内存读取数据或将数据写入内存。访存指令的效率直接影响到程序的整体性能，因为许多计算任务需要频繁地访问内存。

以 LD.B 指令为例，其指令格式为 LD.B Rd, Rj, si12。LD.B 从内存取回一个字节/半字/的数据符号扩展后写入通用寄存器 rd。

其过程如以下表达式所示，首先，通过读取通用寄存器 rj 的值并将 12 位立即数 si12 符号扩展到寄存器长度，然后将两者相加，计算出虚拟地址 vadr。接着，进行地址合规性检查，以确保虚拟地址有效。在此之后，将虚拟地址转换为物理地址 padr，并从这个物理地址对应的内存中加载一个字节的 data。最后，将这个字节 data 符号扩展到寄存器长度，并存储到目标寄存器 rd 中。

```
vadr = GR[rj] + SignExtend(si12, GRLEN)  
AdresComplianceCheck(vadr)  
padr = AdresTranslation(vadr)  
byte = MemoryLoad(padr, BYTE)  
GR[rd] = SignExtend(byte, GRLEN)
```

1.2.2.9 原子访存指令概述

原子访存指令是一种特殊的处理器指令，它能够确保对内存的访问是不可分割和不可中断的。这意味着在执行这类指令的过程中，其他处理器或线程不能干扰该操作，从而保证了数据的一致性和完整性。

以 LL.W 指令为例，其指令格式为 LL.W Rd, Rj, si14。LL.W 指令从内存指定地址取回一个字的数据，符号扩展后写入通用寄存器 rd。

与之配对的 SC.W 指令操作同样宽度的 data 且访问相同的内存地址。访存操作序列原子性的维护机制是，LL.W 执行时记录下访问地址并置上一个标记（LLbit 置为 1）。SC.W 指令执行时会查看 LLbit，仅当 LLbit 为 1 时才真正产生写动作，否则不写。

当软件需要一定成功完成一个原子的“读-修改-写”访存操作序列时，需要构建一个循环来反复执行 LL-SC 指令对直至 SC 成功完成。为了构建这个循环，SC. {W/D} 指令会将其执行成功与否的标志（也可以简单理解为 SC 指令执行时所看到的 LLbit 值）写入到通用寄存器 rd 中返回。

在配对的 LL-SC 执行期间，下列事件会让 LLbit 清 0：

- （1）执行了 ERTN 指令且执行时 CSR.LLBCTL 中的 KLO 位不等于 1；
- （2）其它处理器核或 CacheCoherentI/Omaster 对该 LLbit 对应的地址所在的 Cache 行执行完成了一个 store 操作。

如果 LL-SC 指令对访问地址的存储访问属性不是 Cached，那么执行结果不确定。

需要注意的是，上述指令在计算内存地址时，需要将指令码中的 `si14` 左移 2 位后再与基址相加，即 $vaddr = GR[rj] + SignExtend(\{si14, 2'b0\}, 32)$ 。但是这些指令的汇编表示中所呈现的立即数地址偏移值仍以字节为单位，即其值是指令码中 `si14<<2`。

1.2.2.10 栅障指令概述

栅障指令是一种用于确保处理器中内存访问操作按预期顺序执行的命令。这类指令通过提供同步机制来维护程序的内存一致性模型。数据栅障 (DBAR) 指令用于在加载和存储操作之间建立栅障，确保这些操作的顺序执行。指令栅障 (IBAR) 指令则用于在同一处理器核心内同步存储操作与指令获取过程。

以数据栅障 DBAR 指令为例，其指令格式为 `dbar hint`。DBAR 指令用于完成 load/store 访存操作之间的栅障功能。其携带的立即数 `hint` 用于指示该栅障的同步对象和同步程度。`hint` 值为 0 是默认必须实现的，其指示一个完全功能的同步栅障。只有等到之前所有 load/store 访存操作彻底执行完毕后，“DBAR0”指令才能开始执行；且只有“DBAR0”执行完成后，其后所有 load/store 访存操作才能开始执行。

如果没有专门的功能实现，其它所有 `hint` 值都必须按照 `hint=0` 执行。

1.2.2.11 其他杂项指令概述

杂项指令包括多种用于特殊目的的命令。SYSCALL 指令用于触发系统调用异常，而 BREAK 指令则用于触发断点异常。这两种指令均可以通过其代码域传递额外信息给异常处理程序。此外，还有一组指令用于与 64 位恒定频率计时器 StableCounter 交互具有全局唯一的编号 (CounterID)。相关指令如 `RDCNTV{L/H}.W` 和 `RDCNTID` 可用于读取计时器的低 32 位、高 32 位以及 CounterID 信息。这些指令共同构成了处理特殊任务和事件的基础工具。

1.2.3 基础浮点数指令概述

浮点数指令可以分为浮点运算类指令、浮点比较指令、浮点转换指令、浮点搬运指令、浮点分支指令以及浮点普通访存指令。

1.2.3.1 浮点数据类型

浮点数据类型包括单精度浮点数和双精度浮点数，两者均遵循 IEEE754-2008 标准规范中的定义。

单精度浮点数的宽度为 32 比特，组织为如图 1-5 的格式：

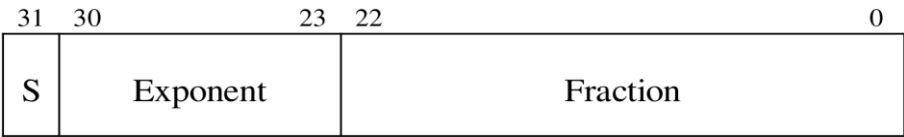


图 1-5 单精度浮点数格式

根据 S、Exponent 和 Fraction 各个域数值的不同，所表示的浮点数数值如

图 1-6 所示：

Exponent	Fraction	S	bit[22]	V
0	=0	0	0	+0
		1	0	-0
0	!=0	0	任意值	非规格化数，值为 $+2^{-126} \times (0.\text{Fraction})$
		1	任意值	非规格化数，值为 $-2^{-126} \times (0.\text{Fraction})$
[1, 0xFE]	任意值	0	任意值	规格化数，值为 $+2^{(\text{Exponent}-127)} \times (1.\text{Fraction})$
		1	任意值	规格化数，值为 $-2^{(\text{Exponent}-127)} \times (1.\text{Fraction})$
0xFF	=0	0	0	正无穷 ($+\infty$)
		1	0	负无穷 ($-\infty$)
0xFF	!=0	任意值	0	发信非数 (Signaling Not a Number, SNaN)
		任意值	1	静默非数 (Quiet Not a Number, QNaN)

图 1-6 单精度浮点数不同数值表示

双精度浮点数的宽度为 64 比特，组织为如图 1-7 的格式：



图 1-7 双精度浮点数格式

根据 S、Exponent 和 Fraction 各个域数值的不同，所表示的浮点数数值如图 1-8 所示：

Exponent	Fraction	S	bit[51]	V
0	=0	0	0	+0
		1	0	-0
0	!=0	0	任意值	非规格化数，值为 $+2^{-1022} \times (0.\text{Fraction})$
		1	任意值	非规格化数，值为 $-2^{-1022} \times (0.\text{Fraction})$
[1, 0x7FE]	任意值	0	任意值	规格化数，值为 $+2^{(\text{Exponent}-1023)} \times (1.\text{Fraction})$
		1	任意值	规格化数，值为 $-2^{(\text{Exponent}-1023)} \times (1.\text{Fraction})$
0x7FF	=0	0	0	正无穷 ($+\infty$)
		1	0	负无穷 ($-\infty$)
0x7FF	!=0	任意值	0	发信非数 (Signaling Not a Number, SNaN)
		任意值	1	静默非数 (Quiet Not a Number, QNaN)

图 1-8 双精度浮点数不同数值表示

有关 $\pm\infty$ 、SNaN 和 QNaN 的具体含义，请查看 IEEE754-2008 标准规范。

部分浮点指令（如浮点转换指令）也会操作定点数据，包括字（Word，简记 W，长度 32b）。字数据类型均采用二进制补码的编码方式。

1.2.3.2 浮点数指令寄存器

浮点数指令编程涉及到的寄存器包括浮点寄存器（Floating-point Register，简称 FR）、条件标志寄存器（Condition Flag Register，简称 CFR）以及浮点控制状态寄存器（Floating-point Control and Status Register，简称 FCSR）。

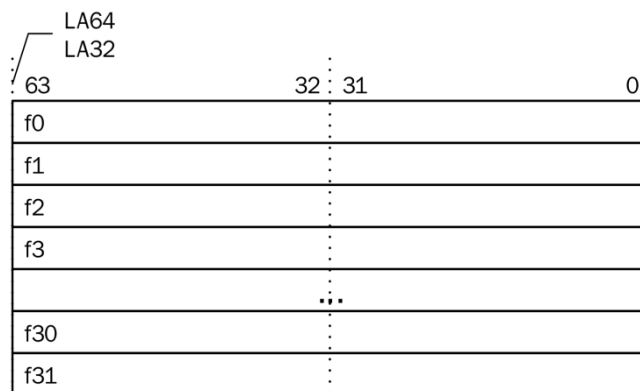


图 1-9 浮点数指令寄存器

浮点寄存器（Floating-point Register，简称 FR）共有 32 个，编号为 f0 到 f31，如上图 1-9 所示，每一个都可以进行读写操作。当仅实现单精度浮点数和字整数的浮点指令时，FR 的位宽为 32 比特。而在通常情况下，无论是在 LA32 还是 LA64 架构中，FR 的位宽都为 64 比特。基础浮点数指令与浮点寄存器之间存在着正交关系，也就是说，从架构的角度来看，任何一条浮点指令的操作数都可以使用这 32 个 FR 中的任意一个。

当浮点寄存器中记录的是一个单精度浮点数或字整数时，数据总是出现在浮点寄存器的[31:0]位上，此时浮点寄存器的[63:32]位可以是任意值。

条件标志寄存器（Condition Flag Register，简称 CFR）共有 1 个，记为 fcc0，可以进行读写操作。CFR 的位宽为 1 比特。浮点比较的结果会写入到条件标志寄存器中，如果比较结果为真，则置 1，否则置 0。浮点分支指令的判断条件来源于条件标志寄存器。

浮点控制状态寄存器（Floating-point Control and Status Register，简称 FCSR）共有 4 个，编号为 fcsr0 到 fcsr3，每个的位宽均为 32 比特。其中 fcsr1 到 fcsr3 是 fcsr0 中部分域的别名，即访问 fcsr1 到 fcsr3 实际上是访问 fcsr0 的某些特定域。当软件写入 fcsr1 到 fcsr3 时，fcsr0 中对应的域会被修改，而其余比特保持不变。其各个域定义如图 1-10 所示。

位	名字	读写	描述
4:0	Enables	RW	浮点操作 VZUI 例外各自允许触发例外陷入的使能位。 比特 4 对应 V，比特 3 对应 Z，比特 2 对应 O，比特 1 对应 U，比特 0 对应 I。
7:5	0	RO	保留域。读返回 0，且软件不允许改变其值。
9:8	RM	RW	舍入模式控制。其包含 4 个合法值，各自含义如下： 0: RNE，对应 IEEE 754-2008 中的 roundTiesToEven； 1: RZ，对应 IEEE 754-2008 中的 roundTowardZero； 2: RP，对应 IEEE 754-2008 中的 roundTowardsPositive； 3: RM，对应 IEEE 754-2008 中的 roundTowardsNegative。
15:10	0	RO	保留域。读返回 0，且软件不允许改变其值。
20:16	Flags	RW	自上次 Flags 域被软件清空后，各类产生但未陷入的浮点操作 VZUI 例外的累计情况。 比特 20 对应 V，比特 19 对应 Z，比特 18 对应 O，比特 17 对应 U，比特 16 对应 I。
23:21	0	RO	保留域。读返回 0，且软件不允许改变其值。
28:24	Cause	RW	最近一次浮点操作所产生的 VZUI 例外的情况。 比特 28 对应 V，比特 27 对应 Z，比特 26 对应 O，比特 25 对应 U，比特 24 对应 I。
31:29	0	RO	保留域。读返回 0，且软件不允许改变其值。

图 1-10 浮点控制状态寄存器各个域定义

FCSR1 是 FCSR0 中 Enables 域的别名，其位置与 FCSR0 中一致。
FCSR2 是 FCSR0 中 Cause 和 Flags 域的别名，各个域的位置与 FCSR0 中一致。
FCSR3 是 FCSR0 中 RM 域的别名，其位置与 FCSR0 中一致。

1.2.3.3 浮点例外

浮点例外是指当浮点处理单元不能以常规的方式处理操作数或浮点计算的结果时，浮点功能部件将产生相应的例外。基础浮点指令支持五个由 IEEE 754-2008 定义的浮点例外：

- (1) 不精确 (Inexact, I)
- (2) 下溢 (Underflow, U)
- (3) 上溢 (Overflow, O)
- (4) 除零 (Division by Zero, Z)
- (5) 非法操作 (Invalid Operation, V)

FCSR0 中的 Cause 域的每一位对应上述的一个例外。每条浮点指令执行结束后会将其例外的产生情况更新至 FCSR0 的 Cause 域中。FCSR0 对于每种浮点例外还包含一个使能位 (Enables 域)。使能位决定了浮点处理单元产生的例外是触发一个例外陷入还是设置一个状态标志。当某个浮点例外产生时，如果其对应的 Enable 位为 1，则将触发一个浮点例外陷入；如果其对应的 Enable 位为 0，则不会触发浮点例外陷入，而是将 FCSR0 中 Flag 域的对应位置 1。

一条浮点指令在执行过程中可以同时产生多个浮点例外。当浮点指令执行过程中产生了浮点例外但没有触发浮点例外陷入的时候，浮点处理单元将生成一个缺省的结果。不同的例外产生缺省结果的方式也不相同，图 1-11 列出了具体的生成规则。

域	描述	舍入模式	缺省结果
I	非精确	任何模式	舍入后的结果或上溢出后的结果
U	下溢	RNE	舍入后的结果，可能是 0, subnormal, 绝对值最小的 normal 数（单精度： $\pm 2^{-126}$ ，双精度： $\pm 2^{-1022}$ ）
		RZ	舍入后的结果，可能是 0, subnormal
		RP	舍入后的结果，可能是 0, subnormal, 最小的正 normal 数（单精度： $+2^{-126}$ ，双精度： $+2^{-1022}$ ）
		RM	舍入后的结果，可能是 0, subnormal, 最大的负 normal 数（单精度： -2^{-126} ，双精度： -2^{-1022} ）
O	上溢	RNE	根据中间结果的符号把结果置为 $+\infty$ 或 $-\infty$
		RZ	根据中间结果的符号把结果置为最大数
		RP	把负上溢修正为最小负数，把正上溢修正为 $+\infty$
		RM	把正上溢修正为最大正数，把负上溢修正为 $-\infty$
Z	被 0 除	任何模式	提供一个相应的带符号的无穷大数
V	非法操作	任何模式	提供一个 QNaN

图 1-11 浮点例外生成规则

1.2.3.4 浮点运算类指令概述

浮点运算类指令主要负责处理浮点数的基本数学运算。

以 FADD.S/D 指令为例，其指令格式为 fadd.s fd, fj, fk 或 fadd.d fd, fj, fk。其将浮点寄存器 fj 中的单精度/双精度浮点数加上浮点寄存器 fk 中的单精度/双精度浮点数，得到的单精度/双精度浮点数结果写入到浮点寄存器 fd 中。浮点加法运算遵循 IEEE 754-2008 标准中的 addition(x, y) 操作规范。

FADD.S:

FR[fd][31:0] = FP32_addition(FR[fj][31:0], FR[fk][31:0])

FADD.D:

FR[fd] = FP64_addition(FR[fj], FR[fk])

1.2.3.5 浮点比较指令概述

浮点数比较指令在处理器的浮点运算单元(FPU)或向量处理单元中实现，用于执行诸如等于、大于、小于等比较操作。

以 FCMP.cond. {S/D} 指令为例，其指令格式为 fcmp.cond.s cc, fj, fk 或 fcmp.cond.d cc, fj, fk。其将比较的结果存入指定的状态码(cc)。这条指令的 cond 有 22 种，这些比较条件以及判断的标准在图 1-12 中列出来。

助记符	cond	含义	True Condition	QNaN 是否报例外	对应 IEEE 754-2008 函数
CAF	0x0	否	无	否	
CUN	0x8	无法比较	UN		compareQuietUnordered
CEQ	0x4	相等	EQ		compareQuietEqual
CUEQ	0xC	相等或无法比较	UN EQ		
CLT	0x2	小于	LT		compareQuietLess
CULT	0xA	小于或无法比较	UN LT		compareQuietLessUnordered
CLE	0x6	小于等于	LT EQ		compareQuietLessEqual
CULE	0xE	小于等于或无法比较	UN LT EQ		compareQuietNotGreater
CNE	0x10	不等	GT LT		
COR	0x14	有序	GT LT EQ		
CUNE	0x18	无法比较或不等	UN GT LT		compareQuietNotEqual
SAF	0x1	否	无	是	
SUN	0x9	不是大于小于或等于	UN		
SEQ	0x5	相等	EQ		compareSignalingEqual
SUEQ	0xD	不是大于或小于	UN EQ		
SLT	0x3	小于	LT		compareSignalingLess
SULT	0xB	不是大于或等于	UN LT		compareSignalingLessUnordered
SLE	0x7	小于等于	LT EQ		compareSignalingLessEqual
SULE	0xF	不是大于	UN LT EQ		compareSignalingNotGreater
SNE	0x11	不等	GT LT		
SOR	0x15	有序	GT LT EQ		
SUNE	0x19	无法比较或不等	UN GT LT		

图 1-12 指令比较条件以及判断的标准

注：UN 表示无法比较、EQ 表示相等、LT 表示小于。当有两个操作数中有至少一个 NaN 时，这两个数就无法比较。

1.2.3.6 浮点转换指令概述

浮点转换指令主要用于将浮点数在单精度、双精度以及定点数之间进行转换。

以 FCVT.S.D 为例，其指令格式为 `fcvt.s.d fd, fj`。其选择浮点寄存器 `fj` 中的双精度浮点数转换为单精度浮点数，得到的单精度浮点数写入到浮点寄存器 `fd` 中。

1.2.3.7 浮点搬运指令概述

浮点搬运指令主要用于从内存中读取浮点数到处理器的浮点寄存器（加载），或者将浮点寄存器中的数据写回到内存中（存储）。

以 FMOV.{S/D} 指令为例，其指令格式 `fmov.s fd, fj` 或 `fmov.d fd, fj`。FMOV.{S/D} 将浮点寄存器 `fj` 的值按单精度/双精度浮点数格式写入到浮点寄存器 `fd` 中，如果 `fj` 的值不是单精度/双精度浮点数格式，则结果不确定。该指令操作是非算术的，不会引发 IEEE754 例外，也不修改浮点控制状态寄存器的 Cause 和 Flags 域。

1.2.3.8 浮点分支指令概述

浮点分支指令（Floating Point Branch Instruction）在计算机体系结构中用于基于浮点数运算的结果来控制程序的流程。这类指令通常依赖于浮点运算单元（FPU, Floating Point Unit）的状态或比较浮点值的结果来决定是否执行跳转。

以 BCEQZ 指令为例，其指令格式 `bceqz cj, offs21`。BCEQZ 对条件标志寄存器 `cj` 的值进行判断，如果等于 0 则跳转到目标地址，否则不跳转。指令的跳转目标地址是将指令码中的 21 比特立即数 `offs21` 逻辑左移 2 位后再符号扩展，所得的偏移值加上该分支指令的 PC。需要注意的是，BCEQZ 指令如果在写汇编时采用直接填入偏移值的方式，则汇编表示中的立即数应填入以字节为单位的偏移值，即指令码中 `offs21<<2`。

1.2.3.9 浮点普通访存指令概述

浮点数访存指令（Floating-point Load/Store Instructions）是处理器架构中用于处理浮点数数据在寄存器和内存之间传输的一类指令。这些指令允许程序将浮点数从内存加载到浮点寄存器中进行运算，或将浮点寄存器中的结果存储回内存。

以 FLD.D 为例，其指令格式为 `fld.d fd, rj, si12`。FLD.D 从内存取回一个双字的数据写入浮点寄存器 `fd`。

其过程如下表达式所述，首先计算虚拟地址 `vaddr`，这是通过寄存器 GR 中的值 `rj` 与 `si12` 符号扩展到 32 位后的结果相加得到的；接着对 `vaddr` 进行地址合规性检查；然后通过地址翻译将 `vaddr` 转换为物理地址 `paddr`；之后从内存中加载双字数据到 `paddr`，并将这个双字赋值给浮点寄存器 FR 中的 `fd`。

```
vaddr = GR[rj] + SignExtend(si12, 32)
AddressComplianceCheck(vaddr)
paddr = AddressTranslation(vaddr)
```

```
doubleword = MemoryLoad(paddr, DOUBLEWORD)
FR[fd] = doubleword
```

1.2.4 特权资源架构概述

龙芯架构中处理器核分为 4 个特权等级 (PrivilegeLeVel, 简称 PLV), 分别是 PLV0~PLV3。处理器核当前处于哪个特权等级由 CSR.CRMD 中 PLV 域的值唯一确定。

所有特权等级中, PLV0 是具有最高权限的特权等级, 也是唯一可以使用特权指令并访问所有特权资源的特权等级。PLV1~PLV3 这三个特权等级, 都不能执行特权指令访问特权资源, 不过三个特权等级在 MMU 采用映射地址翻译模式下具有不同的访问权限。

对于 Linux 系统来说, 架构中仅 PLV0 级可对应核心态, 同时建议以 PLV3 级对应用户态。所有特权指令仅在 PLV0 特权等级下才能访问。仅有一个例外情况, 当 CSR.MISC 中的 RPCNTL1/RPCNTL2/RPCNTL3 配置为 1 时, 可以在 PLV1/PLV2/PLV3 特权等级下执行 CSR.RD 指令读取性能计数器。

1.2.5 存储管理概述

内存物理地址空间范围是: $0 \sim 2^{PALEN}-1$ 。

在 LA32 架构下, PALEN 理论上是一个不超过 36 的正整数, 由实现决定其具体的值, 通常建议为 32。在 LA64 架构下, PALEN 理论上是一个不超过 60 的正整数, 由实现决定其具体的值。系统软件可以通过 CPUCFG 读取配置字 0x1 的 PALEN 域来确定 PALEN 的具体值。

龙芯架构中虚拟地址空间是线性平整的。对于 PLV0 级来说, LA32 架构下虚拟地址空间大小为 2^{32} 字节, LA64 架构下虚拟地址空间大小为 2^{64} 字节。不过对于 LA64 架构来说, 2^{64} 字节大小的虚拟地址空间并不都是合法的, 可以认为存在一些虚拟地址的空洞。

1.2.6 例外与中断概述

1.2.6.1 中断

龙芯架构支持线中断和消息中断两种中断形式, 其中线中断为必须实现的中断形式, 消息中断为可选择实现的中断形式, 是在线中断基础上的扩展。

龙芯架构下每个处理器核内部可记录 13 个线中断, 分别是: 1 个核间中断 (IPI), 1 个定时器中断 (TI), 1 个性能监测计数溢出中断 (PMI), 8 个硬中断 (HWI0~HWI7), 2 个软中断 (SWI0~SWI1)。所有的线中断都是电平中断, 且都是高电平有效。

龙芯架构中每个逻辑处理器核内部可记录 256 个消息中断, 可以包含处理器核外部输入的消息型核间中断和消息型硬中断。一个处理器核内部 CSR.MSGIS0~CSR.MSGIS3 四个 64 位的 CSR 依次记录了从 0~255 号消息中断有无被置起。每个处理器核内部 256 个消息中断的具体来源由实现决定, 软件开发人员需要参阅具体的芯片用户手册以获取相关信息。龙芯架构每个逻辑处理器核内 256 个消息中断之间采用固定优先级, 中断号越大的优先级越高, 即第 255 号消息中断优先级最高, 第 254 号消息中断次之, ……., 第 0 号消息中断优先级最低。

1.2.6.2 例外

TLB 重填例外的入口来自于 CSR.TLBREENTRY。机器错误例外的入口来自于 CSR.MERREENTRY。除上述两种例外之外的例外称为普通例外，其入口地址采用“入口页号 | 页内偏移”的计算方式。这里“|”是按位或运算。所有普通例外入口的入口页号相同，均来自于 CSR.EENTRY。

例外优先级遵循两个基本原则：其一，中断的优先级高于例外；其二，对于例外，取指阶段检测出的优先级最高，译码阶段检测出的优先级次之，执行阶段检测出的优先级再次之。

1.3 2K500 技术规格与特性

1.3.1 CPU 技术规格

龙芯 2K500 芯片是一款基于 LoongArch64 架构的处理器，专为嵌入式应用设计。该 CPU 采用单核设计，拥有单个 LA264 处理器核心，主频范围在 500MHz 到 800MHz 之间。CPU 缓存配置方面拥有 32KB 一级指令缓存、32KB 一级数据缓存、512KB 通用二级缓存。CPU 中内置 2D GPU 图形核心，支持 DVO 显示接口。

在寄存器配置方面，龙芯 2K500 芯片包含 32 个通用寄存器（GPRs），其中每个寄存器在 LA64 架构下均为 64 位宽。通用寄存器包括 \$r0 至 \$r31，其中 \$r0 的内容总是固定为 0，其余寄存器在体系结构层面没有特殊功能，但在使用中有所特定约定。常用寄存器如 \$r1 用于返回地址，\$r2 用于 TLS/线程信息指针，\$r3 为栈指针，\$r22 为帧指针，\$r4 到 \$r11 作为参数寄存器，\$r12 到 \$r20 为临时寄存器，\$r23 到 \$r31 则为静态寄存器。此外，浮点寄存器（FPRs）和向量寄存器（VRs）也是重要组成部分，其中 FPRs 有 32 个，VRs 在 LSX 扩展中使用 \$v0 到 \$v31，在 LASX 扩展中使用 \$x0 到 \$x31。控制状态寄存器（CSRs）仅能在特权模式（PLV0）下访问，用于管理异常处理、TLB 操作和性能监测等任务。这些寄存器配置确保了龙芯 2K500 的高效运行和广泛的应用适应性。

1.3.2 内存及存储技术规格

在内存以及外存规格上，它搭载了 512MB DDR3 内存和 256MB Nand Flash 存储。

1.3.3 系统软件支持

在系统软件支持上，龙芯 2K500 运行 U-Boot 2022 和 Linux 5.10 内核，并兼容 Buildroot 2021 和 OpenHarmony 3.2 文件系统。

1.3.4 工作参数

其工作参数包括 5V/2A 的电源输入、0℃到+70℃的工作温度范围以及 20%到 90%（无凝结）的环境湿度，存储温度则在-40℃到+85℃之间。该产品的尺寸为 93mm×63mm，适用于各种嵌入式系统应用。

第2章 龙芯 2K500 试验箱

2.1 龙芯 2K500 试验箱概述

2.1.1 试验箱设计理念与用途

龙芯 2K500 试验箱是基于龙芯 2K500 处理器设计的一款开发与测试平台,是为了满足教育、科研和产品开发需求而设计的一套完整硬件系统,旨在为用户提供一个稳定、灵活且具有多功能的实验环境。

试验箱的设计理念是支持多种计算任务和数据处理需求,适用于从教学和培训到研究和开发的多种应用场景,核心在于展示和利用龙芯架构的独特优势,帮助用户深入了解并充分发挥龙芯 2K500 芯片的性能和特点,主要体现在以下几个方面:

- (1) 全面性: 试验箱集成了龙芯 2K500 处理器的核心功能,涵盖了从基础外设到高级接口的各种硬件模块,为用户提供了一个完整的开发平台。
- (2) 易用性: 通过模块化设计和友好的用户界面,使得初学者也能快速上手,开展各种实验和开发工作。
- (3) 扩展性: 预留了丰富的扩展接口,方便用户根据需求添加自定义模块或功能板卡。
- (4) 实用性: 紧跟行业发展趋势,集成了多种现代接口,满足实际应用需求。

龙芯 2K500 试验箱的用途包括但不限于:

- (1) 芯片性能测试: 通过一系列实验来评估和优化龙芯 2K500 芯片的性能。
- (2) 应用开发与调试: 提供一个稳定的平台,用于开发和调试基于龙芯 2K500 芯片的应用程序。
- (3) 教学与培训: 作为教学实验设备,帮助学生和工程师了解龙芯架构的设计与实现。
- (4) 研究与创新: 支持学术研究和创新项目,促进新技术和新方法的开发。
- (5) 软件移植与优化: 为操作系统、驱动程序等软件的移植和性能优化提供环境。

2.1.2 试验箱组成结构

龙芯 2K500 试验箱的整体结构设计旨在提供一个功能多样且易于使用的实验环境。开发板集成了丰富的硬件资源,支持多样化的实验需求,实物图如图 2-1。



图 2-1 龙芯 2K500 试验箱实物图

龙芯 2K500 试验箱的组成结构不仅包括核心硬件，还配备了丰富的配套附件和外围设备。以下是详细的组成清单：

(1) 核心硬件：

- a) 试验箱主板：搭载龙芯 2K500 CPU 核心板，集成重力传感器、温湿度传感器、SIM 卡、TF 卡、LED 灯、SATA 接口、以太网接口、4*4 键盘、七段数码管、DAC 接口、RS485 接口、CAN 接口、声卡，及多组扩展 GPIO 接口。
- b) 扩展板：提供蓝牙、ZigBee、RFID 智能卡、点阵、舵机、摄像头、高速电机多项扩展模块。

(2) 配套线缆：

- a) 杜邦线：用于连接各种引脚和接口
- b) 超五类网线：支持高速网络连接
- c) VGA 传输线：连接显示设备

(3) 显示设备：

LCD 屏与主板直接连接，使用对应接口或使用 QT 进行界面的开发。

(4) 控制设备：

红外遥控器，用于远程控制和输入。

(5) 外围设备：

- a) 舵机配件：用于机械控制实验
- b) Debug 数据线：用于程序调试和数据传输
- c) 12V 电源适配器：为整个系统供电

从教学角度来看，这种配置也非常适合课堂使用，教师可以根据课程需求，灵活地安排不同类型的实验，从基础的 GPIO 控制到复杂的网络通信和机械控制实验。

试验箱提供的配件使得用户可以立即开始各种实验和开发工作，而无需额外采购，主板和扩展板的组合为用户提供了丰富的实验可能性，而配套的各种线缆和附件则大大提高了试验箱的实用性和便利性。随试验箱提供的资料库和设计图可以帮助用户进行有效的管理和维护，使其更好地利用龙芯架构进行各种实验。

2. 2 试验箱硬件接口

本节将详细介绍龙芯 2K500 试验箱的硬件接口，包括开发板的主要参数、引脚定义、整体架构以及主板和拓展版上的各种接口和模块。

试验箱的硬件接口直接影响到用户的开发体验和可实现的应用范围，无论是进行基础的嵌入式系统教学，还是开发高级的应用程序，对硬件接口的深入理解都是不可或缺的。

2. 2. 1 开发板参数

龙芯 2K500 试验箱主板基于龙芯 2K500 处理器，采用标准化贴片焊接工艺，接口丰富，具有稳定、安全、可靠等特点，可广泛应用于国防、电力、交通、医疗、金融、通信、能源等行业领域。

开发板参数是了解龙芯 2K500 试验箱性能和功能的重要指标，涵盖了处理器、内存、存储、网络和各种接口的基本信息，具体见表 2-1。

表 2-1 龙芯 2K500 试验箱硬件参数

类型	项目	参数
2K500 核心板	CPU	龙芯 2K500（LoongArch64 架构兼容）
	CPU 核心数	单核
	CPU 主频	500-800MHz
	内存容量	1GB RAM
2K500 主板	存储容量	256MB NAND FLASH
	USB 接口	USB2.0*1、USB3.0*1、USB OTG*1
	网络接口	10/100/1000 兆自适应网口、WIFI*1、移动数据接口*1
	视频接口	VGA 接口*1、4.3 寸触控 LCD 显示屏
	串口	USB 转串口*1、232 串口*2、RS485 串口*1
	模数数模转换	DAC*1、ADC*2
	可编程模块	红绿黄 LED*1 组、SD 卡座*1、陀螺仪*1、蜂鸣器*1、RTC、4*4 键盘、7 段数码管*4、温湿度传感器*1、红外接收*1
	音频接口	拾音器*2、MIC IN 和 LINE OUT 支持

	拓展接口	GPIO 拓展*4、PWM 拓展*4、I2C 拓展*1、串口 TTL 拓展*2、SPI 拓展*1
	其他接口	CAN 总线接口*1、PCIE 接口*1、JTAG 接口*1
	电源	12V 电源接口*1、电源开关*1、复位开关*1
拓展板	拓展板模块	RFID*1、Zigbee*1、16*16 数显矩阵、电机*1、舵机*1、摄像头*1、蓝牙*1
工作参数	工作温度	0℃ ~ +70℃
	环境湿度	20% ~ 90%（无凝结）
	存储温度	-40℃ ~ +85℃

2.2.2 开发板引脚定义

龙芯 2K500 试验箱开发板的引脚定义是硬件设计的核心内容，它详细描述了每个引脚的功能和连接方式。理解这些引脚定义对于进行硬件扩展、外设连接和底层驱动开发至关重要。

由于整体引脚定义角度，本节仅给出部分引脚定义，详情请参考主板、拓展板设计图，及资料库中的龙芯 2K500 处理器用户手册。下面的引脚电路设计图展示了开发板上部分接口和模块的详细连接方式：

上电按钮设计图如图 2-2：

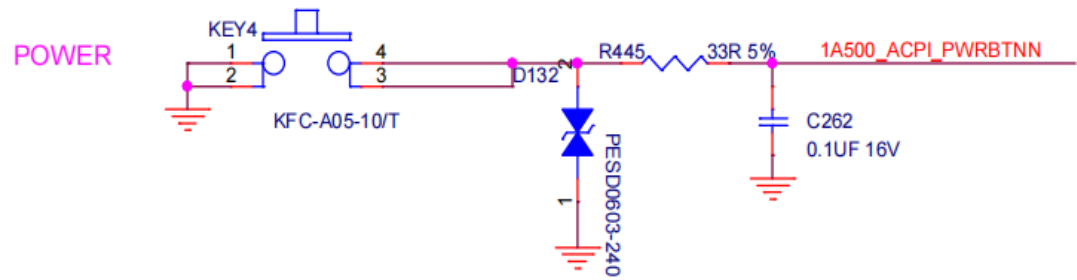


图 2-2 上电按钮设计图

引脚接口 1 & 2 如图 2-3 和 2-4：

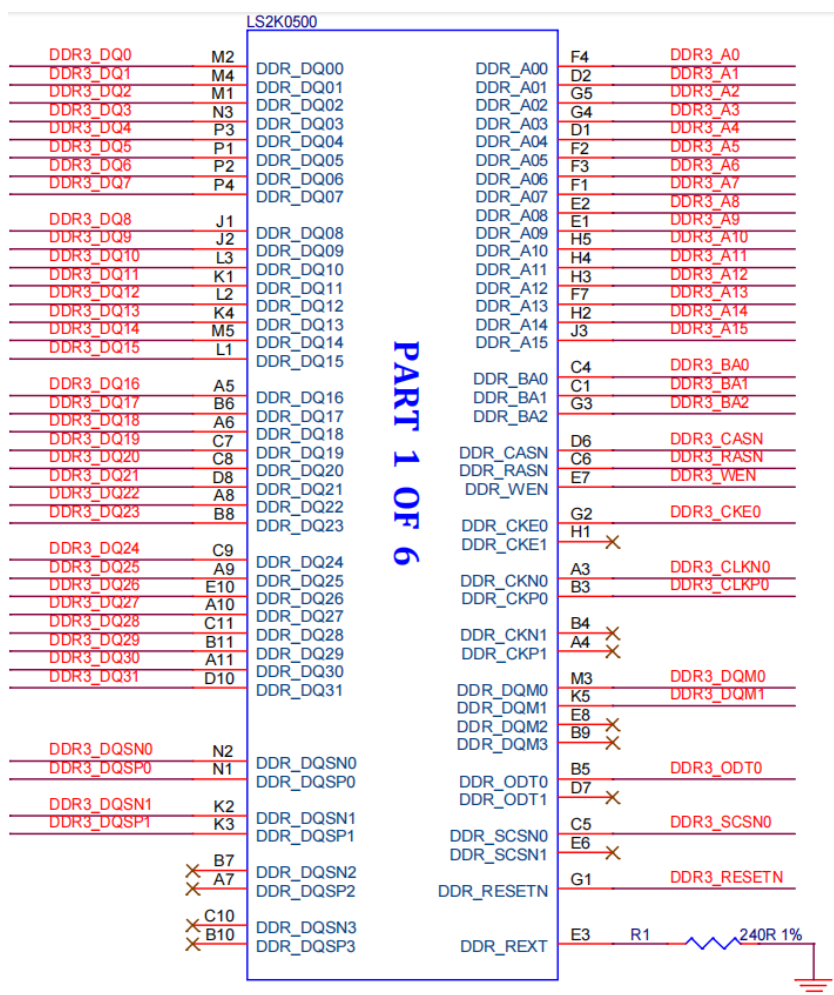


图 2-3 引脚接口 1

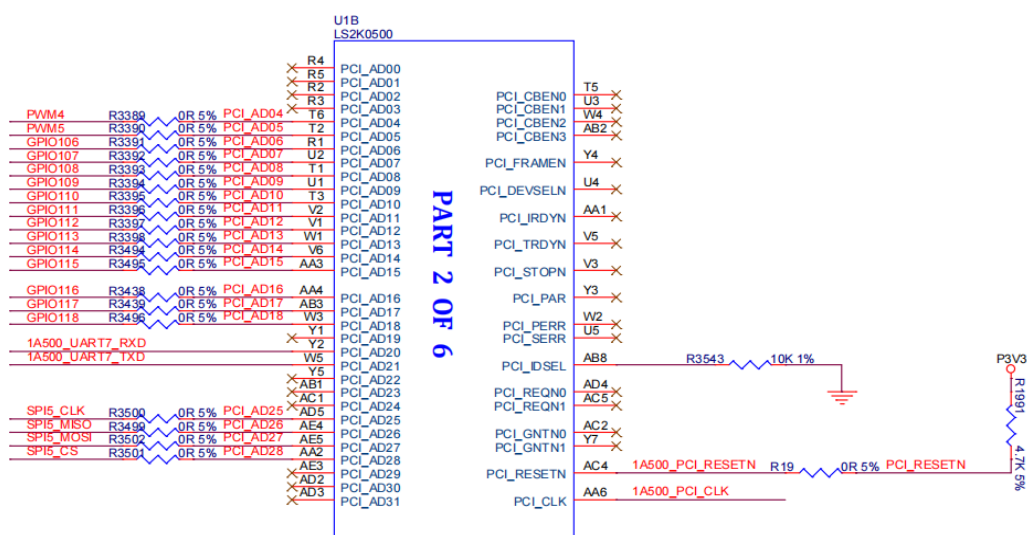


图 2-4 引脚接口 2

引脚接口及方向标注见图 2-5:

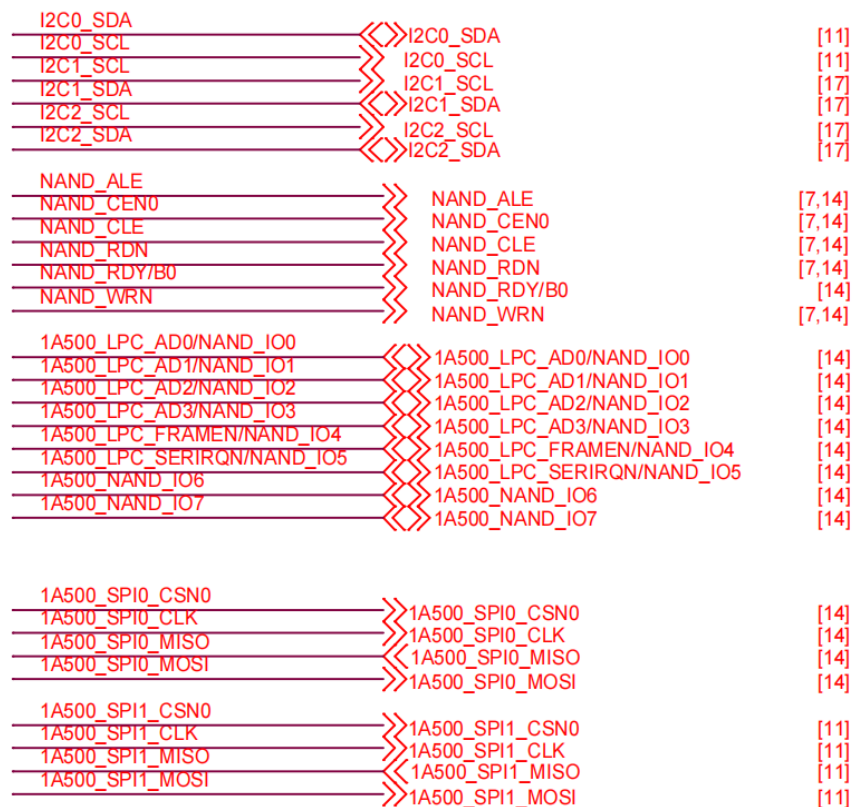


图 2-5 引脚接口及方向标注

USB 接口使用图见图 2-6:

Type A USB2.0(USB2/3)

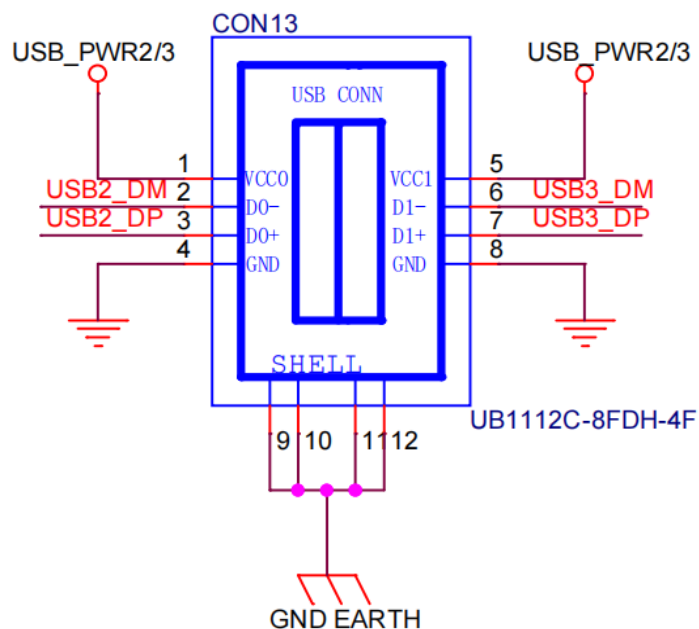


图 2-6 USB 接口使用图

以下是一些关键点，有助于理解这些图表:

- (1) 引脚编号：每个引脚都有唯一的编号，用于在硬件设计和软件编程中精确定位。
- (2) 信号名称：每个引脚旁边的标签表示该引脚的功能或所连接的信号。
- (3) 多功能引脚：某些引脚可能具有多个功能，通过软件配置选择使用哪个功能。
- (4) 电源和接地：注意标记为 VCC、GND 等的引脚，它们提供电源和接地连接。
- (5) 总线结构：注意 SPI、I2C、UART 等总线的引脚分组，其连接方式对于通信协议的实现很重要。
- (6) 特殊功能：一些引脚可能具有中断、PWM 或模拟功能，这些在进行特定应用开发时非常有用。
- (7) 用户手册：通过参考龙芯 2K500 处理器用户手册，可以先查询到所需功能的引脚名称/编号，再通过设计图查看连接方式，从而完成实际开发，开发手册中的部分定义如图 2-7 所示：

信号名称	类型	描述
DDR_DQ[31:0]	I/O	DDR3 SDRAM 数据总线信号
DDR_DQSP[3:0] DDR_DQSN[3:0]	DIFF I/O	DDR3 SDRAM 数据选通
DDR_DQM[3:0]	O	DDR3 SDRAM 数据屏蔽
DDR_A[15:0]	O	DDR3 SDRAM 地址总线信号
DDR_BA[2:0]	O	DDR3 SDRAM 逻辑 BANK 地址信号
DDR_WEN	O	DDR3 SDRAM 写使能信号
DDR_CASN	O	DDR3 SDRAM 列地址选择信号
DDR_RASN	O	DDR3 SDRAM 行地址选择信号
DDR_SCSN[1:0]	O	DDR3 SDRAM 片选信号
DDR_CKE[1:0]	O	DDR3 SDRAM 时钟使能信号

图 2-7 DDR3 引脚定义

理解这些引脚定义需要一定的硬件知识基础。对于初学者来说，可以先关注常用的接口，如 GPIO、I2C 等，随着经验的积累再逐步深入理解其他复杂的接口。

在实际开发中，这些引脚定义图将成为重要参考的参考资料，当连接外部设备、设计扩展板或调试硬件问题时，这些图表将提供必要的信息。通过结合实际的硬件操作，你将能够更好地理解龙芯 2K500 试验箱的硬件结构，为后续的开发工作打下坚实的基础。

2.2.3 开发板整体架构

龙芯 2K500 试验箱开发板的整体架构是理解系统功能和性能的关键。通过分析整体架构，我们可以清晰地看到各个组件之间的关系和数据流动路径，下面我们将通过两张重要的图表来详细说明开发板的架构。

核心板架构图如图 2-8：

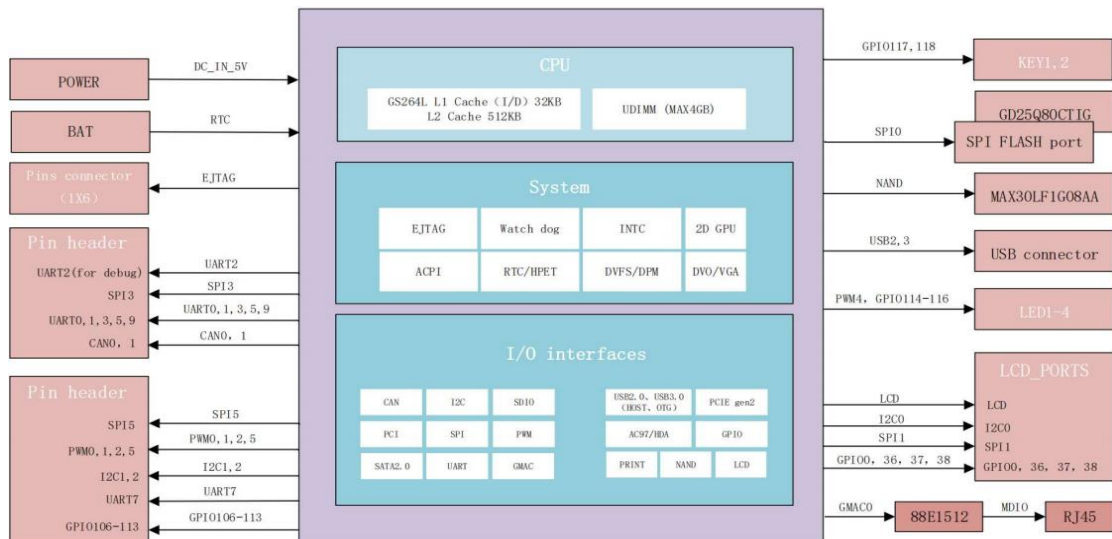


图 2-8 核心板架构图

核心板架构图展示了龙芯 2K500 CPU 与各种接口和功能模块之间的连接关系。主要包括：

- (1) DC_IN_5V: 电源输入，为整个核心板提供电力。
- (2) RTC: 实时时钟，用于维护系统时间。
- (3) EJTAG: 调试接口，用于程序调试和系统诊断。
- (4) 通信接口：UART/CAN/SPI/I2C: 通信接口，UART 用于串行通信，CAN 用于工业控制网络通信，SPI 为高速同步串行接口，I2C 用于连接低速外设。
- (5) GPIO: 通用输入输出接口，用于连接各种外部设备。
- (6) PWM: 脉宽调制接口，用于控制电机或 LED 亮度等。
- (7) NAND: 闪存接口，用于存储操作系统和用户数据。
- (8) USB2/3: 通用串行总线接口，支持高速数据传输和外设连接。
- (9) LCD: 显示接口，用于连接液晶显示器。
- (10) GMAC: 千兆以太网接口，提供高速网络连接。

这些接口和模块围绕龙芯 2K500 CPU，形成了一个功能完整的嵌入式系统，方便地与各种外部设备进行交互。

CPU 晶振频率如图 2-9:

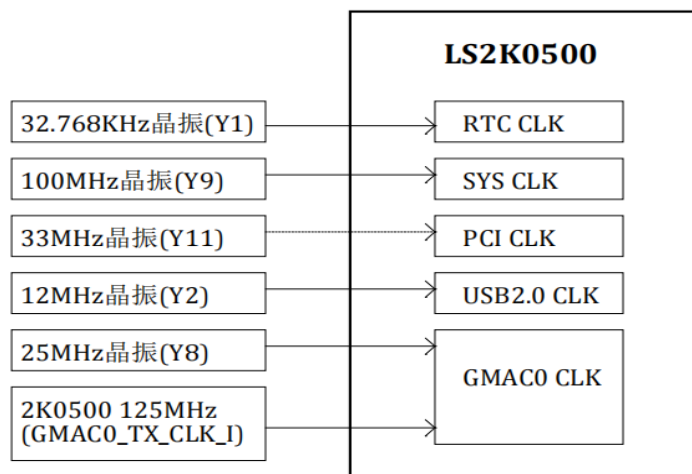


图 2-9 CPU 晶振频率图

CPU 晶振频率图详细展示了龙芯 2K500 CPU 及其相关模块的时钟频率配置：
(1) RTC CLK (实时时钟): 32.768 KHz 这个低频时钟用于保持系统的实时时间，即使在低功耗模式下也能持续运行。

(2) SYS (系统时钟): 100 MHz 作为系统的主时钟，为 CPU 和其他核心模块提供时钟信号。

(3) PCI 时钟: 33 MHz 符合 PCI 总线标准的时钟频率，用于 PCI 设备的通信。

(4) USB 2.0 时钟: 12 MHz 满足 USB 2.0 规范的基准时钟频率。

(5) GMAC0 (千兆以太网控制器) 时钟: 25 MHz 晶振为以太网控制器提供基准时钟，支持高速网络通信；2K500 125 MHz 为 CPU 核心时钟。

该图展示了系统协调各个模块工作的不同晶振，合理的晶振配置在确保系统的正常运行的同时带来了功耗和性能的平衡。

核心板架构图和晶振图有助于开发者进行系统级的设计和优化，为调试和性能分析提供了重要参考。在进行驱动开发、系统调优或是硬件扩展时，这些架构图将非常有帮助。

2.2.4 主板接口及模块

龙芯 2K500 试验箱主板集成了丰富的接口和功能模块，其主要接口及模块如图所示，本节将简要介绍实验书中使用到的接口和模块，帮助用户快速了解和使用这些资源。

主板接口及模块标注图如图 2-10 所示：

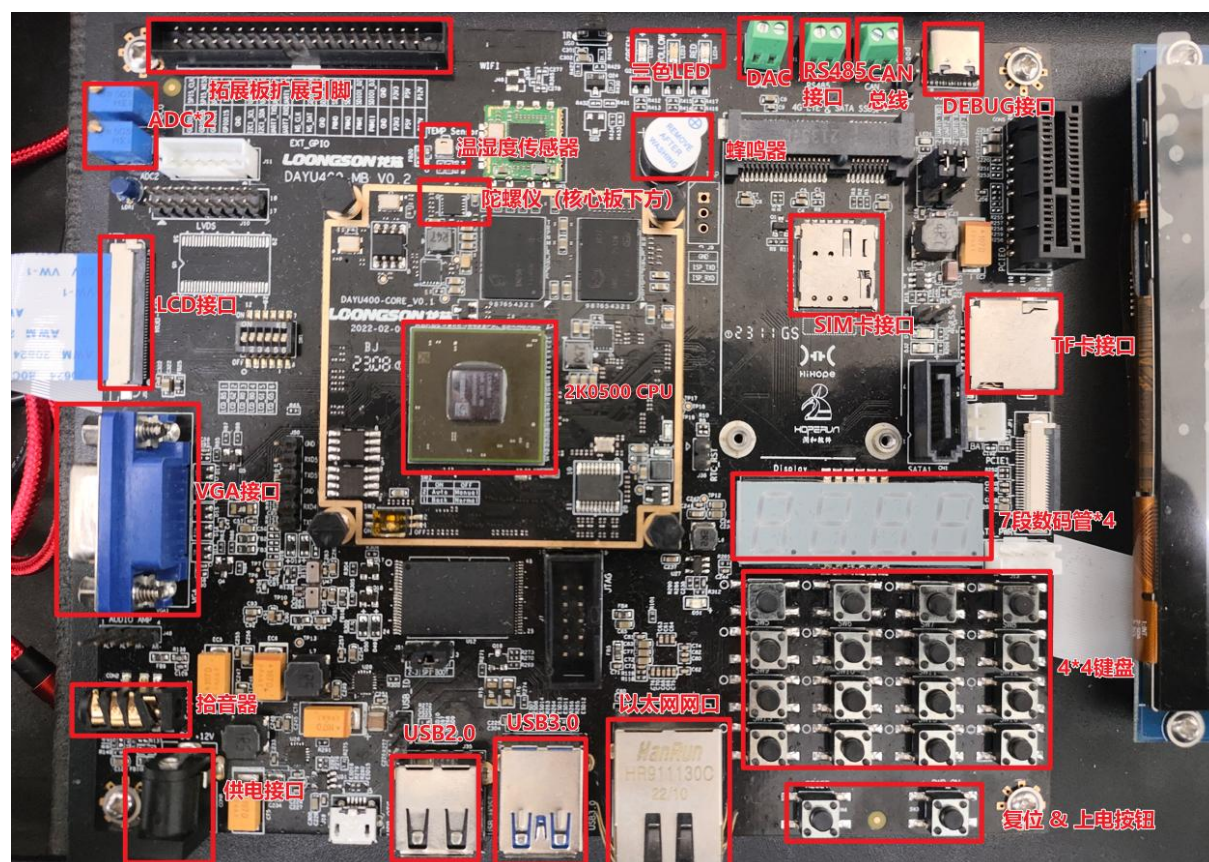


图 2-10 主板接口及模块标注图

(1) 供电口： 提供稳定的 12V 电源输入，确保整个系统的正常运行。

(2) DEBUG 接口：用于系统调试和程序下载，本实验书主要通过该串口查看系统运行信息及调试程序，其原理图如图 2-11 所示：

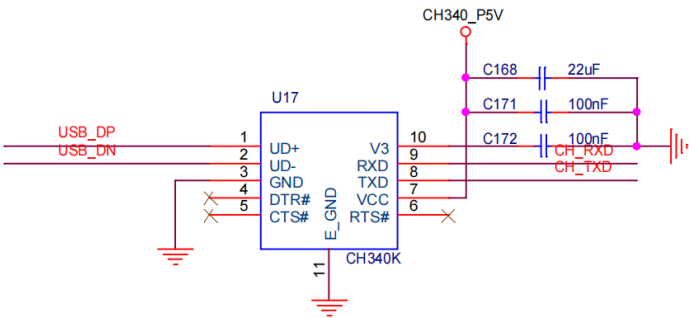


图 2-11 DEBUG 接口原理图

(3) 复位按钮：用于硬件复位系统，在程序卡死或需要重新启动系统时使用。其原理图如图 2-12：

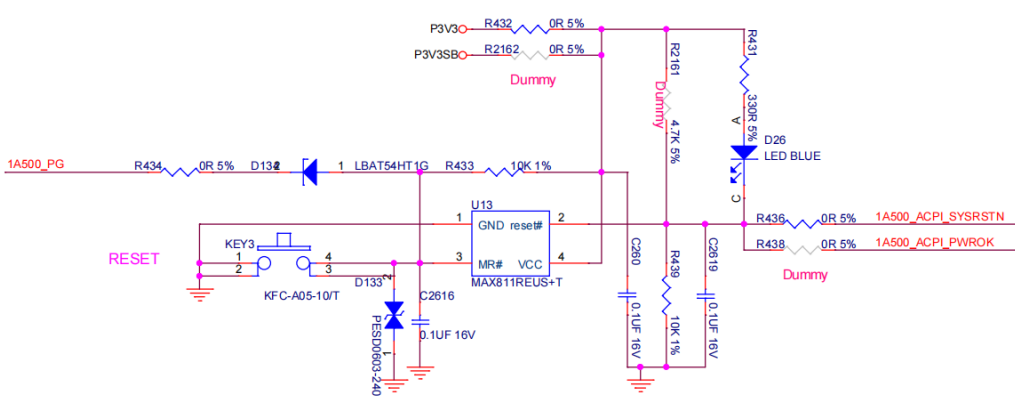


图 2-12 复位按钮原理图

(4) 上电按钮：控制系统上电，即开机操作。其原理图如图 2-13：

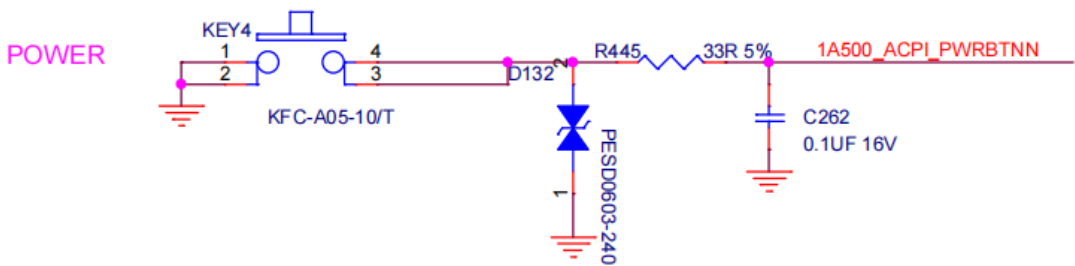


图 2-13 上电按钮原理图

(5) USB2.0 接口：用于连接 USB 设备，支持即插即用功能。在本实验书中的主要作用是连接存储设备和摄像头，存储设备用于拷贝编译后的可执行程序，摄像头会在摄像头实验章节使用到。其原理图如图 2-14：

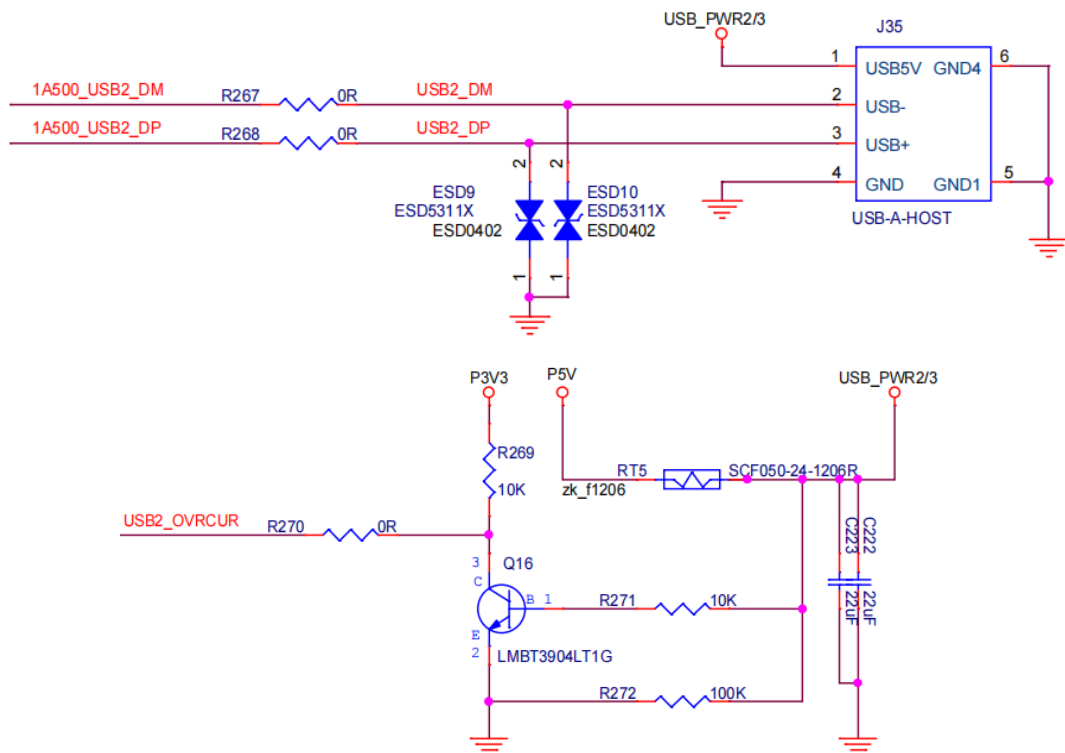


图 2-14 USB2.0 接口原理图

(6) 三色 LED： 可编程控制的红、黄、绿信号指示灯，用于显示系统状态或作为视觉输出。其原理图如图 2-15：

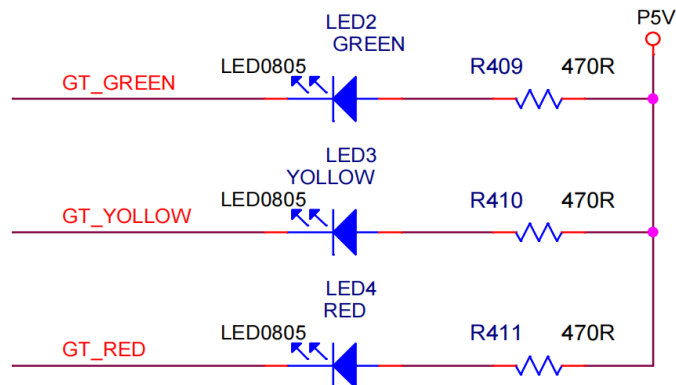


图 2-15 三色 LED 原理图

(7) 蜂鸣器： 提供音频反馈，可用于报警或用户交互。原理图如图 2-16：

BEEP

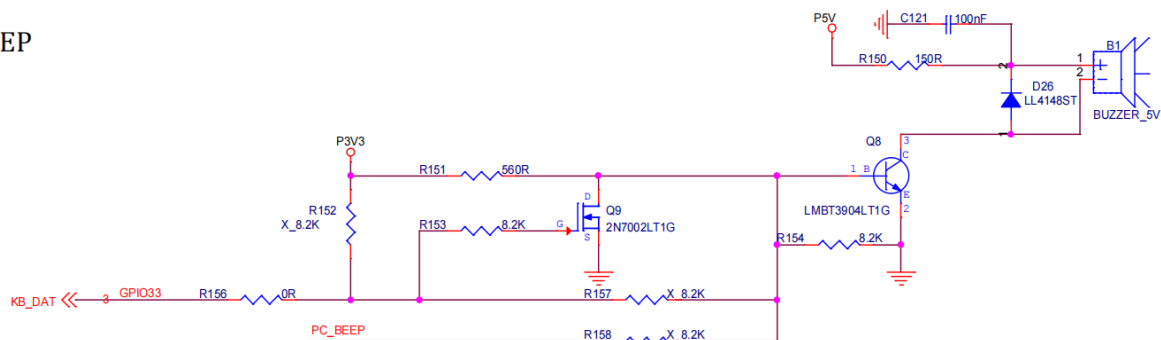
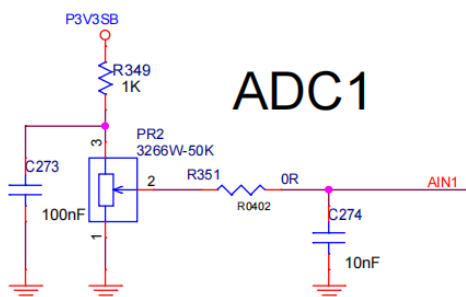
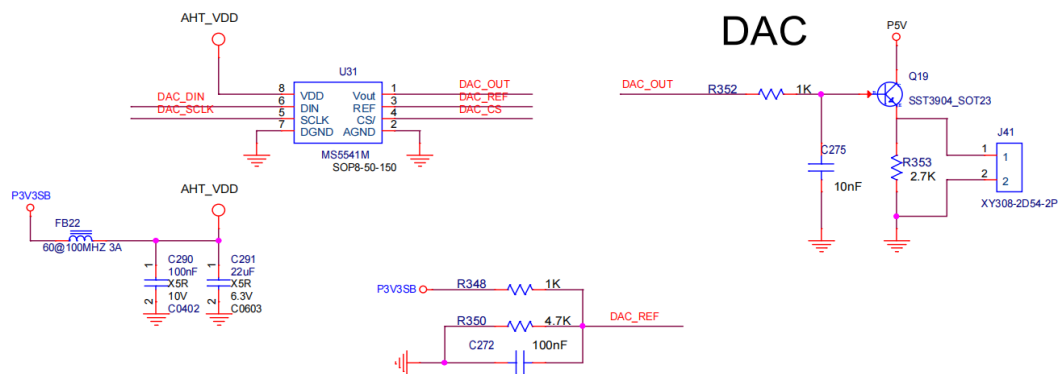


图 2-16 蜂鸣器原理图



(9) DAC (数模转换器): 将数字信号转换为模拟信号。原理图如图 2-18:

The schematic diagram illustrates the DAC interface circuit. It features three input signals: LQ1A18/PR6_START (GPIO132), LQ1A17/PR6_READY (GPIO133), and LQ1A18/PR6_ENABLE (GPIO134). These signals are connected to a DAC circuit. The DAC has three inputs: DAC_DIN, DAC_SCLK, and DAC_CS. The DAC is powered by AHT_VDD. The circuit includes three resistors: R399 (10K), R400 (10K), and R401 (10K). The DAC output is connected to the AHT_VDD pin.



(10) RS485 接口：用于长距离、抗干扰能力强的串行通信，常用于工业控制领域。原理图如图 2-19：

(10) RS485 接口：用于长距离、抗干扰能力强的串行通信，常用于工业控制领域。原理图如图 2-19：

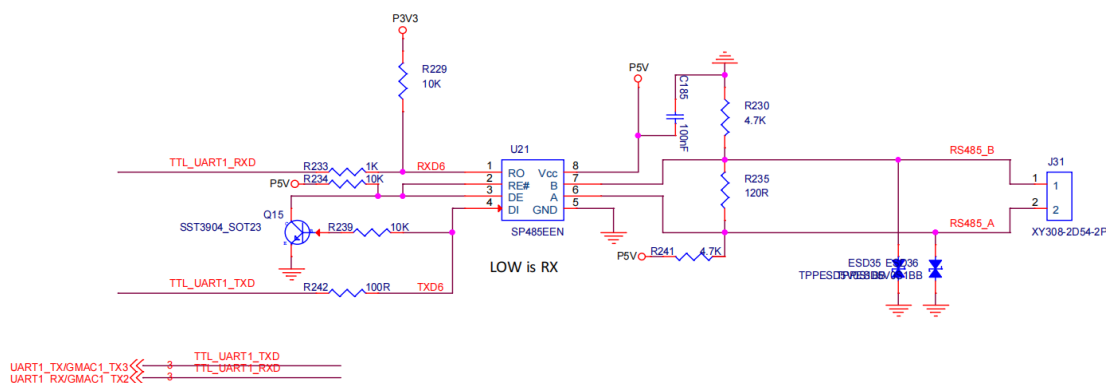


图 2-19 RS485 接口原理图

(11) CAN 总线接口：用于高可靠性的分布式控制系统通信，广泛应用于汽车电子等领域。原理图如图 2-20：

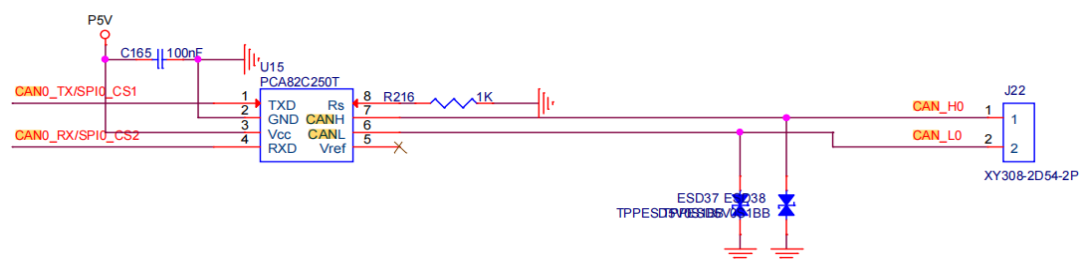


图 2-20 CAN 总线接口原理图

(12) 温湿度传感器：用于环境监测，可以测量周围的温度和湿度。原理图如图 2-21：

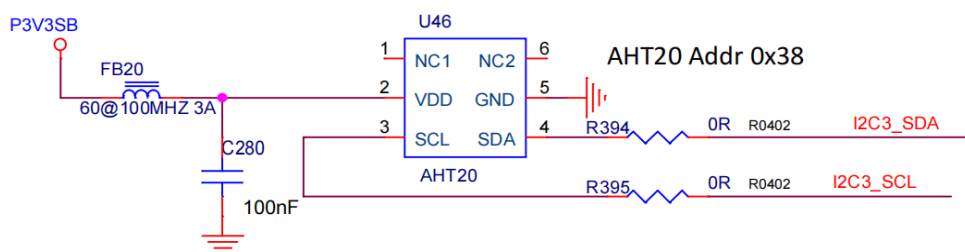


图 2-21 温湿度传感器原理图

(13) 陀螺仪：测量角速度和角度变化，用于姿态检测和运动控制。原理图如图 2-22：

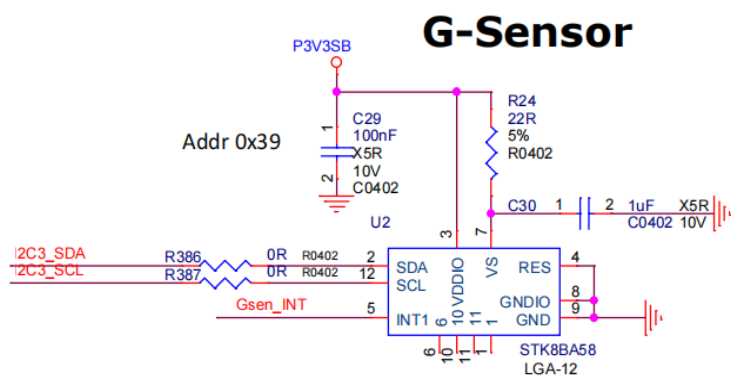


图 2-22 陀螺仪原理图

(14) LCD 接口：连接液晶显示屏，用于图形化用户界面的显示。原理图如图 2-23：

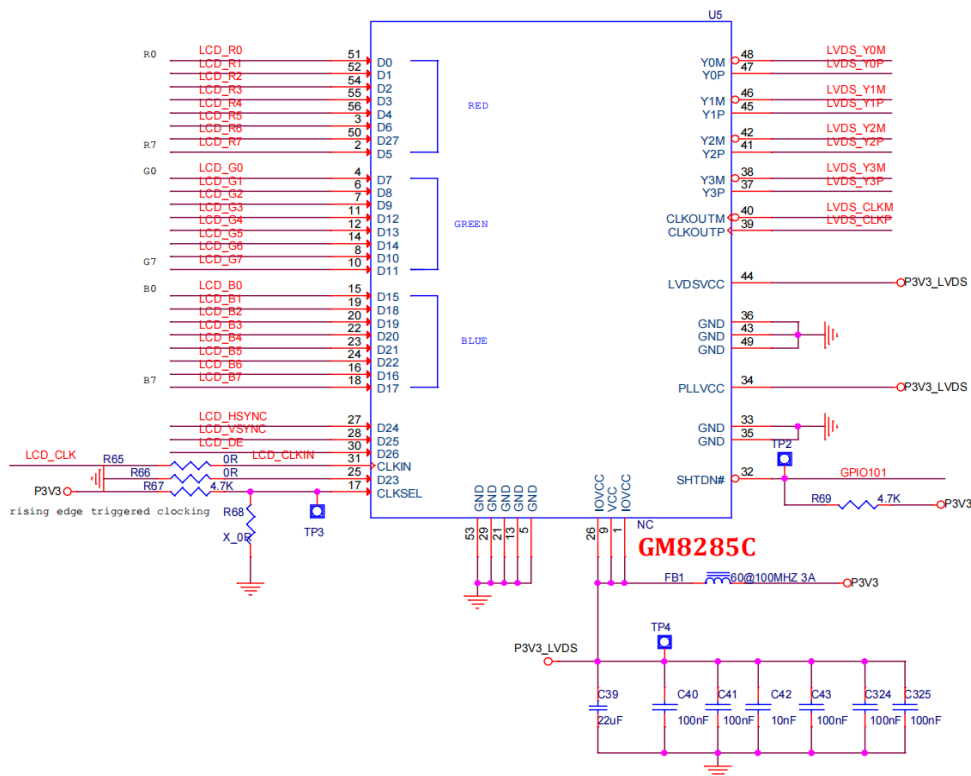


图 2-23 LCD 接口原理图

(15) 7 段数码管：用于显示数字和简单字符，常用于显示计数或状态信息。原理图如图 2-22：

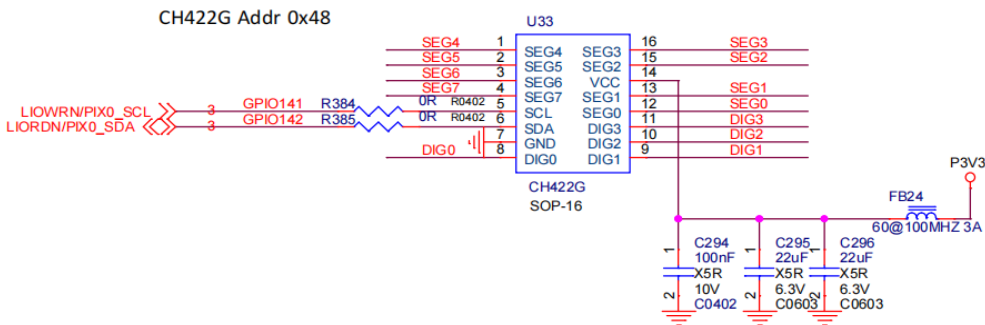


图 2-24 7 段数码管原理图

(16) 拾音器：用于音频输入，可以采集环境声音或语音指令，实验中使用四段式耳机进行连接。原理图如图 2-25：

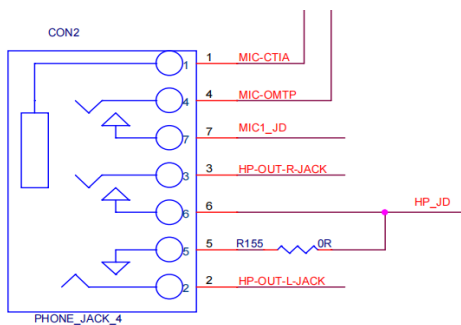


图 2-25 拾音器原理图

(17) 4*4 键盘： 提供 16 个按键的输入矩阵，用于用户交互和数据输入，原理图如图 2-26。

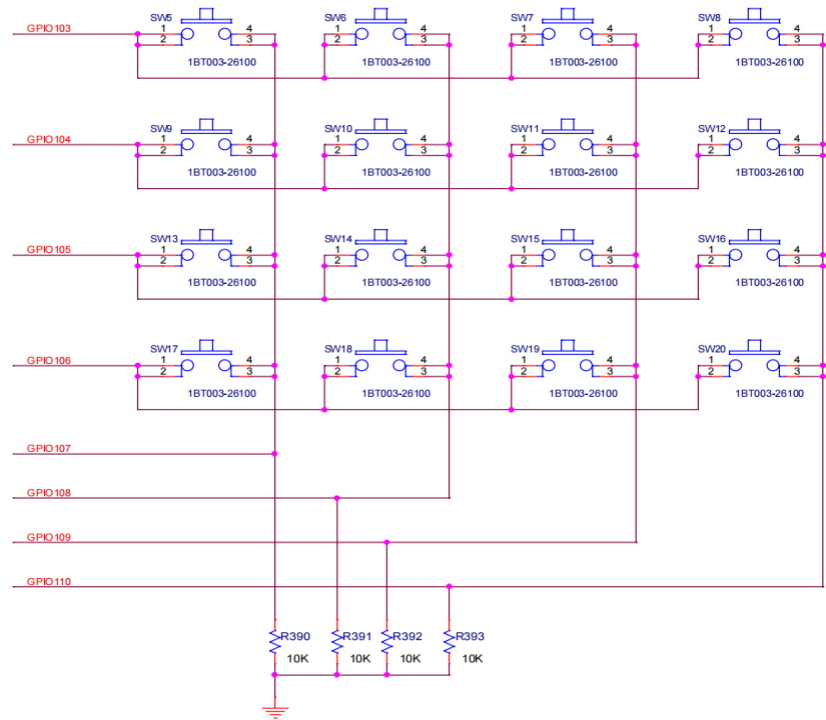


图 2-26 4*4 键盘原理图

(18) 拓展板扩展引脚： 提供额外的 GPIO 接口，用于连接主板上方的拓展板部分。

在使用这些接口和模块时，应当仔细阅读相关文档，了解其工作原理和使用方法，以避免不当操作造成的损坏。

2.2.5 拓展版接口及模块

龙芯 2K500 试验箱的拓展版提供了额外的功能模块和接口，进一步增强了系统的实用性和灵活性。本节将重点介绍实验中使用到的拓展板接口及模块，拓展板实物图如图 2-27。

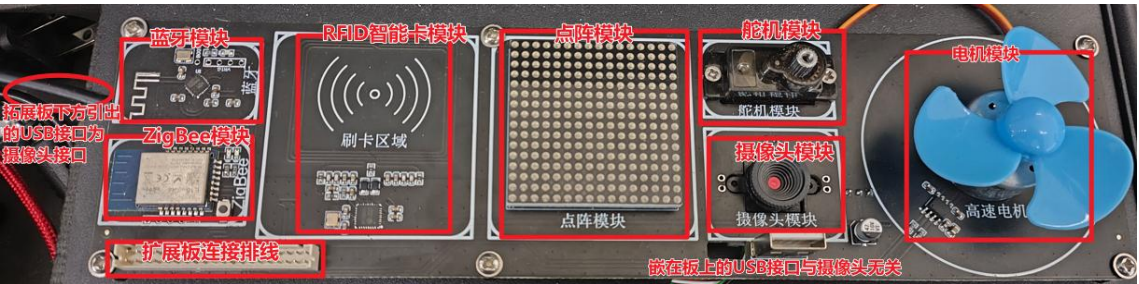
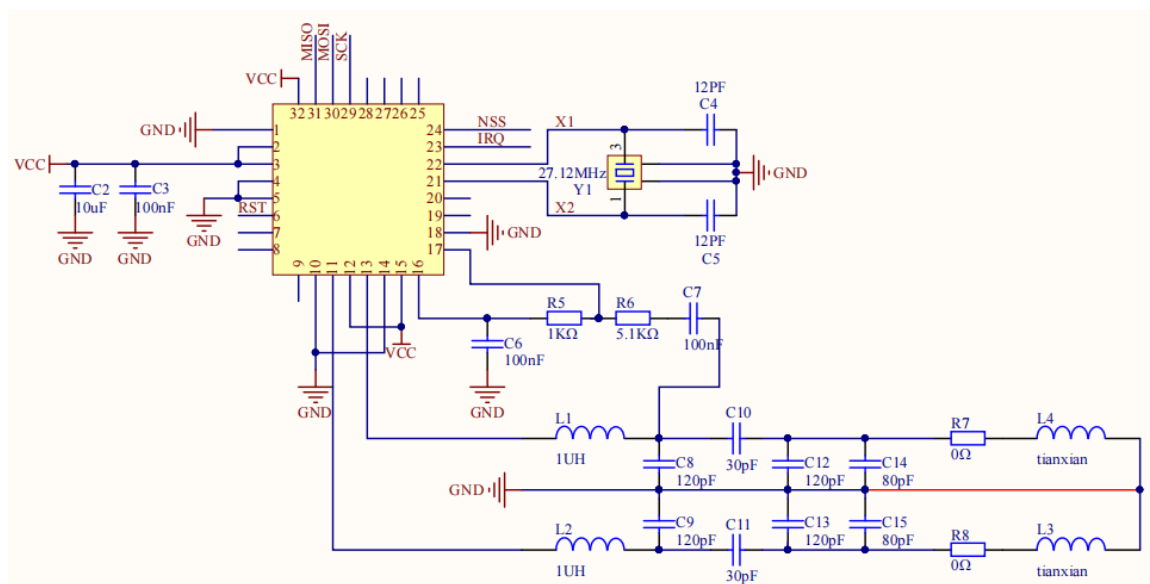


图 2-27 拓展板接口及模块

(1) 扩展板连接排线： 如图 2-28 所示，扩展板连接排线是连接主板和拓展版的关键组件。它传输电源、数据和控制信号，确保两块板之间的通信和协同工作。使用时应注意正确的连接方向和牢固性，以避免接触不良导致的问题，正确的连接方式如下，排线插好后无需拔出也可以收起试验箱，应避免频繁的插拔损伤排

(2) RFID 智能卡模块: RFID (射频识别) 智能卡模块采用 RC522 芯片, 这是一款广泛使用的非接触式读写芯片。其支持 ISO/IEC 14443 A/MIFARE 协议, 使用 SPI 通信接口, 有低功耗、内置收发器、支持多种加密方式等优点。原理图如图 2-27:



(3) 点阵模块：点阵模块为 16*16 大小，使用 AiP1944 芯片控制。AiP1944 是带键盘扫描接口的大笔段 LED 驱动控制专用电路，支持 16 位*16 段（本实验书使用）到 8 位 24 段显示模式。采用功率 CMOS 工艺，内部集成有 MCU 数字接口、数据锁存器、键盘扫描等电路。原理图如图 2-30：

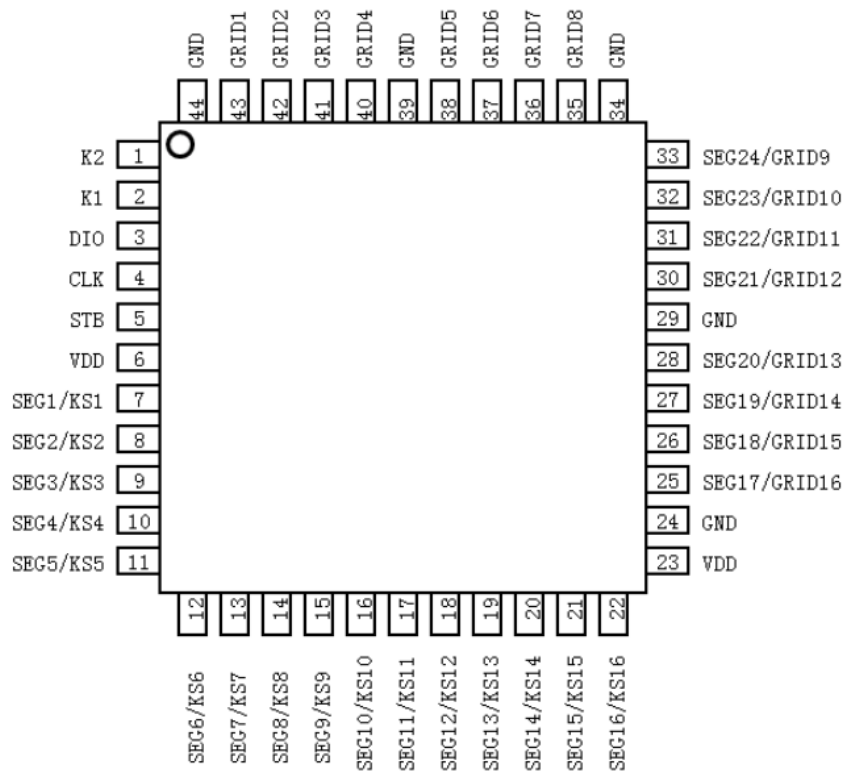


图 2-30 点阵模块原理图

(4) 电机模块：电机模块使用 TC118S 控制芯片，该芯片单通道内置功率 MOS 全桥驱动，支持驱动前进、后退、停止及刹车功能，内置迟滞热效应过热保护功能。原理图如图 2-31：

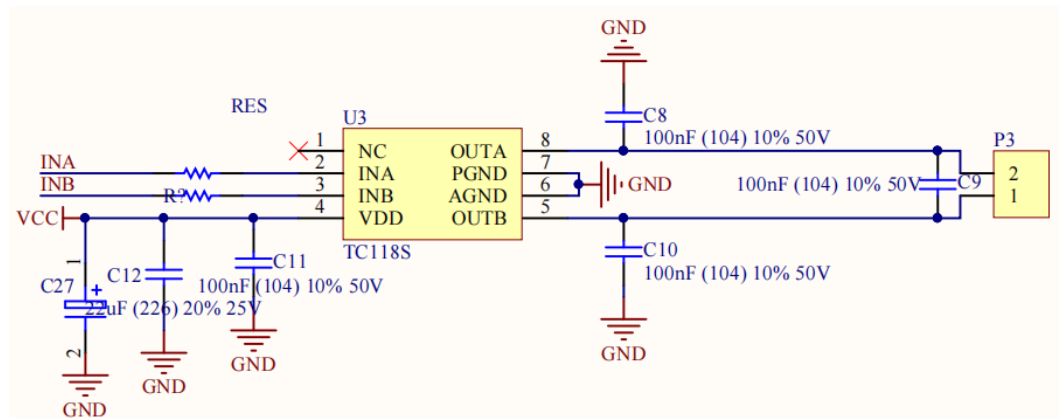


图 2-31 电机模块原理图

这些拓展模块大大增加了龙芯 2K500 试验箱的应用范围，使其能够满足从简单的嵌入式控制到复杂的智能系统开发的各种需求。

2.3 试验箱软件环境

2.3.1 U-boot

U-boot (Universal Boot Loader) 是龙芯 2K500 试验箱中使用的引导加载程序，它在系统启动过程中扮演着至关重要的角色。U-boot 负责初始化硬件、

准备系统环境，并最终加载操作系统内核。

U-boot 的主要功能：

- (1) 硬件初始化：
初始化 CPU、内存控制器、串口等关键硬件。
- (2) 内存检测和配置：
检测系统内存大小及映射。
- (3) 引导选项：
支持从多种存储介质启动，如 NAND Flash、NOR Flash、SD 卡等。
提供交互式命令行界面，允许用户修改启动参数。
- (4) 环境变量管理
- (5) 加载和执行内核

龙芯 2K500 试验箱使用的 U-boot 特性：

- (1) 适配支持 USB1.0 (OHCI)，USB2.0 (EHCI)
- (2) 支持双网口
- (3) 支持 sata
- (4) 显示最大分辨率 1920x1080，支持双屏显示，支持 bmp logo
- (5) 支持 I2C 设备
- (6) 支持 NAND Flash
- (7) 支持 SLC，容量大小：256MB/512MB/1GB
- (8) 支持 SPI Flash 用于启动 BootLoader
- (9) 支持菜单更新系统，包括更新 U-boot、Linux 内核和根文件系统，可通过网络 (tftp) 或 U 盘进行更新

试验箱资料库中配备的资料允许用户自行编译 U-boot，包括修改 NAND 分区大小、修改内核引导参数、修改 logo、修改 CPU 频率与内存频率、在屏幕和串口同步显示调试信息，以及对驱动列表进行编辑等。

2.3.2 系统内核

龙芯 2K500 试验箱采用的是基于 linux-5.10.0 移植开发的 Linux 内核，这个内核针对龙芯 2K500 处理器的 MIPS 架构进行了优化和适配。系统内核是整个操作系统的核心，负责管理硬件资源、提供软件接口，以及确保整个系统的稳定运行。

试验箱使用的内核特性如下：

- (1) 支持 YAFFS2/CRAMFS/NFS/UBIFS/NFS/FAT32 等格式的文件系统
- (2) 支持 SLC NAND Flash 驱动，容量：256MB/512MB/1GB
- (3) 支持看门狗驱动
- (4) 支持 RTC 驱动
- (5) 支持 LED 驱动
- (6) 支持按键驱动
- (7) 支持 SPI 驱动：最大支持 4 个片选
- (8) 支持 I2C 驱动：支持 100KHz，400KHz
- (9) 支持 PWM 驱动
- (10) 支持 USB Host 驱动
- (11) 支持串口：最大支持 12 路串口，波特率稳定支持 460800

- (12)支持 GMAC 千兆以太网驱动: 支持 RTL8211E, 支持 YT8521
- (13)支持显示驱动:RGB 接口可通过芯片转接 LVDS、VGA、HDMI、DVI, 支持双屏显示
- (14)支持 LCD 背光驱动:gpio 控制或 pwm 控制
- (15)支持 PCIE 控制器驱动:支持 usb 3.0 pcie 卡, sata 3.0 pcie 卡, 千兆网卡等
- (16)支持 CAN 驱:最高波特率 1MHz
- (17)支持 GPIO 驱动
- (18)支持 PINCTRL 驱动:用于引脚复用配置
- (19)支持 CLK 驱动
- (20)支持硬件随机数生成器
- (21)支持温度监测:用于读取芯片温度

试验箱资料库中配备的资料允许用户自行编译 linux 内核, 包括内核编译、定制与裁剪、配置驱动、编写并写入驱动等。

2.3.3 文件系统

龙芯 2K500 试验箱使用的是由 Buildroot 构建的基于 BusyBox 的嵌入式 Linux 文件系统。这种组合为嵌入式系统提供了一个轻量级但功能强大的解决方案。

BusyBox 的特点:

- (1)将多个常用 Unix 工具集成到一个小型可执行文件中
- (2)大大减少系统空间占用, 适合资源受限的嵌入式系统
- (3)提供了 shell 环境和大多数基本的 Linux 命令
- (4)可以通过配置选择需要的功能, 进一步优化系统大小

主要组件:

- (1)Shell (ash)
- (2)文件操作工具 (ls, cp, mv 等)
- (3)系统工具 (mount, ifconfig 等)
- (4)文本处理工具 (grep, sed, awk 等)

Buildroot 的优势:

- (1)自动化构建嵌入式 Linux 系统的工具
- (2)可以生成交叉编译工具链、根文件系统、内核和 bootloader
- (3)简化了嵌入式 Linux 系统的构建过程
- (4)提供了丰富的配置选项, 可以根据需求定制系统
- (5)支持多种架构, 包括龙芯 2K500 使用的 MIPS 架构

试验箱资料库中配备的资料同样允许用户自行编译文件系统, 支持用户对文件系统进行二次开发, 如修改页大小、定制网络、适配新 CPU、添加软件包等。

第3章 熟悉开发环境

3.1 实验目的

- (1) 熟悉嵌入式开发环境，学会基于龙芯教学平台的 Linux 或 OpenHarmony 开发环境的配置和使用。
- (2) 掌握通过 TFTP 或 U 盘等方式进行程序下载的方法，通过教学平台提供的 U-boot 界面进行内核，文件系统等固件的更新，成功进入系统界面。

3.2 实验步骤

3.2.1 使用终端工具连接到实验箱

- (1) 下载安装 MobaXterm (Windows) 或 Minicom (Linux)
- (2) 用 UART-USB 线将实验箱与电脑连接，分别接在实验箱的 USB DEBUG 接口和电脑的 USB 接口，如图 3-1 所示。

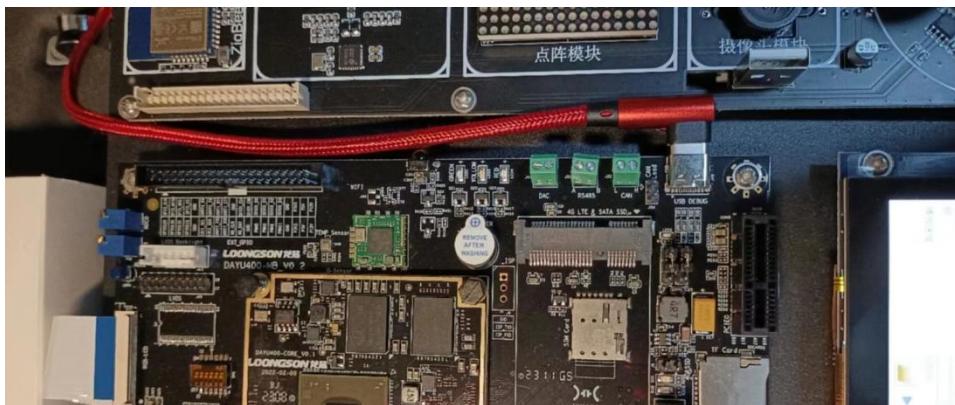


图 3-1 USB DEBUG 接口连接图

3.2.1.1 Windows 下连接

- (1) 下载安装 MobaXterm
- (2) 使用如图 3-2 设置，单击“OK”

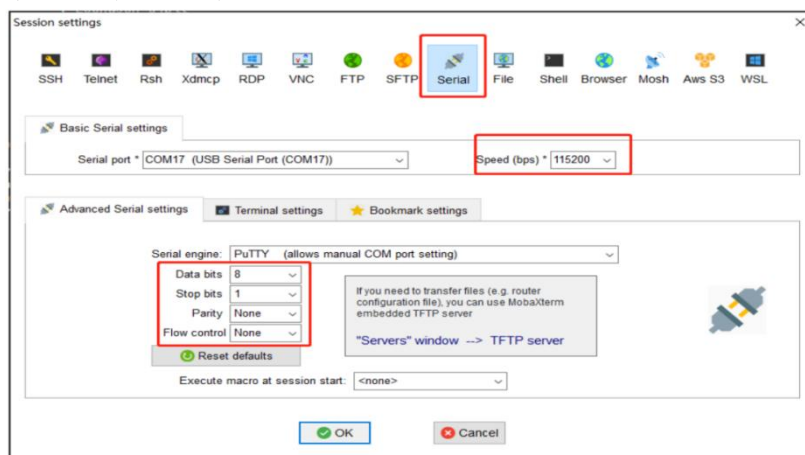


图 3-2 MobaXterm 设置

- (3) 按下开发板上的 PWR ON 按钮即可开机。默认用户为 root，默认密码为 123。

3.2.1.2 Linux 下连接

(1) 首先在 Linux 上安装 minicom

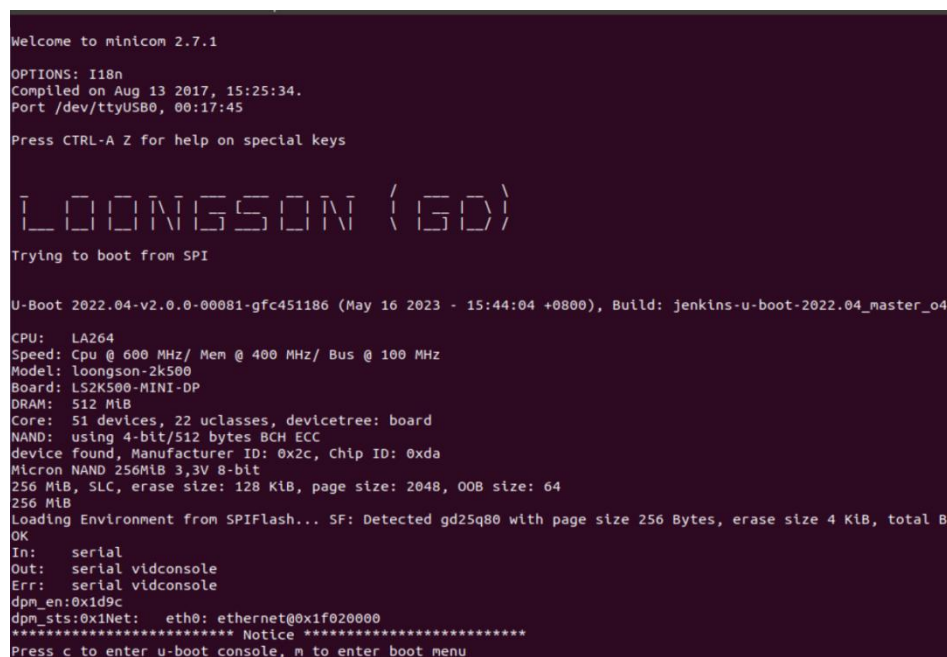
```
sudo apt-get install minicom
```

(2) 然后将实验箱与电脑连接。

(3) 在 shell 中使用下面命令连接试验箱，通常试验箱的设备名为 ttyUSB0，根据实际情况而定。

```
minicom -D /dev/ttyUSB0
```

(4) 启动 minicom 之后，按下开发板上 PWR ON 按钮即可开机，如图 3-3 所示：



```
Welcome to minicom 2.7.1
OPTIONS: I18n
Compiled on Aug 13 2017, 15:25:34.
Port /dev/ttyUSB0, 00:17:45
Press CTRL-A Z for help on special keys

LOONGSON (GO)

Trying to boot from SPI

U-Boot 2022.04-v2.0.0-00081-gfc451186 (May 16 2023 - 15:44:04 +0800), Build: jenkins-u-boot-2022.04_master_o4
CPU: LA264
Speed: Cpu @ 600 MHz/ Mem @ 400 MHz/ Bus @ 100 MHz
Model: loongson-2k500
Board: LS2K500-MINI-DP
DRAM: 512 MiB
Core: 51 devices, 22 uclasses, devicetree: board
NAND: using 4-bit/512 bytes BCH ECC
device found, Manufacturer ID: 0x2c, Chip ID: 0xda
Micron NAND 256MiB 3,3V 8-bit
256 MiB, SLC, erase size: 128 KiB, page size: 2048, OOB size: 64
Loading Environment from SPIFlash... SF: Detected gd25q80 with page size 256 Bytes, erase size 4 KiB, total 8
OK
In: serial
Out: serial vidconsole
Err: serial vidconsole
dpm_en:0x1d9c
dpm_sts:0x1Net: eth0: ethernet@0x1f020000
***** Notice *****
Press c to enter u-boot console, m to enter boot menu
```

图 3-3 minicom 显示开机界面

(5) 如此即成功连接实验箱。

3.2.2 升级内核、文件系统

3.2.2.1 前期准备

(1) 准备一个大于 4G 的 U 盘格式化为 FAT32，并在 U 盘中创建名为 update 的文件夹。

注：Windows 下大于 32G 的 U 盘默认的工具不支持格式化为 FAT32，可使用 Disk Genius 等工具格式化为 FAT32。

(2) 将资料包中的 toolchain-loongarch64-linux-gnugcc8-host-x86_64-2022-07-18.tar.xz 交叉编译工具链压缩包解压到/opt 目录下。

(3) 解压资料包中的 u-boot、linux-5.10-2k500，依次进行编译操作。首先进入 u-boot-2022.04-2k500-cbd-src 目录，执行如下操作：

```
source ./set_env.sh #设置交叉工具链等环境
make loongson_2k500_mini_dp_defconfig #指定板载卡配置
make #开始编译
```

待编译完成后，目录下生成了 u-boot-with-spl.bin 文件，即为成功。
(4) 接着进入 linux-5.10-2k500-cbd-src 目录下，执行如下操作：

```
source ./set_env.sh #设置交叉工具链等环境
make loongson_2k500_defconfig #指定板载卡配置
make #开始编译
```

如果遭遇编译失败，如图 3-4 所示：

Error: Invalid architecture

图 3-4 编译失败报错界面

这是由于是 loongarch 是新的架构，u-boot-tools 包里面的 mkimage 不能识别此架构。所以无法制作 uImage.gz。解决方法是到上一步解压出来的 u-boot 目录下，使用其中 tool 目录下的 mkimage 替换/usr/bin 下的 mkimage，指令如下：

```
sudo cp tools/mkimage /usr/bin/
```

然后再次进行编译即可成功，可以成功得到 uImage.gz 文件。

(5) 最后需要制做文件系统，使用 buildroot 制作一个 busybox 系统。从开发板资料包中获取 buildroot，其中包含了制作 loongson 开发卡文件系统的相关配置文件。

(6) 首先解压 buildroot，并进入到其目录下。使用如下命令生成与 loongson 开发板相关的配置：

```
make loongson2k500_mini_dp_defconfig
```

(7) 接着可以使用如下指令，进入图形化界面查看相关配置：

```
make menuconfig
```

运行之后页面入图 3-5 所示：

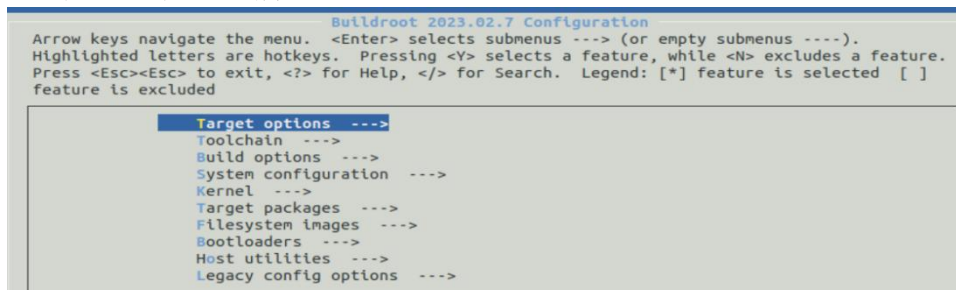


图 3-5 menuconfig 界面设置

一般情况下, Target option、Toolchain、Build options 和 Filesystem images 无需更改。System configuration , Target packages 和 Filesystem images 分别是关于系统配置、系统软件包配置及文件系统制作方式。

在 System configuration 中, 可以设置开机密码, 用户名、终端启动是否需要登录等等。

在 Target packages 下可以根据自己个人需求, 增加或减少一些软件包。

(8) 配置完成之后须保存配置, 即可开始编译文件系统, 使用如下命令:

```
make -j 4 # -j 4 是多核编译选项, 可根据电脑实际情况更改
```

编译时间较长, 一般为 2~3 小时, 须耐心等待。

(9) 编译完成之后, 在 buildroot 目录下的 output/image 目录中, 即可看见编译出来的文件系统产物如图 3-6 所示:

```
rootfs.tar  rootfs.tar.gz  rootfs.ubi  rootfs.ubifs
```

图 3-6 编译完成后生成目标文件

其中 roots.ubi 是可以部署在 NAND 上的文件系统镜像, 将它改名为 rootfs-ubifs-ze.img, 用来部署文件系统。

3.2.2.2 使用 U 盘更新

将上一步中得到的 u-boot-with-spl.bin、uImage、rootfs-ubifs-ze.img 放入 U 盘的 update 目录中, 插入到开发板上。用 minicom 把主机与开发板进行连接, 在启动开发板是按住 M 键, 即可进入 u-boot 设置菜单, 如图 3-7 所示:

```
*** U-Boot Boot Menu ***

[1] System boot select
[2] Update kernel
[3] Update rootfs
[4] Update u-boot
[5] Update ALL
[6] System install or recover
[7] U-Boot console

Press UP/DOWN to move, ENTER to select, ESC/CTRL+C to quit
```

图 3-7 u-boot 设置菜单

选择第 5 个 Update ALL 选项, 一次性更新内核、文件系统和 u-boot, 选择从 U 盘更新, 如图 3-8:

```
*** U-Boot Boot Menu ***

[1] Update all (kernel rootfs uboot) to nand flash (by usb)
[2] Update all (kernel rootfs uboot) to nand flash (by tftp)
[

Press UP/DOWN to move, ENTER to select
```

图 3-8 u-boot 设置

更新完成后，使用 reset 命令重启开发板即可，显示重启界面如图 3-9：

```
=> reset

LOONGSON (GO)

Trying to boot from SPI

U-Boot 2022.04-v2.0.0-00081-gfc451186 (May 16 2023 - 15:44:04 +0800), Build: je4

CPU:   LA264
Speed: Cpu @ 600 MHz/ Mem @ 400 MHz/ Bus @ 100 MHz
Model: loongson-2k500
Board: LS2K500-MINI-DP
DRAM:  512 MiB
Core:  51 devices, 22 uclasses, devicetree: board
NAND:  using 4-bit/512 bytes BCH ECC
device found, Manufacturer ID: 0x2c, Chip ID: 0xda
```

图 3-9 重新启动界面
至此内核、固件与文件系统的升级完成。

第4章 交叉编译与程序运行

4.1 实验目的

(1) 学习掌握交叉编译的方法，利用交叉编译器编译程序，并使用 TFTP 方式进行程序下载。在开发板上运行编译后的程序。

(2) 了解嵌入式系统开发与普通程序开发之间的区别，认识交叉编译的功能原理，掌握嵌入式开发方法如 makefile 的使用等。成功在设备上运行一个简单程序。

4.2 实验步骤

所谓交叉编译，就是在 X86 机器的 ubuntu18.04 的系统中，使用交叉编译工具链，编译软件，然后复制编译出来的可执行文件到板卡中（USB 传输等手段），然后才能在板卡上运行。

4.2.1 编写一个简单程序

```
#include<stdio.h>

int main(){
    int a=1;
    int b=2;
    printf("%d\n",a+b);
    return 0;
}
```

4.2.2 使用交叉编译工具链编译

(1) 在使用交叉编译器之前，需要先配置交叉编译器的环境变量。在任意文件位置处，创建一个 setEnv.sh 文件，文件内容如下：

```
CC_PREFIX=/absolute_route/toolchain-loongarch64-linux-gnu-gcc8-host-x86_64-2022-07-18
export PATH=$CC_PREFIX/bin:$PATH
export LD_LIBRARY_PATH=$CC_PREFIX/lib:$LD_LIBRARY_PATH
export LD_LIBRARY_PATH=$CC_PREFIX/loongarch64-linux-gnu/lib64:$LD_LIBRARY_PATH
export ARCH=loongarch
export CROSS_COMPILE=loongarch64-linux-gnu-
```

需将其中的 absolute_route 替换为交叉编译器所在路径。

(2) 然后编写该程序的 Makefile 文件，需与程序源文件在同一目录下，Makefile 内容如下：

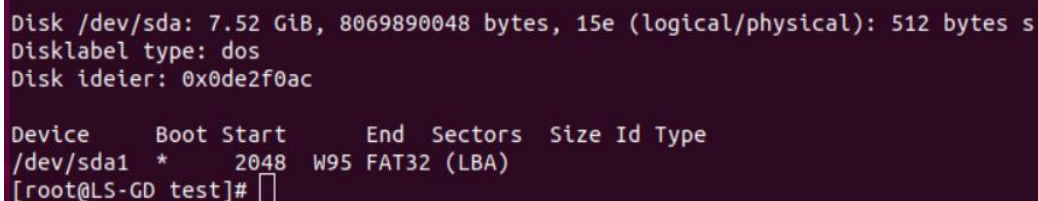
```
target :  
    loongarch64-linux-gnu-gcc -O2 example2_1.c -o example2_1  
clean:  
    rm -rf example2_1
```

(3) 在每次编译程序前，都需要先运行前面的 setEnv.sh 文件设置环境变量，再使用运行 make 完成编译，如下：

```
source ./setEnv.sh  
make
```

完成编译之后，将生成的文件中的 example2_1 文件通过 U 盘或 tftp 传输方式传输到开发板上。这里展示使用 usb 传输。

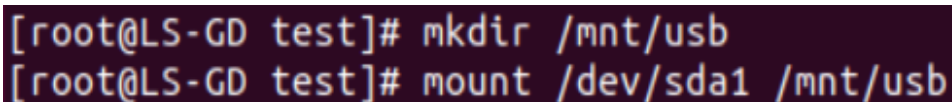
(4) 将文件存入之前准备好的 FAT32 格式 U 盘中，插在开发板的 USB 接口上。使用 fdisk -l 指令，在最下方的设备表格中可以找到 U 盘，如图 4-1：



```
Disk /dev/sda: 7.52 GiB, 8069890048 bytes, 15e (logical/physical): 512 bytes s  
Disklabel type: dos  
Disk identifier: 0x0de2f0ac  
  
Device      Boot Start      End  Sectors  Size Id Type  
/dev/sda1   *      2048    975999   975999  480M W95 FAT32 (LBA)  
[root@LS-GD test]#
```

图 4-1 终端显示挂载的 U 盘

sda1 即为该 U 盘。然后将其挂载。这里将他挂载到 /mnt/usb 目录下，如图 4-2：



```
[root@LS-GD test]# mkdir /mnt/usb  
[root@LS-GD test]# mount /dev/sda1 /mnt/usb
```

图 4-2 挂载 U 盘指令

接着进入 /mnt/usb 目录下，找到 example2_1 文件，运行即可。可以看到输出为 3。

(5) 最后拔出 U 盘前，需要使用如下命令卸载：

```
umount /dev/sda1
```

再拔出 U 盘即可。

第5章 七段数码管实验

5.1 实验目的

(1) 了解教学平台下数码管的用法及控制。掌握七段数码管的显示原理，认识教学平台下数码管的连接方式与控制方法。

(2) 根据掌握的信息进行编程，实现秒表功能，使数码管从 0 开始以秒为单位计数，可以设置停止计数的时间或在到达某个时间后重新开始计数。

5.2 实验步骤

5.2.1 I2C 通用写函数及 CH422 通信

CH422 通过 I2C 进行通信，I2C 两线制包括串行时钟线（SCL）和串行数据线（SDA），SCL 由主器件控制，SDA 实现双向数据传输。I2C 通信协议包括：

-空闲信号：SCL 和 SDA 都保持高电平

-起始信号：SCL 时钟线高电平期间，SDA 数据线出现由高到低的跳变（下降沿）

-结束信号：SCL 时钟线高电平期间，SDA 数据线出现由低到高的跳变（上升沿）

-应答信号：表示数据已被接收，每字节（8 位）数据传输后的下一个时钟信号，在 SCL 高电平期间，如果 SDA 数据线为“低电平”，则表示应答信号；如果 SDA 为“高电平信号”，则表示非应答信号。

I2C 通信的过程可抽象为：起始信号-发送&应答数据-结束信号。CH422 的操作命令约定每次通信中，主器件需发送 2 个字节数据作为操作命令，以下简称字节 1 和字节 2，因此可编写发送指令函数如下：

```
void CH422_WriteByte( unsigned short cmd )    // 写出数据
{
    I2c_Start();    // 启动总线
    I2c_WrByte( ( unsigned char )( cmd>>8 ) );
    I2c_WrByte( ( unsigned char ) cmd );    // 发送数据
    I2c_Stop();    // 结束总线
}
```

本节中仅有发送命令函数以“CH422_”作为前缀，其余函数是 I2C 设备通用函数，在之后的实验仍会用到。

编写起始信号与结束信号代码如下，因为在本实验中仅将 CH422 作为输出设备，因此默认 SDA 与 SCL 为输出方向，代码中的 SDA_D_OUT() 方法被定义为空代码块，DELAY_TIME 设置为 10，即每次置位后保持 10us 以保证数据稳定。

```

#define SDA_D_OUT    {}
#define DELAY_TIME    10
void I2c_Start( void )    // 操作起始
{
    SDA_SET();    // 发送起始条件的数据信号
    SDA_D_OUT;    // 设置 SDA 为输出方向
    SCL_SET();
    SCL_D_OUT;    // 设置 SCL 为输出方向
    udelay(DELAY_TIME);
    SDA_CLR();    //发送起始信号
    udelay(DELAY_TIME);
    SCL_CLR();    //钳住 I2C 总线，准备发送或接收数据
}
void I2c_Stop( void )    // 操作结束
{
    SDA_CLR();
    udelay(DELAY_TIME);
    SCL_SET();
    udelay(DELAY_TIME);
    SDA_SET();    // 发送 I2C 总线结束信号
    udelay(DELAY_TIME);
}

```

涉及的 GPIO 置位/复位操作函数如下，使用 `gpio_set_value` 函数，`my_dev` 是设备私有数据结构体，将在内核模块代码部分介绍。

```

static void SCL_SET(void)
{
    gpio_set_value(my_dev.scl_gpio, 1);
}
static void SCL_CLR(void)
{
    gpio_set_value(my_dev.scl_gpio, 0);
}
static void SDA_SET(void)
{
    gpio_set_value(my_dev.sda_gpio, 1);
}
static void SDA_CLR(void)
{
    gpio_set_value(my_dev.sda_gpio, 0);
}

```

向设备写 1 字节数据的函数如下，该函数将传入的 8 位数据由高位到低位发送至 CH422，且发送完 8 位数据后，在第 9 次时钟脉冲进行接收应答：

```
void I2c_WrByte( unsigned char dat )    // 写一个字节数据
{
    unsigned char i;
    for( i = 0; i != 8; i++ )    // 输出 8 位数据
    {
        if( dat&0x80 ) { SDA_SET(); }
        else { SDA_CLR(); }
        udelay(DELAY_TIME);
        SCL_SET();
        dat<<=1;
        udelay(DELAY_TIME);    // 可选延时
        SCL_CLR();
    }
    SDA_SET();
    udelay(DELAY_TIME);
    SCL_SET();    // 接收应答
    udelay(DELAY_TIME);
    SCL_CLR();
}
```

至此，通过 CH422_SendCmd() 函数已经可以对 CH422 发送操作指令，具体的操作指令在数据手册中有详细介绍，这里只介绍实验中使用的指令：

1. 设置通用输出命令：字节 1 为 0x46，字节 2 为 0x00~0x0F，低 4 位分别对应 OC3~OC0 引脚的输出寄存器，写 0 使引脚输出低电平，写 1 使引脚输出高电平，在本实验中用于分时显示驱动。
2. 加载段数据命令/设置双向输入输出命令：字节 1 为 0x70、0x72、0x74、0x76，分别对应 4 个数码管，字节 2 为 0x00~0xFF，对应引脚 IO7~IO0 的输出寄存器，写 0 使引脚输出低电平，写 1 使引脚输出高电平，因为 CH422 是共阴极数码管，所以写 1 时，对应 LED 灯亮起。

5.2.2 内核模块代码编写

1) 依赖头文件

内核模块依赖头文件如下，可自行创建 ch422g.h 头文件，根据自己的编程习惯对代码进行优化。

```
#include <linux/types.h>
#include <linux/kernel.h>
#include <linux/delay.h>
#include <linux/ide.h>
#include <linux/init.h>
#include <linux/module.h>
```

```

#include <linux/errno.h>
#include <linux/gpio.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/of.h>
#include <linux/of_address.h>
#include <linux/of_gpio.h>
#include <linux/platform_device.h>
#include <linux/miscdevice.h>
#include <linux/i2c.h>
#include <asm/uaccess.h>
#include <asm/io.h>
#include "ch422g.h"

```

2) 私有数据结构体

编写设备驱动时，通常需要编写包含设备信息的私有数据结构体，在本实验中较为简单，其可以为模块代码和操作代码提供 CH422 的 SDA 数据线和 SCL 时钟线的 GPIO 端口号，需要时可以提供其他设备相关信息，在模块初始化代码中进行动态赋值。

my_dev 是测试中的常用名称，可自行更改。

```

struct ch422g_dev{
    dev_t devid;          /* 设备号      */
    void *private_data;    /* 私有数据 */
    int sda_gpio;
    int scl_gpio;
};
struct ch422g_dev misc_ch422g;

```

3) 设备操作函数

CH422 在本实验中作为输出设备，需要覆写 open() 方法用于连接设备和 write() 方法用于修改输出内容，编写设备操作函数 ch422g_open() 和 ch422g_write()。

ch422g_open() 方法较为简单，只需要设置私有数据即可。

```

static int ch422g_open(struct inode *inode, struct file *filp)
{
    filp->private_data = &misc_ch422g; /* 设置私有数据 */
    return 0;
}

```

ch422g_write() 方法使用户可以通过 write() 修改已连接设备的输出，需要用户传入约定的数据首地址和数据大小。

在本实验中，约定该数据为 unsigned char 整型数组，且至少开辟 5 个空间大小，即至少需要传入 5 字节数据，其中，0 号元素作为保留的控制指令，未使用，1~4 号元素为欲显示的数字/符号经转码后的指令（将在测试程序编写指南中介绍），分别对应数码管 1~4。

因为在 ch422g_open() 中向 flip->private_data 存储了设备的私有数据，因此可以在本函数中用对应结构体指针变量进行接收，不过本实验中因代码结构较简单，没有对该指针进行传递使用，而是直接使用了全局的设备结构体。

首先使用 copy_from_user() 函数将用户空间的数据复制到内核空间，并使用局部变量暂存所需的数据，然后通过上文编写的发送指令函数，编辑每个数码管的显示内容。每次先发送 0x46 指令，并在字节 2 将对应的位置 0，其他位置 1，用于分时显示。然后编辑相应数码管，注意指令与数码管之间的对应关系，这里默认传入的 ch1~ch4 是已转码的指令，具体将在测试程序中介绍。

```
static ssize_t ch422g_write(struct file *filp, const char __user *buf, size_t cnt, loff_t *offt)
{
    int retvalue;
    struct ch422g_dev *dev = filp->private_data;
    unsigned char databuf[5], cmd, ch1, ch2, ch3, ch4;
    retvalue = copy_from_user(databuf, buf, cnt);
    if(retvalue < 0) return -EFAULT;
    cmd = databuf[0];
    ch1 = databuf[1];
    ch2 = databuf[2];
    ch3 = databuf[3];
    ch4 = databuf[4];
    CH422_WriteByte( CH422_SET_IO_CMD + (0x07)); //0111
    CH422_WriteByte( CH422_DIG0 | ch1 );
    CH422_WriteByte( CH422_SET_IO_CMD + (0x0b)); //1011
    CH422_WriteByte( CH422_DIG1 | ch2 );
    CH422_WriteByte( CH422_SET_IO_CMD + (0x0d)); //1101
    CH422_WriteByte( CH422_DIG2 | ch3 );
    CH422_WriteByte( CH422_SET_IO_CMD + (0x0e)); //1110
    CH422_WriteByte( CH422_DIG3 | ch4 );
    return 0;
}
```

4) 设备结构体

直接使用 GPIO 端口对设备进行操作时，通常将设备注册为杂项设备（misc device），注册设备时需要使用 Linux 内核中的 struct miscdevice 结构体和 struct fileoperations 结构体。其中：

THIS_MODULE 是 Linux 内核 module.h 中定义的宏，用于引用 struct module 结构。

CH422G_MINOR 和 CH422G_NAME 在本文件中定义，分别为设备号与设备名。
ch422g_open 和 ch422g_write 为上文编写的设备操作函数。

```

#define CH422G_NAME          "ch422g"      /* 名字 */
#define CH422G_MINOR        146           /* 子设备号 */
/* 设备操作函数 */
static struct file_operations misc_ch422g_fops = {
    .owner = THIS_MODULE,
    .open = ch422g_open,
    .write = ch422g_write,
};
/* MISC 设备结构体 */
static struct miscdevice ch422g_miscdev = {
    .minor = CH422G_MINOR,
    .name = CH422G_NAME,
    .fops = &misc_ch422g_fops,
};

```

5) 模块的初始化和卸载

当模块被载入 Linux 内核时，会执行一次模块初始化函数，使用 `__init` 初始化宏，告知系统模块载入后丢弃该函数以释放内存。

使用 `ioremap()` 函数将引脚的物理地址映射为虚拟地址，并使用 `ioread32` 获取虚拟地址对应值，实验中需要配置 GPIO 管脚的复用配置寄存器和输出使能寄存器，将使用的 GPIO 口配置为 GPIO 功能、输出模式。

GPIO 端口的配置寄存器物理地址可以在芯片用户手册 5.5 章节中找到，CH422 使用的 GPIO 管脚为 141 和 142。

```

#define SDA3_GPIO            142
#define SCL3_GPIO            141
#define GPCFG    0x1fe104d4 //0x1fe104b0
/* __iomem 虚拟地址指针，保存虚拟地址*/
static void __iomem *GPCFG_V;

```

```

static int __init ch422g_init(void)
{
    int ret;
    u32 GPFCFG_read, GPFCFG_write;

    // 配置 GPIO 复用
    GPCFG_V = ioremap(GPCFG, sizeof(u32));
    GPFCFG_read = ioread32(GPCFG_V);
    GPFCFG_write = (GPFCFG_read & 0x70077777);
    iowrite32(GPFCFG_write, GPCFG_V);
}

```

复用配置寄存器使用每 4 位的低 3 位用于配置一个 GPIO 管脚的复用功能，

所以将 141 和 142 对应的位置 0，其余位置 1 作为掩码，和该管脚的原值相与，得到的结果为其余管脚原配置不变，管脚 141 和 142 复用为 GPIO 功能（000），然后再写回到该寄存器。

配置私有数据的 GPIO 引脚编号，注册并初始化 GPIO，这里使用的 `gpio_direction_output()` 方法在初始化 GPIO 初始电平的同时将 GPIO 端口配置为输出模式，因此无需配置输出使能寄存器。

```
my_dev.sda_gpio = SDA3_GPIO;
my_dev.scl_gpio = SCL3_GPIO;

gpio_request(my_dev.sda_gpio, "sda3");
gpio_request(my_dev.scl_gpio, "scl3");
ret = gpio_direction_output(my_dev.sda_gpio, 0);
if(ret < 0) printk("can't set sda gpio!\r\n");
ret = gpio_direction_output(my_dev.scl_gpio, 0);
if(ret < 0) printk("can't set scl gpio!\r\n");
CH422_WriteByte( CH422_SYS_CMD | CH422_IO_OE_BIT | CH422_A_SCAN_BIT);
return ret;
}
```

最后进行 misc 设备的注册和 CH422 的系统参数配置，这里将 IO_OE 和 A_SCAN 位置 1 并使用 0x48 命令传递，开启 IO 使能和数码管动态显示，具体作用和其余配置内容可自行查阅芯片数据手册。

```
#define CH422_SYS_CMD    0x4800    // 设置系统参数命令，默认方式
#define CH422_IO_OE_BIT  0x0001    //IO7-IO0 双向输入输出引脚，位 1 允许输出
#define CH422_A_SCAN_BIT 0x0004    //控制动态扫描控制功能，为 0 启用 IO 扩展功能、为 1
数码管动态显示
```

模块的卸载函数较为简单，使用 `__exit` 初始化宏进行声明，首先释放映射的虚拟地址，再卸载 ch422g 设备，最后释放 GPIO 端口即可。

```
static void __exit ch422g_exit(void)
{
    iounmap(GPCFG_V);

    misc_deregister(&ch422g_miscdev);
    gpio_free(my_dev.sda_gpio);
    gpio_free(my_dev.scl_gpio);
}
```

在编写完初始化和卸载函数后，需要使用 `module_init()` 和 `module_exit()` 对相应函数进行注册，才能正常使用 `insmod` 和 `rmmod` 指令。

```
module_init(ch422g_init);
module_exit(ch422g_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR("yangjinrun");
```

至此内核模块代码编写完毕。

1.1.1. 5.2.3 模块代码编译

首先根据实验 2 内容配置好交叉编译链，然后编辑/创建模块代码目录下的 Makefile 文件，配置 KDIR 为本机中 linux 内核源码路径，建议使用绝对路径，以便在任意位置编译内核模块。（注：当前目录下仅存在 ch422g.c 和 Makefile 文件便可以编译内核模块，参考资料中提供了 ch422g.h 头文件，可用于优化代码，如想编译多个源文件为一个模块，则需要多编写一层 obj，如 obj-m:=A.o，下一行增加 A-objs:=B.o C.o，但大部分实验不会用到）

```
obj-m :=ch422g.o
KDIR :=/root/loonarch/linux-5.10-2k500-cbd-src
all:
    make -C $(KDIR) M=$(PWD) modules
clean:
    make -C $(KDIR) M=$(PWD) modules clean
```

配置完成后在当前目录开启终端，输入 make 指令开始编译，如图 5-1 所示：

```
root@htsheep-vm:~/loonarch/drv_src/ch422g_bk# make
make -C /root/loonarch/linux-5.10-2k500-cbd-src M=/root/loonarch/drv_src/ch422g_bk modules
make[1]: Entering directory '/root/loonarch/linux-5.10-2k500-cbd-src'
CC [M] /root/loonarch/drv_src/ch422g_bk/ch422g.o
/root/loonarch/drv_src/ch422g_bk/ch422g.c: In function 'CH422_SCL_CLR':
/root/loonarch/drv_src/ch422g_bk/ch422g.c:83:23: warning: unused variable 'GPFSEL_write' [-Wunused-variable]
    u32 GPFSEL_read, GPFSEL_write;
                        ^~~~~~
```

图 5-1 编译模块代码

编译结束后本目录中出现 ch422g.ko 文件，说明编译成功，如图 5-2：

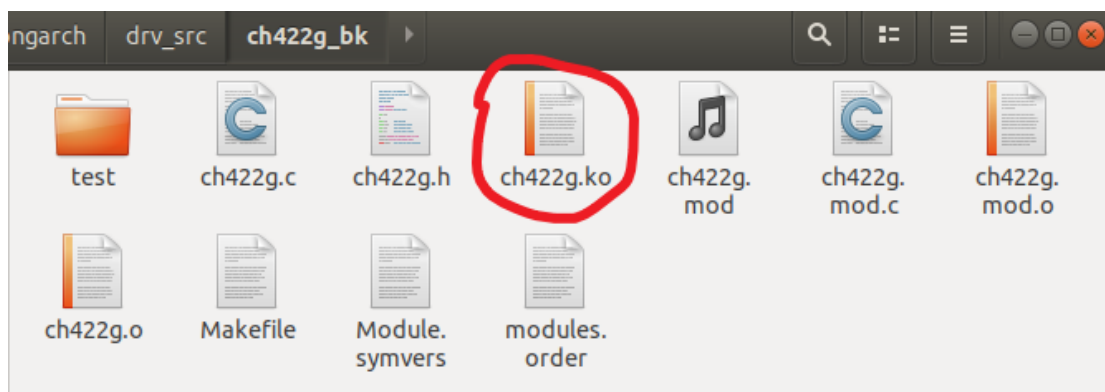


图 5-2 编译生成目标文件

1.1.2. 5.2.4 测试程序编写指南

1) 转码方式

在上文中有介绍过内核模块代码认为通过 `write` 传递进内核的是转码后的指令，这里的转码是指将欲显示的数字/符号转换为 8 位 BCD 码，分别对应 I07~I00 的输出寄存器，即 8 个 LED 灯的亮灭，网络上可以很轻易地搜索到共阴极数码管的转码表，这里提供一个转换数组案例，其中，0~9 号元素分别对应数码管显示数字 0~9。

```
const unsigned char BCD_decode_tab[ 29 ] = { 0X3F, 0X06, 0X5B, 0X4F, 0X66, 0X6D, 0X7D, 0X07,
0X7F, 0X6F, 0X77, 0X7C, 0X58, 0X5E, 0X79, 0X71, 0x00, 0x46, 0x40, 0x41, 0x39, 0x0F, 0x08, 0x76,
0x38, 0x73, 0x80, 0xFF, 0x00 };
```

2) 数码管操作

使用 `open()` 方法以读写方式打开 `/dev/ch422g` 文件即可打开/连接设备：

```
fd = open("/dev/ch422g", O_RDWR);
printf("test: fd=%d\n", fd);
if(fd == -1)
{
    printf("test: Error Opening ch422g\n");
    return(-1);
}
```

想要使数码管亮起，需要创建一个至少 5 字节的数据缓冲区，建议使用至少开辟 5 个空间的 `unsigned char` 数组：

```
unsigned char databuf[5];
```

将转码后的指令存储在 `databuf[1]~databuf[4]`，再使用 `write()` 函数写入对应 `ch422g` 设备的文件描述符 (`fd`)，即可实现显示效果，如果想要某个数码管不亮起，则对应数据写入 `0x00`。

这里给出一个例程，它实现了在数码管上从左到右显示 `count` 的千、百、十、个位：

```
void set_digital_tube(int fd, int count)
{
    for (int i = 4; i >= 1; i--)
    {
        databuf[i] = BCD_decode_tab[count % 10];
        count /= 10;
    }
    write(fd, databuf, sizeof(databuf));
}
```

在测试程序结束之前，需使用 `close()` 方法关闭打开的设备文件描述符。

```
close(fd);
```

3) 测试程序编译

在测试源程序目录下新建/编辑 Makefile 文件，并写入如下内容，其中，`-lpthread` 为可选项，用于参考程序中多线程的编译，如不使用多线程，该编译选项可去除。

```
all: test

clean:

    rm -rf test_ch422g

test:

    loongarch64-linux-gnu-gcc -lpthread -O2 test_ch422g.c -o test_ch422g
```

保存退出后，在目录下开启中断，输入 `make` 命令开始编译，得到可执行测试程序 `test_ch422g`，如图 5-3。

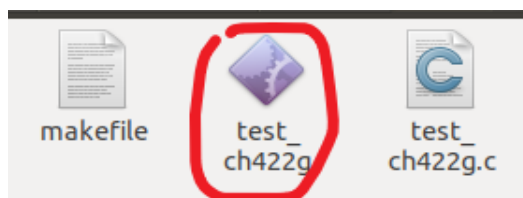


图 5-3 编译生成可执行文件

1. 1. 3. 5. 2. 5 模块加载、运行测试和模块卸载

将编译得到的 `ch422g.ko` 文件和 `test_ch422g` 文件拷贝到 U 盘，将 U 盘插入试验箱，挂载后打开 U 盘，进入存放这两个文件的目录，如图 5-4：

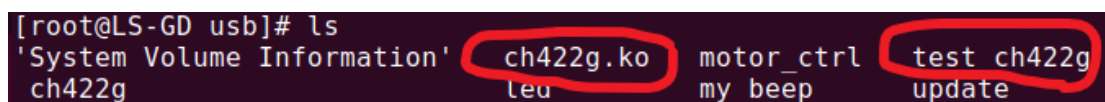


图 5-4 操作设置

使用 `insmod ch422g.ko` 命令加载模块，根据 5.2.3 中编写的代码，会显示如下 debug 信息，如图 5-5：

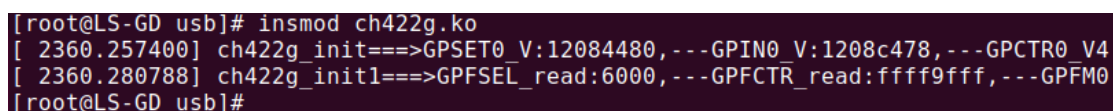


图 5-5 debug 信息

随后便可直接执行输入 `./test_ch422g` 执行测试程序，测试代码功能。

测试结束后，通过 `rmmmod ch422g.ko` 卸载模块，再次执行测试程序，会发现打开 `/dev/ch422g` 失败（前提是要对 `open()` 开启失败进行检查并输出 debug 信息，如上述例程），说明卸载成功，如图 5-6。

```
[root@LS-GD usb]# rmmod ch422g.ko
[root@LS-GD usb]# ./test_ch422g
test: fd=-1
test: Error Opening ch422g
```

图 5-6 运行程序测试

5.3 练习：数码管实现时钟功能

在完成本章基础实验后，学生需要实现一个时钟显示功能。通过扩展实验代码，使得四个七段数码管可以显示当前时间，其中高两位显示小时，低两位显示分钟，从左到右第二个数码管的小数点“.”需常亮作为分隔符。

任务要求：

1. 小时显示：将高两位七段数码管用于显示小时。确保小时数值在 0 到 23 之间循环。
2. 分钟显示：将低两位七段数码管用于显示分钟。确保分钟数值在 0 到 59 之间循环。
3. 分隔符：将从左到右第二个数码管的小数点“.”常亮，以将小时和分钟分隔开。
4. 时间变化：创建小时、分钟、秒共 3 个变量，每秒递增秒数，保证进位正常，将小时和分钟映射到步骤 1 和 2。

扩展任务：

1. 时间格式：支持 12 小时制和 24 小时制的切换。
2. 整点闪烁：时钟时间达到整点时，4 个数码管闪烁 3 次，同时不影响时间正常走动（功能没有显示秒数，可以通过暂时在数码管上显示秒数进行测试）。

第6章 点阵控制实验

6.1 实验目的

了解教学平台下点阵的用法及控制。掌握点阵的显示原理，认识教学平台下点阵的连接方式与控制方法。

根据硬件原理进行编程，使得点阵能够输出字符或画面，实现类似 LED 显示屏的功能，比如循环输出相同字符，按顺序输出字符段等。

6.2 实验步骤

6.2.1 实验箱点阵模块简介

AiP1944 是一款 3 线串口共阴极 16 段 16 位或 24 段 8 位带 16×2 位键盘扫描的 LED 显示驱动控制专用电路，内部集成有 MCU 数字接口、数据锁存器、RC 振荡器等电路，广泛适用于各种 LED 面板场合。

实验箱中 AIP1944 芯片有三线串行接口（CLK，STB，DIO），内置显示 RAM。使用时通过在代码中编写好显示所需的数据编码（32*8）即可在 16*16 的矩阵上显示出来对应的形状。

6.2.2 内核模块代码编写

1) 依赖头文件

模块代码依赖的头文件如下

```
#include <linux/types.h>
#include <linux/kernel.h>
#include <linux/delay.h>
#include <linux/ide.h>
#include <linux/init.h>
#include <linux/module.h>
#include <linux/errno.h>
#include <linux/gpio.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/of.h>
#include <linux/of_address.h>
#include <linux/of_gpio.h>
#include <linux/platform_device.h>
#include <linux/miscdevice.h>
#include <linux/i2c.h>
#include <asm/uaccess.h>
#include <asm/io.h>
#include "aip1944.h"
```

2) 设备结构体

点阵模块 Aip1944 的设备结构体如下，包含设备号，私有数据，以及三个 GPIO 编号信息

```
struct aip_dev{
    dev_t devid;          /* 设备号      */
    void *private_data;    /* 私有数据 */
    int clk_gpio;
    int dio_gpio;
    int stb_gpio;
};
```

3) 设备操作函数

点阵需要编写 open、write、ioctl 三个操作函数

open 函数较为简单，将刚刚创建的设备结构体设置为私有数据即可

```
static int aip_open(struct inode *inode, struct file *filp)
{
    filp->private_data = &misc_aip; /* 设置私有数据 */
    return 0;
}
```

write 函数首先需要获取私有数据，并初始化一个数据缓冲区用于将用户空间中的数据复制到内核空间中，调用 display_arr 函数去显示点阵数据

```
static ssize_t aip_write(struct file *filp, const char __user *buf, size_t cnt, loff_t *offt)
{
    int ret;
    unsigned char databuf[64];
    struct aip_dev *dev = filp->private_data;

    ret = copy_from_user(databuf, buf, cnt);
    if(ret < 0) {
        pr_info("kernel write failed!\r\n");
        return -EFAULT;
    }

    display_arr(databuf);
    return cnt;
}
```

3) 模块初始化和卸载

点阵使用 GPIO50、51 和 115 端口，因此先在复用配置寄存器将该端口配置为 GPIO 功能（000），GPIO50、51 对应的寄存器为 GPIO48~55 复用配置寄存器，

GPIO115 对应的 112-119 复用配置寄存器

```
#define GPIO5548ADDR 0x1fe104a8
#define GPIO119112 0x1fe104c8
static void __iomem *GPIO5548ADDR_V;
static void __iomem *GPIO119112_V;
static int __init aip_init(void)
{
    int ret;
    int i;
    u32 GPIO119112_READ,GPIO119112_WRITE;
    GPIO119112_V = ioremap(GPIO119112,sizeof(u32));
    GPIO119112_READ = ioread32(GPIO119112_V);
    GPIO119112_WRITE = (GPIO119112_READ & 0x77770777);
    iowrite32(GPIO119112_WRITE,GPIO119112_V);

    u32 GPIO5548_READ,GPIO5548_WRITE;
    GPIO5548ADDR_V = ioremap(GPIO5548ADDR,sizeof(u32));
    GPIO5548_READ = ioread32(GPIO5548ADDR_V);
    GPIO5548_WRITE = (GPIO5548_READ & 0x77770077);
    iowrite32(GPIO5548_WRITE,GPIO5548ADDR_V);
}
```

设备卸载需要将映射到虚拟地址的寄存器取消映射，使用 `iounmap` 函数

```
static void __exit aip_exit(void)
{
    write_cmd(0x89);    //显示控制：显示关
    iounmap(GPIO119112_V);
    iounmap(GPIO5548ADDR_V);
    misc_deregister(&aip_miscdev);
    gpio_free(misc_aip.clk_gpio);
    gpio_free(misc_aip.dio_gpio);
    gpio_free(misc_aip.stb_gpio);
}
```

在 `display` 函数中，由于是 16*16 点阵，因此将显示模式设置为 16 位 16 段，对照 AIP1944 数据手册中的显示寄存器地址和显示模式表格。由于为 16 段模式，因此每写两字节数据就要填写两个字节空数据跳过不用的段。点阵从左到右为 SEG1 到 SEG16，从上到下为 GRID1 到 GRID16

```

static void display(unsigned char dat)
{
    unsigned char i;
    write_cmd(0x08);          //显示模式： 16 位 16 段显示
    write_cmd(0x40);          //数据设置： 地址增加
    write_cmd(0xc0);          //地址设定： 0xc0~0xd 对应地址为 00H~0DH
    for(i=0;i<16;i++)
    {
        write(dat);
        write(dat);
        write(0x00);
        write(0x00);
    }
    write_cmd(0x8F);          //显示控制： 显示开
}

```

6.2.3 测试代码编写指南

引入设备头文件，头文件中预定义了几种内置点阵，也可自行编写点阵数据去自定义显示。

```

const unsigned char zhi[] = {
    0x04,0x00,0x7C,0x3E,0x12,0x22,0x10,0x22,0xFF,0x22,0x28,0x22,0x44,0x3E,0x02,0x00,
    0xF8,0x0F,0x08,0x08,0x08,0x08,0xF8,0x0F,0x08,0x08,0x08,0x08,0xF8,0x0F,0x08,0x08
};
const unsigned char neng[] = {
    0x08,0x02,0x24,0x22,0x42,0x12,0xFF,0x0E,0x80,0x02,0x00,0x42,0x7E,0x42,0x42,0x7C,
    0x42,0x00,0x7E,0x22,0x42,0x12,0x42,0x0E,0x7E,0x02,0x42,0x42,0x52,0x42,0x22,0x7C
};

```

```

const unsigned char ke[] = {
    0x10,0x08,0xB8,0x08,0x0F,0x09,0x08,0x09,0x08,0x08,0xBF,0x08,0x08,0x09,0x1C,0x09,
    0x2C,0x08,0x0A,0x78,0xCA,0x0F,0x09,0x08,0x08,0x08,0x08,0x08,0x08,0x08,0x08,0x08
};
const unsigned char ji[] = {
    0x08,0x04,0x08,0x04,0x08,0x04,0xC8,0x7F,0x3D,0x04,0x08,0x04,0x08,0x04,0xA8,0x3F,
    0x18,0x21,0x0C,0x11,0x0B,0x12,0x08,0x0A,0x08,0x04,0x08,0x0A,0x8A,0x11,0x64,0x60
};

```

打开设备，使用 write 向设备中写定义好的点阵数据，即可显示点阵

```

int main(int argc, char** argv)
{
    int aip_fd, ret;
    aip_fd = open("/dev/aip", O_RDWR);
    printf("test: aip_fd=%d\n", aip_fd);
    if (aip_fd == -1)
    {
        printf("test: Error Opening aip1944\n");
        return -1;
    }
    ret = write(aip_fd, ke, sizeof(ke));
    sleep(3);
    ret = write(aip_fd, ji, sizeof(ji));
    sleep(3);

    close(aip_fd);
    return 0;
}

```

例如上图所示代码，所显示点阵为“科”“技”、

6.3 练习：点阵实现任意文字显示

在完成本章基础实验后，学生需要能通过字模软件获取字模信息，并通过点阵显示。可以让多个文字在点阵上依次显示，或滚动显示。

任务要求：

1. 获取字模数据：使用字模生成工具获取待显示文字的点阵数据。学生需要选择合适的字模工具，将文本转换为点阵格式的数据。

2. 实现文字显示

① 固定文字显示：将获取的点阵数据编写到代码中，实现文字的固定显示和多个文字依次显示。

② 滚动文字显示：修改代码中的数据，使得滚动方式显示的文字为自己设置的文字和顺序。

扩展任务：

1. 任意文字支持：修改现有 API，允许输入任意数量的任意文字数据，并根据需要显示或滚动。

2. 参数化控制：实现 API 参数化控制，包括文字的滚动速度、显示位置等。

第7章 RS-485 通讯实验

7.1 实验目的

了解教学平台下 RS485 的用法及控制。掌握 RS485 的电气特性与传输原理，认识教学平台下 RS485 端口的连接方式与控制方法。

根据实际进行编程，实现信息传输功能。如连接 485 转 232 到上位机并打开串口调试助手。在教学平台对应的串口上输入信息回车，在其它开发板或上位机上会显示相应信息，同时可以回复信息并在串口上显示出来。

7.2 RS-485 通讯简介

RS-485（也称作 EIA-485、485 通信）是隶属于 OSI 模型物理层的电气特性规定，为 2 线、半双工、多点通信的标准。其电气特性和 RS-232 不同，用缆线两端的电压差值来表示传递信号。

RS-485 仅仅规定了接受端和发送端的电气特性。它没有规定或推荐任何数据协议，因此在传输的数据、代码的编写上与 RS-232 没有任何不同，同样是根据约定的波特率、数据位、校验位、停止位进行数据收发。RS-485 的特点包括：

（1）接口电平低，不易损坏芯片。RS-485 的电气特性：逻辑“1”以两线间的电压差为 $+ (2 \sim 6)V$ 表示；逻辑“0”以两线间的电压差为 $- (2 \sim 6)V$ 表示。接口信号电平比 RS-232 降低了，不易损坏接口电路的芯片，且该电平与 TTL 电平兼容，可方便与 TTL 电路连接。

（2）传输速率高。10 米时，RS-485 的数据最高传输速率可达 35Mbps，在 1200m 时，传输速度可达 100Kbps。

（3）抗干扰能力强。RS-485 接口是采用平衡驱动器和差分接收器的组合，抗共模干扰能力增强，即抗噪声干扰性好。

（4）传输距离远，支持节点多。RS-485 总线最长可以传输 1200m 以上（速率 $\leq 100Kbps$ ）一般最大支持 32 个节点，如果使用特制的 485 芯片，可以达到 128 个或者 256 个节点，最大的可以支持到 400 个节点。

RS-485 推荐使用在点对点网络中，线型，总线型，不能是星型，环型网络。理想情况下 RS-485 需要 2 个匹配电阻，其阻值要求等于传输电缆的特性阻抗（一般为 120Ω ）。没有特性阻抗的话，当所有的设备都静止或者没有能量的时候就会产生噪声，而且线移需要双端的电压差。没有终接电阻的话，会使得较快速的发送端产生多个数据信号的边缘，导致数据传输出错。

7.3 实验步骤

7.3.1 物理层分析

一般的中控芯片中不具有 485 通信接口，但一般都具有 UART 接口，所以大多数使用 485 通信时，都是使用串口通信通过一个 485 电平转换芯片，然后发出 485 通信信号，在试验箱中，使用的转换芯片为 SP485E。

再次提醒，485 通信仅规定了信号的电气特性，SP485 的作用仅为将 UART 接

口发送的高低电平转换为 485 电气特性的高低电平，编写程序时和其他串口通信没有任何不同。但需要考虑通信的物理层，试验箱通过编号为 J31 的接线柱向外传输转换后的 485 信号，使用时需要注意对方可以接收何种信号，接收的同为 485 信号（如两个试验箱之间通信）则可以直接使用杜邦线连接两个接线柱，如果对方无法接收 485 信号，如试验箱与上位机（PC）通信，则需要将 485 信号通过 485-USB 或 485-232 转换芯片转换后再连接，RS-485 接口原理图如图 7-1。

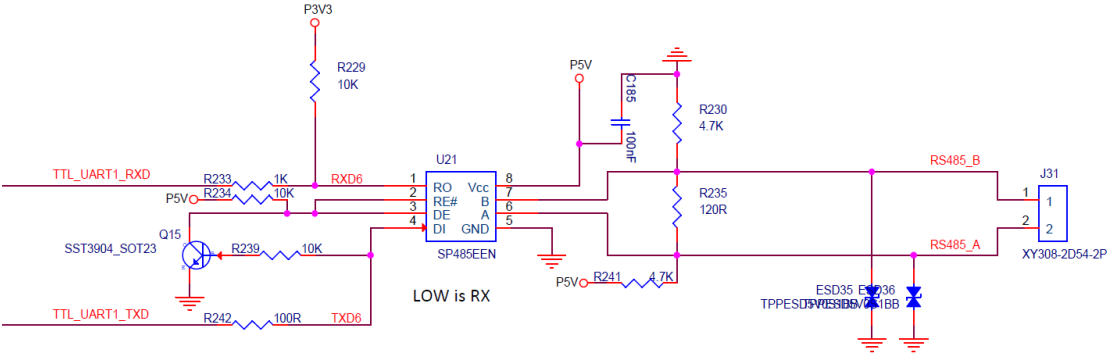


图 7-1 RS-485 接口原理图

查看图 7-2 芯片的真值表可以发现，芯片的三个引脚用于控制数据发送，而试验箱仅通过 UART1 的 TXD 一条线便完成了数据的发送，对齐电路进行分析可以发现，TXD 为高电平（置 1）时，三极管 Q15 导通，RE#和 DE 接地，因为 RE#会取反，所以此时处于数据接收模式，可以从 RO 引脚读取到数据，485 通信在一般状态下均处于数据接收模式，即 TXD=1。但注意此时芯片同时也在输出，如果不考虑输入引脚，此时 AB 引脚输出的数值就是 TXD=1 时芯片发送的数值，分析芯片内部电路可以发现，在 TXD=1 时，DE=0，DI=1，D 未使能输出低电平，AB 的电平由外部电路确定，又因为外部电路中 A 端接上拉电阻 R241，B 端接下拉电阻 R230，AB 端之间接有电阻 R235，故此时电压值 A>B，输出的 485 信号为 1。同理可以分析得到 TXD=0 时，输出的 485 信号为 0，感兴趣的读者可以自行分析。

可以发现，在不考虑输入引脚 RO 时，TXD 的逻辑值就是芯片输出的逻辑值，而常态将 TXD 置 1，不仅让设备在不发送数据时一直处于数据接收模式，而且芯片的输出也为逻辑 1，符合通常串口的通信标准，实现了通信的透明化，编程人员只需要对 UART 接口进行操作。

INPUTS			OUTPUTS		
RE	DE	DI	LINE CONDITION	A	B
X	1	1	No Fault	1	0
X	1	0	No Fault	0	1
X	0	X	X	Z	Z
X	1	X	Fault	Z	Z

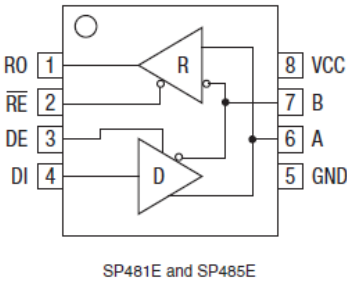


图 7-2 芯片真值表

7.3.2 转换器接线

在本节中，将介绍试验箱通过 485 接口与上位机（PC）通讯的接线方法。需要准备任意具有 485-USB 电平转换芯片的转换设备一个，如果上位机使用 232 通讯，也可以准备 485-232 的转换设备。

因为试验箱主板接线柱的位置不适合操作，建议先将试验箱的主板卸下，卸

下主板前须切断供电，以避免螺丝刀刮擦导致短路，并记得提前拆下与拓展板之间的接线，否则卸下主板后难以拔插。如图 7-3，卸下固定主板的四个螺丝：

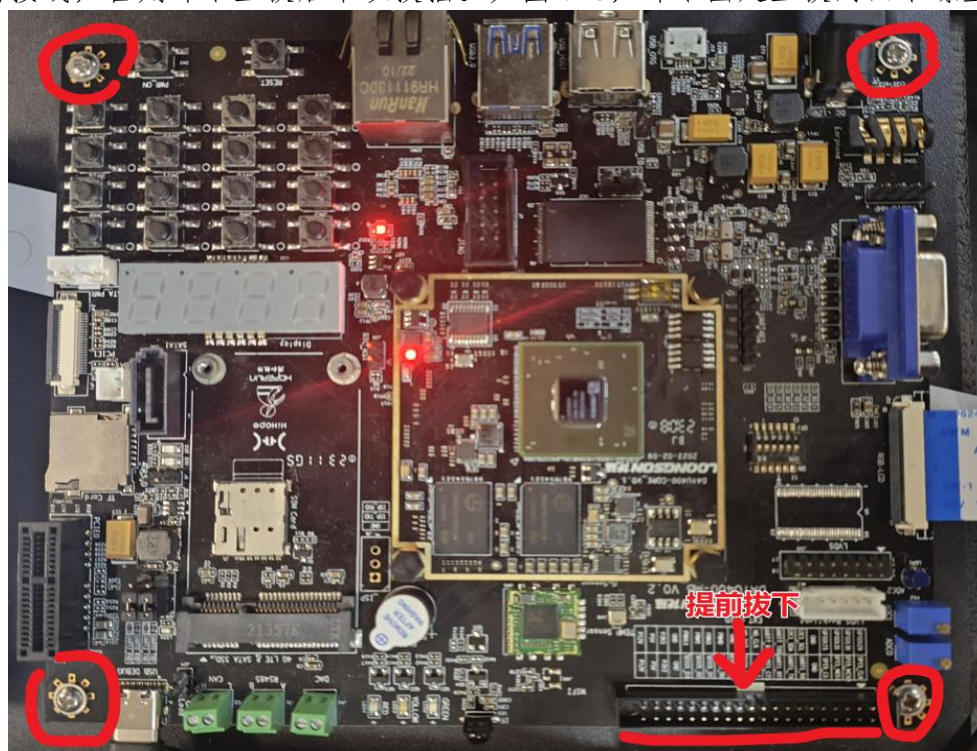


图 7-3 操作指导图

取出两根公对公杜邦线（一端为公，另一端根据转换设备接口选择），将 A 端与 A 端连接，B 端与 B 端连接，与常见串口的 RX 连 TX 不同，485 通信为半双工，通过 A 与 B 的压差表示数据，如图 7-4。

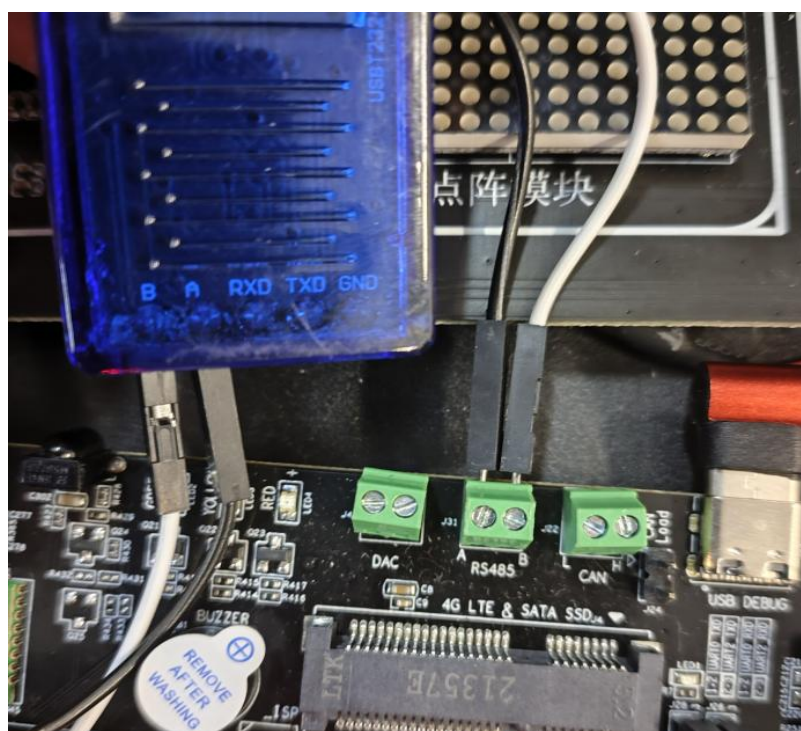


图 7-4 操作指导图

将连接后的转换设备连接 PC 机，可以将其连接至主机或虚拟机，连接主机

可以安装 CH340 等串口驱动后使用 PC 端的串口调试软件，如 XCOM、MobaXterm 等，连接虚拟机则可以直接使用 minicom 等程序进行调试。本实验中将演示这两种方法。

7.3.3 试验箱收发程序

在主机或虚拟机中有现成的软件用于收发数据，但试验箱中的收发数据就需要自行编写了，好在 Linux 有自带的 UART 串口驱动，无需我们编写驱动，我们只需要在用户空间打开对应串口，并通过 `read()` 和 `write()` 即可实现数据的收发。

与 SP485E 芯片连接的串口为 UART1，其通常对应设备 `/dev/ttyS1`。如果无法确定串口对应哪个设备名，可以先使用 `cat /proc/tty/driver/serial`，找到内核中地址为 `0x1FF40400` 的 UART 设备，其对应 UART1，然后使用 `echo message > /dev/ttySx`，每次发送后检查前一个指令中返回的 `rx` 和 `tx`，根据数据的变化判断设备对应的串口。

测试程序预处理内容如下：

```
#include <stdio.h>
#include <fcntl.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
```

创建数据结构线程，使用全局变量 `flag` 控制进程结束。

```
int flag = 1;
void *msg_receive(void *arg)
{
    int ret, fd = *((int *)arg);
    char read_buf[BUFSIZE] = { 0 };
    while (flag)
    {
        ret = read(fd, read_buf, sizeof(read_buf));
        if (ret == 1 && read_buf[0] == '\n') continue;
        printf("num:%d, recv:%s\r\n", ret, read_buf);
    }
}
```

主线程中开启串口对应设备，并创建数据读取线程，将文件描述符地址作为参数传递。

```

int main(int ac,char* av[])
{
    int fd, ret;
    char *filename = "/dev/ttyS1";
    char name[BUFSIZE];
    char buf[BUFSIZE];
    pthread_t pth;
    fd = open(filename, O_RDWR);
    if(fd < 0){
        perror(filename);
        exit(1);
    }
    ret = pthread_create(&pth, NULL, msg_receive, &fd);
    if (ret != 0) {
        printf("test: Error Creating pthread\n");
        return(-1);
    }
}

```

主线程用于数据发送,name 缓冲区为可选,设置在发送失败或输入为"\exit"时退出程序。请注意,如果使用 ctrl+c 等信号方式结束程序,可能导致数据读取线程无法关闭,导致后台出现不可控的数据接收。

```

    printf("Please input your nickname:");
    scanf("%s",name);
    int name_len = strlen(name);
    name[name_len] = '.';
    name[name_len+1] = '\0';
    while(getchar() != '\n'){
        continue;
    }
    printf("\n");
    while(fgets(buf,BUFSIZE,stdin) != NULL){
        buf[strlen(buf)] = '\0';
        if (write(fd,name,strlen(name)) == -1 || write(fd,buf,strlen(buf)) == -1 || \
            strcmp(buf, "\\exit", 5) == 0) {
            flag = 0;
            break;
        }
    }
    close(fd);
    return 0;
}

```

编译程序时请注意，因为使用了多线程编程，所以需要添加编译选项-lpthread，Makefile 如下：

```
all:test
clean:
    rm -rf test_485
test:
    loongarch64-linux-gnu-gcc -lpthread -O2 test_485.c -o test_485
```

7.3.4 与上位机通讯——主机

首先介绍在主机中与试验箱通讯的方式，读者可以自行选择通过主机通讯或通过虚拟机通讯，将接线后的转换器连接上位机，在 VMware 弹出的窗口选择连接到主机，如图 7-5。

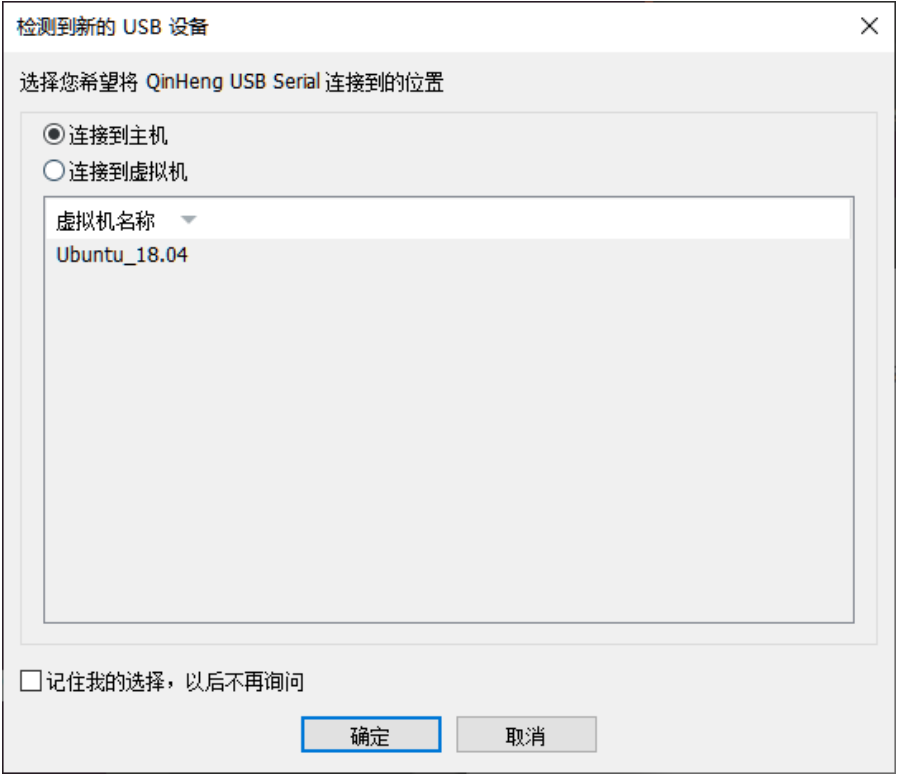


图 7-5 虚拟机连接试验箱

主机中需要提前安装串口驱动，如 CH340，以 windows 10 为例，设备管理器中识别到串口即可，如图 7-6：

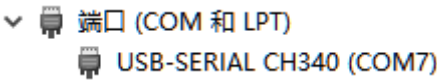


图 7-6 识别串口

开启串口调试软件，如 XCOM，选择识别到的串口，配置波特率为 115200，停止位为 1，数据位为 8，校验位无，开启串口，如图 7-7。

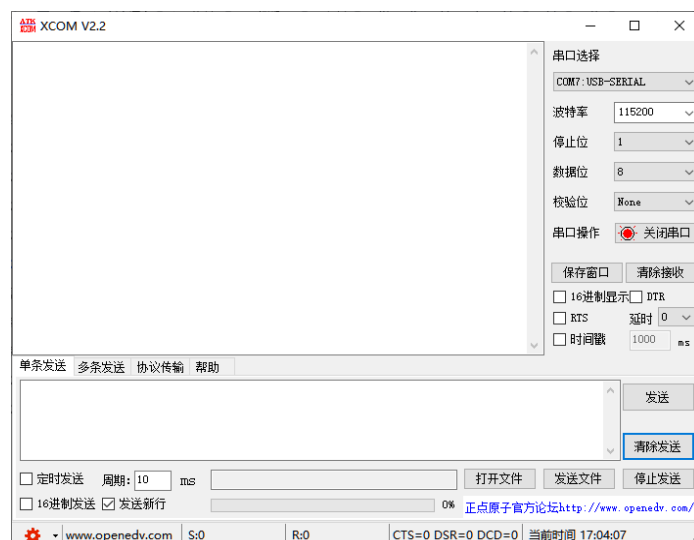


图 7-7 开启串口调试

在试验箱中进行串口的配置：`stty -F /dev/ttyS1 speed 115200 cs8`，并检查配置结果，如图 7-8：

```
[root@LS-GD ~]# stty -F /dev/ttyS1
speed 115200 baud; line = 0;
-brkint -imaxbel
```

图 7-8 检查串口配置

在试验箱上启动上一节编译后的 485 读写程序，发送信息，可以在 XCOM 中查看到试验箱发送的消息，如图 7-9：

```
[root@LS-GD 485]# ./test_485
Please input your nickname:yjr
abc
```

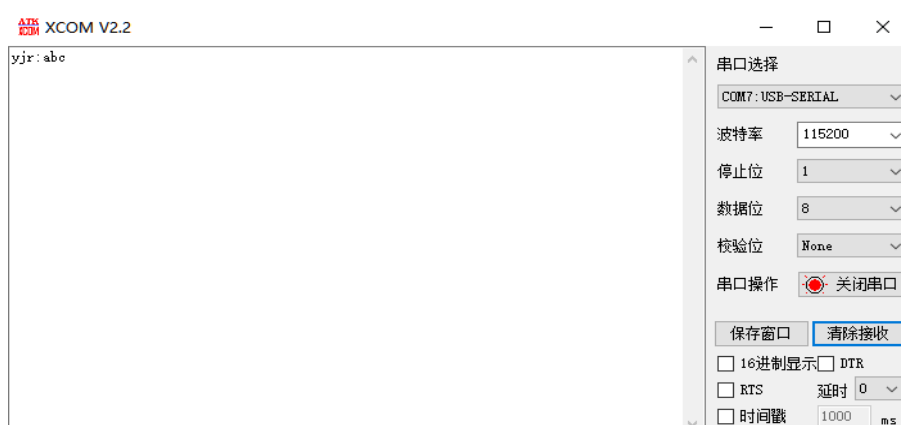


图 7-9 试验箱发送信息串口打印

在 XCOM 中选择“发送新行”，即自动在发送数据末端添加“`\r\n`”，输入信息并点击发送，可以在试验箱中查看到上位机发来的信息，如图 7-10。

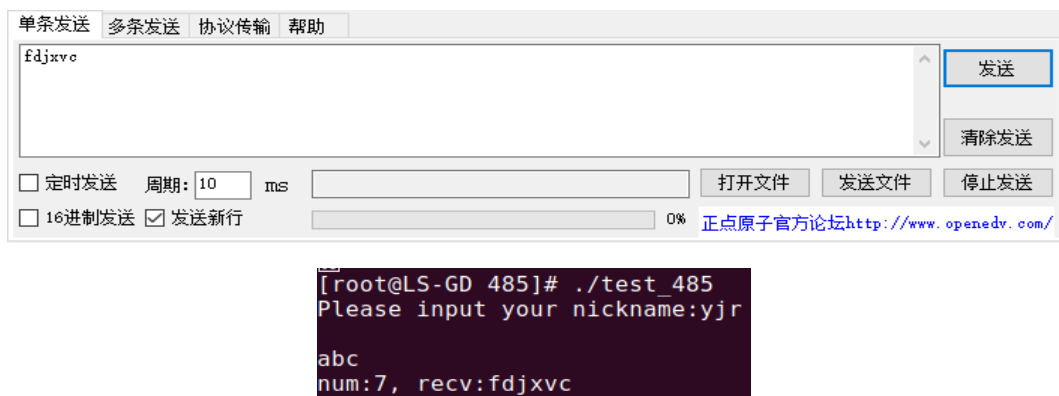


图 7-10 上位机发送信息试验箱中显示信息

7.3.5 与上位机通讯——虚拟机

将转换器连接到虚拟机，一般虚拟机中已经接入了试验箱的 debug 串口，所以接入转换器接口后一般设备名为/dev/ttyUSB1，请根据实际情况选择，见图 7-11。

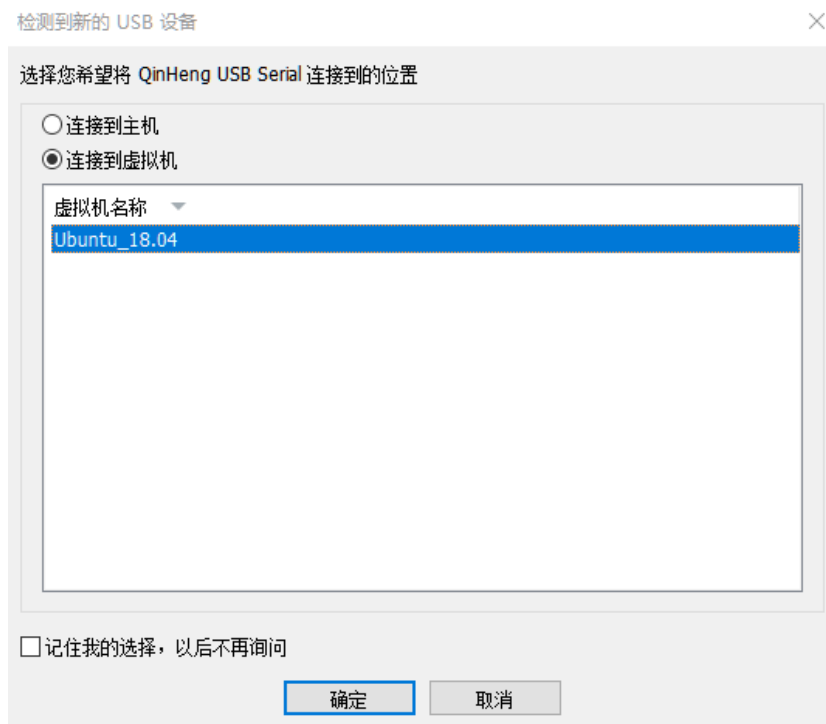


图 7-11 连接到虚拟机

在串口调试软件中先配置为对应的接口，再启动串口调试，以 minicom 为例，使用 minicom -s 配置串口后启动，如图 7-12。

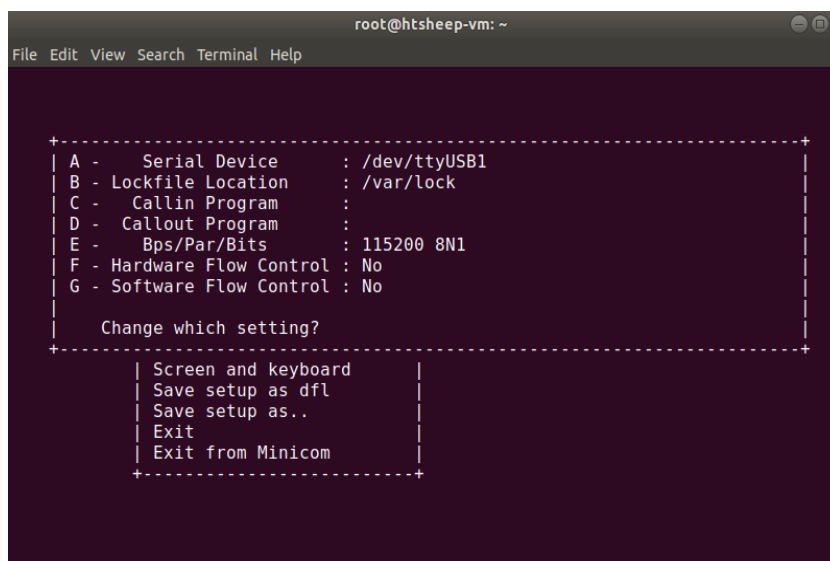


图 7-12 串口配置

选择 exit 使用改配置启动 minicom, 如图 7-13。

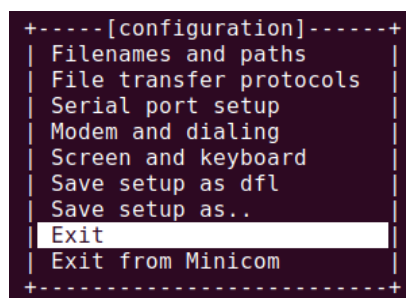


图 7-13 选择 exit

之后的流程和上一节相同，在与试验箱通信的窗口中启动通信程序，两方便可以互发消息，如图 7-14。

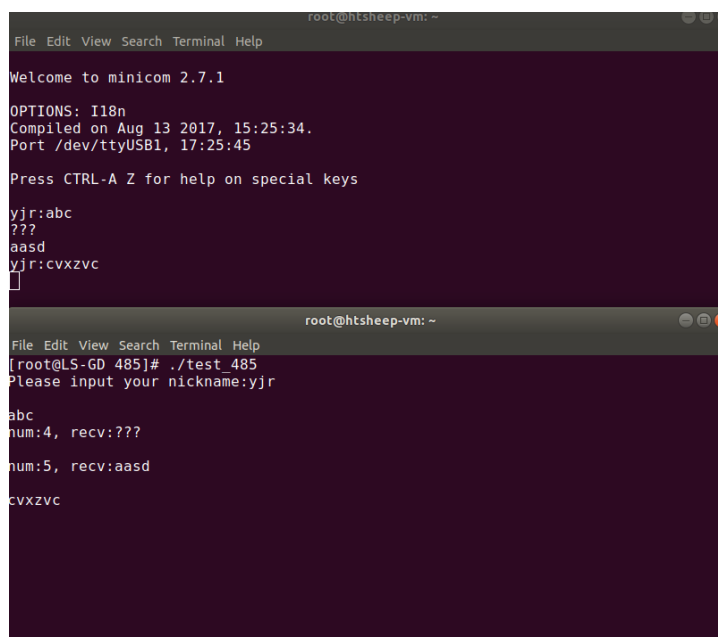


图 7-14 互发信息验证

7.4 练习：RS485 通信实现设备操控

在之前的实验中，学生已经掌握了通过 RS485 通信发送和接收字符串的基础操作。本节将让学生通过 RS485 通信控制外部设备（如数码管、点阵等）。学生可以通过发送 16 进制命令或字符串命令，实现对外部设备的操作控制。

任务要求：

1. 设备控制命令的设计：设计一套 RS485 控制命令，可以通过 16 进制指令或字符串指令控制设备。例如，16 进制命令 55 01 12 01 23 55 设置数码管为 12:23，字符串命令 `settime 12:23` 达到相同效果。

2. 设备接口的实现：学生需要实现接收命令并解析的代码，根据命令内容控制指定设备的显示或动作。

扩展任务：

1. 多设备控制：扩展 RS485 命令的功能，支持同时控制多个设备。例如，学生可以通过一条命令控制数码管和点阵同时显示不同的信息。

2. 双向通信：实现 RS485 双向通信功能，让设备能够反馈当前状态给发送端。例如，发送指令获取数码管当前显示的时间，设备返回当前显示内容。

第8章 LED 实验

8.1 实验目的

了解教学平台下的 LED 灯控制，掌握 LED 灯的电气特性与点亮模式，认识教学平台下对相应 LED 灯的连接方式与控制端口。

进行编程，依靠教学平台上的“红“、“黄“、“绿“三色 LED 灯实现流水灯效果和闪烁，跳变灯以模拟交通灯的工作。

8.2 实验步骤

8.2.1 LED 模块简介

试验箱的主板上放置了三块 0805 贴片 LED 灯，具有低厚度、小基底面和出色的照明性能。它们的颜色分别为绿色、黄色和红色。通过 GPIO 复用配置寄存器可分别控制这三个 LED 灯的明和灭。

8.2.2 LED 模块内核驱动编写

8.2.2.1 依赖头文件

```
#include <linux/types.h>
#include <linux/kernel.h>
#include <linux/delay.h>
#include <linux/ide.h>
#include <linux/init.h>
#include <linux/module.h>
#include <linux/errno.h>
#include <linux/gpio.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/of.h>
#include <linux/of_address.h>
#include <linux/of_gpio.h>
#include <asm/uaccess.h>
#include <asm/io.h>
```

8.2.2.2 声明必要变量

声明了 LED 设备（注册到内核中的设备）的数量，LED 设备的名称，LED 开和关时寄存器应设置的状态以及 LED 对应 GPIO 口的物理地址和虚拟地址变量。

```
#define LED_CNT 1 /* 设备号个数 */
#define LED_NAME "led" /* 名字 */
```

```

#define LEDOFF 0 /* 关 led */
#define LEDON 1 /* 开 led */
#define GPIO_143_136 0x1fe104d4
static void __iomem *gpio_v;

```

8.2.2.3 声明设备结构体

在设备结构体中声明了设备号、cdev、类、device、主设备号、设备节点以及三个 LED 对应的 GPIO 口的编号等信息，并初始化了一个全局 led 设备结构体

```

struct led_dev {
    dev_t devid; /* 设备号 */
    struct cdev cdev; /* cdev */
    struct class *class; /* 类 */
    struct device *device; /* 设备 */
    int major; /* 主设备号 */
    int minor; /* 次设备号 */
    struct device_node *nd; /* 设备节点 */
    int green_gpio;
    int yellow_gpio;
    int red_gpio;
};
struct led_dev led; /* led 设备 */

```

8.2.2.4 定义设备操作函数

首先声明了设备操作函数的结构体，LED 设备对应打开、写入以及释放操作

```

static struct file_operations led_fops = {
    .owner = THIS_MODULE,
    .open = led_open,
    .write = led_write,
    .release = led_release,
};

```

对于打开 LED 设备，只需要将全局 led 设备结构体绑定到设备的私有数据上即可

```

static int led_open(struct inode *inode, struct file *filp) {
    filp->private_data = &led; /* 设置私有数据 */
    return 0;
}

```

对于写入 LED 设备，首先从设备私有数据中获取设备结构体，将写入的数据拷贝

到内核空间，如果 0 号字节为 0 则关闭绿灯，最低位为 1 则开启绿灯，其余两个字节同理，只不过操作的 LED 灯有变化。

```
static ssize_t led_write(struct file *filp, const char __user *buf, size_t cnt,
                        loff_t *offt) {
    int retvalue;
    unsigned char databuf[3];
    struct led_dev *dev = filp->private_data;
    retvalue = copy_from_user(databuf, buf, cnt);
    if (retvalue < 0) {
        printk("kernel write failed!\r\n");
        return -EFAULT;
    }
    if (databuf[0] == LEDON) gpio_set_value(dev->green_gpio, 1);
    else gpio_set_value(dev->green_gpio, 0);

    if (databuf[1] == LEDON) gpio_set_value(dev->yellow_gpio, 1);
    else gpio_set_value(dev->yellow_gpio, 0);

    if (databuf[2] == LEDON) gpio_set_value(dev->red_gpio, 1);
    else gpio_set_value(dev->red_gpio, 0);
    return 0;
}
```

对于释放 LED 设备，仅定义一个函数，无任何实质性实现；可以将此处加入操作，如关闭所有 LED 灯

```
static int led_release(struct inode *inode, struct file *filp) {
    return 0;
}
```

8.2.2.5 声明设备入口以及出口函数

对于设备入口函数

首先定义三个 LED 灯对应 GPIO 口，将寄存器地址映射到内核空间，配置 GPIO 复用配置寄存器，请求 GPIO 口使用权限，并配置 GPIO 口的为输出模式。

```
static int __init led_init(void)
{
    int ret = 0;
    led.green_gpio = 136;
    led.yellow_gpio = 138;
    led.red_gpio = 137;
    u32 gpio_read, gpio_write;
```

```

gpio_v = ioremap(GPIO_143_136, sizeof(u32));
gpio_read = ioread32(gpio_v);
gpio_write = (gpio_read & 0x77777000);
iowrite32(gpio_write, gpio_v);

ret |= gpio_request(led.green_gpio, "green_gpio");
ret |= gpio_request(led.yellow_gpio, "yellow_gpio");
ret |= gpio_request(led.red_gpio, "red_gpio");
ret |= gpio_direction_output(led.green_gpio, 0);
ret |= gpio_direction_output(led.yellow_gpio, 0);
ret |= gpio_direction_output(led.red_gpio, 0);
if (ret < 0) printk("can't set gpio!\r\n");

```

然后分配设备号，如果未手动定义设备号，则由 linux 内核自动分配，随后初始化 cdev，添加 dev 到内核，创建类以及在 linux 中创建设备。

```

if (led.major) { /* 1、创建设备号 */
    led.devid = MKDEV(led.major, 0);
    register_chrdev_region(led.devid, LED_CNT, LED_NAME);
} else { /* 没有定义设备号 */
    alloc_chrdev_region(&led.devid, 0, LED_CNT,
        LED_NAME); /* 申请设备号 */
    led.major = MAJOR(led.devid); /* 获取分配号的主设备号 */
    led.minor = MINOR(led.devid); /* 获取分配号的次设备号 */
}
led.cdev.owner = THIS_MODULE; /* 2、初始化 cdev */
cdev_init(&led.cdev, &led_fops);
cdev_add(&led.cdev, led.devid, LED_CNT); /* 3、添加一个 cdev */
led.class = class_create(THIS_MODULE, LED_NAME); /* 4、创建类 */
if (IS_ERR(led.class)) {
    return PTR_ERR(led.class);
}
/* 5、创建设备 */
led.device = device_create(led.class, NULL, led.devid, NULL, LED_NAME);
if (IS_ERR(led.device)) {
    return PTR_ERR(led.device);
}

return 0;
}

```

对于 LED 模块出口函数

首先，将寄存器从内核空间取消地址映射，释放 GPIO 资源，然后删除系统中 LED 的 cdev 对象，注销掉申请的设备号，然后撤销设备和创建的类即完成模块卸载。

```
static void __exit led_exit(void) {
    iounmap(gpio_v);
    gpio_free(led.green_gpio);
    gpio_free(led.yellow_gpio);
    gpio_free(led.red_gpio);

    /* 注销字符设备驱动 */
    cdev_del(&led.cdev); /* 删除 cdev */
    unregister_chrdev_region(led.devid, LED_CNT); /* 注销设备号 */
    device_destroy(led.class, led.devid);
    class_destroy(led.class);
}
```

8. 2. 3 LED 模块测试代码编写

引入依赖的头文件

```
#include "stdio.h"
#include "unistd.h"
#include "sys/types.h"
#include "sys/stat.h"
#include "fcntl.h"
#include "stdlib.h"
#include "string.h"
```

接收一个参数，控制三个 led 灯的开关，例如 000 全灭，111 全开

```
int main(int argc, char *argv[])
{
    int fd, retvalue;
    char *filename;
    unsigned char databuf[3];

    if (argc != 2) {
        printf("Error Usage!\r\n");
        return -1;
    }
}
```

随后打开 LED 设备并处理输入的参数，将 ASCII 码表示的 0 和 1 转为数字 0 和

1, 将数据写入 LED 设备, 测试结束后关闭 LED 设备。

```
/* 打开 led 驱动 */
fd = open("/dev/led", O_RDWR);
if (fd < 0) {
    printf("file /dev/led open failed!\r\n");
    return -1;
}
databuf[0] = argv[1][0] - '0'; /* 要执行的操作: 打开或关闭 */
databuf[1] = argv[1][1] - '0'; /* 要执行的操作: 打开或关闭 */
databuf[2] = argv[1][2] - '0'; /* 要执行的操作: 打开或关闭 */
printf("databuf[0]:%d,databuf[1]:%d,databuf[2]:%d\r\n", databuf[0],
      databuf[1], databuf[2]);
retvalue = write(fd, databuf, sizeof(databuf));
if (retvalue < 0) {
    printf("LED Control Failed!\r\n");
    close(fd);
    return -1;
}
close(fd); /* 关闭文件 */
return 0;
}
```

8.3 练习: LED 基本实验和设备状态反馈

LED 是嵌入式系统中常用的控制器件。学生至少需要能控制 LED 实现流水灯和呼吸灯效果。通过此实验, 学生可以熟悉基本的 LED 控制逻辑, 并进一步理解如何通过软件灵活控制硬件外设。

任务要求:

1. 流水灯效果实现: 学生需要实现一个简单的流水灯效果, LED 按照一定的顺序依次点亮和熄灭, 循环运行。

2. 呼吸灯效果实现: 学生通过调节 LED 亮度的变化, 模拟“呼吸”效果, 即 LED 的亮度逐渐变亮再逐渐变暗, 形成循环。(亮度的调节需要让 LED 在极短的周期内关闭和开启, 建议选择 40ms 以下。以 25ms 为例, 每过 25ms LED 开启关闭各一次, 控制 LED 开启和关闭所占时间比例即可控制 LED 灯亮度)

扩展任务:

1. 整点 LED 闪烁: 扩展代码, 当第五章实验中数码管显示的时间为整点时, LED 灯闪烁提醒。学生可以设置一个计时器来检测整点, 并触发 LED 闪烁。

2. RS485 通信状态指示: 实现第七章实验 RS485 通信时 LED 的状态指示。当有信息发送时, 绿色 LED 闪烁; 当有信息接收时, 红色 LED 闪烁。

第9章 高速电机实验

9.1 实验目的

了解教学平台下高速电机的用法与控制。掌握高速电机的工作原理，认识教学平台下高速电机的连接方式与控制方法。

根据高速电机的接口与原理编程，实现高速电机的控制程序，能够控制电机的启动，停止，顺时针转与逆时针转，进行转速调节等等。

9.2 TC118S 马达驱动器

TC118S 是一款单通道直流马达驱动器，其支持驱动前进、后退、停止及刹车功能，常应用于玩具马达驱动、电动牙刷、电子锁、电动茶具等领域。

该芯片与电机位于试验箱的拓展板上，其设计图如图 9-1。

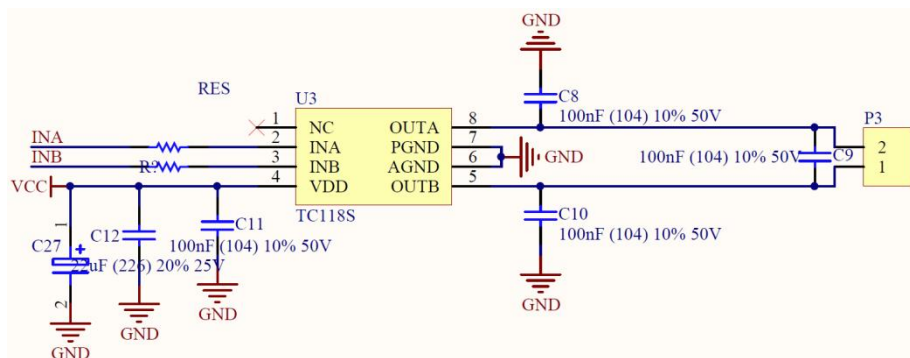


图 9-1 原理图

试验箱通过 INA 和 INB 两个引脚控制电机，其输入/输出逻辑图如图 9-2。

输入		输出		方式
INA	INB	OUTA	OUTB	
L	L	Hi-Z	Hi-Z	待命状态
H	L	H	L	前进
L	H	L	H	后退
H	H	L	L	刹车

图 9-2 输入/输出逻辑图

由于 TC118S 仅作为输入模块，且运动方式只有四种，考虑使用 ioctl 编写电机驱动。

9.3 实验步骤

9.3.1 内核模块代码编写

1) ioctl 与设备控制

ioctl 通过文件描述符和控制命令来控制设备，ioctl 的函数原型如下：

```
int ioctl(int fd, unsigned long request, ...);
```

其中，fd 是 open() 打开的文件描述符，request 是 32 位命令码，可变参数 arg 是当某些命令需要传参时使用，可以传一个整数或一个指针。

命令码的组成格式如图 9-3，其类型由 dir 表示，分为无数据(_IO)、读数据(_IOR)、写数据(_IOW)、读写数据(_IOWR)四种模式。type 指 magic number，通常翻译为“魔数”或“幻数”，可以是任意一个 char 类型字符，用于使 ioctl 命令有唯一设备标识，注意内核中“x”, “y”, “z”已被使用了。nr 指 number，为命令的编号，取值范围 0-255。size 涉及可变参数 arg，用于传递 arg 的数据类型的长度。

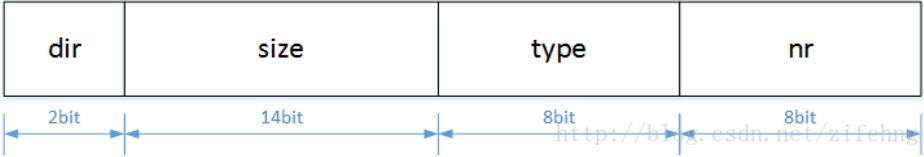


图 9-3 命令码格式

首先定义设备使用的命令码，命令码宏在内核空间和用户空间使用的代码相同，可以为其额外创建一个头文件声明该宏，便于内核代码和用户代码共同使用。直接将命令码声明代码复制到用户空间代码也可以。

```
#define MAGIC_NUMBER 'k'
#define CMD_standby_IO(MAGIC_NUMBER ,1)
#define CMD_forward_IO(MAGIC_NUMBER ,2)
#define CMD_backward_IO(MAGIC_NUMBER ,3)
#define CMD_brake_IO(MAGIC_NUMBER ,4)
```

编写 ioctl 操作函数，通常 ioctl 使用 switch 对不同的命令码进行识别，因为仅需用于切换模式，所以不需要使用可变参数 arg，识别到特定的命令直接对 gpio 端口进行操作即可。

```
static long motor_ioctl(struct file *file, unsigned int cmd, unsigned long arg) {
    switch(cmd) {
        case CMD_standby:
            gpio_set(0,0); break;
        case CMD_forward:
            gpio_set(1,0); break;
        case CMD_backward:
            gpio_set(0,1); break;
        case CMD_brake:
            gpio_set(1,1); break;
        default:
            break;
    };
    return 0;
}
```

编写 gpio 端口控制函数，仅需对 INA 和 INB 端口进行控制，两次操作之间建议延时至少 10us。

```

#define DELAY_TIME      10
static void gpio_set(int ina,int inb)
{
    gpio_set_value(misc_motor.ina_gpio, ina);
    udelay(DELAY_TIME);
    gpio_set_value(misc_motor.inb_gpio, inb);
}

```

使用的 `misc_motor` 是设备的私有数据结构体。

```

struct motor_dev{
    dev_t devid;          /* 设备号      */
    void *private_data;   /* 私有数据 */
    int ina_gpio;
    int inb_gpio;
};
struct motor_dev misc_motor;

```

再编写设备的 `open` 函数。

```

static int motor_open(struct inode *inode, struct file *filp)
{
    filp->private_data = &misc_motor; /* 设置私有数据 */
    return 0;
}

```

2) 模块初始化和卸载

配置 MISC 设备结构体，子设备号在当前 MISC 设备使用的子设备号之外任选即可。

```

#define MOTOR_NAME      "motor" /* 名字      */
#define MOTOR_MINOR     150     /* 子设备号 */
static struct file_operations misc_motor_fops = {
    .owner = THIS_MODULE,
    .open = motor_open,
    .unlocked_ioctl = motor_ioctl,
};
static struct miscdevice motor_miscdev = {
    .minor = MOTOR_MINOR,
    .name = MOTOR_NAME,
    .fops = &misc_motor_fops,
};

```

编写模块初始化函数，首先注册 misc 设备，再进行复用配置寄存器的配置，本次使用的两个 GPIO 端口为 84 和 85，分别对应 INA 和 INB。

```
#define INA_GPIO 84
#define INB_GPIO 85
#define GPCFG 0x1fe104b8 //GPIO 87-80
static void __iomem *GPCFG_V;
static int __init motor_init(void)
{
    int ret;
    int i;
    u32 GPCFG_read, GPCFG_write;
    ret = misc_register(&motor_miscdev);
    if(ret < 0){
        printk("misc device register failed!\r\n");
        return -EFAULT;
    }
    GPCFG_V = ioremap(GPCFG, sizeof(u32));
    GPCFG_read = ioread32(GPCFG_V);
    GPCFG_write = (GPCFG_read & 0x77007777);
    iowrite32(GPCFG_write, GPCFG_V);
}
```

最后配置和申请 GPIO 端口，并将两个端口均置为低电平，即默认电机为待机模式。

```
misc_motor.ina_gpio = INA_GPIO;
misc_motor.inb_gpio = INB_GPIO;
gpio_request(misc_motor.ina_gpio, "ina");
gpio_request(misc_motor.inb_gpio, "inb");
ret = gpio_direction_output(misc_motor.ina_gpio, 0);
if(ret < 0) printk("can't set ina gpio!\r\n");
ret = gpio_direction_output(misc_motor.inb_gpio, 0);
if(ret < 0) printk("can't set inb gpio!\r\n");
return 0;
}
```

然后编写模块的卸载函数，注册初始化和卸载函数。

```
static void __exit motor_exit(void) {
    misc_deregister(&motor_miscdev);
    iounmap(GPCFG_V);
    gpio_free(misc_motor.ina_gpio);
    gpio_free(misc_motor.inb_gpio);
}
```

```
module_init(motor_init);
module_exit(motor_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR("yangjinrun");
```

9.3.2 测试代码编写指南

用户空间代码使用的命令码要和内核空间相同，建议使用一个特定的头文件进行规定，也可以直接声明在测试代码中。

```
#define MAGIC_NUMBER    'k'
#define CMD_standby _IO(MAGIC_NUMBER    ,1)
#define CMD_forward  _IO(MAGIC_NUMBER    ,2)
#define CMD_backward _IO(MAGIC_NUMBER    ,3)
#define CMD_brake    _IO(MAGIC_NUMBER    ,4)
```

代码测试方法较为简单，使用 open 打开设备后，使用 ioctl 直接传递命令码宏即可实现电机的模式控制。

```
int main(int argc, char** argv) {
    int fd,ret;
    fd = open("/dev/motor", O_RDWR);
    printf("test: fd=%d\n", fd);
    if(fd == -1) {
        printf("test: Error Opening motor\n");
        return(-1);
    }
    ioctl(fd,CMD_forward); sleep(3);
    ioctl(fd,CMD_standby); sleep(3);
    ioctl(fd,CMD_backward); sleep(3);
    ioctl(fd,CMD_brake); sleep(3);
    close(fd);
}
```

9.4 练习：上位机控制高速电机

在本节中将专注于结合电机的多种功能，如待命、前进、后退和刹车等操作，学生需要熟练控制高速电机的基本功能，深入理解嵌入式系统对外部设备的控制。

任务要求：

1. 电机基本控制功能：编写代码，实现电机的待命、前进、后退和刹车功能。

通过使用 RS485 章节中的设备操控功能,通过上位机发送不同的控制指令来触发这些功能,并观察电机的相应反应。

2. 功能切换与测试: 确保能够准确控制电机的状态切换,例如从前进状态切换到后退,或从运行状态切换到刹车状态。

扩展任务:

电机反馈与状态显示: 通过数码管、点阵或 LED 显示电机的当前状态,显示内容可自行设计,使控制逻辑更加直观。

第10章 蜂鸣器实验

10.1 实验目的

了解蜂鸣器的工作原理及使用方法。掌握蜂鸣器的触发方式与电气特性，认识教学平台下蜂鸣器的连接方式与控制端口。

根据教学平台下蜂鸣器的连接方式进行编程，实现蜂鸣器响动，可以拓展实现各种简单的声音组合如警笛声等。

10.2 试验箱蜂鸣器简介

蜂鸣器分为有源蜂鸣器和无源蜂鸣器，“源”指的是震荡源。有源蜂鸣器内部带有震荡源，只需要通电就会工作；无源蜂鸣器内部无震荡源，直流电无法使其发声，需要使用方波驱动。

试验箱使用的是有源蜂鸣器，一端上拉，另一端连接电路较为复杂，仅需了解 GPIO33 引脚置 0 时，蜂鸣器另一端接地即可，因此使用时将 GPIO33 置 0 便可以使蜂鸣器响起，其原理图如图 10-1。

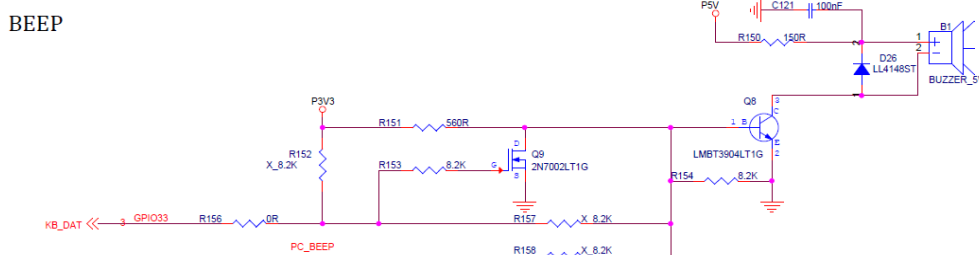


图 10-1 蜂鸣器原理图

10.3 实验步骤

10.3.1 内核模块代码编写

1) 依赖头文件

模块代码依赖的头文件如下：

```
#include <linux/types.h>
#include <linux/kernel.h>
#include <linux/delay.h>
#include <linux/ide.h>
#include <linux/init.h>
#include <linux/module.h>
#include <linux/errno.h>
#include <linux/gpio.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/of.h>
```

```
#include <linux/of_address.h>
#include <linux/of_gpio.h>
#include <asm/uaccess.h>
#include <asm/io.h>
```

2) 设备结构体

蜂鸣器的设备结构体 struct beep_dev 如下，包含设备号、主设备号、次设备号、GPIO 编号等信息。其中，cdev、class、device、device_node 结构体均为 Linux 内核结构体，用于设备注册。

```
struct beep_dev {
    dev_t devid; /* 设备号 */
    struct cdev cdev; /* cdev */
    struct class *class; /* 类 */
    struct device *device; /* 设备 */
    int major; /* 主设备号 */
    int minor; /* 次设备号 */
    struct device_node *nd; /* 设备节点 */
    int beep_gpio; /* beep 所使用的 GPIO 编号 */
};
```

3) 设备操作函数

蜂鸣器需要编写 open()、write()、release() 操作。

open() 函数较为简单，将刚刚创建的设备结构体设置为私有数据即可：

```
static int beep_open(struct inode *inode, struct file *filp) {
    filp->private_data = &beep; /* 设置私有数据 */
    return 0;
}
```

write() 函数是用户空间用于修改设备状态的方法，对于有源蜂鸣器而言，传入的操作指令只需要为“开启”或“关闭”即可，因此只需要 1 个整数以 1 和 0 表示开启和关闭指令，注意这里将 1 作为开启指令是为用户编程习惯考虑，驱动实现时接收到指令 1 时，要将端口置 0 才能开启蜂鸣器。

为统一规范起见，建议使用 unsigned char 数组从用户空间拷贝数据，不过只需要开辟 1 个空间即可，根据传入的数据对 GPIO 端口进行操作，注意使用 else if 而非 else，仅对传入的指令为 0 或 1 时作出反应。

```
#define BEEPOFF 0 /* 关蜂鸣器 */
#define BEEPON 1 /* 开蜂鸣器 */
```

```

static ssize_t beep_write(struct file *filp, const char __user *buf, size_t cnt, loff_t *offt) {
    int retvalue;
    unsigned char databuf[1];
    unsigned char beepstat;
    struct beep_dev *dev = filp->private_data;
    retvalue = copy_from_user(databuf, buf, cnt);
    if (retvalue < 0) {
        printk("kernel write failed!\r\n");
        return -EFAULT;
    }
    beepstat = databuf[0]; /* 获取状态值 */
    if (beepstat == BEEPON) gpio_set_value(dev->beep_gpio, 0); /* 打开蜂鸣器 */
    else gpio_set_value(dev->beep_gpio, 1); /* 关闭蜂鸣器 */
    return 0;
}

```

release() 是关闭/释放设备时调用的方法，这里只进行返回 0，认为任何情况下释放设备都是成功的。

```

static int beep_release(struct inode *inode, struct file *filp) {
    return 0;
}

```

4) 模块初始化和卸载

蜂鸣器使用 GPIO33 端口，因此先在复用配置寄存器将该端口配置为 GPIO 功能（000），GPIO33 对应的寄存器为 GPIO32~39 的复用配置寄存器，使用每 4 位的低 3 位配置功能，先将物理地址转换为虚拟地址，读出寄存器原值后将 33 号端口对应配置清 0，即使用一个其他端口配置位置 1、33 号端口配置位置 0 的掩码相与，再将结果写回到对应地址，完成管脚的配置。

```

#define GPMUX 0x1fe104a0
static void __iomem *GPFMUX0_V;
static int __init beep_init(void)
{
    int ret = 0;
    u32 GPFMUX_READ, GPFMUX_WRITE;
    GPFMUX0_V = ioremap(GPMUX, sizeof(u32));
    GPFMUX_READ = ioread32(GPFMUX0_V);
    GPFMUX_WRITE = (GPFMUX_READ & 0x77777707);
    iowrite32(GPFMUX_WRITE, GPFMUX0_V);
}

```

配置设备结构体的 GPIO 端口号，申请 GPIO 端口号，并将端口配置为高电平，即默认关闭蜂鸣器。

```

beep.beep_gpio = 33;
gpio_request(beep.beep_gpio, "sda3");
/* 设置 GPIO5_IO01 为输出, 并且输出高电平, 默认关闭 BEEP */
ret = gpio_direction_output(beep.beep_gpio, 1);
if (ret < 0) {
    printk("can't set gpio!\r\n");
}

```

然后注册字符设备驱动。首先创建设备号, 如果主设备号已定义, 则可以直接用主设备号通过 MKDEV() 宏获取设备号, 再进行设备号的注册。未定义主设备号则会进入 else 语句, 会申请一个设备号, 然后通过设备号获取主设备号和次设备号, 保存在设备结构体中。

再对设备结构体中的 cdev、class、device 结构体进行初始化, 使用 beep_fops 对 cdev 进行初始化并添加 cdev, 最后创建类和设备后模块便初始化完毕。

```

if (beep.major) { /* 1、创建设备号 */
    beep.devid = MKDEV(beep.major, 0);
    register_chrdev_region(beep.devid, BEEP_CNT, BEEP_NAME);
} else { /* 没有定义设备号 */
    alloc_chrdev_region(&beep.devid, 0, BEEP_CNT,
        BEEP_NAME); /* 申请设备号 */
    beep.major = MAJOR(beep.devid); /* 获取分配号的主设备号 */
    beep.minor = MINOR(beep.devid); /* 获取分配号的次设备号 */
}
printk("beep major=%d,minor=%d\r\n", beep.major, beep.minor);
beep.cdev.owner = THIS_MODULE; /* 2、初始化 cdev */
cdev_init(&beep.cdev, &beep_fops);
cdev_add(&beep.cdev, beep.devid, BEEP_CNT); /* 3、添加一个 cdev */
beep.class = class_create(THIS_MODULE, BEEP_NAME); /* 4、创建类 */
if (IS_ERR(beep.class)) {
    return PTR_ERR(beep.class);
}
beep.device =
    device_create(beep.class, NULL, beep.devid, NULL, BEEP_NAME); /* 5、创建设备 */
if (IS_ERR(beep.device)) {
    return PTR_ERR(beep.device);
}
return 0;
}

```

```

/* 设备操作函数 */
static struct file_operations beep_fops = {
    .owner = THIS_MODULE,
    .open = beep_open,
    .write = beep_write,
    .release = beep_release,
};

```

模块的卸载函数较为简单，需要删除系统中蜂鸣器的 cdev 对象，注销掉申请的设备号，然后撤销设备和创建的类即完成模块卸载。编写完初始化和卸载方法后记得在最后将这两个方法注册为模块的入口和出口函数。

```

static void __exit beep_exit(void) {
    iounmap(GPFMUX0_V);
    /* 注销字符设备驱动 */
    cdev_del(&beep.cdev); /* 删除 cdev */
    unregister_chrdev_region(beep.devid, BEEP_CNT); /* 注销设备号 */
    device_destroy(beep.class, beep.devid);
    class_destroy(beep.class);
}
module_init(beep_init);
module_exit(beep_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR("yangjinrun");

```

至此内核模块代码编写完毕，编写 Makefile 文件后编译得到 .ko 文件即可加载和卸载蜂鸣器内核模块到试验箱。

```

obj-m :=beep.o
KDIR :=${MY_KDIR}
all:
    make -C $(KDIR) M=$(PWD) modules
clean:
    make -C $(KDIR) M=$(PWD) modules clean

```

10.3.2 测试代码编写指南

加载蜂鸣器内核模块后，可在用户空间使用 open() 函数以读写方式开启 /dev/beep 设备文件以获取用于操作蜂鸣器的文件描述符 fd。

```

fd = open("/dev/beep", O_RDWR);
if(fd < 0){
    printf("file %s open failed!\r\n", argv[1]);
    return -1;
}

```

通过 write() 向打开的文件写入数据以控制蜂鸣器, 建议使用 unsigned char 数组作为指令暂存区, 内核驱动在使用时, 会读取传入地址开始的第 1 个字节作为指令, 可自行根据这一特点编写程序。这里给出例程 beep_control, 实现了对蜂鸣器的单次操控, 并通过该例程实现了简易的开启和关闭蜂鸣器方法(例程使用了字符串常量, 但要注意在龙芯架构中, 字符串常量默认为有符号类型, 尽管使用的指令范围 0-1 在有符号和无符号类型对应整数范围的交集内, 但并不建议这样使用)。

```

void beep_on(int fd) {
    beep_control(fd, "1");
}
void beep_off(int fd) {
    beep_control(fd, "0");
}
void beep_control(int fd, unsigned char *ctl) {
    int ret;
    ret = write(fd, ctl, 1);
    if (ret < 0) {
        close(fd);
        perror("Beep Control Failed!\n");
    }
}

```

10.4 练习：蜂鸣器实现闹钟功能

有源蜂鸣器是嵌入式开发中重要的提示设备, 简单的控制和尖锐的声音使得其功能使用方便且应用范围广, 如 Debug、状态指示、提醒功能等, 本节中要求学生通过蜂鸣器实现闹钟功能。

任务要求:

1. 蜂鸣器报时: 在数码管显示时间为整点时, 蜂鸣器响 1-2 次(类似电子表)。
2. 闹钟功能: 编写代码, 用上位机通过 RS485 通信设定闹钟(指令自行设计和识别, 如 setAlarm 12:12:00), 时间逻辑部分可使用第五章数码管实验中的时钟, 到达闹钟时间时, 蜂鸣器响起。

扩展任务：

1. 闹钟类型：通过设计不同指令，为闹钟设计“每天”、“只响一次”等类型，为响铃声音设计“急鸣”、“长鸣”等类型。（有能力的同学可使用 RTC 设备获取固定日期和时间，开发板 RTC 型号为 sd2068，操作方式为 RTC，设备地址为 0x32）

2. 闹钟的关闭：简单学习第 18 章键盘内容，了解按键的检测方式，闹钟响起后，如果按下特定按键，则关闭闹钟，否则响铃 3 分钟后自动关闭，并且在闹钟设定时间后 10 分钟再响。

第11章 陀螺仪实验

11.1 实验目的

了解陀螺仪的工作原理与使用方法。掌握教学实验平台上陀螺仪的功能特性与输出的数据结构，认识其连接方式与控制端口。

根据教学平台上陀螺仪的信息编写程序，实现一个简单的定位功能。使得在摇晃开发板时，程序输出的 XYZ 数据会不断变化。

11.2 STK8BA58 简介

试验箱使用的设备是 STK8BA58，STK8BA58 是一款带数字输出（I2C）的 $\pm 2g/\pm 4g/\pm 8g$ 三轴线性加速度计，是一款电容式 MEMS 传感器，有 0g 偏移和增益误差补偿功能，采样率可配置，可以通过中断引脚改变传感器数值。在本实验中仅使用其输出的 XYZ 数据。

STK8BA58 使用 I2C 通信，与实验三不同的是，在本实验中我们将 STK8BA58 作为读写设备，在代码部分将会补充 I2C 的通用读取数据代码。

STK8BA58 支持两种类型的中断：新数据中断和斜率变化中断，在本实验中将会对中断线的注册、中断处理函数的声明进行介绍，选择何种中断实现何种目的可以自行查阅 STK8BA58 芯片手册进行编写。

11.3 实验步骤

11.3.1 I2C 通用操作函数

I2C 设备的通用写函数已经在实验三中给出，本节将给出 I2C 设备的通用读函数，并会在本节末给出所有 I2C 设备读写函数的声明。

首先，因为涉及从 SDA 线读取来自从设备的数据，需要编写配置 SDA 端口为输入模式/输出模式的方法和从 SDA 线读取电平数据的函数：

```
static void SDA_D_OUT(void) {
    gpio_direction_output(my_dev.sda_gpio, 1);
}

static void SDA_D_IN(void) {
    gpio_direction_input(my_dev.sda_gpio);
}

static int SDA_READ(void)
{
    return gpio_get_value(my_dev.sda_gpio);
}
```

在主机接收数据后，要向从机发送响应位，响应位包括 ACK 和 NACK 两种。

```

void I2c_Ack( void ) {
    SCL_CLR();
    SDA_D_OUT();
    SDA_CLR();
    udelay(DELAY_TIME);
    SCL_SET();
    udelay(DELAY_TIME);
    SCL_CLR();
}

void I2c_NAck( void ) {
    SCL_CLR();
    SDA_D_OUT();
    SDA_SET();
    udelay(DELAY_TIME);
    SCL_SET();
    udelay(DELAY_TIME);
    SCL_CLR();
}

```

最后编写读取字节函数，因为响应位是 ACK 或 NACK 在不同芯片手册的通信协议中约定不同，因此要通过传入的 ack 来选择响应方式，在例程中，传入非 0 值则发送 ACK，传入 0 则发送 NACK。

```

unsigned char I2c_RdByte( int ack ) {
    unsigned char i, ret = 0;
    SDA_D_IN();
    for (i = 0; i < 8; i++) {
        SCL_CLR();
        udelay(DELAY_TIME);
        SCL_SET();
        ret <<= 1;
        if (SDA_READ()) ret++;
        udelay(DELAY_TIME);
    }
    if (ack) I2c_Ack();
    else I2c_NAck();
    return ret;
}

```

至此，I2C 通用读写函数便介绍完毕，其中 SDA_D_OUT，SDA_D_IN 用来控制 SDA 引脚方向，SDA_READ 用来获取 SDA 引脚状态，SDA_SET，SDA_CLR，SCL_SET，SCL_CLR 用来控制两条线，I2c_Ack 和 I2c_NAck 用于在读取 1 字节后发送响应位，I2c_WrByte，I2c_RdByte 则是最顶层的读写 1 字节函数，如果想要更高的

效率，可将除 I2c_前缀之外的底层函数改为宏定义，或者使用 inline 修饰符。

11.3.2 内核模块代码编写

1) 依赖头文件

首先导入内核模块文件依赖的头文件：

```
#include <linux/types.h>
#include <linux/kernel.h>
#include <linux/delay.h>
#include <linux/ide.h>
#include <linux/init.h>
#include <linux/module.h>
#include <linux/errno.h>
#include <linux/gpio.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/of.h>
#include <linux/of_address.h>
#include <linux/of_gpio.h>
#include <linux/platform_device.h>
#include <linux/miscdevice.h>
#include <linux/i2c.h>
#include <asm/uaccess.h>
#include <asm/io.h>
#include "stk8ba.h"
```

stk8ba.h 文件中包含芯片的寄存器地址和对应的 GPIO 端口声明：

```
#define STK8BA_CHIP_ID 0x00          /* 芯片 ID 寄存器    */
#define STK8BA_XOUT1  0x02
#define STK8BA_XOUT2  0x03
#define STK8BA_YOUT1  0x04
#define STK8BA_YOUT2  0x05
#define STK8BA_ZOUT1  0x06
#define STK8BA_ZOUT2  0x07
#define STK8BA_INTMAP1    0x19
#define STK8BA_INTMAP2    0x1a
#define STK8BA_INTSTS1    0x09
#define STK8BA_INTSTS2    0x0A
#define STK8BA_EVENTINFO1 0x0B
#define STK8BA_DATASETUP 0x13
```

```
#define STK8BA_SWRST 0x14
#define STK8BA_INTEN1 0x16
#define STK8BA_INTEN2 0x17
#define STK8BA_INTCFG1 0x20
#define STK8BA_INTCFG2 0x21
#define STK8BA_INTFCFG 0x34
#define STK8BA_OFSTX 0x38
#define STK8BA_OFSTY 0x39
#define STK8BA_OFSTZ 0x3A
#define SDA_GPIO 68
#define SCL_GPIO 69
#define INT_GPIO 100
```

2) STK8BA58 寄存器读写函数

不同于 CH422 第一个发送的字节有非常多的选择，常见的 I2C 设备需要在第一个字节指定设备地址（7 位）和读写方式（1 位），I2C 设备的设备地址由厂商设定，一般可以在数据手册中查找到，STK8BA58 的设备地址为 0x18：

设备地址手册中一般会给出低 7 位的表示，但在实际传输的第一个字节是使用高 7 位表示设备地址，最低位置 0 表示写设备，置 1 表示读设备，因此可以编写声明如下：

```
#define DEVICE_ADDR 0x18
#define DEVICE_WRITE_ADDR((DEVICE_ADDR << 1) | 0)
#define DEVICE_READ_ADDR ((DEVICE_ADDR << 1) | 1)
```

这样在编写不同设备驱动时仅需修改设备地址即可。

不同的数据手册会用不同的方式介绍通信协议，如 STK8BA58 数据手册中则使用图片方式进行介绍，如图 11-2：

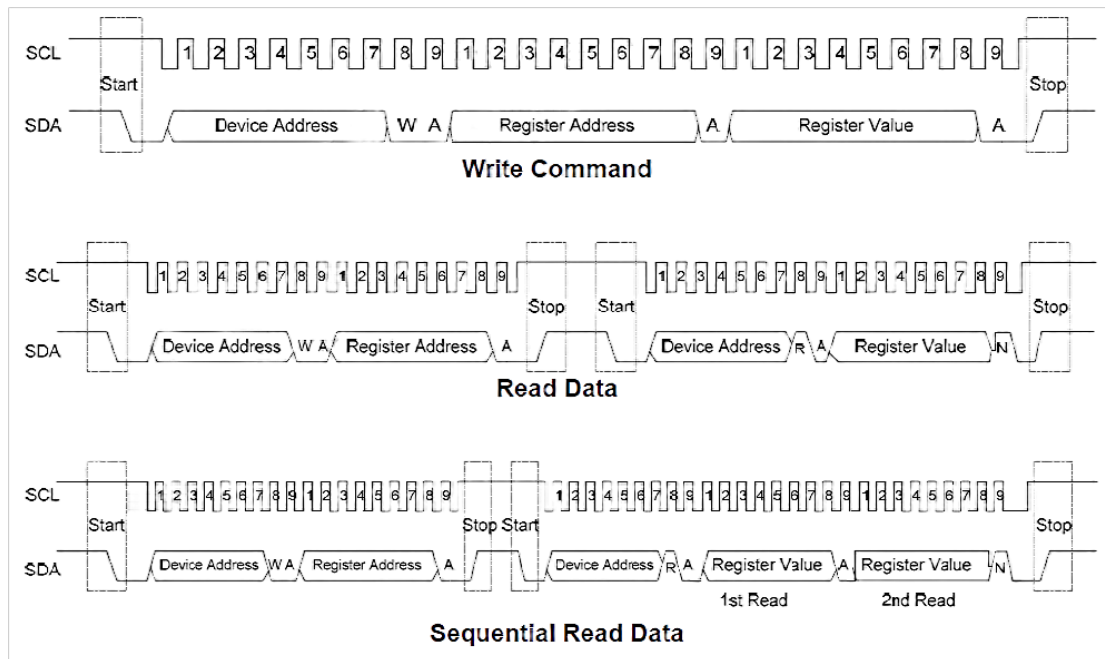


图 11-2 不同通信模式

可以看到，在写设备时，先发送开始信号，字节 1 发送设备地址+写位，字节 2 发送欲写入的寄存器地址，字节 3 发送该寄存器的新值，再发送停止信号即完成对指定寄存器的写入。

而读寄存器有两种方式，但都需要先在写模式下指定寄存器地址，如读取单个寄存器时，先发送开始信号，字节 1 发送设备地址+写位，字节 2 发送欲读取的寄存器地址，然后发送结束信号，紧接着再发送开始信号，字节 1 发送设备地址+读位，然后接收 8 位数据并响应 NACK，完成 1 字节的读取。同理第三张图是对同一寄存器进行连续读取，期间发送 ACK 响应，并在结束时发送 NACK 响应。

由手册内容，可编写读写寄存器函数如下：

```
static unsigned char STK8BA_Read_Reg(unsigned char reg) {
    unsigned char val;
    I2c_Start();
    I2c_WrByte(my_dev.write_addr);
    I2c_WrByte(reg);
    I2c_Stop();

    I2c_Start();
    I2c_WrByte(my_dev.read_addr);
    val = I2c_RdByte(NACK);
    I2c_Stop();
    return val;
}
```

```

void STK8BA_Write_Reg( unsigned char reg_addr, unsigned char new_value ) {
    I2c_Start();    // 启动总线
    I2c_WrByte(my_dev.write_addr);
    I2c_WrByte(reg_addr);
    I2c_WrByte(new_value);
    I2c_Stop();    // 结束总线
}

```

可以选择将设备的读写命令写进设备的私有数据,这里命名为了 write_addr 和 read_addr, 可以看到在使用读写字节函数进行抽象后, STK8BA58 的读写寄存器函数和手册中完全一致, 响应位则可以自行编写宏声明:

```

#define ACK    1
#define NACK   0

```

3) 设备操作函数和设备结构体

首先编写设备的私有数据结构体, 其中保存了 SDA、SCL 端口号和中断线, 以及读写地址, 具体值会在模块初始化中动态赋值。

```

struct stk8ba_dev {
    dev_t devid;          /* 设备号      */
    void *private_data;    /* 私有数据 */
    int sda_gpio;
    int scl_gpio;
    int irq_num;
    unsigned char write_addr;
    unsigned char read_addr;
};

struct stk8ba_dev my_dev;

```

编写 open 函数, 仅需简单地设置私有数据。

```

static int stk8ba_open(struct inode *inode, struct file *filp) {
    filp->private_data = &my_dev; /* 设置私有数据 */
    return 0;
}

```

编写 read 函数, 在这里直接将 xyz 对应的 6 个寄存器数据进行返回, 但直接输出是难以看出实际数据的, 需要用户自行转换, 将在测试程序编写指南中介绍。

```

static void stk8ba_read_xyz(unsigned char *data) {
    data[0] = STK8BA_Read_Reg(STK8BA_XOUT1);
    data[1] = STK8BA_Read_Reg(STK8BA_XOUT2);
    data[2] = STK8BA_Read_Reg(STK8BA_YOUT1);
    data[3] = STK8BA_Read_Reg(STK8BA_YOUT2);
    data[4] = STK8BA_Read_Reg(STK8BA_ZOUT1);
    data[5] = STK8BA_Read_Reg(STK8BA_ZOUT2);
}

static ssize_t stk8ba_read(struct file *filp, char __user *buf, size_t cnt, loff_t *off)
{
    unsigned char data[6];
    struct stk8ba_dev *dev = (struct stk8ba_dev *)filp->private_data;
    stk8ba_read_xyz(data);
    return copy_to_user(buf, data, cnt);
}

```

编写 write 函数，这里只是为了介绍写入函数的用法，但实际使用时不建议这样编写，让用户空间可以随意写入寄存器是非常危险的，如果仅为了控制设备模式，推荐覆写 ioctl 方法。

```

static ssize_t stk8ba_write(struct file *filp, const char __user *buf, size_t cnt, loff_t *offt) {
    int ret;
    unsigned char databuf[2];
    struct stk8ba_dev *dev = filp->private_data;
    ret = copy_from_user(databuf, buf, cnt);
    if(ret < 0) {
        pr_info("kernel write failed!\r\n");
        return -EFAULT;
    }
    STK8BA_Write_Reg(databuf[0], databuf[1]);
    return 0;
}

```

编写 release 函数。

```

static int stk8ba_release(struct inode *inode, struct file *filp) {
    return 0;
}

```

配置设备操作函数和 misc 设备结构体，其中 DEVICE_NAME 和用户空间 open 时对应的设备名相同，子设备号则可以自行查找 misc 设备（主设备号 10）对应的已使用子设备号，避开已使用的设备号即可。

```

#define DEVICE_NAME      "stk8ba" /* 名字 */
#define DEVICE_MINOR     148      /* 子设备号 */
/* 设备操作函数 */
static struct file_operations stk8ba_ops = {
    .owner = THIS_MODULE,
    .open = stk8ba_open,
    .read = stk8ba_read,
    .write = stk8ba_write,
    .release = stk8ba_release,
};
/* MISC 设备结构体 */
static struct miscdevice stk8ba_miscdev = {
    .minor = DEVICE_MINOR,
    .name = DEVICE_NAME,
    .fops = &stk8ba_ops,
};

```

4) 模块注册和设备中断

首先进行 misc 设备的注册，然后配置 GPIO 口的复用配置寄存器，将所有使用到的端口配置为 GPIO 模式。

```

static int __init stk8ba_init(void)
{
    int ret;
    u32 GPCFG1_read, GPCFG1_write;
    u32 GPCFG2_read, GPCFG2_write;
    ret = misc_register(&stk8ba_miscdev);
    if(ret < 0){
        printk("misc device register failed!\r\n");
        return -EFAULT;
    }

    GPCFG1_V = ioremap(GPCFG1, sizeof(u32));
    GPCFG1_read = ioread32(GPCFG1_V);
    GPCFG1_write = (GPCFG1_read & 0x77707777);
    iowrite32(GPCFG1_write, GPCFG1_V);
    GPCFG2_V = ioremap(GPCFG2, sizeof(u32));
    GPCFG2_read = ioread32(GPCFG2_V);
    GPCFG2_write = (GPCFG2_read & 0x77007777);
    iowrite32(GPCFG2_write, GPCFG2_V);
}

```

这里配置了 3 个 GPIO 端口，查看开发板设计图 11-3 可以看到，芯片使用了三条线路，SDA 和 SCL 查阅手册可以找到其对应 GPIO68 和 69，而 INT 则对应 GPIO100。

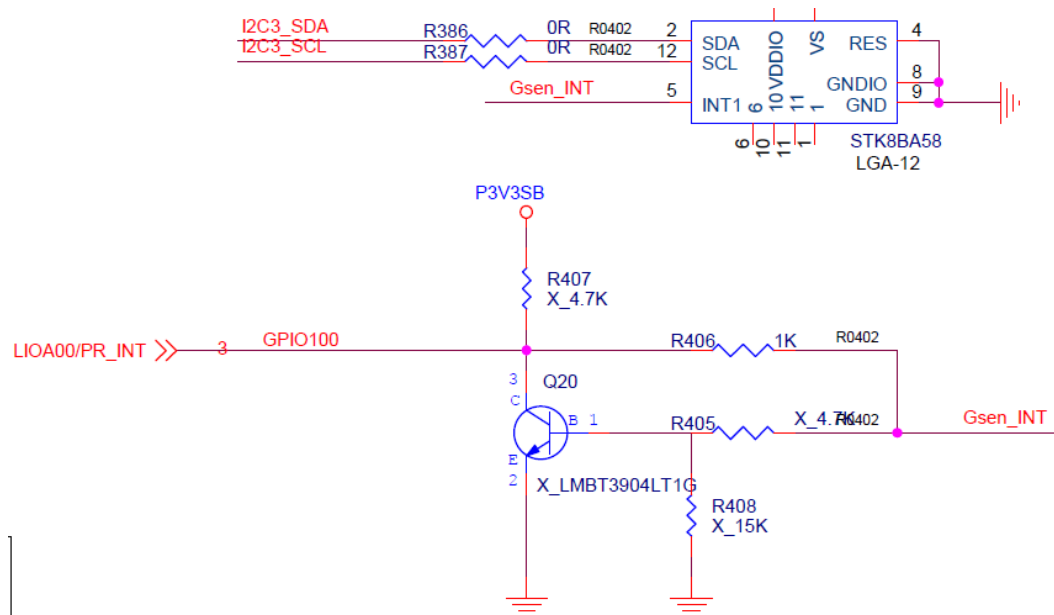


图 11-3 GPIO 口

申请 GPIO 口，并配置私有数据对象，注意中断号需要使用 gpio_to_irq 方法将 GPIO 端口号转换为中断号。

```
gpio_request(SDA_GPIO, "stk8ba_sda");
gpio_request(SCL_GPIO, "stk8ba_scl");
gpio_request(INT_GPIO, "stk8ba_int");

my_dev.sda_gpio = SDA_GPIO;
my_dev.scl_gpio = SCL_GPIO;
my_dev.irq_num = gpio_to_irq(INT_GPIO);
my_dev.write_addr = DEVICE_WRITE_ADDR;
my_dev.read_addr = DEVICE_READ_ADDR;
pr_info("stk8ba_init gpio int irq num:%d\n", my_dev.irq_num);
```

将 SDA 和 SCL 配置为输出模式。

```
ret = gpio_direction_output(my_dev.sda_gpio, 1);
if(ret < 0) {
    printk("can't set sda gpio!\r\n");
}
ret = gpio_direction_output(my_dev.scl_gpio, 1);
if(ret < 0) {
    printk("can't set scl gpio!\r\n");
}
```

配置 STK8BA58 的中断映射和使能寄存器，注意，这里将所有方式（有效运动中断、任意运动中断、新数据中断）中断均映射到中断线，且开启了 3 个轴和新数据的中断使能，用于参考，具体可参考数据手册进行选择使用。

然后申请中断号，这里会指定中断处理函数 `read_handler`，这里函数名可自己修改，但函数的返回值和参数固定，可参考例程。

```
    STK8BA_Write_Reg(STK8BA_INTMAP1, 0x5);
    STK8BA_Write_Reg(STK8BA_INTMAP2, 0x1);
    STK8BA_Write_Reg(STK8BA_INTEN1, 0x7);
    STK8BA_Write_Reg(STK8BA_INTEN2, 0x10);

    ret = request_irq(my_dev.irq_num, read_handler,
                     IRQF_TRIGGER_FALLING|IRQF_TRIGGER_RISING, "stk8ba-irq", &my_dev);
    return ret;
}
```

中断处理函数的返回值为 `irqreturn_t`，接收 `int irq`，`void *dev_id` 作为参数，如果使用 STK8BA58 的新数据中断，因为该中断触发非常频繁，最短 250us 即可触发一次，请谨慎使用 `pr_info()` 等向用户空间打印的 debug 方法。

```
static irqreturn_t read_handler(int irq, void *dev_id){
    //struct stk8ba_dev *dev = (struct stk8ba_dev *)dev_id;
    return IRQ_RETVAL(IRQ_HANDLED);
}
```

最后编写模块卸载函数，卸载 `misc` 设备并撤销所有映射的地址和申请的资源。

```
static void __exit stk8ba_exit(void) {
    iounmap(GPCFG1_V);
    iounmap(GPCFG2_V);
    misc_deregister(&stk8ba_miscdev);
    gpio_free(my_dev.sda_gpio);
    gpio_free(my_dev.scl_gpio);
    free_irq(my_dev.irq_num, &my_dev);
}

module_init(stk8ba_init);
module_exit(stk8ba_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR("yangjinrun");
```

11.3.3 测试代码编写指南

在 STK8BA58 数据手册中可以查询到 XYZ 轴对应寄存器的保存数据，查询手册图 11-4 可知，重力数据以 12 位补码数据保存，NEW_位置 1 时说明读取到新数据。

02h	XOUT1	XOUT[3:0]	reserved	NEW_X	00h
03h	XOUT2	XOUT[11:4]			00h
04h	YOUT1	YOUT[3:0]	reserved	NEW_Y	00h
05h	YOUT2	YOUT[11:4]			00h
06h	ZOUT1	ZOUT[3:0]	reserved	NEW_Z	00h
07h	ZOUT2	ZOUT[11:4]			00h

图 11-4 XYZ 轴对应寄存器

因此解码该寄存器数据时，需要使用至少 2 字节大小的整型类型变量。

请注意，设备使用的是 12 位补码，如果使用 short 保存数据，那么如果数值为负数，short 的高 4 位仍然为 0，得到的数据还是正数，显然这不是我们想要的结果。将补码转为原码再转为补码是较为直观的解决方法，但较为复杂。另一种巧妙的方法是当补码为负数时，将高位额外位全部置 1，如使用 short 接收则是将高 4 位置 1，感兴趣可以自行验证其正确性。

以下给出了解码 12 位补码的例程，并给出了简单的使用方法，主函数简单地读取寄存器数据并输出：

```
short axis_out_encode(unsigned char out1, unsigned char out2) {
    short val;
    val = (out1 >> 4) | (out2 << 4);
    if (val & (1 << 11)) {
        val |= 0xF000;
    }
    return val;
}
```

```
int main(int argc, char *argv[])
{
    int fd;
    char *filename = "/dev/stk8ba";
    unsigned char databuf[6];
    int ret = 0;
    unsigned char xout1,xout2,yout1,yout2,zout1,zout2;
    short x, y, z;
    int t = 20;
    fd = open(filename, O_RDWR);
    if(fd < 0) {
        printf("can't open file %s\r\n", filename);
        return -1;
    }
}
```

```

while (t--) {
    ret = read(fd, databuf, sizeof(databuf));
    if(ret == 0) { /* 数据读取成功 */
        xout1 = databuf[0];
        xout2 = databuf[1];
        yout1 = databuf[2];
        yout2 = databuf[3];
        zout1 = databuf[4];
        zout2 = databuf[5];
        x = axis_out_encode(xout1, xout2);
        y = axis_out_encode(yout1, yout2);
        z = axis_out_encode(zout1, zout2);
    }
    printf("x:%d, y:%d, z:%d\n", x, y, z);
    usleep(500000); /* 500ms */
}
close(fd); /* 关闭文件 */
return 0;
}

```

11.4 练习：陀螺仪姿态控制和判定

本实验通过转动试验箱来获取不同的重力数据，进而理解陀螺仪的工作原理。由于陀螺仪固定在试验箱内，学生需要转动整个箱子来使数据发生变化，掌握如何读取和处理这些变化的数据。

任务要求：

方向控制：基于陀螺仪数据，模拟控制试验箱内的设备。例如当倾斜到一定角度时，控制 LED 灯点亮或熄灭。

扩展任务：

姿态判定与报警：设定一定的倾斜角度阈值，当箱子旋转角度超过阈值时，蜂鸣器响起，提示可能出现的危险姿态。

第12章 CAN 总线通信实验

12.1 实验目的

了解 CAN 总线的工作原理及使用方法。掌握教学平台上 CAN 总线接口位置，认识其可行的连接方式。

根据实际编写程序，实现两块开发板间的 CAN 总线通信，或者也可以运用 CAN 总线通信控制外设。

12.2 CAN 总线介绍

控制器局域网（Controller Area Network, CAN）是一种串行通信协议，最初由德国公司 Bosch 在 1986 年开发，旨在满足汽车电子系统之间的高效通信需求。CAN 总线使用差分信号传输数据，即两个互补的信号线（CAN_H 和 CAN_L）。这种差分传输减少了对电磁干扰的敏感性，提高了通信的稳定性。

SocketCAN 是 Linux 内核中的一个子系统，用于支持 CAN（Controller Area Network）总线的通信。它提供了一套用于用户空间应用程序与 CAN 总线进行通信的 API（应用程序接口），通过这个 API，开发者可以方便地进行 CAN 总线的配置、数据传输和控制。

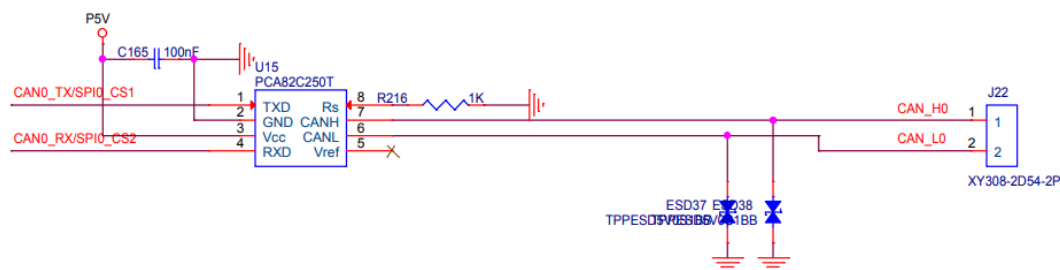


图 12-1 CAN 接口原理图

如图 12-1，龙芯 2K0500 集成了四路 CAN 接口控制器。CAN 总线是由发送数据线 TX 和接收数据线 RX 构成的串行总线，可发送和接收数据。器件与器件之间进行双向传送，最高传送速率 1Mbps。本实验中使用 SocketCAN 进行 CAN 总线通讯，实现一个服务端通过 CAN 总线发送消息，一个客户端通过 CAN 总线接收消息。

12.3 实验代码编写

12.3.1 服务端

1) 引入必要头文件

这些头文件包含了与 SocketCAN 相关的定义和函数声明，以及一些常用的系统调用。

```
#include <linux/can.h>
#include <linux/can/raw.h>
#include <net/if.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ioctl.h>
#include <sys/socket.h>
#include <unistd.h>
```

2) 声明主函数以及创建 Socket

声明 socketfd 文件描述符索引以及必要变量，并创建 CAN 协议的 socket 将其赋值给 socketfd。struct ifreq ifr 是用于设置 CAN Socket 接口的相关信息。具体而言，这里的目的是设置 CAN Socket 的接口名字（can0），并使用 ioctl 获取相关的接口索引。这样就能够将 Socket 绑定到指定的 CAN 接口上，从而进行 CAN 通信。之后定义 CAN 数据帧 can_frame

```
int main() {
    printf("SocketCAN send demo made by Shandong university");
    int ret;
    int sockfd, nbytes;
    struct sockaddr_can addr;
    struct ifreq ifr;
    struct can_frame frame;
    sockfd = socket(PF_CAN, SOCK_RAW, CAN_RAW);
```

3) 绑定 socket 到接口

将 can0 这个 CAN 接口名赋值给 ifr 中的属性，使用 ioctl 获取 can0 的接口索引，addr.can_family = AF_CAN 设置 CAN 地址族，AF_CAN 是一个宏，用于表示 Socket 的地址族，表明这是一个用于 CAN 通信的 Socket。之后设置之前获取到的 can 接口索引，并完成 socket 绑定

```
strcpy(ifr.ifr_name, "can0");
ioctl(sockfd, SIOCGIFINDEX, &ifr);
addr.can_family = AF_CAN;
addr.can_ifindex = ifr.ifr_ifindex;
ret = bind(sockfd, (struct sockaddr *)&addr, sizeof(addr));
```

4) 设置 socket 选项

程序设置了一些 Socket 选项。CAN_RAW_LOOPBACK 选项被设置为关闭回环模式，CAN_RAW_FILTER 选项未设置过滤器，即接收所有 CAN 消息。

```
int loopback = 0; //0 表示关闭, 1 表示开启(默认)
setsockopt(sockfd, SOL_CAN_RAW, CAN_RAW_LOOPBACK, &loopback, sizeof(loopback));
setsockopt(sockfd, SOL_CAN_RAW, CAN_RAW_FILTER, NULL, 0);
```

5) 准备 CAN 数据帧

设置了帧的 ID、数据长度和数据内容。

```
frame.can_id = 0x11;
frame.can_dlc = 2;
frame.data[0] = 'Y';
frame.data[1] = 'N';
```

6) 循环发送 CAN 消息

每发送一条 CAN 消息休眠一秒并打印日志

```
while (1) {
    nbytes = write(sockfd, &frame, sizeof(frame));
    printf("nbytes=%d\n", nbytes);
    if (nbytes != sizeof(frame)) {
        printf("Send Error frame write ret:%d\n!",nbytes);
        break; //发送错误,退出
    }
    sleep(1);
}
close(sockfd);
return 0;
}
```

12.3.2 客户端

1) 引入必要头文件

这些头文件包含了与 SocketCAN 相关的定义和函数声明,以及一些常用的系统调用。

```
#include <linux/can.h>
#include <linux/can/raw.h>
#include <net/if.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ioctl.h>
#include <sys/socket.h>
#include <unistd.h>
```

2) 声明主函数以及创建 Socket

声明 sockfd 文件描述符索引以及必要变量, 并创建 CAN 协议的 socket 将其赋值给 sockfd。struct ifreq ifr 是用于设置 CAN Socket 接口的相关信息。具体而言, 这里的目的是设置 CAN Socket 的接口名字 (can0), 并使用 ioctl 获取相关的接口索引。这样就能够将 Socket 绑定到指定的 CAN 接口上, 从而进行 CAN 通信。之后定义 CAN 数据帧 can_frame

```
int main()
{
    printf("SocketCAN receive demo made by Shandong university");
    int ret;
    int sockfd, nbytes;
    struct sockaddr_can addr;
    struct ifreq ifr;
    struct can_frame frame;
    struct can_filter rfilter;
    sockfd = socket(PF_CAN, SOCK_RAW, CAN_RAW); //创建套接字
```

3) 绑定 socket 到接口

将 can0 这个 CAN 接口名赋值给 ifr 中的属性, 使用 ioctl 获取 can0 的接口索引, addr.can_family = AF_CAN 设置 CAN 地址族, AF_CAN 是一个宏, 用于表示 Socket 的地址族, 表明这是一个用于 CAN 通信的 Socket。之后设置之前获取到的 can 接口索引, 并完成 socket 绑定

```
strcpy(ifr.ifr_name, "can0");
ioctl(sockfd, SIOCGIFINDEX, &ifr); //指定 can0 设备
addr.can_family = AF_CAN;
addr.can_ifindex = ifr.ifr_ifindex;
ret = bind(sockfd, (struct sockaddr *)&addr, sizeof(addr)); //将套接字与 can0 绑定
```

4) 设置 socket 选项 (与服务端有所区分)

程序设置了一些 Socket 选项。CAN_RAW_LOOPBACK 选项被设置为关闭回环模式。除此以外, 设置 CAN Socket 支持 CAN FD (Flexible Data-rate) 帧。CAN FD 是一种扩展了标准 CAN 帧的协议, 提供了更高的数据传输速率和更大的数据帧容量。

```
int loopback = 0; //0 表示关闭, 1 表示开启(默认)
setsockopt(sockfd, SOL_CAN_RAW, CAN_RAW_LOOPBACK, &loopback, sizeof(loopback));
//设置过滤规则
int canfd_on = 1;
setsockopt(sockfd, SOL_CAN_RAW, CAN_RAW_FD_FRAMES,
           &canfd_on, sizeof(canfd_on));
```

5) 循环接收 CAN 消息

每接收到对应消息便打印日志，输出数据帧中的 CAN_ID，长度以及 data 中首个字节并以十六进制显示。

```
while (1) {
    nbytes = read(sockfd, &frame, sizeof(frame));
    //接收报文//显示报文
    if (nbytes > 0) {
        printf("ID=0x%X DLC=%d data[0]=0x%X\n", frame.can_id,
               frame.can_dlc, frame.data[0]);
    }
}
close(sockfd);
return 0;
}
```

12.4 练习：CAN 总线基本实验

上述代码需要至少两个开发板，通过杜邦线连接开发板上的 CAN 总线接口，其中一个开发板执行服务端程序，其余执行客户端程序，服务端可以通过 CAN 总线控制其他开发板。

任务要求：

1. 硬件连接：使用 CAN 线缆将两个试验箱的 CAN 总线接口连接在一起，确保物理连接正确无误。
2. 基础通信：一个试验箱发送字符串/数据，另一个试验箱接收并显示到终端。

扩展任务：

设备控制：约定控制指令，服务端发送控制命令，客户端接收、解析并执行，如控制指定 LED 的亮灭。

第13章 音频实验

13.1 实验目的

了解实验平台上如何设置音频输入输出。掌握教学平台上拾音器与耳机接口的连接方式与控制端口。

根据教学平台上设备配置编写程序,实现录音、音频播放、音量调节等功能。

13.2 实验步骤

13.2.1 拾音器与耳机接口

如图 13-1 所示,在主板左下角靠近开发板电源接口的位置,用红圈画出的部分就是拾音器,其使用四段式接口,需要准备一个四段式的带麦耳机。

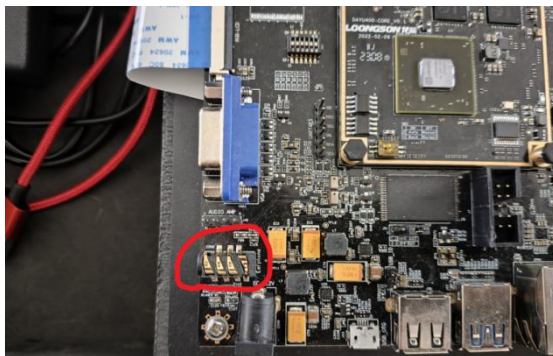


图 13-1 耳机接口

四段式接口和插入效果如图 13-2 所示。

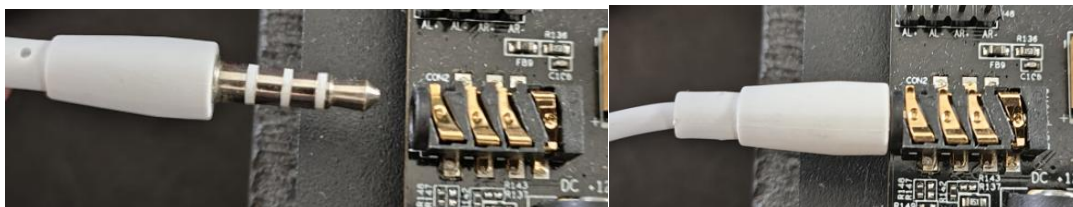


图 13-2 耳机插入效果图

13.2.2 系统声卡配置

接下来配置系统声卡,首先在 root 目录下创建 asound.state 文件,如图 13-3。

```
[root@LS-GD ~]# vim asound.state
```

图 13-3 创建 asound.state

在文件中配置如下内容:

```
state.Loongson {
  control.1 {
    iface MIXER
    name 'Master Playback Volume'
    value.0 87
    value.1 87
    comment {
      access 'read write'
      type INTEGER
      count 2
      range '0 - 87'
      dbmin -6525
      dbmax 0
      dbvalue.0 0
      dbvalue.1 0
    }
  }
  control.2 {
    iface MIXER
    name 'Master Playback Switch'
    value.0 true
    value.1 true
    comment {
      access 'read write'
      type BOOLEAN
      count 2
    }
  }
  control.3 {
    iface MIXER
    name 'Loopback Mixing'
    value Disabled
    comment {
      access 'read write'
      type ENUMERATED
      count 1
      item.0 Disabled
      item.1 Enabled
    }
  }
}
```

```

control.4 {
    iface MIXER
    name 'Mic Playback Volume'
    value.0 31
    value.1 31
    comment {
        access 'read write'
        type INTEGER
        count 2
        range '0 - 31'
        dbmin -3450
        dbmax 1200
        dbvalue.0 1200
        dbvalue.1 1200
    }
}

control.5 {
    iface MIXER
    name 'Mic Playback Switch'
    value.0 false
    value.1 false
    comment {
        access 'read write'
        type BOOLEAN
        count 2
    }
}

control.6 {
    iface MIXER
    name 'Capture Volume'
    value.0 63
    value.1 63
    comment {
        access 'read write'
        type INTEGER
        count 2
        range '0 - 63'
        dbmin -1725
        dbmax 3000
        dbvalue.0 3000
        dbvalue.1 3000
    }
}

```

```

control.7 {
    iface MIXER
    name 'Capture Switch'
    value.0 true
    value.1 true
    comment {
        access 'read write'
        type BOOLEAN
        count 2
    }
}

control.8 {
    iface MIXER
    name 'Mic Boost Volume'
    value.0 3
    value.1 3
    comment {
        access 'read write'
        type INTEGER
        count 2
        range '0 - 3'
        dbmin 0
        dbmax 3000
        dbvalue.0 3000
        dbvalue.1 3000
    }
}

control.9 {
    iface MIXER
    name 'Beep Playback Volume'
    value.0 31
    value.1 31
    comment {
        access 'read write'
        type INTEGER
        count 2
        range '0 - 31'
        dbmin -3450
        dbmax 1200
        dbvalue.0 1200
        dbvalue.1 1200
    }
}

```

```
control.10 {
    iface MIXER
    name 'Beep Playback Switch'
    value.0 false
    value.1 false
    comment {
        access 'read write'
        type BOOLEAN
        count 2
    }
}

control.11 {
    iface CARD
    name 'Mic Jack'
    value false
    comment {
        access read
        type BOOLEAN
        count 1
    }
}

control.12 {
    iface CARD
    name 'Headphone Jack'
    value false
    comment {
        access read
        type BOOLEAN
        count 1
    }
}

control.13 {
    iface PCM
    name 'Playback Channel Map'
    value.0 0
    value.1 0
    comment {
        access read
        type INTEGER
        count 2
        range '0 - 36'
    }
}
```

```

control.14 {
    iface PCM
    name 'Capture Channel Map'
    value.0 0
    value.1 0
    comment {
        access read
        type INTEGER
        count 2
        range '0 - 36'
    }
}
}

```

然后执行命令 `alsactl -f ./asound.state restore`，声卡配置便完成，这一步每次重启系统都需要重新执行，可以选择编写脚本快速完成配置或让系统启动时自动配置声卡，如图 13-4。

```
[root@LS-GD ~]# alsactl -f ./asound.state restore
```

图 13-4 配置声卡

13.2.3 播放和录音测试

将任意音频文件拷贝到试验箱中，使用 `aplay` [音频文件路径]播放音频文件，成功在耳机中听到音频文件的声音说明配置成功，如图 13-5。

```
[root@LS-GD ~]# aplay music.wav
Playing WAVE 'music.wav' : Unsigned 8 bit, Rate 22050 Hz, Mono
```

图 13-5 播放音乐

如图 13-6，使用 `arecord` 命令进行音频的录制，以下是一个使用例，录制一个双声道的 16 位 16KHz 持续 5 秒钟的 wav 格式音频文件，命令为：

```
arecord --file-type=wav --channel=2 --format=S16_LE --rate=16000 -
-duration=5 /tmp/record.wav
```

```
[root@LS-GD ~]#
--duration=5 /tmp/record.wave-type=wav --channel=2 --format=S16 LE --rate=1600
Recording WAVE '/tmp/record.wav' : Signed 16 bit Little Endian, Rate 16000 Hz, o
```

图 13-6 录制音频

使用 `aplay` [录制文件路径]播放录制的音频，成功听到录音内容说明麦克风正常使用，如图 13-7。

```
[root@LS-GD ~]# aplay /tmp/record.wav
Playing WAVE '/tmp/record.wav' : Signed 16 bit Little Endian, Rate 16000 Hz, Sto
```

图 13-7 录制播放的音乐

13.3 练习：音频设备的简易使用

本实验的主要任务是使用试验箱的音频模块实现简单的音频播放和录音功能。学生将通过系统声卡配置，使用内置的麦克风和耳机接口进行音频的输入和输出，掌握基本的音频处理流程。

任务要求：

1. 音频播放配置系统声卡，使用耳机连接拾音器接口，使用指令播放预设的音频，如拷贝到试验箱中的歌曲文件。
2. 录音功能：配置系统麦克风接口，使用指令进行录音，将录制的音频文件保存到试验箱，回放录制的音频文件，验证录音效果。

第14章 摄像头采集实验

14.1 实验目的

了解实验教学平台上摄像头的处理方法与显示。掌握教学平台上摄像头与通过 ADC 接口连接的 LCD 屏幕的布线与工作原理。

根据教学平台上提供的接口与功能编写程序，实现对摄像头采集画面的实时播放，也可以进一步实现推流功能，将摄像头采集的视频流推送到其他设备。

14.2 实验步骤

14.2.1 实验代码编写

本实验为 QT 项目，编写代码前需要确保虚拟机中有 QT Creator 和试验资料中的 Qt-5.15.2-LA64.tar.gz 的解压后文件夹。

Qt 的安装和小节(1)的配置流程可以参考如下教程进行操作：

https://blog.csdn.net/Chuangke_Andy/article/details/122928669

1) 构建工具配置

首先，可以直接新建 QT 项目也可以打开实验资料中的 QT 项目，build type 选择 qmake，其他均选择默认，如图 14-1。

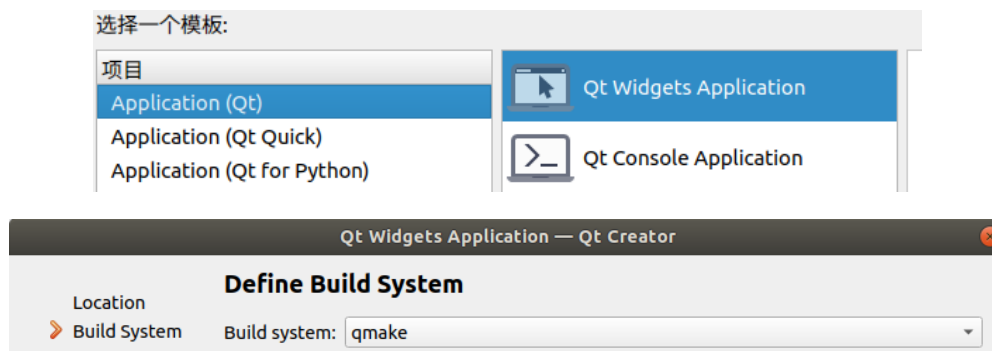


图 14-1 QT 项目配置

在顶端“工具”栏中找到并点击选项，如图 14-2。

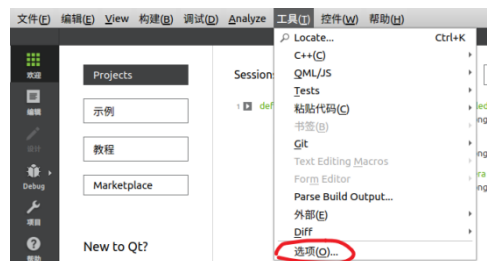


图 14-2 QT 配置

选择 Kits，在 Qt Versions 栏的手动（Manual）中添加新 Qt 版本，如图 14-3。

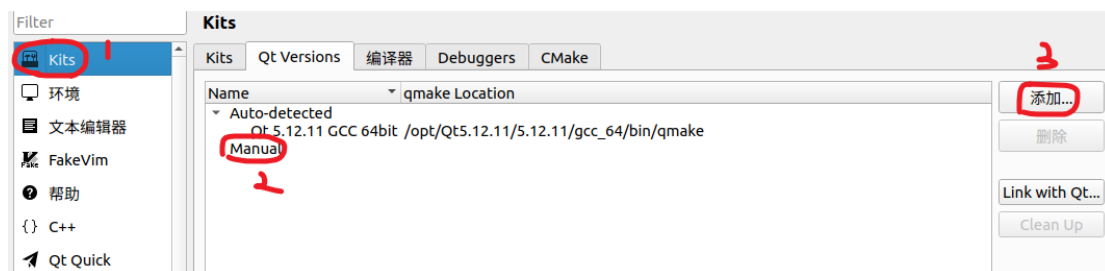


图 14-3 添加新 QT 版本

找到并选择解压得到的 Qt-5.15.2-LA64 目录下的 bin/qmake 文件即可，其他无需改动，如图 14-4。

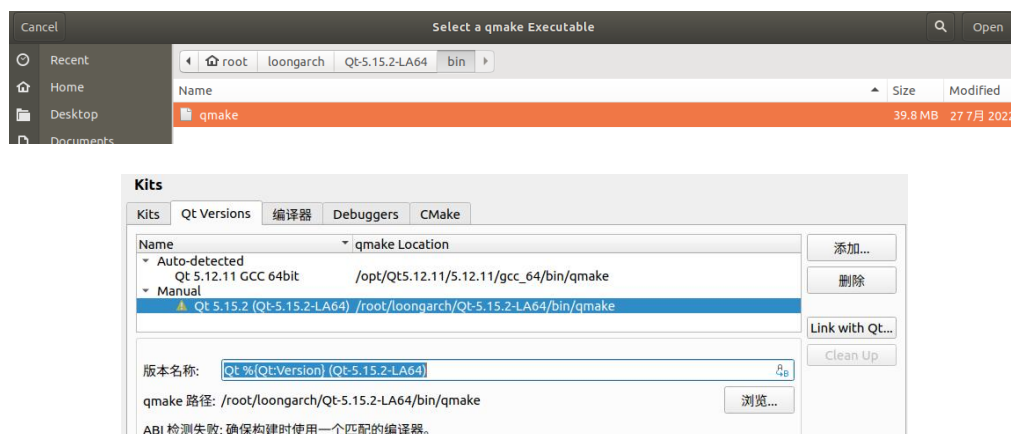


图 14-4 配置 QT

再选择“编译器”栏，将 GCC 配置到手动（Manual）的 C++ 栏中，编译器使用 loongarch64-linux-gnu-g++, 在交叉编译工具包的 bin 目录下，如图 14-5。

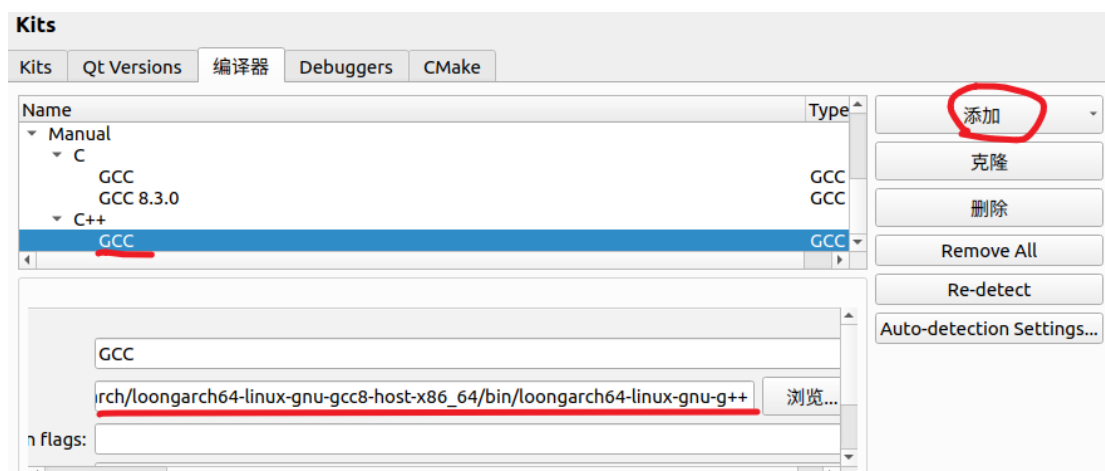


图 14-5 配置交叉编译工具

最后在 Kits 栏中添加新配置（软件可能会自动生成一个配置，没有就手动创建），命名为 LS-Qt 5.15.2，如图 14-6。

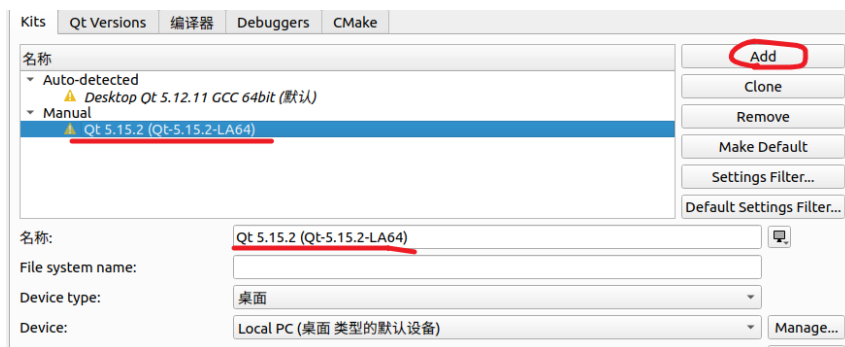


图 14-6 添加配置

修改其 C 编译器为空，C++编译器为刚刚配置的交叉编译器，Qt 版本选择刚配置的版本，保存配置即可，如图 14-7。

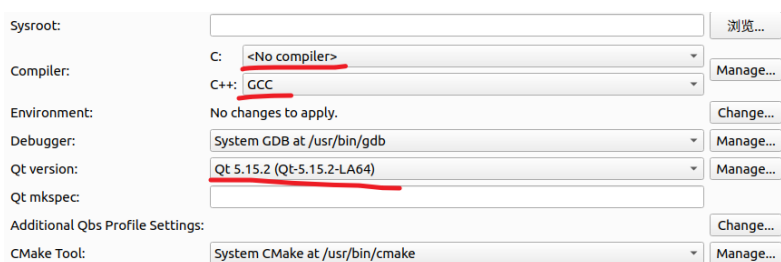


图 14-7 选择 C++编译器

在项目中可以点击添加该配置并且存在构建配置（如图）即为配置成功，如图 14-8。



图 14-8 完成配置

2) 项目的创建和编写

因本项目较复杂，部分代码（如 UI 设计）不会详细分析，建议在 Qt 中打开资料库中的项目作为参考，具体操作为文件-打开文件或项目-选择 项目名.pro 文件（资料中为 v4l2.pro 文件），见图 14-9。

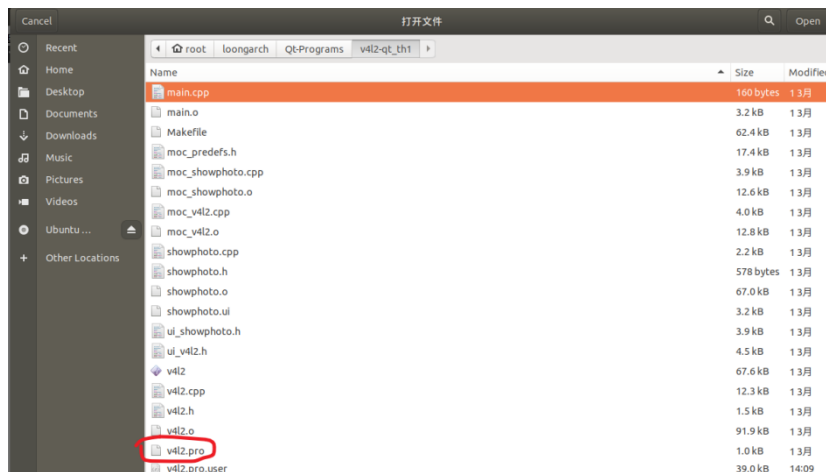


图 14-9 项目文件

打开项目后项目文件如图 14-10:

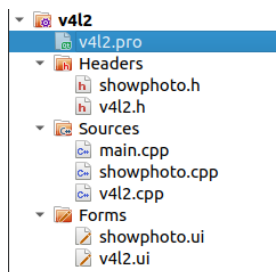


图 14-10 项目文件展示

3) v4l2.pro

该文件是项目的配置文件，用于确定项目使用的库、项目结构等。

```
QT      += core gui
greaterThan(QT_MAJOR_VERSION, 4): QT += widgets
CONFIG += c++11
DEFINES += QT_DEPRECATED_WARNINGS
SOURCES += \
    main.cpp \
    showphoto.cpp \
    v4l2.cpp
HEADERS += \
    showphoto.h \
    v4l2.h
FORMS += \
    showphoto.ui \
    v4l2.ui
# Default rules for deployment.
qnx: target.path = /tmp/${TARGET}/bin
else: unix:!android: target.path = /opt/${TARGET}/bin
!isEmpty(target.path): INSTALLS += target
```

4) Showphoto. h

该文件用于声明展示摄像头的界面类，用于声明界面类中所使用的方法（主要为 showphoto() 方法）和成员变量（主要用于查找摄像头）。

```
#ifndef SHOWPHOTO_H
#define SHOWPHOTO_H
#include <QMainWindow>
#include <QDir>

namespace Ui {
class showphoto;
}

class showphoto : public QMainWindow
{
    Q_OBJECT
public:
    explicit showphoto(QWidget *parent = nullptr);
    ~showphoto();
private slots:
    void on_pushButton_back_clicked();
    void on_pushButton_next_clicked();
    void on_pushButton_front_clicked();
private:
    Ui::showphoto *ui;
    QDir dir; // 目录对象
    QStringList fileList; // 存储文件列表
    int currentIndex = 0; // 当前显示的文件索引
};

#endif // SHOWPHOTO_H
```

5) V4l2. h

该文件声明了摄像头类和缓冲区结构，用于初始化摄像头参数和存储其缓冲区数据，并将数据打印到界面。

```
#ifndef V4L2_H
#define V4L2_H
#include <QMainWindow>
#include <stdio.h>
#include <sys/types.h>
#include <sys/stat.h>
```

```

#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <linux/videodev2.h>
#include <string.h>
#include <sys/mman.h>
#include <signal.h>
#include <poll.h>
#include <linux/fb.h>
#include <stdlib.h>
#include <string.h>
#include <QTimer>
#include <showphoto.h>

#define video_width 640 //800
#define video_height 480 //600

QT_BEGIN_NAMESPACE
namespace Ui { class v4l2; }
QT_END_NAMESPACE

typedef struct Frame_Buffer{
    unsigned char buf[video_width*video_height*3];
    int length;
}FrameBuffer;

class v4l2 : public QMainWindow
{
    Q_OBJECT
public:
    v4l2(QWidget *parent = nullptr);
    ~v4l2();
private slots:
    void on_pushButton_open_clicked();
    void on_pushButton_take_clicked();
    void on_pushButton_photos_clicked();
private:
    Ui::v4l2 *ui;
    char *userbuff[4]; /* 存放摄像头设备内核缓冲区映射后的用户内存地址 */
    int userbuff_length[4]; /* 保存映射后的数据长度，释放缓存时要用 */
    int show_flag = 1; /* 1:保存到本地 2:显示在 lcd 上 */
    int video_fd;
    int lcd_fd;
    int lcd_xres, lcd_yres;
    int lcd_realvirtual;
    int start = 0;

```

```

    QTimer *timer;
    void yuv_to_rgb(unsigned char *yuv, unsigned char *rgb);
    int v4l2_open();    //初始化相机参数
    int v4l2_close();
public slots:
    void video_show();
};
#endif // V4L2_H

```

6) Main. cpp

主函数较简单，创建了摄像头类并持续打印摄像头数据到界面。

```

#include "v4l2.h"
#include <QApplication>
int main(int argc, char *argv[]) {
    QApplication a(argc, argv);
    v4l2 w;
    w.show();
    return a.exec();
}

```

7) Showphoto. cpp

实现了摄像头界面的显示和按钮功能，但并没有接接触屏 API，按键功能仅作为代码参考。

```

#include "showphoto.h"
#include "ui_showphoto.h"
#include <v4l2.h>
showphoto::~showphoto() {
    delete ui;
}
void showphoto::on_pushButton_back_clicked() {
    this->close();
    v4l2 *v = new v4l2();    //new 会分配内存
    v->show();
}

```

```

showphoto::showphoto(QWidget *parent) : QMainWindow(parent), ui(new Ui::showphoto) {
    ui->setupUi(this);
    dir.setPath("./photo/"); // 设置目录路径
    dir.setFilter(QDir::Files | QDir::NoDotAndDotDot); // 过滤条件，只获取文件
    dir.setNameFilters(QStringList() << "*.jpg"); // 文件名过滤，只获取 jpg 文件
    dir.setSorting(QDir::Time | QDir::Reversed); // 根据修改时间进行排序，逆序排列
    fileList = dir.entryList(); // 获取文件列表
    if(fileList.length() > 0){
        QString filePath = dir.filePath(fileList[currentIndex]); // 获取文件路径
        QPixmap pixmap(filePath); // 创建图片对象
        ui->label->setPixmap(pixmap); // 在控件上显示图片
        currentIndex++; // 更新当前显示的文件索引
        if (currentIndex >= fileList.size()) {
            currentIndex = 0;
        }
    }
}

/* 下一张图片 */
void showphoto::on_pushButton_next_clicked()
{
    if(fileList.length() > 0){
        QString filePath = dir.filePath(fileList[currentIndex]); // 获取文件路径
        QPixmap pixmap(filePath); // 创建图片对象
        ui->label->setPixmap(pixmap); // 在控件上显示图片
        currentIndex++; // 更新当前显示的文件索引
        if (currentIndex >= fileList.size()) {
            currentIndex = 0;
        }
    }
}

/* 上一张图片 */
void showphoto::on_pushButton_front_clicked()
{
    if(fileList.length() > 0){
        QString filePath = dir.filePath(fileList[currentIndex]); // 获取文件路径
        QPixmap pixmap(filePath); // 创建图片对象
        ui->label->setPixmap(pixmap); // 在控件上显示图片
        currentIndex--; // 更新当前显示的文件索引
        if (currentIndex < 0) {
            currentIndex = fileList.size()-1;
        }
    }
}
}

```

8) V4l2. cpp

该文件实现了主要的各项功能，因为功能较复杂较多，在指导书中不过多赘述，如下分别实现了 v4l2 类的初始化和摄像头显示。

```
#include "v4l2.h"
#include "ui_v4l2.h"
#include <QtGui/QScreen>
#include <QDateTime>
#include <QDebug>
#include <QImage>

v4l2::v4l2(QWidget *parent) : QMainWindow(parent), ui(new Ui::v4l2) {
    printf("v4l2 ui 运行\n");
    ui->setupUi(this);
    timer = new QTimer(); //每隔固定的时间显示一帧
    connect(timer, SIGNAL(timeout()), this, SLOT(video_show()));
    if(0 == v4l2_open()){
        printf("打开相机成功!\n");
        timer->start(10);
        start = 1;
        ui->pushButton_open->setText("关闭");
    }
    // 获取当前目录
    QString currentDir = QDir::currentPath();
    // 拼接 photo 文件夹路径
    QString photoDirPath = currentDir + "/photo";
    // 创建 QDir 对象
    QDir photoDir(photoDirPath);
    // 判断 photo 文件夹是否存在
    if (!photoDir.exists()){
        // 创建 photo 文件夹
        if (photoDir.mkdir(photoDirPath))
            printf("创建 photo 文件夹成功\n");
        else
            printf("创建 photo 文件夹失败\n");
    }
    else
        printf("photo 文件夹已存在\n");
}

v4l2::~v4l2()
{
    delete ui;
}
```

```

void v4l2::video_show()
{
    QPixmap pix;
    FrameBuffer framebuf,bufData;
    /* 采集图片数据 */
    //定义结构体变量，用于获取内核队列数据
    struct v4l2_buffer buffer;
    buffer.type = V4L2_BUF_TYPE_VIDEO_CAPTURE; //设置获取视频流
    /* 从内核中捕获好的输出队列中取出一个 */
    if(0 == ioctl(video_fd, VIDIOC_DQBUF, &buffer)){ /* 获取一帧显示,显示在 label 控件上 */
        memcpy(bufData.buf,(unsigned char *)userbuff[buffer.index],buffer.bytesused);
        yuv_to_rgb(bufData.buf,framebuf.buf);
        QImage img_stream = QImage(framebuf.buf,640,480,QImage::Format_RGB888);
        QImage img=img_stream.scaled(video_width,video_height);
        ui->label->setPixmap(QPixmap::fromImage(img));
    }
    /* 将使用后的缓冲区放回到内核的输入队列中 (VIDIOC_QBUF) */
    if(0 > ioctl(video_fd, VIDIOC_QBUF, &buffer)){
        perror("返回队列失败! ");
    }
}
}

```

14. 2. 2 编译

在 Qt 中，在项目的左下角点击锤子图标或者在上方“构建”栏选择构建均可构建项目，而后可以在项目目录下找到可执行文件，如图 14-11、图 14-12。

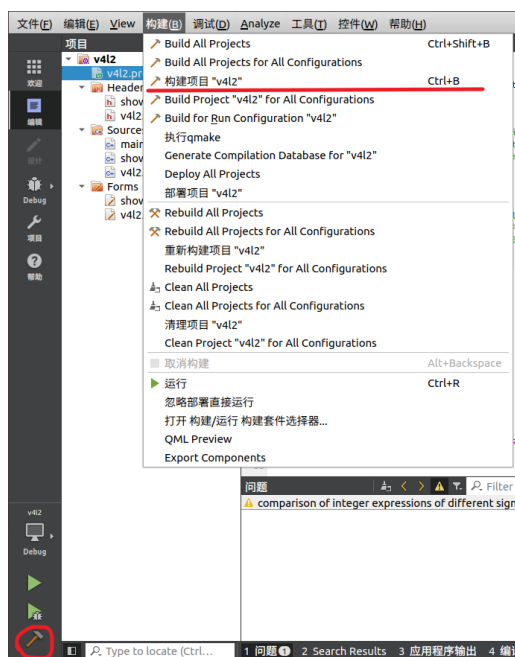


图 14-11 构建项目

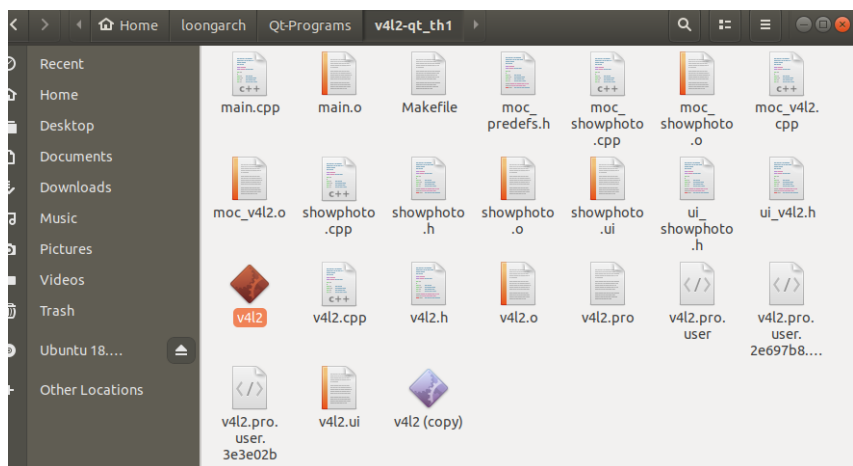


图 14-12 编译得到目标文件

14.2.3 摄像头采集测试

编译编写的程序后，将得到的可执行文件拷贝到试验箱中，在执行该文件之前，首先应在系统中查找并关闭 logo_player 进程，使用如下命令：

```
ps aux | grep logo_player
```

如图 14-13 所示，可以看到该进程的进程号为 139（第二栏），使用 kill -9 [进程号]便可以关闭该进程。

```
[root@LS-GD ~]# ps aux | grep logo_player
root      139  13.8  5.1 71488 43392 ?        S      Feb02   5:29 ./logo_player
root      217   0.0  0.1  3056  1600 ttyS2    S+     00:09   0:00 grep logo_plr
```

```
[root@LS-GD ~]# kill -9 139
```

图 14-13 查看并关闭进程

执行 kill 命令后，可以看到试验箱的 LCD 屏动画停止，说明成功关闭。然后将摄像头的 USB 插头插入开发板主板中，如图 14-14 所示：

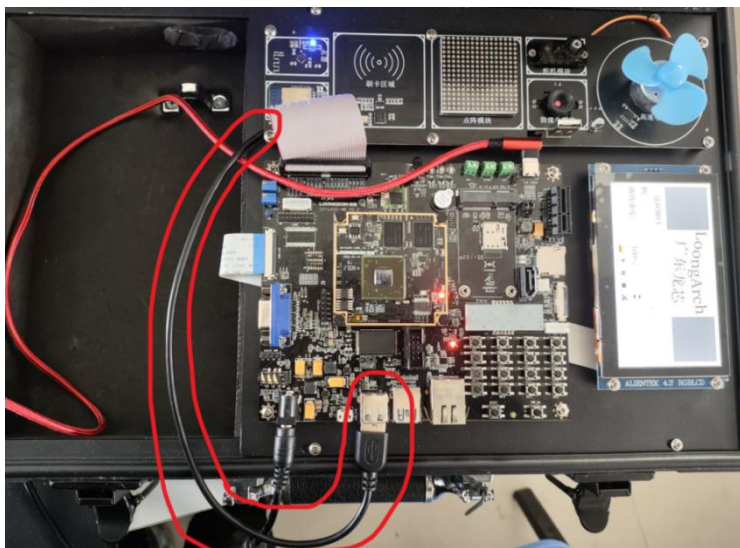


图 14-14 插入摄像头 USB 插头

最后执行编译的可执行程序，可以看到 LCD 屏幕上开始显示摄像头拍摄的内容，如图 14-15。

```
[root@LS-GD ~]# ./v4l2
QStandardPaths: XDG_RUNTIME_DIR not set, defaulting to '/tmp/runtime-root'
ts_setup() failed (No such file or directory)
v4l2 ui运行
支持格式: Motion-JPEG, MJPG
该格式默认帧率 30 fps
支持的分辨率有:
1280 x 720
640 x 480
320 x 240
320 x 180
800 x 600
848 x 480
640 x 360
1280 x 800
```

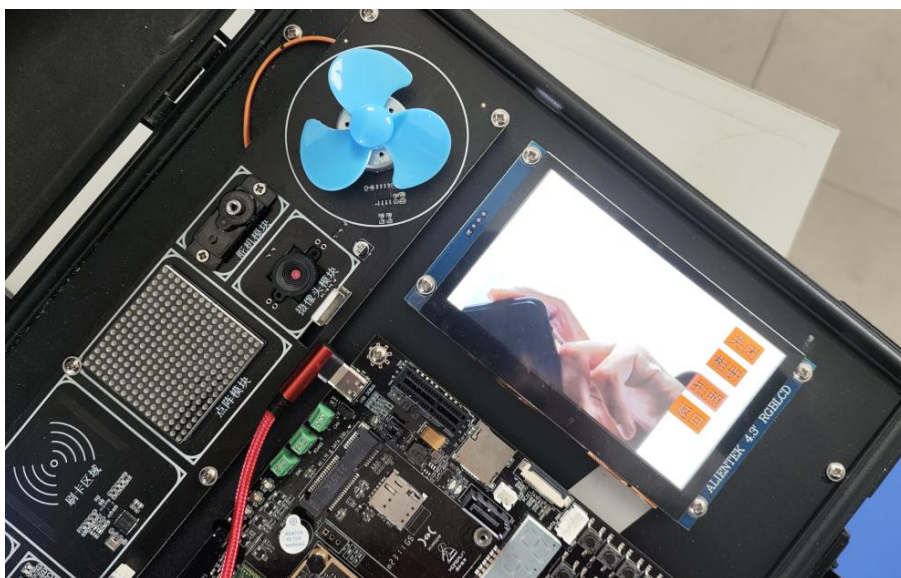


图 14-15 摄像头拍摄测试

14.3 练习：摄像头实现拍照功能

本节需要通过试验箱的摄像头模块实现拍照功能，大致代码在实验代码中已经配置，学生可以通过 RS485、CAN、按键等方式进行触发，将采集的图像数据导出，并在外部设备上确认。

任务要求：

1. 摄像头初始化：编写代码以正确初始化摄像头模块，确保摄像头能够被试验箱识别并正常工作。
2. 拍照功能实现：将 RS485 或按键等代码合并入此项目（因为使用的编译器相同，因此代码无需过多修改），通过命令行或按键触发，调用代码中原有的摄像头拍照功能。
3. 图像导出与确认：导出拍摄的图像文件到外部设备（如 U 盘或网络存储），在外部设备上打开并确认。

扩展任务：

定时拍照：实现定时拍照功能，设定时间间隔后摄像头自动拍摄并保存图像。

第15章 NFC 模块实验

15.1 实验目的

了解 NFC 模块的使用及读写功能实现。掌握教学平台上 NFC 模块的功能特性、连接方式与控制模式。

根据 NFC 的工作特性与技术原理编写程序，实现对 NFC 卡片的读写功能，在终端中输出读取的信息或写入终端中输入的信息。

15.2 RFID 简介

RFID(射频识别: Radio Frequency Identification)俗称"电子标签", 是一种非接触式的自动识别技术, 它通过射频信号自动识别目标对象并获取相关数据, 识别工作无须人工干预, 作为条形码的无线版本, RFID 技术具有条形码所不具备的防水、防磁、耐高温、使用寿命长、读取距离大、标签上数据可以加密、存储数据容量更大、存储信息更改自如等优点, 其应用将给零售、物流等产业带来革命性变化。

试验箱上对应模块为 RC522, 其上标有“刷卡区域”文字, 位于拓展板上, 使用时要注意插紧排线, 使用到的管脚包括 CLK、MISO、MOSI、NSS、RST, 如图 15-1。

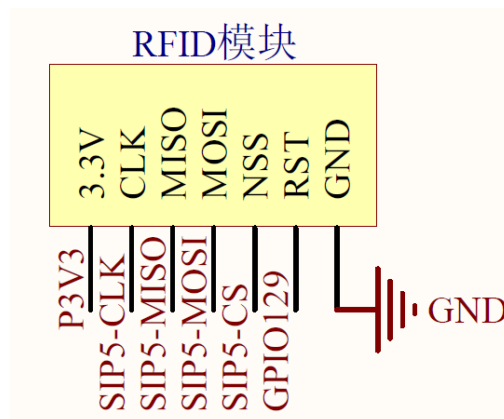


图 15-1 RFID 引脚

SPI 通信中 RC522 模块作为从机, SPI 时钟 CLK 由单片机产生, MOSI 为主板发送给 RC522 信息的引脚, MISO 为模块发送给单片机信息的引脚。NSS 为片选信号。

本实验中使用软件模拟 SPI 通信, SPI 通信时序图如图 15-2:

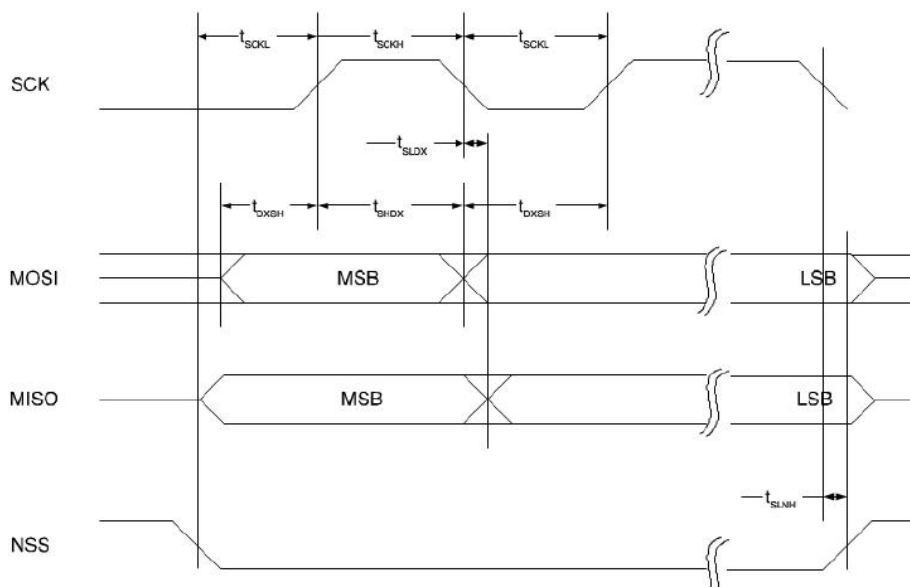


图 15-2 SPI 通信时序图

如果要对某个 SPI 外设进行读写，要对这个设备使能，即片选功能，在这里 RC522 片选端为低电平即为选中，如果有多个 RC522，要注意，选中某个的时候，其 NSS 为低电平，其他为高电平。

SPI 进行读写的时候，因为是同步通信，所以所需的同步信号需由 CLK（也叫 SCK）时钟线提供，简单的说就是 CLK 高低电平的转换。

传送数据时：在 CLK 在上升沿或下降沿时发送器 MOSI 发送数据，在紧接着的下降沿或上升沿时接收器 MISO 读取数据，完成一位数据的传送，八个时钟周期完成一个字节的传送。

与 RC522 通信时，即与 RC522 的寄存器进行通信，阅读其手册，可查阅其读写方式，首先查看其读取方式如图 15-3：

	byte 0	byte 1	byte 2	to	byte n	byte n+1
MOSI	adr 0	adr 1	adr 2		adr n	00
MISO	X	data 0	data 1		data n-1	data n

图 15-3 SPI 通信读取方式

可见主机首先发送 1 字节地址，从机在下一个字节返回地址指向的，并且支持连续读取，不过通常用不到连续读取功能，在大部分情况下，会编写两个底层基础方法，一个用于主机发送 1 字节，同时忽略接收到的 1 字节数据，另一个用于主机发送 1 字节无意义数据，并接收 1 字节数据。

再查看其写入方式如图 15-4：

	byte 0	byte 1	byte 2	to	byte n	byte n+1
MOSI	adr 0	data 0	data 1		data n-1	data n
MISO	X	X	X		X	X

图 15-4 SPI 通信写入方式

主机连续发送两个字节，分别为地址和新值，即完成数据写入。

15.3 实验步骤

15.3.1 基本 API

RFID 的内核代码较复杂，本实验中仅对修改为软件模拟的两个底层基础方法和顶层方法进行详细介绍，其余部分感兴趣的同学可自行搜索调查。

RFID 的基本 API 在网上已有很多实现，这里推荐其中一个参考网址：<https://blog.csdn.net/lzzzzzzm/article/details/120362536>，但请注意，除了 2 个基础方法和顶层方法，其余方法可能存在些许不同，请不要直接运行参考网址的代码，建议跟着指导书敲一遍。

首先创建两个文件 rc522_api.c 和 rc522_api.h，用于编写 rc522 的基础 API。

1) rc522_api.h

首先是 rc522_api.h，前部分与参考网址中的相差无几，是 RC522 的基本指令、寄存器地址、各类基本数值等。

```
#ifndef __rc522_h__
#define __rc522_h__

#define PCD_IDLE                0x00
#define PCD_AUTHENT             0x0E
#define PCD_RECEIVE             0x08
#define PCD_TRANSMIT            0x04
#define PCD_TRANSCEIVE         0x0C
#define PCD_RESETPHASE         0x0F
#define PCD_CALCCRC            0x03

#define PICC_REQIDL             0x26
#define PICC_REQALL            0x52
#define PICC_ANTICOLL1         0x93
#define PICC_ANTICOLL2         0x95
#define PICC_AUTHENT1A         0x60
#define PICC_AUTHENT1B         0x61
#define PICC_READ               0x30
#define PICC_WRITE              0xA0
#define PICC_DECREMENT          0xC0
#define PICC_INCREMENT          0xC1
#define PICC_RESTORE            0xC2
#define PICC_TRANSFER           0xB0
#define PICC_HALT               0x50
#define DEF_FIFO_LENGTH        64
#define MAXRLEN 18

#define RFU00                   0x00
#define CommandReg              0x01
#define ComIEnReg               0x02
```

```

#define DivlEnReg          0x03
#define ComIrqReg          0x04
#define DivIrqReg          0x05
#define ErrorReg           0x06
#define Status1Reg         0x07
#define Status2Reg         0x08
#define FIFODataReg        0x09
#define FIFOLevelReg       0x0A
#define WaterLevelReg      0x0B
#define ControlReg         0x0C
#define BitFramingReg      0x0D
#define CollReg            0x0E
#define RFU0F              0x0F

// PAGE 1
#define RFU10              0x10
#define ModeReg            0x11
#define TxModeReg          0x12
#define RxModeReg          0x13
#define TxControlReg       0x14
#define TxAutoReg          0x15
#define TxSelReg           0x16
#define RxSelReg           0x17
#define RxThresholdReg     0x18
#define DemodReg           0x19
#define RFU1A              0x1A
#define RFU1B              0x1B
#define MifareReg          0x1C
#define RFU1D              0x1D
#define RFU1E              0x1E
#define SerialSpeedReg     0x1F

// PAGE 2
#define RFU20              0x20
#define CRCResultRegM      0x21
#define CRCResultRegL      0x22
#define RFU23              0x23
#define ModWidthReg        0x24
#define RFU25              0x25
#define RFCfgReg           0x26
#define GsNReg             0x27
#define CWGsCfgReg         0x28
#define ModGsCfgReg        0x29
#define TModeReg           0x2A
#define TPrescalerReg      0x2B

```

```

#define    TReloadRegH          0x2C
#define    TReloadRegL          0x2D
#define    TCounterValueRegH    0x2E
#define    TCounterValueRegL    0x2F
// PAGE 3
#define    RFU30                 0x30
#define    TestSel1Reg           0x31
#define    TestSel2Reg           0x32
#define    TestPinEnReg          0x33
#define    TestPinValueReg       0x34
#define    TestBusReg            0x35
#define    AutoTestReg           0x36
#define    VersionReg            0x37
#define    AnalogTestReg         0x38
#define    TestDAC1Reg           0x39
#define    TestDAC2Reg           0x3A
#define    TestADCReg            0x3B
#define    RFU3C                 0x3C
#define    RFU3D                 0x3D
#define    RFU3E                 0x3E
#define    RFU3F                 0x3F

```

然后是 API 中返回值状态和 RC522 管脚的声明。

```

#define    MI_OK                 0
#define    MI_NOTAGERR           (-1)
#define    MI_ERR                (-2)
#define    GPIO_CLK             125
#define    GPIO_MISO             126
#define    GPIO_MOSI             127
#define    GPIO_CS                128
#define    GPIO_RST              129

```

最后进行会在模块代码中使用的函数的外部声明。

```

extern char PcdReset(void);
extern char PcdRequest(unsigned char req_code,unsigned char *pTagType);
extern void PcdAntennaOn(void);
extern void PcdAntennaOff(void);
extern char M500PcdConfigISOType(unsigned char type);
extern char PcdAnticoll(unsigned char *pSnr);
extern char PcdSelect(unsigned char *pSnr);

```

```

extern char PcdAuthState(unsigned char auth_mode,unsigned char addr,unsigned char *pKey,unsigned
char *pSnr);
extern char PcdWrite(unsigned char addr,unsigned char *pData);
extern char PcdRead(unsigned char addr,unsigned char *pData);
extern char PcdHalt(void);
extern struct spi_device *rc522_spi;
extern unsigned char ReadRawRC(unsigned char Address);
#endif

```

2) rc522_api.c

然后进行 API 源码的编写，引入依赖的头文件，并在文件首声明部分较杂乱的函数，避免部分函数无法寻找到需要的函数声明，以下介绍的函数可以不分先后地编写，如果出现声明无法找到，在这一部分声明所需的函数即可。

```

#include "rc522_api.h"
#include <linux/delay.h>
#include <linux/spi/spi.h>
#include <linux/gpio.h>
void ClearBitMask(unsigned char reg,unsigned char mask);
void WriteRawRC(unsigned char Address, unsigned char value);
void SetBitMask(unsigned char reg,unsigned char mask);
char PcdComMF522(unsigned char Command,
                unsigned char *pInData,
                unsigned char InLenByte,
                unsigned char *pOutData,
                unsigned int  *pOutLenBit);
void CalulateCRC(unsigned char *pIndata,unsigned char len,unsigned char *pOutData);
unsigned char ReadRawRC(unsigned char Address);
void PcdAntennaOn(void);

```

再编写除 MISO 之外四个引脚的软件置位清空函数，和 MISO 的读取函数。

```

static void SET_RC522RST(void) {
    gpio_set_value(GPIO_RST, 1);
}
static void CLR_RC522RST(void) {
    gpio_set_value(GPIO_RST, 0);
}
static void SET_SPI_CS(void) {
    gpio_set_value(GPIO_CS, 1);
}

```

```

static void CLR_SPI_CS(void) {
    gpio_set_value(GPIO_CS, 0);
}
static void SET_SPI_CLK(void) {
    gpio_set_value(GPIO_CLK, 1);
}
static void CLR_SPI_CLK(void) {
    gpio_set_value(GPIO_CLK, 0);
}
static void SET_SPI_MOSI(void) {
    gpio_set_value(GPIO_MOSI, 1);
}
static void CLR_SPI_MOSI(void) {
    gpio_set_value(GPIO_MOSI, 0);
}
static int READ_SPI_MISO(void) {
    return gpio_get_value(GPIO_MISO);
}

```

编写两个基本函数之一的读取寄存器函数 ReadRawRC，和 I2C 相反，在 SPI 中通过最高位判断该操作为读/写，最高位为 1 时说明为读操作，并通过第 1~6 共 6 位表示寄存器地址，得到实际发送的 1 字节 ucAddr。拉高片选并发送 ucAddr，发送结束后不用对 MOSI 管脚进行操作，无论是 1 或是 0，接下来发送的 1 字节均无实意，只需要读取的 1 字节，读取到从机发送的 1 字节后返回即可。

```

unsigned char ReadRawRC(unsigned char Address) {
    unsigned char i, ucAddr;
    unsigned char ucResult=0;
    CLR_SPI_CLK();
    CLR_SPI_CS();
    ucAddr = ((Address << 1) & 0x7E) | 0x80;
    for (i = 0; i < 8; i++) {
        if ((ucAddr & 0x80) == 0x80) SET_SPI_MOSI();
        else CLR_SPI_MOSI();
        SET_SPI_CLK();
        ucAddr <<= 1;
        CLR_SPI_CLK();
        udelay(10);
    }
    SET_SPI_CS();
    SET_SPI_CLK();
    return ucResult;
}

```

然后是写寄存器函数，同样将寄存器地址放在 1 字节的中间 6 位，最高位为 0，表示为写操作并发送，直接忽略从机发来的数据。然后再发送新值 value，同样忽略从机发来的数据，写操作结束。

```
void WriteRawRC(unsigned char Address, unsigned char value) {
    unsigned char i, ucAddr;
    CLR_SPI_CLK();
    CLR_SPI_CS();
    ucAddr = ((Address<<1)&0x7E);
    for (i = 0; i < 8; i++) {
        if ((ucAddr & 0x80) == 0x80) SET_SPI_MOSI();
        else CLR_SPI_MOSI();
        SET_SPI_CLK();
        ucAddr <<= 1;
        CLR_SPI_CLK();
        udelay(10);
    }
    for (i = 0; i < 8; i++) {
        if ((value & 0x80) == 0x80) SET_SPI_MOSI();
        else CLR_SPI_MOSI();
        SET_SPI_CLK();
        value <<= 1;
        CLR_SPI_CLK();
        udelay(10);
    }
    SET_SPI_CS();
    SET_SPI_CLK();
}
```

然后编写两个基础函数，其用于将掩码指定的位置 1 或清 0。

```
void SetBitMask(unsigned char reg,unsigned char mask) {
    char tmp = 0x0;
    tmp = ReadRawRC(reg);
    WriteRawRC(reg,tmp | mask); // set bit mask
}

void ClearBitMask(unsigned char reg,unsigned char mask) {
    char tmp = 0x0;
    tmp = ReadRawRC(reg);
    WriteRawRC(reg, tmp & ~mask); // clear bit mask
}
```

编写重置函数，用于重置 RC522 模块。

```

char PcdReset(void) {
    unsigned char a;
    SET_RC522RST();
    udelay(10);
    CLR_RC522RST();
    udelay(10);
    SET_RC522RST();
    udelay(10);
    a = ReadRawRC(0x02);
    WriteRawRC(CommandReg,PCD_RESETPHASE);
    ndelay(10);
    WriteRawRC(ModeReg,0x3D);
    WriteRawRC(TReloadRegL,30);
    WriteRawRC(TReloadRegH,0);
    WriteRawRC(TModeReg,0x8D);
    WriteRawRC(TPrescalerReg,0x3E);
    WriteRawRC(TxAutoReg,0x40);
    return MI_OK;
}

```

编写 PcdComMF522 函数，其功能为通过 RC522 与 ISO14443 卡通讯。

```

char PcdComMF522(unsigned char Command, unsigned char *pInData, unsigned char InLenByte,
    unsigned char *pOutData, unsigned int *pOutLenBit) {
    char status = MI_ERR;
    unsigned char irqEn    = 0x00;
    unsigned char waitFor = 0x00;
    unsigned char lastBits;
    unsigned char n;
    unsigned int i;
    switch (Command) {
        case PCD_AUTHENT:
            irqEn    = 0x12;
            waitFor = 0x10;
            break;
        case PCD_TRANSCEIVE:
            irqEn    = 0x77;
            waitFor = 0x30;
            break;
        default:
            break;
    }
    WriteRawRC(ComIEnReg,irqEn|0x80);
}

```

```

ClearBitMask(ComIrqReg,0x80);
WriteRawRC(CommandReg,PCD_IDLE);
SetBitMask(FIFOLevelReg,0x80);
for (i=0; i<InLenByte; i++) WriteRawRC(FIFODataReg, pInData[i]);
WriteRawRC(CommandReg, Command);
if (Command == PCD_TRANSCEIVE) SetBitMask(BitFramingReg,0x80);
i = 2000;
do {
    n = ReadRawRC(ComIrqReg);
    i--;
} while ((i!=0) && !(n&0x01) && !(n&waitFor));
ClearBitMask(BitFramingReg,0x80);
if (i!=0) {
    if(!(ReadRawRC(ErrorReg)&0x1B)) {
        status = MI_OK;
        if (n & irqEn & 0x01) status = MI_NOTAGERR;
        if (Command == PCD_TRANSCEIVE) {
            n = ReadRawRC(FIFOLevelReg);
            lastBits = ReadRawRC(ControlReg) & 0x07;
            if (lastBits) *pOutLenBit = (n-1)*8 + lastBits;
            else *pOutLenBit = n*8;
            if (n == 0) n = 1;
            if (n > MAXRLEN) n = MAXRLEN;
            for (i=0; i<n; i++) pOutData[i] = ReadRawRC(FIFODataReg);
        }
    }
    else status = MI_ERR;
}
SetBitMask(ControlReg,0x80); // stop timer now
WriteRawRC(CommandReg,PCD_IDLE);
return status;
}

```

nnaOn 和 PcdAntennaOff，用于开启和关闭天线。

```

void PcdAntennaOn(void) {
    unsigned char i;
    i = ReadRawRC(TxControlReg);
    if (!(i & 0x03)) SetBitMask(TxControlReg, 0x03);
}

void PcdAntennaOff(void) {
    ClearBitMask(TxControlReg, 0x03);
}

```

编写寻卡方法 PcdRequest。

```
char PcdRequest(unsigned char req_code,unsigned char *pTagType) {
    char status;
    unsigned int unLen;
    unsigned char ucComMF522Buf[MAXRLEN];
    ClearBitMask(Status2Reg,0x08);
    WriteRawRC(BitFramingReg,0x07);
    SetBitMask(TxControlReg,0x03);
    ucComMF522Buf[0] = req_code;
    status = PcdComMF522(PCD_TRANSCEIVE,ucComMF522Buf,1,ucComMF522Buf,&unLen);
    if ((status == MI_OK) && (unLen == 0x10)) {
        *pTagType      = ucComMF522Buf[0];
        *(pTagType+1) = ucComMF522Buf[1];
    }
    else status = MI_ERR;
    return status;
}
```

编写防冲撞函数 PcdAnticoll。

```
char PcdAnticoll(unsigned char *pSnr) {
    char status;
    unsigned char i,snr_check=0;
    unsigned int unLen;
    unsigned char ucComMF522Buf[MAXRLEN];
    ClearBitMask(Status2Reg,0x08);
    WriteRawRC(BitFramingReg,0x00);
    ClearBitMask(CollReg,0x80);
    ucComMF522Buf[0] = PICC_ANTICOLL1;
    ucComMF522Buf[1] = 0x20;
    status = PcdComMF522(PCD_TRANSCEIVE,ucComMF522Buf,2,ucComMF522Buf,&unLen);
    if (status == MI_OK) {
        for (i=0; i<4; i++) {
            *(pSnr+i) = ucComMF522Buf[i];
            snr_check ^= ucComMF522Buf[i];
        }
        if (snr_check != ucComMF522Buf[i]) status = MI_ERR;
    }
    SetBitMask(CollReg,0x80);
    return status;
}
```

编写选卡函数 PcdSelect。

```

char PcdSelect(unsigned char *pSnr) {
    char status;
    unsigned char i;
    unsigned int unLen;
    unsigned char ucComMF522Buf[MAXRLEN];
    ucComMF522Buf[0] = PICC_ANTICOLL1;
    ucComMF522Buf[1] = 0x70;
    ucComMF522Buf[6] = 0;
    for (i=0; i<4; i++) {
        ucComMF522Buf[i+2] = *(pSnr+i);
        ucComMF522Buf[6] ^= *(pSnr+i);
    }
    CalculateCRC(ucComMF522Buf,7,&ucComMF522Buf[7]);
    ClearBitMask(Status2Reg,0x08);
    status = PcdComMF522(PCD_TRANSCEIVE,ucComMF522Buf,9,ucComMF522Buf,&unLen);
    if ((status == MI_OK) && (unLen == 0x18)) status = MI_OK;
    else status = MI_ERR;
    return status;
}

```

编写验证卡片密码函数 PcdAuthState

```

char PcdAuthState(unsigned char auth_mode,unsigned char addr,
    unsigned char *pKey,unsigned char *pSnr) {
    char status;
    unsigned int unLen;
    unsigned char i,ucComMF522Buf[MAXRLEN];
    ucComMF522Buf[0] = auth_mode;
    ucComMF522Buf[1] = addr;
    for (i=0; i<6; i++) ucComMF522Buf[i+2] = *(pKey+i);
    for (i=0; i<6; i++) ucComMF522Buf[i+8] = *(pSnr+i);
    status = PcdComMF522(PCD_AUTHENT,ucComMF522Buf,12,ucComMF522Buf,&unLen);
    if ((status != MI_OK) || (!(ReadRawRC(Status2Reg) & 0x08))) status = MI_ERR;
    return status;
}

```

编写读卡函数 PcdRead。

```

char PcdRead(unsigned char addr,unsigned char *pData) {
    char status;
    unsigned int unLen;
    unsigned char i,ucComMF522Buf[MAXRLEN];

```

```

ucComMF522Buf[0] = PICC_READ;
ucComMF522Buf[1] = addr;
CalulateCRC(ucComMF522Buf,2,&ucComMF522Buf[2]);
status = PcdComMF522(PCD_TRANSCEIVE,ucComMF522Buf,4,ucComMF522Buf,&unLen);
if ((status == MI_OK) && (unLen == 0x90)) {
    for (i=0; i<16; i++) *(pData+i) = ucComMF522Buf[i];
}
else status = MI_ERR;
return status;
}

```

编写写卡函数 PcdWrite。

```

char PcdWrite(unsigned char addr,unsigned char *pData)
{
    char status;
    unsigned int unLen;
    unsigned char i,ucComMF522Buf[MAXRLEN];

    ucComMF522Buf[0] = PICC_WRITE;
    ucComMF522Buf[1] = addr;
    CalulateCRC(ucComMF522Buf,2,&ucComMF522Buf[2]);

    status = PcdComMF522(PCD_TRANSCEIVE,ucComMF522Buf,4,ucComMF522Buf,&unLen);
    if ((status != MI_OK) || (unLen != 4) || ((ucComMF522Buf[0] & 0x0F) != 0x0A))
    {
        status = MI_ERR;
    }

    if (status == MI_OK)
    {
        for (i=0; i<16; i++) ucComMF522Buf[i] = *(pData+i);
        CalulateCRC(ucComMF522Buf,16,&ucComMF522Buf[16]);

        status = PcdComMF522(PCD_TRANSCEIVE,
                               ucComMF522Buf,18,ucComMF522Buf,&unLen);
        if ((status != MI_OK) || (unLen != 4) || ((ucComMF522Buf[0] & 0x0F) != 0x0A))
        {
            status = MI_ERR;
        }
    }
    return status;
}

```

编写休眠函数 PcdHalt。

```

char PcdHalt(void) {
    char status;
    unsigned int unLen;
    unsigned char ucComMF522Buf[MAXRLEN];
    ucComMF522Buf[0] = PICC_HALT;
    ucComMF522Buf[1] = 0;
    CalculateCRC(ucComMF522Buf,2,&ucComMF522Buf[2]);
    status = PcdComMF522(PCD_TRANSCEIVE,ucComMF522Buf,4,ucComMF522Buf,&unLen);
    return MI_OK;
}

```

编写 CRC 校验码计算函数 CalculateCRC。

```

void CalculateCRC(unsigned char *pIndata,unsigned char len,unsigned char *pOutData)
{
    unsigned char i,n;
    ClearBitMask(DivIrqReg,0x04);
    WriteRawRC(CommandReg,PCD_IDLE);
    SetBitMask(FIFOLevelReg,0x80);
    for (i=0; i<len; i++)
    {   WriteRawRC(FIFODataReg, *(pIndata+i));   }
    WriteRawRC(CommandReg, PCD_CALCCRC);
    i = 0xFF;
    do
    {
        n = ReadRawRC(DivIrqReg);
        i--;
    }
    while ((i!=0) && !(n&0x04));
    pOutData[0] = ReadRawRC(CRCResultRegL);
    pOutData[1] = ReadRawRC(CRCResultRegM);
}

```

编写 ISO14443 类型配置函数 M500PcdConfigISOType。

```

char M500PcdConfigISOType(unsigned char type)
{
    if (type == 'A')                //ISO14443_A
    {
        ClearBitMask(Status2Reg,0x08);
        WriteRawRC(ModeReg,0x3D);//3F
        WriteRawRC(RxSelReg,0x86);//84
        WriteRawRC(RFCfgReg,0x7F);  //4F
    }
}

```

```

        WriteRawRC(TReloadRegL,30);//tmoLength);// TReloadVal = 'h6a =tmoLength(dec)
        WriteRawRC(TReloadRegH,0);
        WriteRawRC(TModeReg,0x8D);
        WriteRawRC(TPrescalerReg,0x3E);
        ndelay(1000);
        PcdAntennaOn();
    }
    else{ return -1; }

    return MI_OK;
}

```

至此，RC522 基本 API 编写完毕。

15. 3. 2 内核模块代码编写

1) 依赖头文件

```

#include <linux/kernel.h>
#include <linux/module.h>
#include <linux/slab.h>
#include <linux/input.h>
#include <linux/init.h>
#include <linux/errno.h>
#include <linux/delay.h>
#include <linux/miscdevice.h>
#include <linux/platform_device.h>
#include <asm/uaccess.h>
#include <linux/poll.h>
#include <linux/wait.h>
#include <linux/interrupt.h>
#include <linux/sched.h>
#include <asm/atomic.h>
#include <linux/mutex.h>
#include <linux/spi/spi.h>
#include <linux/workqueue.h>
#include <linux/gpio.h>
#include "rc522_api.h"

```

2) 全局变量声明

进行全局的宏、变量声明，声明 ioctl 的操作码，命令码在无数据的情况下

也可以不使用声明宏_I0，这样默认高 2 位为 0，可以视为为无数据命令码，无 size 位，使用字符'\0' 作为魔数的命令码。GPCFG 用于配置复用配置寄存器。

```
#define CHANGE_PASSWD 1
#define CHANGE_BLOCK 2
#define READ_CARD 3
#define WRITE_CARD 4
#define CHANGE_KEY 5
#define GET_ID 6

#define GPCFG10x1fe104cc//120-127
#define GPCFG20x1fe104d0 //128-135

static void __iomem *GPCFG1_V;
static void __iomem *GPCFG2_V;
```

声明全局的缓冲区，用于数据读写。其中使用块的默认密码 0xff，全局声明当前读的块 KuaiN 和操作符 operationcard。

```
typedef unsigned char uchar;
uchar
NewKey[16]={0xff,0xff,0xff,0xff,0xff,0xff,0xff,0x07,0x80,0x69,0x00,0x00,0x00,0x00,0x00,0x00};

/*static DECLARE_WAIT_QUEUE_HEAD(rc522_wait);*/
static unsigned char Read_Data[16]={0x00};
static unsigned char read_data_buff[16];

static uchar PassWd[6]={0xff, 0xff, 0xff, 0xff, 0xff, 0xff};
static uchar WriteData[16];
static unsigned char RevBuffer[30];
static unsigned char MLastSelectedSnr[4];

uint KuaiN;
uchar operationcard;
```

3) file_operations 结构体

进行 file_operations 结构体对象及对应方法的声明。

首先是 open 和 read 方法，open 方法中对 RC522 进行初始化，read 方法中先指定操作符为 READ_CARD，然后通过 rc522_loop_work 进行操作符的统一处理，再将读取到的数据拷贝到用户空间，这里的 rc522_loop_work 函数将稍后介绍。

```

static int rc522_open(struct inode *inode, struct file *filp) {
    InitRc522();
    pr_info("rc522 start work!\n");
    return 0;
}

static ssize_t rc522_read(struct file *filp, char *buf, size_t count, loff_t *f_pos) {
    operationcard = READ_CARD;
    if(rc522_loop_work(operationcard))
        return 0;
    pr_info("card info:%2.2X\n", Read_Data[0]);
    if(copy_to_user(buf, read_data_buff, sizeof(read_data_buff))) {
        pr_info("copy card number to userspace err\n");
        return 0;
    }
    return sizeof(read_data_buff);
}

```

RC522 初始化函数，其先检查是否能与 RC522 通信，然后进行天线和 ISO 类型的初始化。

```

void InitRc522(void) {
    unsigned char a;
    PcdReset();
    a = ReadRawRC(TReloadRegL);
    if(a != 30) pr_info("What?, NO RC522:%d\n", a);
    else pr_info("RC522 exist\n");
    PcdAntennaOff();
    PcdAntennaOn();
    M500PcdConfigISOType('A');
}

```

然后编写 write 和 release 方法，write 方法会先检查块是否合法，读取用户空间数据后，通过 rc522_loop_work 统一执行操作。

```

static ssize_t rc522_write(struct file *filp, const char *buf, size_t count, loff_t *f_pos) {
    if(KuaiN == 0) {
        pr_info("block[0] is reserved, can't write\n");
        return 0;
    }
    if(KuaiN < 0 || KuaiN > 63) {
        pr_info("block[%d] unreachable, please set the write block first", KuaiN);
        return -0;
    }
}

```

```

        if ((KuaiN % 4) == 3) {
            pr_info("block[%d] is key block, not data block\n", KuaiN);
            return -0;
        }
        memset(WriteData, 0, sizeof(WriteData));
        if (copy_from_user(WriteData, (char *)buf, count)) {
            pr_info("%s, [line %d] copy from user err.\n", __FILE__, __LINE__);
            return 0;
        }
        operationcard = WRITE_CARD;
        pr_info("writing...\n");
        if(rc522_loop_work(operationcard))
            return -EFAULT;
        return 0;
    }
}

```

编写 ioctl 函数, 其中 READ 和 WRITE 通过 read() 和 write() 系统调用实现, 不进行操作。

```

static long rc522_ioctl(struct file *file, unsigned int cmd, unsigned long arg) {
    switch(cmd) {
        case CHANGE_PASSWD:
            operationcard = CHANGE_PASSWD;
            if (copy_from_user(PassWd, (char *)arg, sizeof(PassWd))) {
                pr_info("%s:change pass word err", __func__);
                return -EFAULT;
            }
            break;
        case CHANGE_BLOCK:
            if (arg < 0 || arg > 63) {
                pr_info("block number err %lu", arg);
                return -EFAULT;
            }
            KuaiN = (int)arg;
            break;
        case READ_CARD:
            break;
        case WRITE_CARD:
            break;
        case CHANGE_KEY:
            operationcard = CHANGE_KEY;
            break;
    }
}

```

```

        case GET_ID:
            operationcard = GET_ID;
            if(!rc522_loop_work(operationcard)){
                if (copy_to_user((char *)arg, MLastSelectedSnr,4)) {
                    pr_info("%s, [line %d] copy to user err.\n", __FILE__, __LINE__);
                    return -EFAULT;
                }
            }
            else
                return -EFAULT;
            break;
        default:
            break;
    }
    return 0;
}

```

最后创建 file_operations 结构体对象 rc522_fops。

```

static struct file_operations rc522_fops = {
    .owner = THIS_MODULE,
    .open = rc522_open,
    .release = rc522_release,
    .read = rc522_read,
    .write = rc522_write,
    .unlocked_ioctl = rc522_ioctl,
};

```

4) rc522_loop_work

rc522_loop_work 是该驱动顶层最核心的执行函数，其先执行不同指令中重合的相同操作，再根据不同指令执行对应的操作。

首先要进行寻卡操作，如果没有成功寻卡，则重置 RC522，再次寻卡，寻卡 3 次（约 3ms）均失败则提示未找到卡，返回错误信息。

```

static char rc522_loop_work(uchar opnd) {
    char *pdata = read_data_buff;
    char status;
    int t = 0;
    while ((status = PcdRequest(PICC_REQIDL,&RevBuffer[0])) != MI_OK) {
        PcdReset();
        PcdAntennaOff();
        mdelay(1);
    }
}

```

```

        PcdAntennaOn();
        t++;
        if (t > 3) {
            pr_info("search card: no card\n");
            return -EFAULT;
        }
    }
}

```

进行防冲撞操作，获取最终进行读写的卡的序号。然后通过 PcdSelect 选择该卡。

```

status=PcdAnticoll(&RevBuffer[2]);
if(status!=MI_OK) {
    pr_info("get card nu: no number\n");
    return -EFAULT;
}
memcpy(MLastSelectedSnr,&RevBuffer[2],4);
status=PcdSelect(MLastSelectedSnr);
if(status!=MI_OK) {
    pr_info("select card: no card\n");
    return -EFAULT;
}

```

然后便针对不同操作执行不同操作。对于 GET_ID，因为卡序号已经获得，使用 PcdHalt 使模块休眠，然后返回即可。对于 READ_CARD，首先验证块密码，这里使用所有块的默认密码 0xff，无需用户输入，然后进行读卡操作，读取成功后则拷贝读取到的数据到缓冲区，并打印 debug 信息。

```

if (opnd == GET_ID) {
    PcdHalt();
    return 0;
}
else if (opnd == READ_CARD) {
    status=PcdAuthState(PICC_AUTHENT1A,KuaiN,PassWd,MLastSelectedSnr);
    if(status!=MI_OK) {
        pr_info("read authorize card err\n");
        return -EFAULT;
    }
    status=PcdRead(KuaiN,Read_Data);
    if(status!=MI_OK) {
        pr_info("read card err\n");
        return -EFAULT;
    }
}

```

```

        else {
            int i;
            memcpy(pdata, Read_Data, sizeof(Read_Data));
            /*wake_up_interruptible(&rc522_wait);*/
            pr_info("read block %d info:", KuaiN);
            for(i = 0; i < 16; i++) {
                pr_info("%02.2X",pdata[i]);
            }
            pr_info("\n");
        }
    }
}

```

对于 CHANGE_KEY, 先验证块密码, 然后将新密码写入, 这里为了避免误操作, 默认的 NewKey 数值也为 0xff, 感兴趣的读者可以尝试编写修改密码和用户输入并验证密码部分。对于 WRITE_CARD, 先验证块密码, 然后写入新值。最后统一执行休眠函数 PcdHalt。

```

else if (opnd == CHANGE_KEY) {
    status=PcdAuthState(PICC_AUTHENT1A,KuaiN,PassWd,MLastSelectedSnr);
    if(status!=MI_OK) {
        pr_info("card authorize err\n");
        return -EFAULT;
    }
    status=PcdWrite(KuaiN,&NewKey[0]);
    if(status!=MI_OK) {
        pr_info("change password err\n");
        return -EFAULT;
    } else pr_info("set password success\n");
} else if (opnd == WRITE_CARD) {
    status=PcdAuthState(PICC_AUTHENT1A,KuaiN,PassWd,MLastSelectedSnr);
    if(status!=MI_OK) {
        pr_info("write authrioze err\n");
        return -EFAULT;
    }
    status=PcdWrite(KuaiN,&WriteData[0]);
    if(status!=MI_OK) {
        pr_info("write data err\n");
        return -EFAULT;
    } else pr_info("write data to block %d sucess\n", KuaiN);
}
PcdHalt();
return 0;
}

```

5) 模块初始化和卸载

首先创建 MISC 设备结构体对象，用于设备注册。

```
static struct miscdevice rc522_misc_device = {
    .minor = MISC_DYNAMIC_MINOR,
    .name = "rfid_dev",
    .fops = &rc522_fops,
};
```

然后进行模块初始化代码的编写，首先配置 5 个引脚的复用配置寄存器，然后申请 5 个 GPIO 口配置数据方向并赋初值，最后注册 MISC 设备，并初始化 RC522 相关配置。

```
static int __init RC522_init(void)
{
    int ret;
    u32 GPCFG1_read, GPCFG1_write;
    u32 GPCFG2_read, GPCFG2_write;
    pr_info("rfid_rc522 module init.\n");

    GPCFG1_V = ioremap(GPCFG1, sizeof(u32));
    GPCFG1_read = ioread32(GPCFG1_V);
    GPCFG1_write = (GPCFG1_read & 0x00077777);
    iowrite32(GPCFG1_write, GPCFG1_V);

    GPCFG2_V = ioremap(GPCFG2, sizeof(u32));
    GPCFG2_read = ioread32(GPCFG2_V);
    GPCFG2_write = (GPCFG2_read & 0x77777700);
    iowrite32(GPCFG2_write, GPCFG2_V);

    gpio_request(GPIO_RST, "rst");
    ret = gpio_direction_output(GPIO_RST, 0);
    if(ret < 0) {
        printk("can't set rst gpio!\r\n");
        return ret;
    }

    gpio_request(GPIO_CS, "cs");
    ret = gpio_direction_output(GPIO_CS, 1);
    if(ret < 0) {
        printk("can't set rst gpio!\r\n");
        return ret;
    }
}
```

```

        gpio_request(GPIO_CLK, "clk");
        ret = gpio_direction_output(GPIO_CLK, 1);
        if(ret < 0) {
            printk("can't set rst gpio!\r\n");
            return ret;
        }
        gpio_request(GPIO_MISO, "miso");
        ret = gpio_direction_input(GPIO_MISO);
        if(ret < 0) {
            printk("can't set miso gpio!\r\n");
            return ret;
        }
        gpio_request(GPIO_MOSI, "mosi");
        ret = gpio_direction_output(GPIO_MOSI, 1);
        if(ret < 0) {
            printk("can't set mosi gpio!\r\n");
            return ret;
        }
        ret = misc_register(&rc522_misc_device);
        if(ret < 0) {
            pr_info("device register failed with %d.\n",ret);
            return ret;
        }

        KuaiN = 1;
        PcdReset();
        PcdAntennaOff();    // 关闭天线
        mdelay(10);
        PcdAntennaOn();    // 开启天线
        mdelay(10);
        return 0;
    }

```

编写卸载函数，取消虚拟地址映射，释放 GPIO 口，并撤销 MISC 设备。最后注册模块初始化和卸载函数。

```

static void __exit RC522_exit(void) {
    pr_info("rc522 driver exit\n");
    iounmap(GPCFG1_V);
    iounmap(GPCFG2_V);

    gpio_free(GPIO_RST);
    gpio_free(GPIO_CS);
}

```

```

        gpio_free(GPIO_CLK);
        gpio_free(GPIO_MISO);
        gpio_free(GPIO_MOSI);
        misc_deregister(&rc522_misc_device);
    }
    module_init(RC522_init);
    module_exit(RC522_exit);
    MODULE_AUTHOR("yangjinrun");
    MODULE_LICENSE("Dual BSD/GPL");

```

15.3.3 测试程序

这里给出一个寻卡-读取卡号-读取块-写块-再次读取块的例程。首先包含头文件，声明命令码。

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <sys/types.h>
#include <unistd.h>
#include <errno.h>
#include <arpa/inet.h>
#include <sys/time.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <math.h>
static enum IO_CMD {
    READ_CARD = 3,
    CHANGE_PASSWD = 1,
    CHANGE_BLOCK = 2,
    CHANGE_KEY = 5,
    WRITE_CARD = 4,
    GET_ID = 6,
};

```

开启 RC522，然后等待 1 秒 RC522 初始化。

```

int main(int argc, char** argv)
{
    int rc522_fd;

```

```

int i, read_num, write_num;
char r[256];
char a[16];
rc522_fd = open("/dev/rfid_dev", O_RDWR);
printf("test: rc522_fd=%d\n", rc522_fd);
if(rc522_fd == -1) {
    printf("test: Error Opening rc522\n");
    return(-1);
}
printf("test: wait begin\n");
sleep(1); //wait
printf("test: wait done\n");

```

读取到卡后会先打印第 0 块的当前值，然后写入 3 位 1，再次读取卡后结束程序。

```

ioctl(rc522_fd, GET_ID, &(a[0])); //参数 3: 选第 0 块 */
printf("%d%d%d%d", a[0], a[1], a[2], a[3]);
while (1) {
    read_num = read(rc522_fd, r, 0);
    printf("read_num=%d\n", read_num);
    for (i = 0; i < read_num; i++) printf("%x ", r[i]);
    printf("\n");
    if(read_num > 0){
        write_num = write(rc522_fd, "\1\1\1", 3);
        printf("write_num=%d\n", write_num);
        read_num = read(rc522_fd, r, 0);
        printf("read again read_num=%d\n", read_num);
        for (i = 0; i < read_num; i++) printf("%x ", r[i]);
        printf("\n");
        break;
    }
    sleep(1);
}
close(rc522_fd);
return 0;
}

```

15.4 练习：NFC 的基本读写

本节中，学生需要通过命令行或程序实现对卡片内容的读写操作，并验证数据的正确性。

任务要求：

1.NFC 卡内容写入：设计存储字符串指令，使用上位机通过 RS485 发送指令到试验箱，试验箱识别指令并向 NFC 卡中写入” Hello world”。

2.NFC 卡内容读取：编写代码，刷卡后读取 1 中 NFC 卡存储位置，正确读取到” Hello world” 并将内容打印到控制台。

第16章 湿度传感器模块实验

16.1 实验目的

了解湿度传感器的方法及控制方式。掌握教学平台上湿度传感器的功能特性，明确其连接方法与数据结构。

根据湿度传感器的工作原理与输出模式编写程序，实现简单的湿度监控功能，如实时显示湿度值并在湿度高于或低于某个阈值时输出报警信息。

16.2 AHT20 简介

AHT20 是一款完全标定、高性价比、长期稳定性优异、响应迅速、抗干扰能力强的温湿度传感器，其嵌入了适于回流的双列扁平无引脚 SMD 封装，传感器输出经过标定的数字信号，使用标准 I2C 格式。

AHT20 广泛应用于智能家居、消费电子、医疗、汽车、工业、气象等领域，手册中给出的最大工作范围和建议工作范围（正常范围）如图 16-1，长期工作在正常范围以外可能导致信号暂时性漂移（60 小时后偏移+3%RH），恢复正常范围后会缓慢自恢复到校正状态。

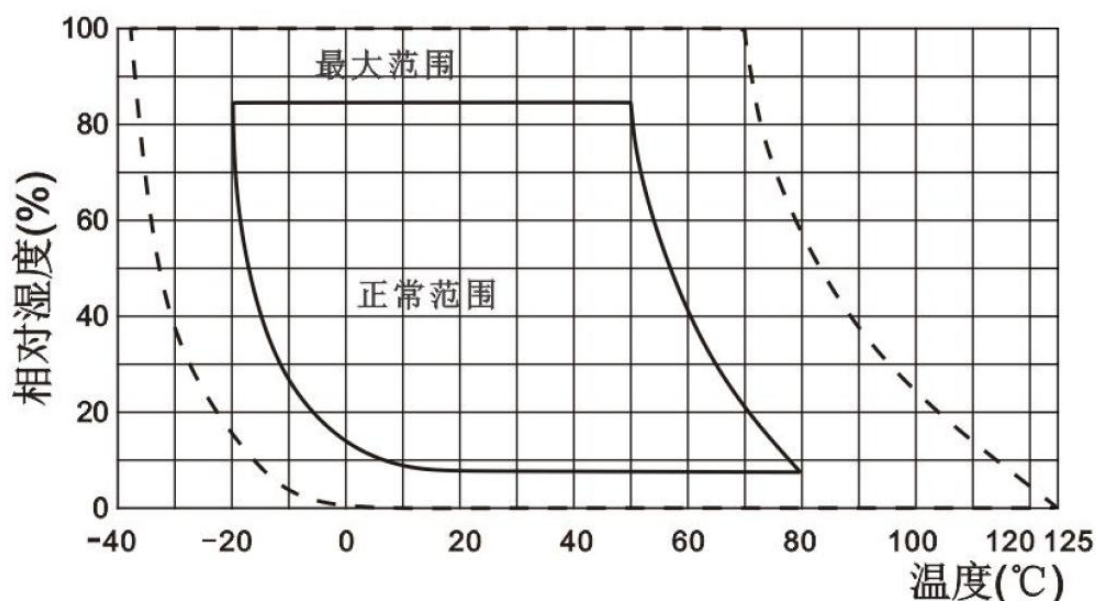


图 16-1 AHT 工作范围

16.3 实验步骤

16.3.1 内核模块代码编写

- 1) 依赖头文件

```

#include <linux/types.h>
#include <linux/kernel.h>
#include <linux/delay.h>
#include <linux/ide.h>
#include <linux/init.h>
#include <linux/module.h>
#include <linux/errno.h>
#include <linux/gpio.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/of.h>
#include <linux/of_address.h>
#include <linux/of_gpio.h>
#include <linux/platform_device.h>
#include <linux/miscdevice.h>
#include <linux/i2c.h>
#include <asm/uaccess.h>
#include <asm/io.h>

```

2) AHT20 读写函数

AHT20 使用标准 I2C 格式通信，I2C 设备的通用读写函数已在重力传感器实验中完整介绍，这里仍然使用之前编写的 I2C 读写函数，仅需对设备名、子设备号（只要是主设备号为 10 的杂项设备中未使用的子设备号均可）、设备地址进行修改，因为 AHT20 在本开发板上仍挂载在 I2C3 总线上，所以使用的 GPIO 端口仍为 68 和 69。

AHT20 使用标准 I2C 格式，因此每次通信的第一个字节要发送设备地址+读/写位，根据手册内容可以得到设备地址为 0x38。

```

#define DEVICE_NAME      "aht20"    /* 名字 */
#define DEVICE_MINOR     147        /* 子设备号 */

#define SDA_GPIO         68
#define SCL_GPIO         69

```

设备私有数据结构体也和之前相似。

```

struct aht20_dev {
    dev_t devid;          /* 设备号 */
    void *private_data;   /* 私有数据 */
    int sda_gpio;
    int scl_gpio;
    unsigned char write_addr;
    unsigned char read_addr;
};

```

手册中对 AHT20 通信格式的说明如图 16-2:

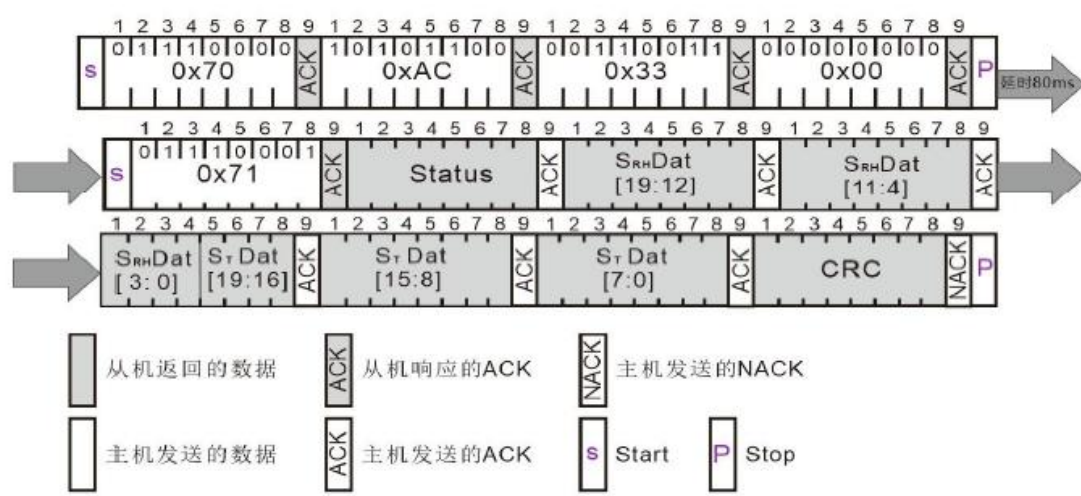


图 16-2 AHT20 通信格式

在手册中并未明确提到设备地址为 0x38，但可以由写地址为 0x70、读地址为 0x71 分析得出设备地址为 $(0x70 \gg 1) = 0x38$ 。

```
#define DEVICE_ADDR    0x38
#define DEVICE_WRITE_ADDR((DEVICE_ADDR << 1) | 0)
#define DEVICE_READ_ADDR ((DEVICE_ADDR << 1) | 1)
```

通过通信图,可知从 AHT20 获取温湿度数据的流程如下:发送写设备命令后,先后发送 0xAC, 0x33, 0x00 作为测量指令,等待 80ms 测量结束后,发送读数据命令,然后连续接收 7 字节内容,其中 1 字节状态位、20 位温度数据、20 位湿度数据、1 字节 CRC 校验码。

根据该流程,可抽象出发送指令、读取数据两个方法:

```
void AHT20_SendCmd( unsigned char *cmd_buf, size_t cnt ){
    int i;
    I2c_Start();    // 启动总线
    I2c_WrByte(my_dev.write_addr);
    for ( i = 0; i < cnt; i++ ) I2c_WrByte( ( unsigned char )( cmd_buf[i] ) );
    I2c_Stop();    // 结束总线
}

void AHT20_Recv( unsigned char *buf, size_t cnt ) {
    int i;
    I2c_Start();
    I2c_WrByte(my_dev.read_addr);
    for ( i = 0; i < cnt; i++ ) buf[i] = I2c_RdByte(cnt - i - 1);
    I2c_Stop();
}
```

这两个方法提供了较高的自由度,可以发送任意字节的指令、接收任意字节

的数据，需要注意的是在接收数据时，通信说明中规定接收完最后一字节后响应 NACK，在之前编写的 I2c_RdByte() 方法中，传入非 0 数据均发送 ACK，传入 0 发送 NACK，因此这里传入的值为 cnt-i-1，在接收最后一字节时传入 0。

最后在 read 方法中编写总流程，进入函数后会进行短暂延迟，然后发送测量指令，延迟 80ms 后，接收 7 字节数据并拷贝到用户空间，不在内核中进行解码工作。

```
static ssize_t aht20_read(struct file *filp, char __user *buf, size_t cnt, loff_t *offt){
    unsigned char databuf[7], cmd[3];
    mdelay(50);
    cmd[0] = 0xac;
    cmd[1] = 0x33;
    cmd[2] = 0x00;
    AHT20_SendCmd(cmd, 3);
    mdelay(80);
    AHT20_Recv(databuf, 7);
    return copy_to_user(buf, databuf, cnt);
}
```

因为只将 AHT20 作为输出设备，所以无需覆写 write 方法，仅需编写 open、read、release 即可。

```
static int aht20_open(struct inode *inode, struct file *filp) {
    filp->private_data = &my_dev; /* 设置私有数据 */
    return 0;
}

static int aht20_release(struct inode *inode, struct file *filp) {
    return 0;
}
```

3) 模块注册和卸载

首先配置 misc 设备结构体：

```
static struct file_operations aht20_ops = {
    .owner = THIS_MODULE,
    .open = aht20_open,
    .read = aht20_read,
    .release = aht20_release,
};

static struct miscdevice aht20_miscdev = {
    .minor = DEVICE_MINOR,
    .name = DEVICE_NAME,
    .fops = &aht20_ops,
};
```

编写模块初始化函数，首先进行 misc 设备的注册。

```
#define GPCFG 0x1fe104b0
static void __iomem *GPCFG_V;
static int __init aht20_init(void)
{
    int ret;
    u32 GPCFG_read, GPCFG_write;

    ret = misc_register(&aht20_miscdev);
    if(ret < 0){
        printk("misc device register failed!\r\n");
        return -EFAULT;
    }
}
```

然后配置复用配置寄存器，将 GPIO68、69 配置为 GPIO 模式（000）。

```
GPCFG_V = ioremap(GPCFG, sizeof(u32));
GPCFG_read = ioread32(GPCFG_V);
GPCFG_write = (GPCFG_read & 0x77007777);
iowrite32(GPCFG_write, GPCFG_V);
```

最后配置设备私有数据对象的管脚和读写地址，申请并初始化 GPIO 管脚为输出模式。

```
my_dev.sda_gpio = SDA_GPIO;
my_dev.scl_gpio = SCL_GPIO;
my_dev.write_addr = DEVICE_WRITE_ADDR;
my_dev.read_addr = DEVICE_READ_ADDR;

gpio_request(my_dev.sda_gpio, "sda3");
gpio_request(my_dev.scl_gpio, "scl3");
ret = gpio_direction_output(my_dev.sda_gpio, 1);
if(ret < 0) {
    printk("can't set sda gpio!\r\n");
}

ret = gpio_direction_output(my_dev.scl_gpio, 1);
if(ret < 0) {
    printk("can't set scl gpio!\r\n");
}
return ret;
}
```

编写模块卸载函数，撤销虚拟地址映射，注销 misc 设备，并释放申请的 GPIO 端口。

最后注册模块的初始化、卸载函数，便完成内核模块代码编写。

```
static void __exit aht20_exit(void) {  
    iounmap(GPCFG_V);  
    misc_deregister(&aht20_miscdev);  
    gpio_free(my_dev.sda_gpio);  
    gpio_free(my_dev.scl_gpio);  
}  
module_init(aht20_init);  
module_exit(aht20_exit);  
MODULE_LICENSE("GPL");  
MODULE_AUTHOR("yangjinrun");
```

16.3.2 测试代码编写指南

首先声明所需的数据，并打开设备，因为温度、湿度数据各占 20 位，所以至少需要用 4 字节的整型类型变量接收。

```
int main(int argc, char *argv[]) {  
    int fd;  
    char *filename = "/dev/aht20";  
    unsigned char readByte[6]; // 忽略 CRC 校验  
    int ret = 0, i;  
    unsigned int H1=0; // Humidity  
    unsigned int T1=0;  
    fd = open(filename, O_RDWR);  
    if(fd < 0) {  
        printf("can't open file %s\r\n", filename);  
        return -1;  
    }  
}
```

手册中可查阅到 20 位数据的解码方式，注意这里前一段的计算结果为浮点值，如果想确保更高的计算精度，建议仍使用整型计算，即提前将两个算式中括号后的 100 和 200 提前乘到括号中，并加乘所需分辨率，计算方式如图 16-3。

计算温湿度值：
相对湿度转换公式：

$$RH[\%] = \left(\frac{S_{RH}}{2^{20}} \right) * 100\%$$

温度转换公式：

$$T[^\circ\text{C}] = \left(\frac{S_T}{2^{20}} \right) * 200 - 50$$

示例：S_T: 0x2FFAB, 转换为十进制数 196523, T = (196523/1048576)*200-50 = -12.5°C

图 16-3 计算温湿度值

手册中有说明 AHT20 对湿度、温度测量的分辨率分别为 0.024%RH 和 0.01℃，那么就可以考虑让湿度精确到 1 位小数，即 0.1%RH，将“xxx.x”的结果转换为整数则需要额外乘 10，最终得到的湿度公式为 $(SRH * 1000) \gg 20$ ，得到的十进制结果的千、百、十、个位则表示 xxx.x% 中对应的数字。

同理，考虑温度如果想保留 2 位小数，那么 ST 就要先乘 20000 再除以 2^{20} ，但得到的数据是 20 位，如果使用 4 字节整型存储，则剩余 $32 - 20 = 12$ 位，而 $2^{12} = 4096$ ，所以乘以 4096 以上的数值就有上溢风险，因此可以考虑在计算时使用强制类型转换转为 64 位整数进行计算，最后右移 20 位解除上溢风险。但为兼容性考虑，实验例程使用 32 位整型进行处理，将温度精确到 0.1℃。

注意在解码数据时，第 4 个字节中高 4 位是湿度的低 4 位，获取湿度数据时可以直接靠移位屏蔽掉低 4 位的温度数据，而低 4 位是温度的高 8 位，在获取温度数据时记得将高 4 位置 0。

结束程序时，记得关闭打开的设备。

```
while (1) {
    ret = read(fd, readByte, sizeof(readByte));
    if((readByte[0] & 0x68) == 0x08) { /* 数据读取成功 */
        H1 = readByte[1];
        H1 = (H1 << 8) | readByte[2];
        H1 = (H1 << 8) | readByte[3];
        H1 = H1 >> 4;

        H1 = (H1 * 1000) >> 20;

        T1 = readByte[3];
        T1 = T1 & 0x0000000F;
        T1 = (T1 << 8) | readByte[4];
        T1 = (T1 << 8) | readByte[5];

        T1 = ((T1 * 2000) >> 20) - 500;
        printf("Temp:%d%d.%d\r\n", T1/100, (T1/10)%10, T1%10);
        printf("Hum:%d%d.%d\r\n", H1/100, (H1/10)%10, H1%10);
    }
    else
    {
        printf("数据读取失败\n");
    }
    sleep(2);
}
close(fd); /* 关闭文件 */
return 0;
}
```

16.4 练习：温湿度传感器的基本应用

本节中，学生需要将温湿度传感器读取到的数据与其他模块进行结合，如显示、报警等。

任务要求：

1. 数据记录与显示：编写一个定时读取温湿度数据的功能，每隔一定时间记录当前的数据，并保存到文件中或显示在数码管上。

2. 温湿度阈值报警：为温湿度传感器数据添加上下限阈值，当温度或湿度超过设定值时，LED 灯闪烁或蜂鸣器发出警报提示。

第17章 WIFI 无线网卡使用实验

17.1 实验目的

了解 WIFI 的基本概念与原理，掌握其配置与使用。学习教学平台上 WIFI 模块的功能特性，明确其连接端口与数据结构。

根据 WIFI 模块的实际功能，实现网卡设备的发现、启动和对无线网络的扫描、连接等功能。

17.2 WIFI 模块简介

试验箱中使用的 wifi 模块为 BL-R8723BT1 模块，其直接接线至 USB2.0，支持 USB 无线网卡，WiFi 符合标准 IEEE802.11b/g/n，频率范围为 2.400GHz-2.4835GHz，原理图如图 17-1。

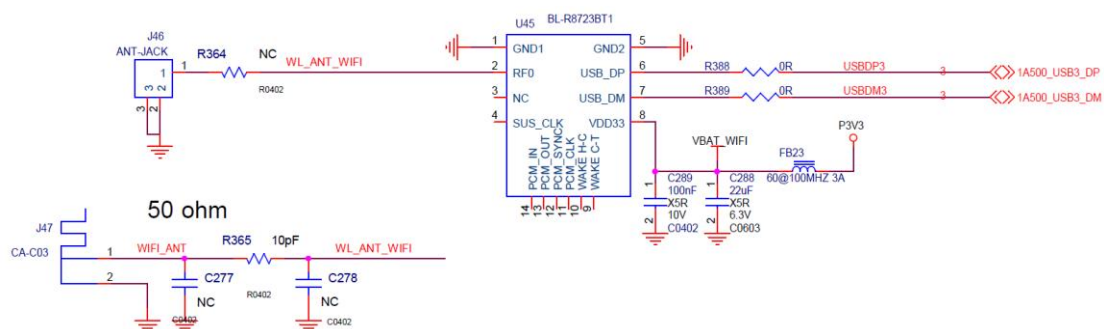


图 17-1 WIFI 模块原理图

实验中使用的设备为 RTL8188CU 网卡，建议使用网卡芯片产商为 Realtek 的网卡，即 RTL 为前缀的芯片型号，如 rtl8192xx、rtl8188xx、rtl8723xx 等，内核支持 Realtek 的驱动版本较多，配置较方便，具体可在实验步骤的内核配置中查看内核支持的芯片型号。不推荐使用免驱版网卡，因为内核会将其视为存储设备，需要通过 usb_modeswitch 或其他工具进行转换后才能使用，如想要使用免驱版网卡，需自行进行 usb_modeswitch 等工具的交叉编译，并在开发板中进行 USB 模式的切换（可以参考其文档，或 README 等文件）。

图 17-2 为指导书中使用的网卡，网卡型号为 RLT8188CU，为 Realtek 官方生产的网卡。



图 17-2 RLT8188CU 网卡

实验主要通过命令行对网卡设备进行启动、配置、连接等操作。

17.3 实验步骤

17.3.1 内核配置与编译

试验箱使用的内核默认没有开启 USB 无线网卡功能，需要手动开启。

在试验箱使用内核的根目录（编译模块使用的内核目录）下开启终端，输入 make menuconfig 后回车，进入内核可视化配置页面。

在 menuconfig 下，方向键上下用于切换选项，空格用于确定是否配置某个功能，"*"为编译进内核，"M"为编译为模块，为空则不编译进内核。方向键左右用于切换操作，回车用于执行操作，如果某个目录存在子目录（选项后有"--->"），使用 Select 操作可以进入子目录，Exit 用于返回上级目录，Help 查看帮助，Sava 和 Load 用于保存和读取配置（可指定配置文件名）。本实验不以内核配置为主，仅介绍内核配置的主要功能，如图 17-3。

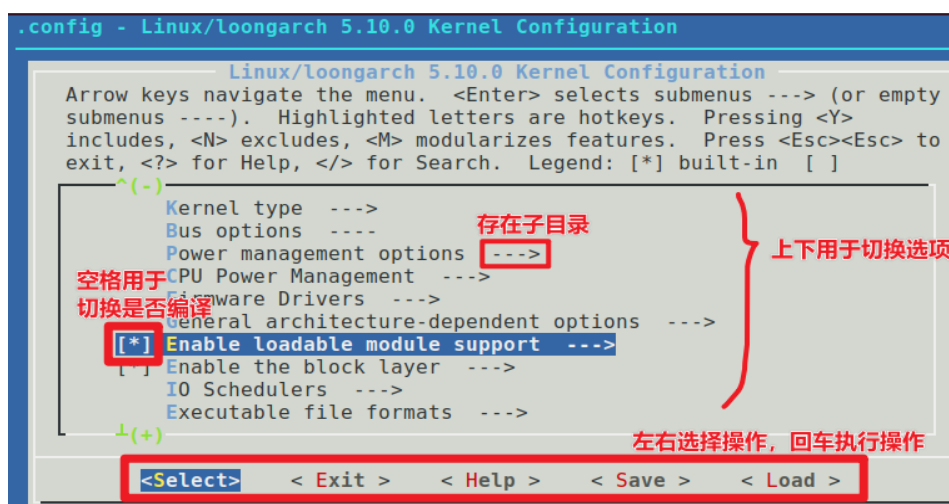


图 17-3 menuconfig 配置

首先进入目录 Device Drivers，进入 USB support 配置，如果 USB support 左侧方括号内不为"*"，将光标移到其上并按空格将其配置为"*"，即编译进内核，然后才能配置其子配置项。

后续操作中如果出现部分选项不存在，同理请先开启上层配置选项，然后才会显示其子配置项（同一页面中也可能存在配置的依赖关系），如图 17-4。

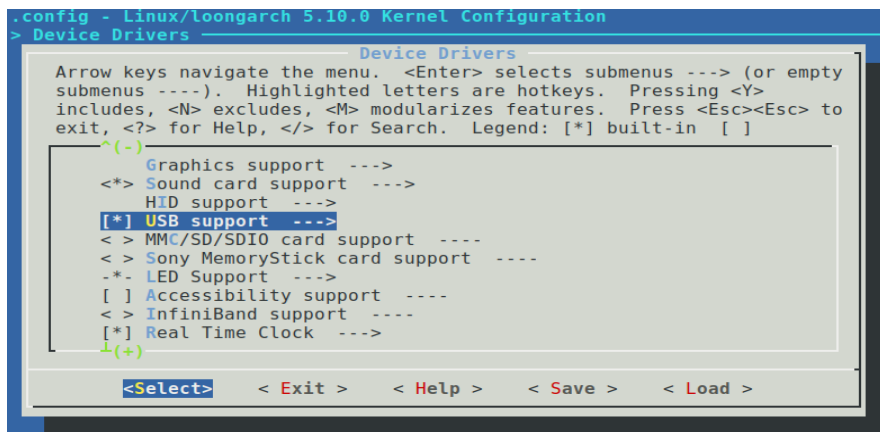


图 17-4 menuconfig 配置

然后配置 USB 支持设备，如图 17-5~17-8:

Device Drivers --->

[*] USB support --->

- <*> Support for Host-side USB
- <*> EHCI HCD (USB 2.0) support
- <*> OHCI HCD (USB 1.1) support
- <*> ChipIdea Highspeed Dual Role Controller
- [*] ChipIdea host controller
- [*] ChipIdea driver debug

```
--- USB support
[ ] USB LED Triggers
-*- USB ULPI PHY interface support
< > USB GPIO Based Connection Detection Driver
<*> Support for Host-side USB
[*] PCI based USB host interface
[ ] USB announce new devices
*** Miscellaneous USB options ***
[*] Enable USB persist by default
[ ] Limit USB device initialization to only a few retries
+ (+)
```

图 17-5 menuconfig 配置

```
<*> USB Monitor
*** USB Host Controller Drivers ***
< > Cypress C67x00 HCD support
< > xHCI HCD (USB 3.0) support
<*> EHCI HCD (USB 2.0) support
-*- Root Hub Transaction Translators
[*] Improved Transaction Translator scheduling
< > Support for Freescale on-chip EHCI USB controller
<*> Generic EHCI driver for a platform device
< > OXU210HP HCD support
+ (+)
```

图 17-6 menuconfig 配置

```
^ (-)
< > OXU210HP HCD support
< > ISP116X HCD support
< > FOTG210 HCD support
< > MAX3421 HCD (USB-over-SPI) support
<*> OHCI HCD (USB 1.1) support
< > OHCI support for PCI-bus USB controllers
<*> Generic OHCI driver for a platform device
< > UHCI HCD (most Intel and VIA) support
< > SL811HS HCD support
< > R8A66597 HCD support
+ (+)
```

图 17-7 menuconfig 配置

```
^ (-)
<*> Synopsys PCIe-based HAPS Platforms
<*> Generic OF Simple Glue Layer
< > DesignWare USB2 DRD Core Support
<*> ChipIdea Highspeed Dual Role Controller
[*] ChipIdea device controller
[*] ChipIdea host controller
< > NXP ISP 1760/1761 support
*** USB port drivers ***
< > USB Serial Converter support ----
*** USB Miscellaneous drivers ***
+ (+)
```

图 17-8 menuconfig 配置

再配置支持 Wifi 设备，请注意到达该配置项的路径. 如图 17-9:

```
Device Drivers --->
  [*] Network device support --->
    [*] Wireless LAN --->
      <*> IEEE 802.11 for Host AP (Prism2/2.5/3 and
WEP/TKIP/CCMP)
        [*] Support downloading firmware images with Host
AP driver
        [*] Support for non-volatile firmware download
```

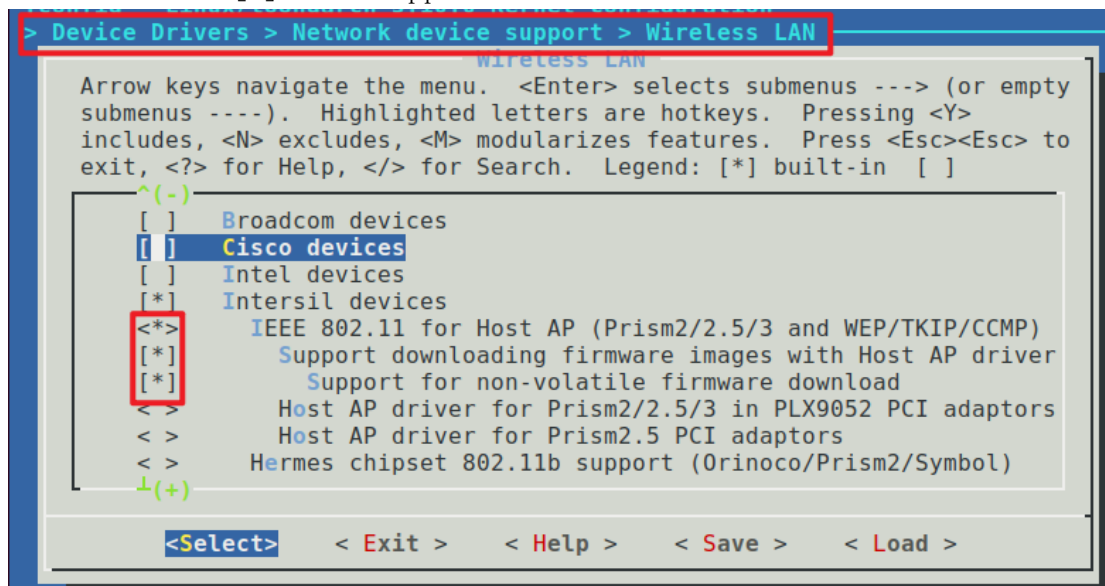


图 17-9 menuconfig 配置

配置支持 IEEE 802.11，注意要返回首层，如图 17-10:

```
[*] Networking support --->
  -- Wireless --->
    <*> cfg80211 - wireless configuration API
      [*] cfg80211 wireless extensions compatibility
    <*> Generic IEEE 802.11 Networking Stack (mac80211)
```

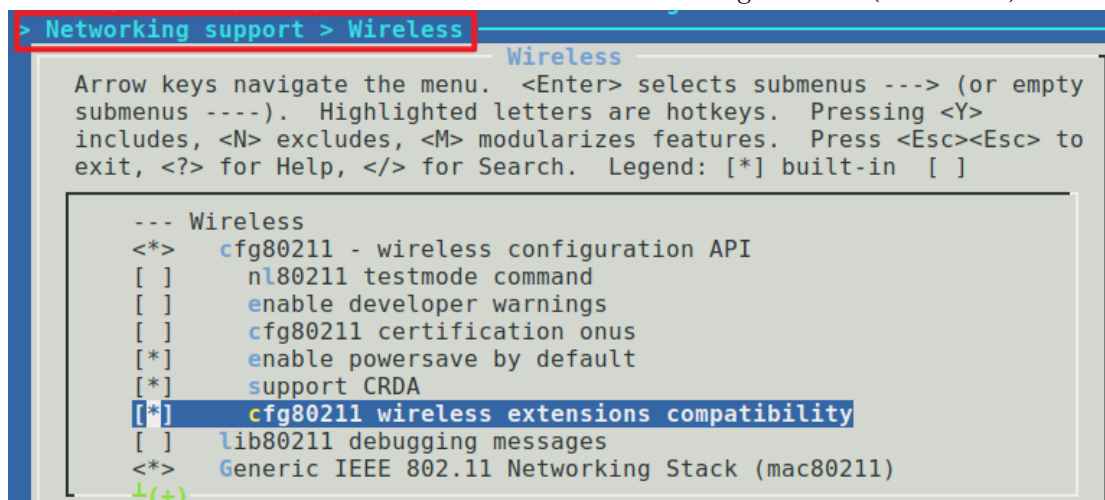


图 17-10 menuconfig 配置

最后配置网卡驱动，建议使用的网卡的芯片对应驱动在内核中默认支持（可

以通过查询网卡的产品编号来查询网卡的对应芯片及所需驱动，如 WN823N 需要 rtl8192fu 驱动），如果没有对应的网卡驱动，可以自行下载驱动源码，将其复制到内核中并配置 Kconfig，然后在 menuconfig 中选择新增的驱动将其编译进内核即可。

因为测试时使用的是 RTL8188CU 网卡，因此配置路径如下，如图 17-11：

```
Device Drivers --->
[*] Network device support --->
    [*] Wireless LAN --->
        <*> Realtek wifi famliy of devices --->
            <*> Realtek RTL8192CU/RTL8188CU USB .....
```

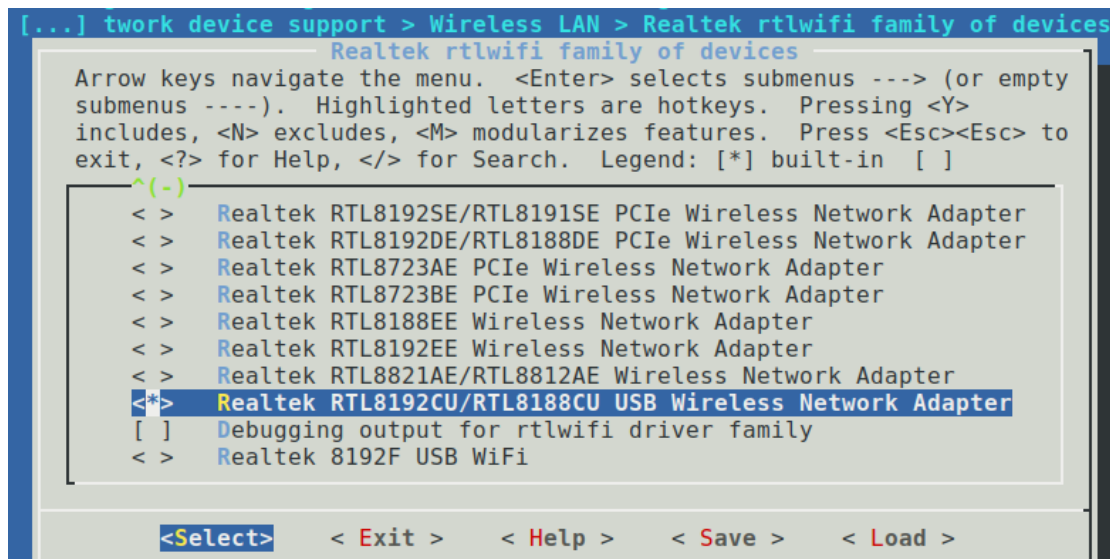


图 17-11 menuconfig 配置

建议使用 Realtek 芯片的网卡，内核支持的对应芯片种类较多。如果使用的是其他芯片的网卡，可以在上级目录中查找，开启对应 xxx devices 选项即可看到其子配置项，可以打开配置后查看其子配置项有什么芯片的驱动，找到网卡对应芯片后选择编译进内核即可。

上一级目录中也包括部分 Realtek 芯片的驱动，如果对应的芯片在这里能找到可以配置进内核，如图 17-12。

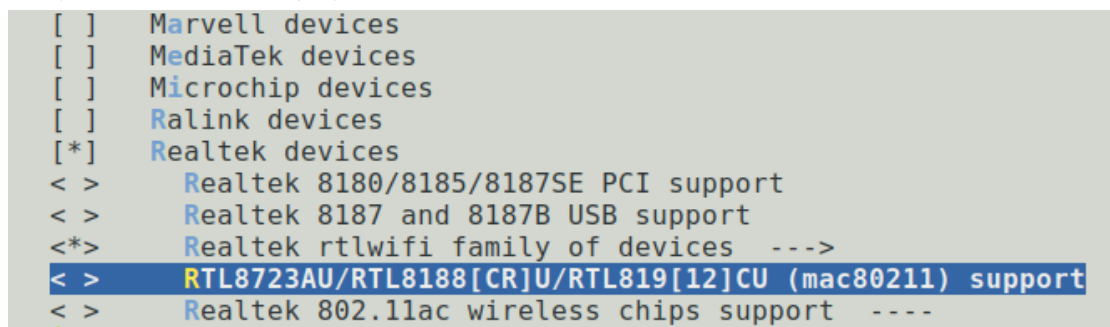


图 17-12 menuconfig 配置

然后逐级返回，退出根目录时选择 Yes 保存配置，如图 17-13。

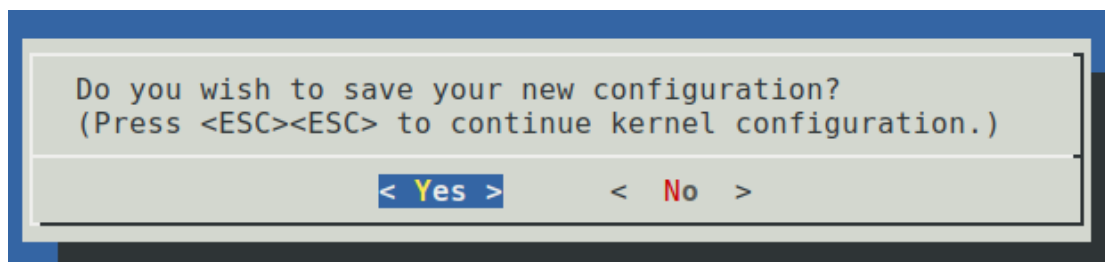


图 17-13 保存配置并退出

随后的操作与正常编译内核相同，使用 `make uImage` 命令编译镜像，将得到的内核镜像烧录进开发板即可。

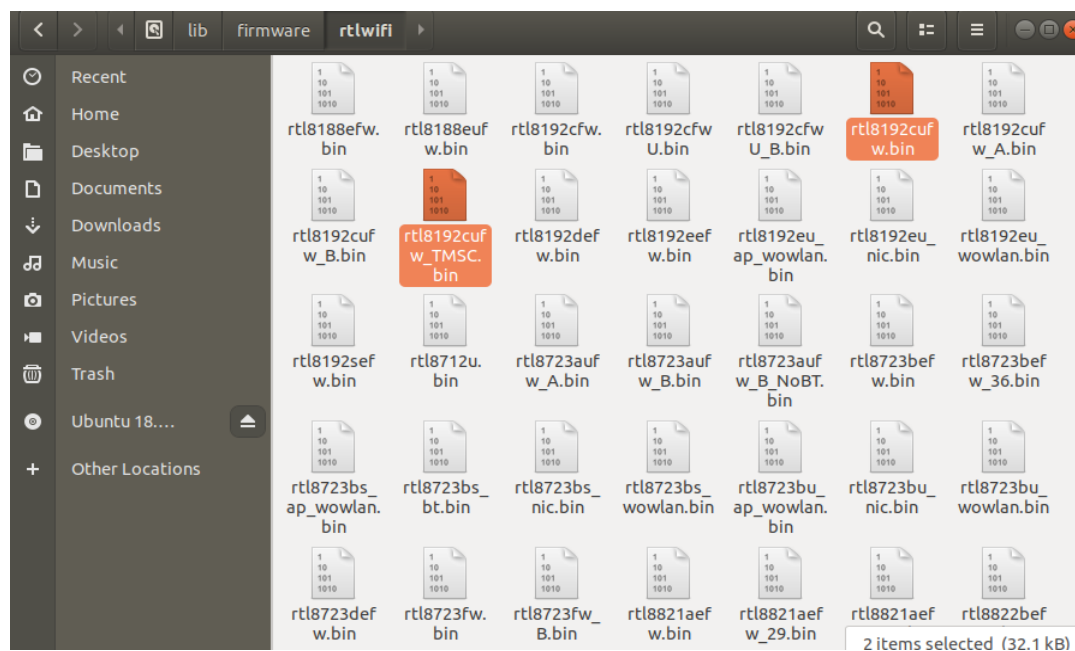
17.3.2 配置网卡并连接到网络

启动烧录新内核后的开发板，插入网卡后查看输出的信息，如图 17-14。

```
[ 819.157477] usb 1-3: new high-speed USB device number 6 using ehci-platform
[ 819.322987] rtl8192cu: Chip version 0x10
[ 819.428755] rtl8192cu: Board Type 0
[ 819.432577] rtl usb: rx_max size 15360, rx_urb_num 8, in_ep 1
[ 819.439592] rtl8192cu: Loading firmware rtlwifi/rtl8192cufw_TMSC.bin
[ 819.475133] usb 1-3: Direct firmware load for rtlwifi/rtl8192cufw_TMSC.bin failed with error -2
[ 819.513687] usb 1-3: Direct firmware load for rtlwifi/rtl8192cufw.bin failed with error -2
[ 819.559072] rtlwifi: Loading alternative firmware rtlwifi/rtl8192cufw.bin
[ 819.577606] rtlwifi: Selected firmware is not available
```

图 17-14 插入网卡打印信息

开发板有可能对 `rtlwifi/rtl8192xxxx.bin` 报错，这是因为缺少网卡固件，可以直接在虚拟机的 ubuntu 中 `/lib/firmware/rtlwifi/` 下找到对应固件，如图 17-15，将其复制到开发板的 `/lib/firmware/rtlwifi/` 下（如果没有 `firmware` 和 `rtlwifi` 目录，直接创建即可）即可。



```
[root@LS-GD rtlwifi]# ls
rtl8192cufw.bin  rtl8192cufw_TMSC.bin
[root@LS-GD rtlwifi]#
```

图 17-15 寻找对应固件

插入网卡，输出信息如图 17-16：

```
[ 1086.077539] usb 1-3: new high-speed USB device number 7 using ehci-platform
[ 1086.242904] rtl8192cu: Chip version 0x10
[ 1086.349627] rtl8192cu: Board Type 0
[ 1086.354246] rtl_usb: rx_max size 15360, rx_urb_num 8, in_ep 1
[ 1086.360151] rtl8192cu: Loading firmware rtlwifi/rtl8192cufw_TMSC.bin
```

图 17-16 输出信息

输入 `ifconfig -a`，如果配置顺利，则会显示如图 17-17 信息，说明成功识别到网卡。

```
wlan0      Link encap:Ethernet  HWaddr FC:8F:C4:0C:15:0B
            BROADCAST MULTICAST  MTU:1500  Metric:1
            RX packets:0 errors:0 dropped:0 overruns:0 frame:0
            TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
            collisions:0 txqueuelen:1000
            RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

[root@LS-GD ~]#
```

图 17-17 成功识别网卡

输入 `ifconfig wlan0 up`，启动网卡，如图 17-18，如果在这一步报错，请检查固件文件。

```
[root@LS-GD ~]# ifconfig wlan0 up
[ 118.685954] rtl8192cu: MAC auto ON okay!
[ 118.731351] rtl8192cu: Tx queue select: 0x05
[root@LS-GD ~]#
```

图 17-18 启动网卡

因为内核里的 `iwconfig` 工具功能较差，不支持 WPA 密码，如果想要使用带密码的 wifi 则需要借助其他工具，如 `wpa_supplicant` 等，而使用其他工具需要工具源码和交叉编译，较复杂，这里推荐使用无密码的 wifi 进行测试。

开启一个无密码的 wifi/热点，输入 `iwconfig wlan0 essid "name"`，name 为热点名称，如果先开启的 wifi/热点，再进行的配置，会打印如下信息，说明成功连接，如图 17-19。

```
[root@LS-GD ~]# iwconfig wlan0 essid "HTSheep"
[root@LS-GD ~]# [ 1293.337982] wlan0: authenticate with 1e:32:07:de:d5:95
[ 1293.380624] wlan0: send auth to 1e:32:07:de:d5:95 (try 1/3)
[ 1293.489599] wlan0: send auth to 1e:32:07:de:d5:95 (try 2/3)
[ 1293.508547] wlan0: authenticated
[ 1293.513929] wlan0: associate with 1e:32:07:de:d5:95 (try 1/3)
[ 1293.621585] wlan0: associate with 1e:32:07:de:d5:95 (try 2/3)
[ 1293.633663] wlan0: RX AssocResp from 1e:32:07:de:d5:95 (capab=0x1421 status=0 aid=2)
[ 1293.647507] IPv6: ADDRCONF(NETDEV_CHANGE): wlan0: link becomes ready
[ 1293.654041] wlan0: associated
```

图 17-19 打印信息

再输入 `iwconfig wlan0 ap auto`，配置接入点为自动选择，如图 17-20。

```
[root@LS-GD ~]# iwconfig wlan0 ap auto
[root@LS-GD ~]#
```

图 17-20 配置接入点

再进行 IP、子网掩码和网关的配置，在其他设备连接该 wifi/热点，查看网关 ip，以 windows 为例，如图 17-21：

```
无线网络适配器 WLAN:

连接特定的 DNS 后缀 . . . . . :
描述. . . . . : Intel(R) Wi-Fi 6 AX201 160MHz
物理地址. . . . . :
DHCP 已启用 . . . . . : 是
自动配置已启用. . . . . : 是
IPv4 地址 . . . . . : 192.168.232.26(首选)
子网掩码 . . . . . : 255.255.255.0
获得租约的时间 . . . . . : 2023年11月25日 20:10:09
租约过期的时间 . . . . . : 2023年11月25日 21:10:08
默认网关. . . . . : 192.168.232.138
DHCP 服务器 . . . . . : 192.168.232.138
DNS 服务器 . . . . . : 192.168.232.138
TCP/IP 上的 NetBIOS . . . . . : 已启用
```

图 17-21 查看网关

查看到网关（gateway）为 192.168.232.138、子网掩码为 255.255.255.0，则可以选择 IP 地址为 192.168.232.x，输入 `ifconfig wlan0 192.168.232.28 netmask 255.255.255.0`，以手动配置 IP 和子网掩码。注意这里的网关和掩码要根据实际的值修改，如图 17-22。

```
[root@LS-GD ~]# ifconfig wlan0 192.168.232.28 netmask 255.255.255.0
[root@LS-GD ~]#
```

图 17-22 配置 IP 和子网掩码

然后配置默认网关，输入 `route add default gw 192.168.232.138`，网关 IP 需根据实际情况修改，使用 `route -n` 以验证，如图 17-23。

```
[root@LS-GD ~]# route add default gw 192.168.232.138
[root@LS-GD ~]# route -n
Kernel IP routing table
Destination      Gateway          Genmask         Flags Metric Ref    Use Iface
0.0.0.0          192.168.232.138 0.0.0.0         UG    0      0      0 wlan0
192.168.232.0    0.0.0.0         255.255.255.0   U     0      0      0 wlan0
[root@LS-GD ~]#
```

图 17-23 配置默认网关

此时使用 `ping` 命令查看本机和网关 IP 是否能 ping 通，并查看外网是否能 ping 通（如 `www.baidu.com`），均能 ping 通则说明配置成功，如图 17-24。

```
[root@LS-GD ~]# ping 192.168.232.28
PING 192.168.232.28 (192.168.232.28): 56 data bytes
64 bytes from 192.168.232.28: seq=0 ttl=64 time=0.370 ms
64 bytes from 192.168.232.28: seq=1 ttl=64 time=0.345 ms

[root@LS-GD ~]# ping 192.168.232.138
PING 192.168.232.138 (192.168.232.138): 56 data bytes
64 bytes from 192.168.232.138: seq=0 ttl=64 time=22.618 ms
64 bytes from 192.168.232.138: seq=1 ttl=64 time=21.907 ms
```

```
[root@LS-GD ~]# ping www.baidu.com
PING www.baidu.com (110.242.68.3): 56 data bytes
64 bytes from 110.242.68.3: seq=0 ttl=52 time=40.233 ms
64 bytes from 110.242.68.3: seq=1 ttl=52 time=59.083 ms
```

图 17-24 ping 测试

如果出现域名无法 ping 通的情况, 请检查 DNS 配置, 即/etc/resolv.conf, 可以选择在最前面增加 nameserver <其他设备连接此 wifi 使用的 DNS 地址>, 如上面 windows 的查询结果中默认 DNS 和网关为同一地址, 则如下配置 (根据实际情况配置), 如图 17-25。

```
[root@LS-GD ~]# cat /etc/resolv.conf
# This file is managed by man:systemd-resolved(8). Do not edit.
#
# This is a dynamic resolv.conf file for connecting local clients directly to
# all known uplink DNS servers. This file lists all configured search domains.
#
# Third party programs should typically not access this file directly, but only
# through the symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a
# different way, replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.
nameserver 192.168.232.138 根据情况配置
nameserver 114.114.114.114
nameserver 8.8.8.8
# Too many DNS servers configured, the following entries may be ignored.
nameserver 1.1.1.1
nameserver 1.0.0.1
nameserver 8.8.4.4
search .
```

图 17-25 检查 DNS 配置

如果需要 DNS 配置永久生效, 则编辑/etc/systemd/resolved.conf, 将 DNS 选项取消注释, 并依次输入 DNS 地址, 以空格分隔, 配置完后重启即可, 可以达到启动自动配置/etc/resolv.conf 的效果, 如图 17-26。

```
[root@LS-GD ~]# cat /etc/systemd/resolved.conf
# This file is part of systemd.
#
# systemd is free software; you can redistribute it and/or modify it
# under the terms of the GNU Lesser General Public License as published by
# the Free Software Foundation; either version 2.1 of the License, or
# (at your option) any later version.
#
# Entries in this file show the compile time defaults.
# You can change settings by editing this file.
# Defaults can be restored by simply deleting this file.
#
# See resolved.conf(5) for details

[Resolve]
# Some examples of DNS servers which may be used for DNS= and FallbackDNS=:
# Cloudflare: 1.1.1.1 1.0.0.1 2606:4700:4700::1111 2606:4700:4700::1001
# Google:      8.8.8.8 8.8.4.4 2001:4860:4860::8888 2001:4860:4860::8844
# Quad9:       9.9.9.9 2620:fe::fe
DNS=192.168.232.138 114.114.114.114 8.8.8.8 1.1.1.1 1.0.0.1 8.8.4.4
FallbackDNS=1.1.1.1 8.8.8.8 1.0.0.1 8.8.4.4 2606:4700:4700::1111 2001:4860:4860::8888 2606:4700:4700::1001 2001:4860:4860::8844
```

图 17-26 使 DNS 配置永久生效

如果仍然无法 ping 通域名, 但可以 ping 通域名对应的 ip 地址 (可以用其他能 ping 通域名的设备查看域名对应的 ip 地址), 可能是设备连接稳定性较

差，可以重新插拔、重新配置后再次测试。

17. 3. 3 TCP 程序测试

使用一个在局域网内进行 TCP 通信的程序进行测试，测试程序如下，将开发板作为客户端，实现了数据的接收和发送，注意编译时需要增加-lpthread 选项。

```
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <arpa/inet.h>
#include <netinet/in.h>
#include <pthread.h>
typedef struct sockaddr SA;
typedef struct sockaddr_in SIN;
typedef struct {
    int sockfd;
    SIN server_conn;
} ServerInfo;
```

```
void *task_read(void *arg)
{
    ServerInfo *server = (ServerInfo *)arg;
    char *server_ip = inet_ntoa(server->server_conn.sin_addr);
    while (1)
    {
        char read_buff[512] = {0};
        int len = read(server->sockfd, read_buff, sizeof(read_buff));
        if (len > 0)
        {
            printf("ServerIP:%s 接收信息: %s\n", server_ip, read_buff);
        }
        else
        {
            printf("ServerIP:%s 已断开连接\n", server_ip);
            pthread_exit(NULL);
        }
    }
}
```

```

int main(int argc,char *argv[]){
    int sockfd = socket(AF_INET, SOCK_STREAM, 0);

    if (argc < 3) {
        printf("arg error!\n");
        return -1;
    }

    ServerInfo server = {0};
    server.sockfd = sockfd;
    server.server_conn.sin_family = AF_INET;    //ipv4
    server.server_conn.sin_port = htons(atoi(argv[2])); //参数传递端口号 port
    server.server_conn.sin_addr.s_addr = inet_addr(argv[1]); //参数传递 ip 地址
    connect(sockfd, (SA *)&server.server_conn, sizeof(SA));
    pthread_t id = 0;
    pthread_create(&id, NULL, task_read, (void *)&server);
    while (1) {
        char write_buff[512] = {0};
        scanf("%s", write_buff);
        write(sockfd, write_buff, strlen(write_buff));
    }
    return 0;
}

```

首先主机和开发板需要连到同一个局域网下，可以是手机热点。主机新建 TCP Server，以 TCP/UDP Socket 调试工具为例，创建监听端口为 9999 的服务器端，如图 17-27。



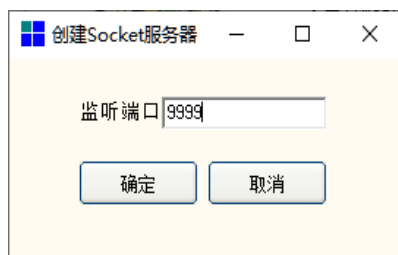


图 17-27 配置 socket 调试工具

在开发板上连接到主机端的 IP+监听端口，这里主机端的局域网 IP 为 192.168.232.26，开发板的 IP 为 192.168.232.28。发送数据给主机，主机可以正确接收，如图 17-28、17-29。

```
[root@LS-GD wifi]# ./client 192.168.232.26 9999
123465165342
456f4sadsf
```

图 17-28 发送数据

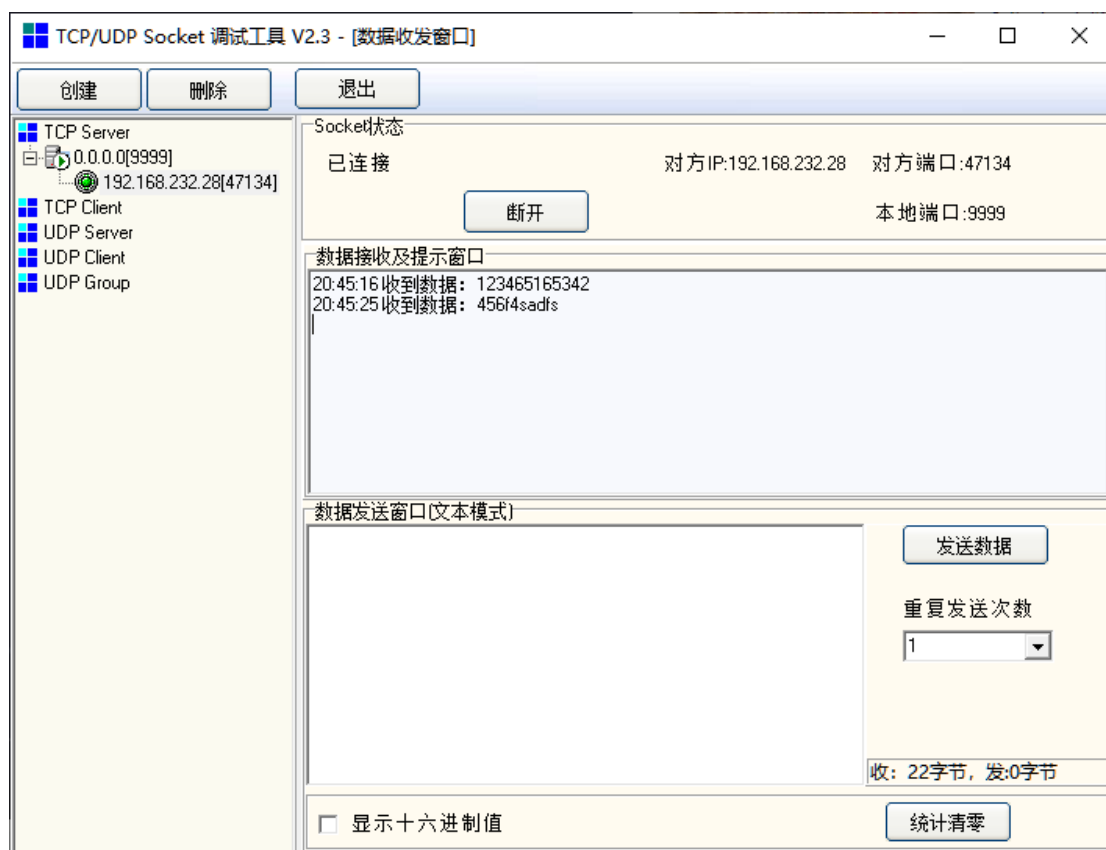


图 17-29 调试工具打信息

主机发送数据给客户端，客户端可以正常接收，如图 17-30。



图 17-30 主机发送数据给客户端

如果其中一个方向的通信出现问题，请关闭主机防火墙再次尝试。

17.4 练习：设备控制和多设备通信

本节中学生需要通过局域网进行设备控制，并通过服务端广播的方式实现多设备通信及多设备控制。

任务要求：

1. 基础局域网通信：确认开发板已通过无线网卡连接至同一局域网。
2. 编写代码，实现通过局域网接收来自上位机的控制指令，控制开发板上的设备，如数码管显示特定数字或 LED 灯状态改变。

扩展任务：

1. 多设备通信：将开发板和上位机通信拓展到多设备通信，一个开发板作为服务端，其余开发板和上位机作为客户端；服务端保存所有客户端 IP，在接收到任一客户端信息后，将其转发到所有其他客户端。
2. 多设备协同控制：通过服务端发送指令，控制所有接入的客户端的设备，如 LED 灯状态改变。（因为可以只需要客户端进行指令解析，所以可以将上位机 IP 作为控制 IP，服务端接收到后转发控制信息到其他所有客户端，并且服务端也同时执行命令，开发板不足时可以这样解决）

第18章 矩阵键盘模块实验

18.1 实验目的

了解矩阵键盘的基本功能与原理。掌握教学平台上矩阵键盘的连接方式、支持的输入模式与输出的数据结构。

根据矩阵键盘的功能特性编写程序，实现一个点击计数程序，每按下一次键盘就可以在数码管上进行一次数字更新。也可以使用矩阵键盘对其他设备进行控制，如 LED 灯，蜂鸣器等。

18.2 实验步骤

18.2.1 矩阵键盘模块简介

龙芯试验箱上具备 4x4 矩阵键盘模块用于输入设备的交互，它由 16 个按钮组成，每个按钮都是型号为 1BT003-26100 的按钮。该矩阵键盘采用了行列扫描的方式来检测按钮的状态。在键盘模块上，有 4 条行线和 4 条列线与 GPIO 引脚相连。具体地，行线与 GPIO103 到 GPIO106 相对应，而列线与 GPIO107 到 GPIO110 相对应。通过使用矩阵键盘模块，可以方便地进行按钮输入的检测和处理。通过扫描行列的方式，可以检测到用户按下的具体按钮，并据此执行相应的操作，原理图如图 18-1。

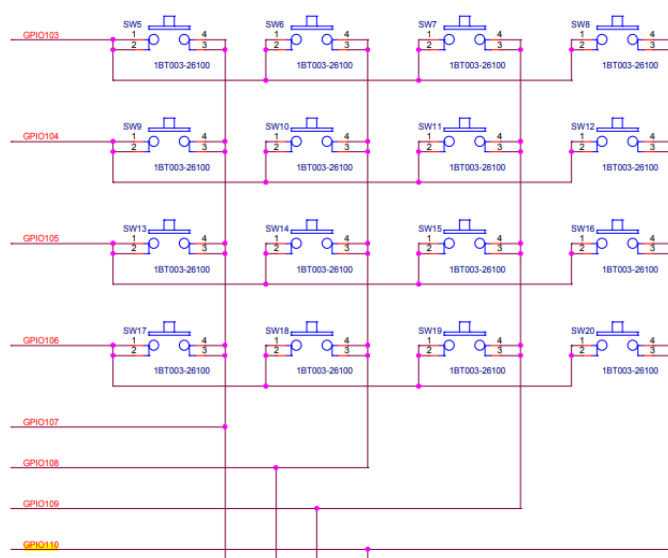


图 18-1 矩阵键盘原理图

18.2.2 内核驱动代码编写

18.2.2.1 引入依赖头文件

```
#include <linux/types.h>
#include <linux/kernel.h>
```

```

#include <linux/delay.h>
#include <linux/ide.h>
#include <linux/init.h>
#include <linux/module.h>
#include <linux/errno.h>
#include <linux/gpio.h>
#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/of.h>
#include <linux/of_address.h>
#include <linux/of_gpio.h>
#include <linux/semaphore.h>
#include <asm/uaccess.h>
#include <asm/io.h>

```

18.2.2.2 定义设备结构体

key_gpio[8], key 所使用的 GPIO 编号, 一个包含 8 个整数的数组, 用于存储与按键相关的 GPIO 引脚编号; keyvalue: 按键值, 使用原子类型的变量来存储按键的状态值。

```

/* key 设备结构体 */
struct key_dev{
    dev_t devid;          /* 设备号 */
    struct cdev cdev;      /* cdev */
    struct class *class;   /* 类 */
    struct device *device; /* 设备 */
    int major;             /* 主设备号 */
    int minor;             /* 次设备号 */
    struct device_node *nd; /* 设备节点 */
    int key_gpio[8];        /* key 所使用的 GPIO 编号 */
    atomic_t keyvalue;      /* 按键值 */
};
struct key_dev keydev;    /* key 设备 */

```

18.2.2.3 驱动入口函数

然后注册字符设备驱动。首先创建设备号, 如果主设备号已定义, 则可以直接用主设备号通过 MKDEV() 宏获取设备号, 再进行设备号的注册。未定义主设备号则会进入 else 语句, 会申请一个设备号, 然后通过设备号获取主设备号和次设备号, 保存在设备结构体中。

```

static int __init mykey_init(void)
{
    /* 初始化原子变量 */
    atomic_set(&keydev.keyvalue, INVAKEY);

    /* 注册字符设备驱动 */
    /* 1、创建设备号 */
    if (keydev.major) {          /* 定义了设备号 */
        keydev.devid = MKDEV(keydev.major, 0);
        register_chrdev_region(keydev.devid, KEY_CNT, KEY_NAME);
    } else {                    /* 没有定义设备号 */
        alloc_chrdev_region(&keydev.devid, 0, KEY_CNT, KEY_NAME); /* 申请设备号 */
        keydev.major = MAJOR(keydev.devid); /* 获取分配号的主设备号 */
        keydev.minor = MINOR(keydev.devid); /* 获取分配号的次设备号 */
    }
}

```

再对设备结构体中的 cdev、class、device 结构体进行初始化,使用 key_fops 对 cdev 进行初始化并添加 cdev, 最后创建类和设备后模块便初始化完毕。

```

    /* 2、初始化 cdev */
    keydev.cdev.owner = THIS_MODULE;
    cdev_init(&keydev.cdev, &key_fops);

    /* 3、添加一个 cdev */
    cdev_add(&keydev.cdev, keydev.devid, KEY_CNT);

    /* 4、创建类 */
    keydev.class = class_create(THIS_MODULE, KEY_NAME);
    if (IS_ERR(keydev.class)) {
        return PTR_ERR(keydev.class);
    }

    /* 5、创建设备 */
    keydev.device = device_create(keydev.class, NULL, keydev.devid, NULL, KEY_NAME);
    if (IS_ERR(keydev.device)) {
        return PTR_ERR(keydev.device);
    }

    return 0;
}

```

```

/* 设备操作函数 */
static struct file_operations key_fops = {
    .owner = THIS_MODULE,
    .open = key_open,
    .read = key_read,
    .write = key_write,
    .release = key_release,
};

```

18.2.2.4 驱动出口函数

解除之前通过 `ioremap` 函数映射，遍历 8 个 GPIO 引脚，调用 `gpio_free` 函数释放这些 GPIO 资源。从内核中删除字符设备结构 `cdev`，注销之前分配的设备号，销毁之前创建的设备节点，销毁之前创建的设备类。

```

static void __exit mykey_exit(void){
    int i;
    iounmap(gpio_111_104_v);
    iounmap(gpio_103_96_v);
    /* 注销字符设备驱动 */
    for(i=0;i<8;i++)
    {
        gpio_free(keydev.key_gpio[i]);
    }
    cdev_del(&keydev.cdev);/* 删除 cdev */
    unregister_chrdev_region(keydev.devid, KEY_CNT);/* 注销设备号 */

    device_destroy(keydev.class, keydev.devid);
    class_destroy(keydev.class);
}

module_init(mykey_init);
module_exit(mykey_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR("yangjinrun");

```

18.2.2.5 设备初始化函数

定义设备所使用的 GPIO 口的物理地址和存储设备 GPIO 口经过映射后的虚拟地址所需变量

```

#define GPIO_96_103 0x1fe104c0
#define GPIO_104_111 0x1fe104c4

static void __iomem *gpio_103_96_v;
static void __iomem *gpio_111_104_v;

```

将 GPIO 口从物理地址空间映射到虚拟地址空间

```

static int keyio_init(void) {
    int ret;
    u32 gpio_read,gpio_write;
    gpio_103_96_v = ioremap(GPIO_96_103,sizeof(u32));
    gpio_read = ioread32(gpio_103_96_v);
    gpio_write = (gpio_read & 0x07777777);
    iowrite32(gpio_write,gpio_103_96_v);

    gpio_111_104_v = ioremap(GPIO_104_111,sizeof (u32));
    gpio_read = ioread32(gpio_111_104_v);
    gpio_write = (gpio_read & 0x70000000);
    iowrite32(gpio_write,gpio_111_104_v);
}

```

设置设备 GPIO 数组

```

keydev.key_gpio[0] = 103;
keydev.key_gpio[1] = 104;
keydev.key_gpio[2] = 105;
keydev.key_gpio[3] = 106;
keydev.key_gpio[4] = 107;
keydev.key_gpio[5] = 108;
keydev.key_gpio[6] = 109;
keydev.key_gpio[7] = 110;

```

如果设备已经被初始化过，则不再重新初始化

```

if(g_flag)
    goto L1;

```

请求 GPIO 口资源以及设定 GPIO 口输出使能寄存器状态，将列线设置为输出模式，将行线设置为输入模式。当按键按下时，导通，即可在行线对应的 GPIO 口中看到寄存器状态从 0 变为 1，初始化完成后，将 flag 置 1 防止重复初始化

```

/* 初始化 key 所使用的 IO */
ret = gpio_request(keydev.key_gpio[0], "key0"); /* 请求 IO */
gpio_direction_input(keydev.key_gpio[0]); /* 设置为输入 */
ret = gpio_request(keydev.key_gpio[1], "key1"); /* 请求 IO */
gpio_direction_input(keydev.key_gpio[1]); /* 设置为输入 */
ret = gpio_request(keydev.key_gpio[2], "key2"); /* 请求 IO */
gpio_direction_input(keydev.key_gpio[2]); /* 设置为输入 */
ret = gpio_request(keydev.key_gpio[3], "key3"); /* 请求 IO */
gpio_direction_input(keydev.key_gpio[3]); /* 设置为输入 */
ret = gpio_request(keydev.key_gpio[4], "key4"); /* 请求 IO */
gpio_direction_output(keydev.key_gpio[4], 0); /* 设置为输出 */
ret = gpio_request(keydev.key_gpio[5], "key5"); /* 请求 IO */
gpio_direction_output(keydev.key_gpio[5], 0); /* 设置为输出 */
ret = gpio_request(keydev.key_gpio[6], "key6"); /* 请求 IO */
gpio_direction_output(keydev.key_gpio[6], 0); /* 设置为输出 */
ret = gpio_request(keydev.key_gpio[7], "key7"); /* 请求 IO */
gpio_direction_output(keydev.key_gpio[7], 0); /* 设置为输出 */
g_flag = 1;
L1:
return 0;
}

```

18. 2. 2. 6 设备打开函数 open

该函数比较简单，主要实现两个功能，其一设置设备私有数据，其二调用了设备初始化函数完成初始化

```

static int key_open(struct inode *inode, struct file *filp){
    int ret = 0;
    filp->private_data = &keydev; /* 设置私有数据 */
    ret = keyio_init(); /* 初始化按键 IO */
    if (ret < 0) {
        return ret;
    }
    return 0;
}

```

18. 2. 2. 7 设备读函数 read

从设备的 8 个 GPIO 口中依次读出数据，拷贝到用户空间

```

static ssize_t key_read(struct file *filp, char __user *buf, size_t cnt, loff_t *offt) {
    int ret = 0;
    int keyvalue[8];
    int i;
    struct key_dev *dev = filp->private_data;
    for(i=0;i<8;i++)
    {
        keyvalue[i] = gpio_get_value(dev->key_gpio[i]);
    }
    ret = copy_to_user(buf, keyvalue, sizeof(keyvalue));
    return ret;
}

```

18.2.2.8 设备写函数

GPIO 口中 4 条列线设置为输出模式，因此只有列线对应的 GPIO 能被设定数据，所以仅需从用户空间接收 4 个字节的数据，并将它们设定到列线对应的 GPIO 寄存器上

```

static ssize_t key_write(struct file *filp, const char __user *buf, size_t cnt, loff_t *offt) {
    int ret;
    unsigned char databuf[4];
    struct key_dev *dev = filp->private_data;
    ret = copy_from_user(databuf, buf, cnt);
    if(ret < 0) {
        printk("kernel write failed!\r\n");
        return -EFAULT;
    }
    gpio_set_value(dev->key_gpio[4], databuf[0]);
    gpio_set_value(dev->key_gpio[5], databuf[1]);
    gpio_set_value(dev->key_gpio[6], databuf[2]);
    gpio_set_value(dev->key_gpio[7], databuf[3]);
    return 0;
}

```

18.2.3 设备测试程序编写

定义必备变量以及缓冲区

```

int main(int argc, char *argv[])
{
    int has_press = 0;
    int fd, ret;
}

```

```
char *filename;
int keyvalue[8];
unsigned char databuf[4];
```

检查传参数量

```
if (argc < 2) {
    printf("Error Usage!\r\n");
    return -1;
}
```

打开设备

```
/* 打开 key 驱动 */
fd = open("/dev/key", O_RDWR);
if (fd < 0) {
    printf("file /dev/key open failed!\r\n");
    return -1;
}
```

将传入的参数如 1000，代表将 GPIO107 设置为 1 即高电平，108 到 110 设置为低电平，也就是列线第一列拉高，其他拉低电平，检测第一列上是否有按键按下

```
strcpy(databuf, argv[1]);
for (int i = 0; i < 4; i++) {
    databuf[i] = '0';
}
ret = write(fd, databuf, sizeof(databuf));
if (ret < 0) {
    printf("Key Control Failed!\r\n");
    close(fd);
    return -1;
}
```

循环读取设备获取 GPIO 寄存器状态，当有按键按下时 GPIO103~GPIO106 也就是 keyvalue[0]~[3]会发生变化

```
/* 循环读取按键值数据! */
while (1) {
    read(fd, keyvalue, sizeof(keyvalue));
    printf("KEY0 Press, value = %#X-%#X-%#X-%#X-%#X-%#X-%#X-%#X\r\n",
        keyvalue[0], keyvalue[1], keyvalue[2], keyvalue[3],
        keyvalue[4], keyvalue[5], keyvalue[6], keyvalue[7]);
}
```

```
        sleep(1);
    }

    ret = close(fd); /* 关闭文件 */
    if (ret < 0) {
        printf("file %s close failed!\r\n", argv[1]);
        return -1;
    }
    return 0;
}
```

18.3 练习：设备控制和复杂输入

本节需要学生掌握如何使用矩阵键盘模块进行输入，并通过编程实现对不同按键操作的响应。基础任务将重点放在设备控制上，而扩展任务则涉及到更复杂的按键操作模式。

任务要求：

1. 键盘输入处理：编写代码，处理矩阵键盘的输入，并在终端显示按键值或状态。
2. 基本设备控制编写代码，实现通过矩阵键盘的按键控制开发板上的设备。例如，按下特定的按键，控制 LED 灯的开关或数码管的显示内容。

扩展任务：

复杂输入处理：实现对复杂按键操作的处理，如长按、多次快速按键等。例如，长按某个键达到特定时间启动某项特殊功能，或多次快速按键切换设备模式。

第 19 章红外遥控实验

19.1 实验目的

了解红外遥控的基本原理与控制方式。掌握教学平台上 ZigBee 模块的连接方式和输入输出模式。

根据 ZigBee 的功能特性与红外通信设计，实现红外遥控功能。使得开发板在接收到红外信号时可以做出相应反应，如 LED 灯闪烁，数码管计数变化等。

19.2 红外通信协议 NEC 简介

红外通信协议是一种基于红外线的传输技术，广泛使用的家电遥控器几乎都是采用的红外线传输技术。由于红外线的波长较短，对障碍物的衍射能力差，适合应用在需要短距离无线通信场合点对点的直接线数据传输。

本实验中采用的红外通信协议为 NEC 协议，单次发送由引导码+地址码+地址码反码+数据码+数据码反码+结束码 6 部分组成。地址码可以理解成品牌码，通常而言一个遥控器的地址码是固定的。数据码是遥控器上的按键对应的码，每一个按键对应一个数据码，接收端根据数据码来判断接收的按键是哪个。反码是校验用的，保证数据传输准确。实际编程解码时也可以不校验。

NEC 协议中以 38KHz 载波对编码后的波形进行调制，所有的数据由 0 和 1 组成，根据脉冲时间长短来编解码，逻辑 1 位宽为 2.25ms，由 562.5μs 的有效脉冲 + 1.6875ms 的空闲间隔组成；逻辑 0 的位宽为 1.12ms，由 562.5μs 的有效脉冲 + 562.5μs 的空闲间隔组成，如图 19-1。

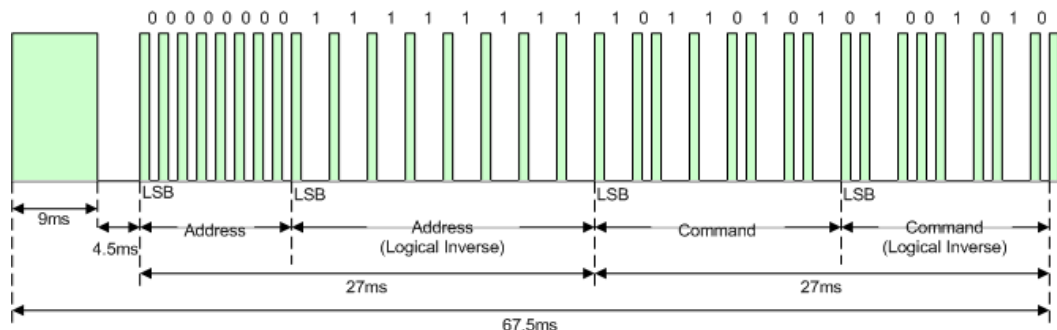


图 19-1 NEC 协议波形图

19.3 实验步骤

19.3.1 编写设备驱动函数

1) 定义数据码与遥控器对应按键的映射表 key_code.h

每一个遥控器按键由数据码，Linux 中定义的 key_code，以及按键名组成。

```
struct key_table_t {
    unsigned char key_val;
    unsigned char key_code;
    unsigned char key_name[20];
};
```

将遥控器所有的按键按照顺序依次写入程序中，制成一个查找表，同时定义按键数量

```
struct key_table_t key_table[] = {
    { 0x40, KEY_UP, "UP" },          { 0x19, KEY_DOWN, "DOWN" },
    { 0x7, KEY_LEFT, "LEFT" },       { 0x9, KEY_RIGHT, "RIGHT" },
    { 0x15, KEY_ENTER, "ENTER" },    { 0xc, KEY_1, "Num:1" },
    { 0x18, KEY_2, "Num:2" },        { 0x5e, KEY_3, "Num:3" },
    { 0x8, KEY_4, "Num:4" },          { 0x1c, KEY_5, "Num:5" },
    { 0x5a, KEY_6, "Num:6" },        { 0x42, KEY_7, "Num:7" },
    { 0x52, KEY_8, "Num:8" },        { 0x4a, KEY_9, "Num:9" },
    { 0x16, KEY_0, "Num:0" },        { 0x45, KEY_A, "A" },
    { 0x46, KEY_B, "B" },            { 0x47, KEY_C, "C" },
    { 0x44, KEY_D, "D" },            { 0x43, KEY_E, "E" },
    { 0xd, KEY_F, "F" },
};

#define KEY_NUM sizeof(key_table) / sizeof(key_table[0])
#endif
```

定义查找函数，遍历上面定义的查找表，根据数据码查找对应的按键，并返回在查找表中的索引。

```
static unsigned int get_key_table_id(unsigned int kv) {
    int id;
    for (id = 0; id < KEY_NUM; id++) {
        if (kv == key_table[id].key_val) {
            if (KEY_NUM >= id)
                return id;
        }
    }
    return 0xFFFF;
}
```

2) 引入必要头文件 irdrv.c

```
#include <linux/types.h>
#include <linux/kernel.h>
#include <linux/delay.h>
#include <linux/ide.h>
#include <linux/init.h>
#include <linux/module.h>
#include <linux/errno.h>
#include <linux/gpio.h>
```

```

#include <linux/cdev.h>
#include <linux/device.h>
#include <linux/of.h>
#include <linux/of_address.h>
#include <linux/of_gpio.h>
#include <linux/semaphore.h>
#include <linux/timer.h>
#include <linux/of_irq.h>
#include <linux/irq.h>
#include <asm/uaccess.h>
#include <asm/io.h>
#include <linux/input.h>
#include "key_code.h"

```

3) 定义必要宏及变量

IR_GPIO 用于表示与红外传感器相关的 GPIO 引脚号，定义 GPIO 寄存器所对应的地址，以及输入设备的名字。

```

#define IR_GPIO 121
#define GPCFG 0x1fe104cc
#define KEYINPUT_NAME "keyinput" /* 名字 */

```

定义了一系列用于红外驱动模块的时间参数的宏。这些宏与红外信号的持续时间有关，用于确定红外信号的高低电平持续时间的上下限。

```

#define START_TIME_MAX 11000
#define START_TIME_MIN 5000
#define L_TIME_MAX 660
#define L_TIME_MIN 360
#define L_TIME_MAX2 660
#define L_TIME_MIN2 360
#define H_TIME_MAX 1880
#define H_TIME_MIN 1480

```

为 GPCFG_V 分配合适的地址，使其指向相关的硬件寄存器或内存映射区域。

```

static void __iomem *GPCFG_V;

```

定义一个红外设备的数据结构以及全局变量，其中 ir_dev 结构体包含了设备的各种属性如设备号，字符设备结构，设备类，设备节点等，my_dev 是一个具体的设备实例。

```

struct ir_dev {
    dev_t devid; /* 设备号 */
    struct cdev cdev; /* cdev */
    struct class *class; /* 类 */
    struct device *device; /* 设备 */
    struct device_node *nd; /* 设备节点 */
    int major; /* 主设备号 */
    void *private_data; /* 私有数据 */
    int irq_num;
    struct input_dev *inputdev;
    int last_key_code;
};

static struct ir_dev my_dev;
static struct work_struct work;

```

定义一个名为 `infrared_pwm_t` 的结构体，用于表示红外波形的一些参数。同时，创建一个名为 `infrared_pwm` 的结构体实例，并进行初始化。

再定义一个名为 `nec_decode_buf_t` 的结构体，用于表示红外 NEC 数据的解析信息。同时创建一个名为 `nec_buf` 的结构体实例，并进行初始化。

```

struct infrared_pwm_t {
    long long previous; // 记录上一次的时间，64bit
    int flag; // 表示每个方波周期的开始
    long long start_time; // 周期的起始时间
    int low_time; // 低电平时间
    int high_time; // 高电平时间
};

struct infrared_pwm_t infrared_pwm = // 红外波形采集
{
    .flag = 0,
    .previous = 0,
    .start_time = 0,
    .low_time = 0,
    .high_time = 0,
};

```

定义一个名为 `nec_decode_buf_t` 的结构体，用于表示红外 NEC 数据的解析信息。同时创建一个名为 `nec_buf` 的结构体实例，并进行初始化。

```

/*----- 红外 NEC 数据解析结构体 -----*/
struct nec_decode_buf_t {
    int flag; // 表示 NEC 数据开始
    unsigned times[128]; // 记录每帧的时间
    int num; // 表示第几帧
};

struct nec_decode_buf_t nec_buf = {
    .flag = 0,
    .num = 0,
};

```

定义接收红外数据的缓冲区，不包括开头的引导码，所以是 5 个字节

```
u8 InfraredRxBuff[5] = { 0 }; //存放红外线接收的数据值，其中[4]表示标志位。=0 失败，=1 成功
```

4) 定义设备红外解码函数 `infrared_nec_decode`

`repeat` 表示当前是否接收重复码，即用户是否长按，`id` 用于保存当前按键在查找表中的索引

```

static void infrared_nec_decode(int period, struct ir_dev *dev)
{
    int i, j;
    unsigned char temp;
    int repeat = 0;
    int id = 0xFFFF;
}

```

如果当前波的起始时间差大于 13000ns 小于 14000ns 则视为接收到引导码，视为一个新的周期，开始接收之后传输的红外数据

```

if ((period > 13000) && (period < 14000)) {
    nec_buf.flag = 1;
    nec_buf.num = 0;
    repeat = 0;
    dev->last_key_code = 0;
    return;
}

```

如果当前数据缓冲区没有存满 4 个字节即 32bit，则将新来的数据存储到缓冲区中并记录波形持续时间

```

if (nec_buf.num < 32) {
    nec_buf.times[nec_buf.num++] = period;
}

```

如果存满了 32bit 且处于正确的状态下, 则对 32bit 的缓冲区中的时间数据进行解码, 根据波形持续的时间将他们视为 0 或者 1 并组装成 4 个字节放入解码 buffer 中。解码 buffer 中第三个字节为数据码, 调用查找函数查找出对应的按键在查找表中的 ID。将当前的 ID 放到设备私有数据中

```
if (nec_buf.num == 32 && nec_buf.flag) {
    for (i = 0; i < 4; i++) // 一共 4 个字节
    {
        temp = 0;
        for (j = 0; j < 8; j++) {
            if ((nec_buf.times[i * 8 + j] > 2100) &&
                (nec_buf.times[i * 8 + j] < 2500)) {
                temp |= 1 << j;
            }
        }
        InfraredRxBuff[i] = temp;
    }

    id = get_key_table_id(InfraredRxBuff[2]);
    dev->last_key_code = id;
    printk("Rx: %02x  %02x  %02x  %02x", InfraredRxBuff[0],
           InfraredRxBuff[1], InfraredRxBuff[2], InfraredRxBuff[3]);
    printk("id: %d\r\n", id);
    nec_buf.flag = 0;
}
```

如果当前周期大于 10500ns 小于 13500ns, 视为接收到重复信号, 如果不在接收红外数据的周期中, 将缓冲区全置位 0xff, 并标明当前 repeat=1。如果处于 repeat 也就是接收重复信号的情况下, 将 id 设为设备最近一次接收到的 id

```
if ((period > 10500) && (period < 13500)) {
    if (!nec_buf.flag) {
        // printk(PRINTK_GRADE "Repetitive signal\n");
        memset(InfraredRxBuff, 0xFF, sizeof(InfraredRxBuff));
        repeat = 1;
    }
}

if (repeat) {
    id = dev->last_key_code;
}
```

如果 id 不是非法 id, 则使用 input_report_key 向 linux 内核上报输入信息。

```

        if (id != 0xFFFF) {
            input_report_key(my_dev.inputdev, key_table[id].key_code,
                            1); //report press
            udelay(20);
            input_report_key(my_dev.inputdev, key_table[id].key_code,
                            0); //report release
            input_sync(my_dev.inputdev);
        }
    }
}

```

5) 定义设备中断函数 `infrared_interrupt`

定义两个时间差变量，包含上一次接收到信号产生中断的时间差以及当前波形的持续时间，调用内核获取到当前时间并保存。如果当前红外波 `flag=0`，则开始接收一个新的红外波

```

static irqreturn_t infrared_interrupt(int irq, void *dev) {
    unsigned previous_offset; // 上一次的时间
    unsigned start_offset; // 波型的起始时间差
    long long now = ktime_to_us(ktime_get());

    /* 从当前时刻开始接收一个下降沿开始的方波周期 */
    if (0 == infrared_pwm.flag) {
        infrared_pwm.start_time = now;
        infrared_pwm.flag = 1;
    }
}

```

计算当前中断和上一次中中断的时间差，并保存现在的时间

```

/* 计算两次下降沿的时差 */
previous_offset = now - infrared_pwm.previous;
infrared_pwm.previous = now;

```

由于红外接收传感器在接收到红外信号时产生一个 38KHz 的信号，所以接收时需要过滤，使信号变为一个低电平信号，也就是忽略信号，直接中断处理完成返回即可

```

/* 过滤红外接收器自产生 38KHz 的信号，周期大约是 26us */
if (previous_offset < 60) {
    return IRQ_HANDLED;
}

```

用现在的时间减去当前波开始的时间，得到时间差，如果当前就是波开始的时刻，则中断处理完成，返回

```

/* 下降沿开始的时差，也就是一个周期的时间 */
start_offset = now - infrared_pwm.start_time;
/* 消除上次持续的信号 */
if (start_offset == 0) {
    return IRQ_HANDLED;
}

```

如果当前时波结束的时间，则完成一个波的采集

```

/* 完成一个周期的数据采集 */
infrared_pwm.flag = 0;

```

将红外波的持续时间作为参数送入解码函数中进行解码，解码完成即中断处理完成

```

/* NEC 解码 */
infrared_nec_decode(start_offset, dev);
return IRQ_HANDLED;
}

```

6) 声明设备初始化函数 `ir_init`

映射 GPIO 寄存器地址到虚拟地址空间中，使能 GPIO 通用配置寄存器

```

static int ir_init(void)
{
    unsigned char i = 0;
    char name[10];
    int ret = 0;
    u32 GPCFG_read, GPCFG_write;

    GPCFG_V = ioremap(GPCFG, sizeof(u32));
    GPCFG_read = ioread32(GPCFG_V);
    GPCFG_write = (GPCFG_read & 0x77777707);
    iowrite32(GPCFG_write, GPCFG_V);
}

```

声明 GPIO 已被使用，并设置 GPIO 为输入模式

```

ret = gpio_request(IR_GPIO, "ir_gpio");
pr_info("ir_init====ret:%d\r\n", ret);
gpio_direction_input(IR_GPIO);

```

为红外设备的 GPIO 引脚设置中断，并注册一个中断服务程序函数 `infrared_interrupt`。当红外信号的 GPIO 引脚上升沿时，中断服务程序将被调

用。

```
my_dev.irq_num = gpio_to_irq(IR_GPIO);
//IRQF_TRIGGER_RISING
ret = request_irq(my_dev.irq_num, infrared_interrupt,
                  IRQF_TRIGGER_RISING, "ir-irq",
                  &my_dev); //IRQF_TRIGGER_FALLING|IRQF_TRIGGER_RISING
if (ret < 0) {
    printk("irq %d request failed!\r\n", my_dev.irq_num);
    return -EFAULT;
}
```

分配一个输入设备结构体，设置输入设备的名称为 KEYINPUT_NAME，设置了对键盘事件（EV_KEY）和重复事件（EV_REP）的支持。

```
my_dev.inputdev = input_allocate_device();
my_dev.inputdev->name = KEYINPUT_NAME;
my_dev.inputdev->evbit[0] = BIT_MASK(EV_KEY) | BIT_MASK(EV_REP);
for (i = 0; i < KEY_NUM; i++) {
    input_set_capability(my_dev.inputdev, EV_KEY,
                        key_table[i].key_code);
}
```

用于注册之前配置好的输入设备，将其注册到输入系统中，以便能够接收和处理键盘事件。

```
/* 注册输入设备 */
ret = input_register_device(my_dev.inputdev);
if (ret) {
    printk("register input device failed!\r\n");
    return ret;
}
return 0;
}
```

7) 设备退出函数

释放之前通过 request_irq 请求的中断，释放之前通过 gpio_request 请求的 GPIO 引脚。取消之前通过 ioremap 映射的内存区域，注销注册的输入设备且释放分配的输入设备。

```

static void __exit ir_exit(void) {
    free_irq(my_dev.irq_num, &my_dev);
    gpio_free(IR_GPIO);
    iounmap(GPCFG_V);

    /* 释放 input_dev */
    input_unregister_device(my_dev.inputdev);
    input_free_device(my_dev.inputdev);
}

module_init(ir_init);
module_exit(ir_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR("yangjinrun");

```

19.3.2 编写设备驱动测试代码

1) 引入必要头文件

```

#include "stdio.h"
#include "unistd.h"
#include "sys/types.h"
#include "sys/stat.h"
#include "sys/ioctl.h"
#include "fcntl.h"
#include "stdlib.h"
#include "string.h"
#include <poll.h>
#include <sys/select.h>
#include <sys/time.h>
#include <signal.h>
#include <fcntl.h>
#include <linux/input.h>
#include "../key_code.h"

```

2) 定义查找函数以及必要变量

定义一个 input_event 变量，存放输入事件信息

```

/* 定义一个 input_event 变量，存放输入事件信息 */
static struct input_event inpuvent;

```

```

static unsigned int get_key_table_id(unsigned int code)
{
    int id;
    for (id = 0; id < KEY_NUM; id++) {
        if (code == key_table[id].key_code) {
            if (KEY_NUM >= id)
                return id;
        }
    }
    return 0xFFFF;
}

```

3) 编写主函数

测试函数接收一个参数，打开红外遥控输入设备在内核中对应的 input/eventX，如/dev/input/event0

```

int main(int argc, char *argv[]) {
    int fd, id = 0;
    int err = 0;
    char *filename;
    filename = argv[1];
    if (argc != 2) {
        printf("Error Usage!\r\n");
        return -1;
    }
    fd = open(filename, O_RDWR);
    if (fd < 0) {
        printf("Can't open file %s\r\n", filename);
        return -1;
    }
}

```

不断循环读取 event 中的输入事件，并使用查找函数查找出来对应的按键，将按键信息打印到终端，如图 19-2。

```

while (1) {
    err = read(fd, &inputevent, sizeof(inputevent));
    if (err > 0) { /* 读取数据成功 */
        id = get_key_table_id(inputevent.code);
        switch (inputevent.type) {
            case EV_KEY:
                if (inputevent.code < BTN_MISC) { /* 键盘键值 */
                    printf("key %s %s\r\n", key_table[id].key_name,
                        inputevent.value ? "press" : "release");
                }
            }
        }
    }
}

```

```

        }
        break;
/* 其他类型的事件，自行处理 */
case EV_REL:
        break;
case EV_ABS:
        break;
case EV_MSC:
        break;
case EV_SW:
        break;
    }
} else {
    //printf("reading data failed\r\n");
}
}
return 0;
}

```

```

[ 212.273514] Rx: 00 ff 16 e9
[ 212.273533] id: 14
key Num:0 press
key Num:0 release

```

```

[ 219.923781] Rx: 00 ff 15 ea
[ 219.923801] id: 4
key ENTER press
key ENTER release

```

图 19-2 测试

19.4 练习：红外设备控制

本节练习需要学生掌握如何使用红外遥控模块进行设备控制。

任务要求：

设备控制：基于接收到的红外信号，实现对开发板上设备的控制。例如，使用遥控器的按钮来开关 LED 灯或调整数码管显示的内容。

第20章 LibTorch 构建编译实验

20.1 实验目的

了解 LibTorch 在 LoongArch64 架构下的适配背景与意义，认识交叉编译构建深度学习框架的基本流程。

掌握在 x86 架构的 Linux 主机上，通过交叉编译的方式为 LoongArch64 架构构建 LibTorch 动态库的完整方法。

20.2 LibTorch for LoongArch64 简介

LibTorch 是 PyTorch 的 C++ 前端，能够在 C++ 环境中构建和运行 PyTorch 模型，而无需依赖 Python 运行时。PyTorch 凭借直观的 API 设计和活跃的开发者社区，显著降低了算法研究与工程落地的技术门槛，是深度学习领域广受欢迎的开源框架。然而，针对 LoongArch64 架构的 PyTorch 官方社区支持暂时完全缺失，尤其是缺乏适配资源受限设备的轻量化运行时库（如 LibTorch），导致开发者无法在该架构上直接复用 PyTorch 高效的模型训练与部署流程。

通过将 LibTorch 移植到 LoongArch64 架构，可以将深度学习框架从服务器端无缝部署到龙芯嵌入式开发板上，借助 PyTorch 进行深度学习功能的开发，包括模型推理、训练以及自定义算子开发。由于 LibTorch 直接在 C++ 环境下运行，能够更充分地利用 C++ 本身的高性能特性，在资源受限的新架构设备上实现高效的深度学习计算，尤其是在边缘计算和嵌入式系统中的推理阶段，效率提升非常明显。

本章介绍在 x86 架构的 Linux 主机上，通过交叉编译的方式为 LoongArch64 架构构建 LibTorch 的完整流程。该流程已在全新虚拟机环境下测试通过，适用于 Ubuntu 18.04 LTS 和 Ubuntu 22.04 LTS 系统，构建产物为 LoongArch64 架构通用的 LibTorch 动态库。本实验详细相关文件及完整文档见 Gitee 仓库：<https://gitee.com/sdu-aes-lab/libtorch-for-loongarch64>

20.3 实验步骤

20.3.1 安装 Miniconda

首先推荐使用 Anaconda 或者 Miniconda 进行 Python 的虚拟环境创建与管理，避免影响同一台设备上其他项目的正常工作。Miniconda 提供了一个轻量级的 Python 环境管理解决方案，尤其适合需要灵活创建、隔离和管理不同项目环境的场景。

创建工作目录并下载 Miniconda 安装脚本，官方源下载命令如下：

```
mkdir -p workspace-pytorch && cd workspace-pytorch
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
```

若网速较慢，国内可以使用清华镜像源来下载：

```
wget -c https://mirrors.tuna.tsinghua.edu.cn/anaconda/miniconda/Miniconda3-py39_25.3.1-Linux-x86_64.sh
```

下载完成后，执行安装脚本。建议安装时不要使用 root 用户：

```
bash Miniconda3-latest-Linux-x86_64.sh
```

根据提示按回车继续，然后接受 license terms(许可条款)，输入 yes，回车两次进行安装：

```
Preparing transaction: done
Executing transaction: done
entry_point.py:256: DeprecationWarning: Python 3.14 will, by default, filter extracted tar archives and
reject files or modify their metadata.
Do you wish to update your shell profile to automatically initialize conda?
This will activate conda on startup and change the command prompt when activated.
If you'd prefer that conda's base environment not be activated on startup,
run the following command when conda is activated:
conda config --set auto_activate_base false

You can undo this by running `conda init --reverse $SHELL`? [yes|no]
[no] >>> no
```

安装完成后，需要设置环境变量确保终端能够调用 conda 命令，执行如下操作：

```
echo 'export PATH="$HOME/miniconda3/bin:$PATH"' >> ~/.bashrc
```

输入以下命令查看 Miniconda 版本，如果能成功显示则说明安装成功：

```
conda -V
```

20.3.2 获取 PyTorch 源码

在工作目录下下载完整的 PyTorch 及其所有依赖项的源码。PyTorch 源码依赖于多个第三方库，这些库通过 Git Submodule 的形式被包含进来，因此需要确保所有必需的外部依赖被正确下载和配置。

克隆 PyTorch 源码仓库，执行如下操作：

```
mkdir -p ~/workspace-pytorch/ && cd ~/workspace-pytorch
git clone https://github.com/pytorch/pytorch.git pytorch-source
```

```
os-liyy@os-student: ~/workspace-pytorch
(base) os-liyy@os-student:~$ mkdir -p ~/workspace-pytorch/ && cd ~/workspace-pytorch
(base) os-liyy@os-student:~/workspace-pytorch$ git clone https://github.com/pytorch/pytorch.git pytorch-source
正克隆到 'pytorch-source'...
remote: Enumerating objects: 1065112, done.
remote: Counting objects: 100% (275/275), done.
remote: Compressing objects: 100% (153/153), done.
remote: Total 1065112 (delta 216), reused 122 (delta 122), pack-reused 1064837 (from 3)
接收对象中: 100% (1065112/1065112), 1020.54 MiB | 1.23 MiB/s, 完成.
处理 delta 中: 100% (845042/845042), 完成.
(base) os-liyy@os-student:~/workspace-pytorch$
```

图 20-1 git clone 原始仓库

切换到 v1.12.0 版本并确保所有子模块同步完整克隆，执行如下操作：

```
cd pytorch-source
git checkout v1.12.0
git submodule sync
git submodule update --init --recursive --no-recommend-shallow
```

其中--init 用于初始化子模块，--recursive 用于递归更新所有嵌套的子模块，--no-recommend-shallow 确保获取完整的子模块历史。

```
子模块路径 'third_party/googletest': 检出 '5c71164113271002200c0f310000c21001'
子模块路径 'third_party/psind': 检出 '072586a71b55b7f8c584153d223e95687148a900'
子模块路径 'third_party/threadpool': 检出 'a134dd5d4cee80cce15db81a72e7f929d71dd413'
子模块路径 'third_party/pybind11': 检出 '8de7772cc72daca8e947b79b83fea46214931604'
子模块路径 'third_party/python-enum': 检出 '4cfedc426c4e2fc52e3f5c2b4297e15ed8d6b8c7'
子模块路径 'third_party/python-peachpy': 检出 'f45429b087dd7d5bc78bb40dc7cf06425c252d67'
子模块路径 'third_party/python-six': 检出 '15e31431af97e5e64b80af0a3f598d382bcd49a'
子模块路径 'third_party/sleef': 检出 'e0a003ee838b75d11763aa9c3ef17bf71a725bfff'
子模块路径 'third_party/tbb': 检出 'a51a90bc609bb73db8ea13841b5cf7aa43444da9'
子模块路径 'third_party/tensorpipe': 检出 '52791a2fd214b2a9dc5759d36725909c1daa7f2e'
子模块 'third_party/googletest' (https://github.com/google/googletest.git) 已对路径 'third_party/tensorpipe/third_party/googletest' 注册
子模块 'third_party/libnop' (https://github.com/google/libnop.git) 已对路径 'third_party/tensorpipe/third_party/libnop' 注册
子模块 'third_party/libuv' (https://github.com/libuv/libuv.git) 已对路径 'third_party/tensorpipe/third_party/libuv' 注册
子模块 'third_party/pybind11' (https://github.com/pybind/pybind11.git) 已对路径 'third_party/tensorpipe/third_party/pybind11' 注册
正克隆到 '/home/os-liyy/workspace-pytorch/pytorch-source/third_party/tensorpipe/third_party/googletest'...
正克隆到 '/home/os-liyy/workspace-pytorch/pytorch-source/third_party/tensorpipe/third_party/libnop'...
正克隆到 '/home/os-liyy/workspace-pytorch/pytorch-source/third_party/tensorpipe/third_party/libuv'...
正克隆到 '/home/os-liyy/workspace-pytorch/pytorch-source/third_party/tensorpipe/third_party/pybind11'...
子模块路径 'third_party/tensorpipe/third_party/googletest': 检出 'aee0f9d9b5b87796ee8a0ab26b7587ec30e8858e'
子模块路径 'third_party/tensorpipe/third_party/libnop': 检出 '910b55815be16109f04f4180e9adee14fb4ce281'
子模块路径 'third_party/tensorpipe/third_party/libuv': 检出 '1dff88e5161c8a5c59276d2070d2e304e4dcb242'
子模块路径 'third_party/tensorpipe/third_party/pybind11': 检出 'a23996fce38ff6ccfbcd09f1e63f2c4be5ea2ef'
子模块 'tools/clang' (https://github.com/wjakob/clang-cindex-python3) 已对路径 'third_party/tensorpipe/third_party/pybind11/tools/clang' 注册
正克隆到 '/home/os-liyy/workspace-pytorch/pytorch-source/third_party/tensorpipe/third_party/pybind11/tools/clang'...
子模块路径 'third_party/tensorpipe/third_party/pybind11/tools/clang': 检出 '6a00cbc4a9b8e68b71caf7f774b3f9c753ae84d5'
子模块路径 'third_party/zstd': 检出 'aec56a52fbab207fc639a1937d1e708a282edca8'
(base) os-liyy@os-student:~/workspace-pytorch/pytorch-source$
```

图 20-2 递归初始化所有子模块

同步完成后，利用以下命令检查子模块状态：

```
git submodule status --recursive
```

输出内容应与项目仓库中提供的 submodule_status.txt 一致（https://gitee.com/sdu-aes-lab/libtorch-for-loongarch64/blob/master/submodule_status/submodule_status.txt）。若遇到子模块不对应且出现不适配情况，可去子模块目录 pytorch-source/third_party/[submodule_name]下手动切换到指定的 Git Commit。

```
(base) os-liyy@os-student:~/workspace-pytorch/pytorch-source$ git submodule status --recursive
7e1e1fe3858c63c251c637ae41a20de425dde96f android/libs/fbjni (v0.1.0-12-g7e1e1fe)
4dfe081cf6bcd15db339cf2680b9281b8451eeb3 third_party/FP16 (4dfe081)
b408327ac2a15ec3e43352421954f5b1967701d1 third_party/FXdiv (b408327)
c07e3a0400713d546e0dea2d5466dd22ea389c73 third_party/NNPACK (c07e3a0)
7d2a4e9931a82adc3814275b6219a03e24e36b4c third_party/QNNPACK (heads/master)
ae108ef49aa5623b896fc93d4298c49d1750d9ba third_party/XNNPACK (remotes/origin/test_294865623-2079-gae108ef49)
e991355c02b93fe17713efe04cbc2e278e00fdbd third_party/benchmark (v1.5.5)
5916273f79a21551890fd3d56fc5375a78d1598d third_party/cpuinfo (5916273)
d106ddb991a56c3df1b6d51b2409e36ba8181ce4 third_party/cub (1.9.10)
```

图 20-3 子模块版本信息

20.3.3 配置交叉编译工具链

所谓交叉编译，就是在 x86 架构的主机上使用交叉编译工具链，编译出面向 LoongArch64 架构目标设备的二进制代码。这里使用龙芯开源社区发布的最新交叉工具链。

下载 LoongArch64 交叉编译工具链，执行如下操作：

```
cd ~/workspace-pytorch

wget      ftp.loongnix.cn/toolchain/gcc/release/loongarch/gcc8/loongson-gnu-toolchain-8.3-x86_64-
loongarch64-linux-gnu-rc1.6.tar.xz
```

```
长度： 36028144 (34M) [application/x-xz]
正在保存至： 'loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6.tar.xz'

loongson-gnu-toolch 100%[=====>] 34.36M  3.84MB/s   用时 9.8s

2025-03-20 11:00:34 (3.51 MB/s) - 已保存 'loongson-gnu-toolchain-8.3-x86_64-loongarch64-
linux-gnu-rc1.6.tar.xz' [36028144/36028144]
```

图 20-3 获取交叉编译工具链

解压工具链：

```
tar -xJvf loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6.tar.xz
```

解压完成后，工具链位于`~/workspace-pytorch/loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6`目录下。该工具链包含 loongarch64-linux-gnu-gcc、loongarch64-linux-gnu-g++等交叉编译器，以及目标架构的头文件和库文件。

注：该工具链提供的 gcc 版本为 8.3。若需使用更新版本的 gcc（如 gcc 9.4.0 或更高版本），或需要自行构建交叉编译工具链，通过手动下载并编译 Binutils、Glibc、GCC 以及 Linux 内核头文件，并在 QEMU 或实体开发板上进行测试验证。使用更高版本 gcc 时，可能需要对部分源文件手动添加缺失的头文件引用（如`#include <cstdint>`、`#include <stdexcept>`等），具体可参考项目仓库的完整文档。

20.3.4 创建 Python 虚拟环境

使用 Miniconda 创建 Python 虚拟环境并安装编译所需的依赖包，执行如下操作：

```
conda create -n pytorch-loongarch64-env python=3.9
conda activate pytorch-loongarch64-env
conda install -y astunparse numpy ninja pyyaml mkl mkl-include \
    setuptools cmake cffi typing_extensions future six requests \
    dataclasses pkg-config libuv
```

也可以使用项目仓库中提供的 `pytorch-loongarch64-env.yaml` 文件直接创建环境：

```
conda env create -f pytorch-loongarch64-env.yaml
```

创建完成后，使用以下命令激活虚拟环境：

```
conda activate pytorch-loongarch64-env
```

20.3.5 编写 CMake 工具链文件

根据 LoongArch64 的架构特点调整编译选项和设置，编写 CMake 工具链文件，用于指定目标架构和交叉编译工具链路径。在工作目录下创建 loongarch64_setup.cmake 文件，内容如下：

```
set(CMAKE_SYSTEM_NAME Linux)
set(CMAKE_SYSTEM_PROCESSOR loongarch64)

if(DEFINED ENV{LOONGARCH64_ROOT_PATH})
    file(TO_CMAKE_PATH $ENV{LOONGARCH64_ROOT_PATH}
LOONGARCH64_ROOT_PATH)
else()
    message(FATAL_ERROR "LOONGARCH64_ROOT_PATH env must be defined")
endif()

set(LOONGARCH64_ROOT_PATH ${LOONGARCH64_ROOT_PATH}
    CACHE STRING "root path to loongarch64 toolchain")

set(CMAKE_C_COMPILER "${LOONGARCH64_ROOT_PATH}/bin/loongarch64-linux-gnu-gcc")
set(CMAKE_CXX_COMPILER "${LOONGARCH64_ROOT_PATH}/bin/loongarch64-linux-gnu-
g++")

set(CMAKE_FIND_ROOT_PATH "${LOONGARCH64_ROOT_PATH}/loongarch64-linux-gnu")

set(CMAKE_FIND_ROOT_PATH_MODE_PROGRAM NEVER)
set(CMAKE_FIND_ROOT_PATH_MODE_LIBRARY ONLY)
set(CMAKE_FIND_ROOT_PATH_MODE_INCLUDE ONLY)
set(CMAKE_FIND_ROOT_PATH_MODE_PACKAGE ONLY)

set(CMAKE_C_FLAGS "-march=loongarch64")
set(CMAKE_CXX_FLAGS "-march=loongarch64")

# cache flags
set(CMAKE_C_FLAGS "${CMAKE_C_FLAGS}" CACHE STRING "c flags")
set(CMAKE_CXX_FLAGS "${CMAKE_CXX_FLAGS}" CACHE STRING "c++ flags")
set(USE_GLOO OFF)
```

该工具链文件通过 CMAKE_SYSTEM_PROCESSOR 指定目标处理器架构为 loongarch64，并通过 LOONGARCH64_ROOT_PATH 环境变量定位交叉编译工具链的安装路径。CMAKE_FIND_ROOT_PATH_MODE 系列变量控制 CMake 在查找库文件和

头文件时的搜索策略，设置为 ONLY 表示仅在工具链的 sysroot 中查找，避免误用主机上的 x86 库文件。

20.3.6 预编译 protoc 和 sleef 库

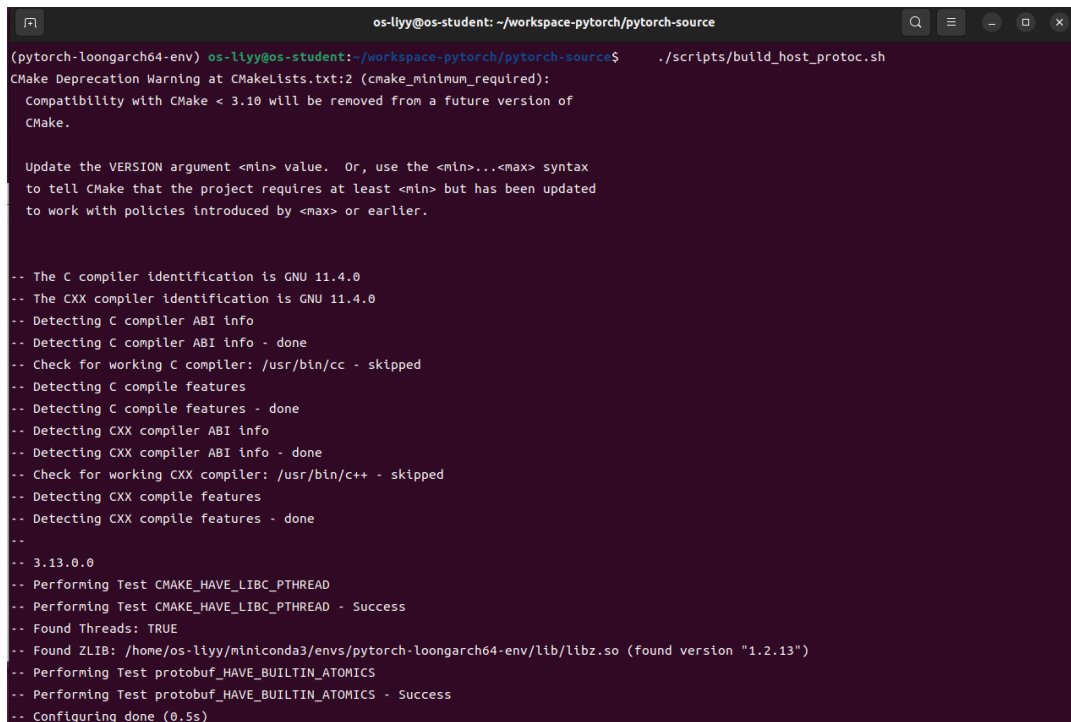
在进行交叉编译之前，需要预先使用 x86 编译工具链编译两个宿主机端的工具库：protoc（Protocol Buffers 编译器）和 sleef（SIMD 数学函数库的宿主机版本）。这两个库在交叉编译过程中需要在主机端运行，因此必须使用 x86 工具链编译。

预编译 protoc 库，在 pytorch-source 目录下执行如下操作：

```
cd ~/workspace-pytorch/pytorch-source/  
conda activate pytorch-loongarch64-env  
./scripts/build_host_protoc.sh
```

如果执行时报错找不到 cmake 命令，则需要先安装 cmake：

```
sudo apt-get update  
sudo apt-get install cmake
```

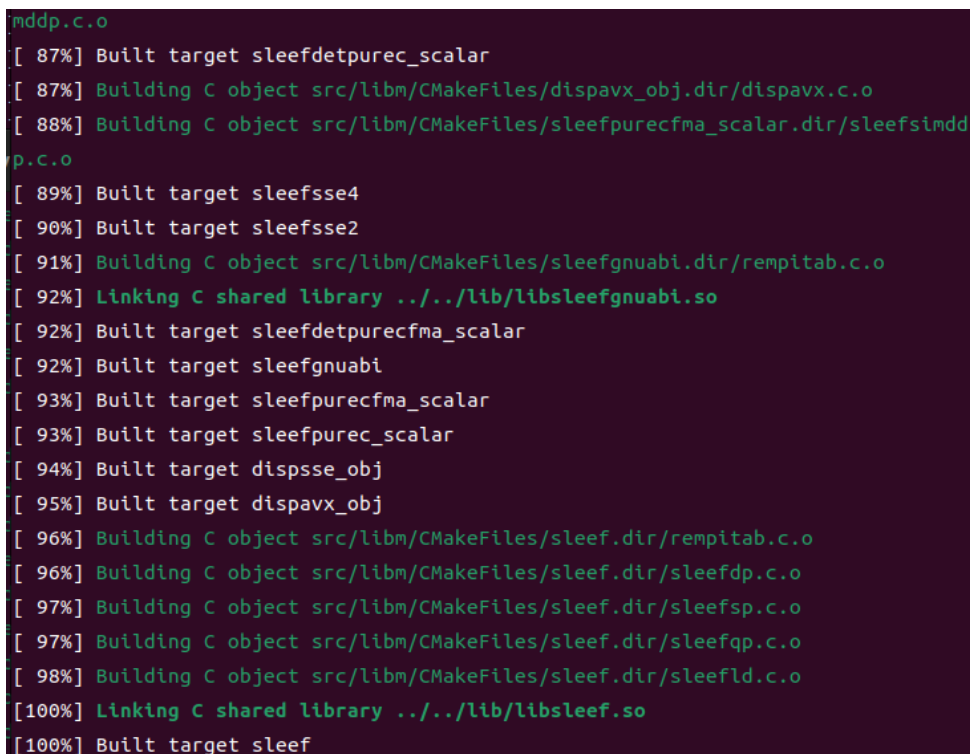


```
os-liyy@os-student: ~/workspace-pytorch/pytorch-source  
(pytorch-loongarch64-env) os-liyy@os-student:~/workspace-pytorch/pytorch-source$ ./scripts/build_host_protoc.sh  
CMake Deprecation Warning at CMakeLists.txt:2 (cmake_minimum_required):  
  Compatibility with CMake < 3.10 will be removed from a future version of  
  CMake.  
  
Update the VERSION argument <min> value. Or, use the <min>...<max> syntax  
to tell CMake that the project requires at least <min> but has been updated  
to work with policies introduced by <max> or earlier.  
  
-- The C compiler identification is GNU 11.4.0  
-- The CXX compiler identification is GNU 11.4.0  
-- Detecting C compiler ABI info  
-- Detecting C compiler ABI info - done  
-- Check for working C compiler: /usr/bin/cc - skipped  
-- Detecting C compile features  
-- Detecting C compile features - done  
-- Detecting CXX compiler ABI info  
-- Detecting CXX compiler ABI info - done  
-- Check for working CXX compiler: /usr/bin/c++ - skipped  
-- Detecting CXX compile features  
-- Detecting CXX compile features - done  
--  
-- 3.13.0.0  
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD  
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD - Success  
-- Found Threads: TRUE  
-- Found ZLIB: /home/os-liyy/miniconda3/envs/pytorch-loongarch64-env/lib/libz.so (found version "1.2.13")  
-- Performing Test protobuf_HAVE_BUILTIN_ATOMICS  
-- Performing Test protobuf_HAVE_BUILTIN_ATOMICS - Success  
-- Configuring done (0.5s)
```

图 20-4 预构建 protoc

编译 sleef 库，在 pytorch-source/third_party/sleef 目录下执行如下操作：

```
cd ~/workspace-pytorch/pytorch-source/third_party/sleef
mkdir -p _host && cd _host
cmake .. -DCMAKE_BUILD_TYPE=Release -DBUILD_TESTS=OFF
make -j$(nproc)
```



```
nddp.c.o
[ 87%] Built target sleefdetpurec_scalar
[ 87%] Building C object src/libm/CMakeFiles/dispavx_obj.dir/dispavx.c.o
[ 88%] Building C object src/libm/CMakeFiles/sleefpurecfma_scalar.dir/sleefsimdd
p.c.o
[ 89%] Built target sleefsse4
[ 90%] Built target sleefsse2
[ 91%] Building C object src/libm/CMakeFiles/sleefgnuabi.dir/rempitab.c.o
[ 92%] Linking C shared library ../../lib/lib sleefgnuabi.so
[ 92%] Built target sleefdetpurecfma_scalar
[ 92%] Built target sleefgnuabi
[ 93%] Built target sleefpurecfma_scalar
[ 93%] Built target sleefpurec_scalar
[ 94%] Built target dispasse_obj
[ 95%] Built target dispavx_obj
[ 96%] Building C object src/libm/CMakeFiles/sleef.dir/rempitab.c.o
[ 96%] Building C object src/libm/CMakeFiles/sleef.dir/sleefdp.c.o
[ 97%] Building C object src/libm/CMakeFiles/sleef.dir/sleefsp.c.o
[ 97%] Building C object src/libm/CMakeFiles/sleef.dir/sleefqp.c.o
[ 98%] Building C object src/libm/CMakeFiles/sleef.dir/sleefld.c.o
[100%] Linking C shared library ../../lib/lib sleef.so
[100%] Built target sleef
```

图 20-5 预构建编译 sleef

编译完成后，protoc 的可执行文件位于 pytorch-source/build_host_protoc/bin/目录下，sleef 的库文件位于 pytorch-source/third_party/sleef/_host/目录下。这两个路径将在后续的交叉编译 cmake 配置中使用。

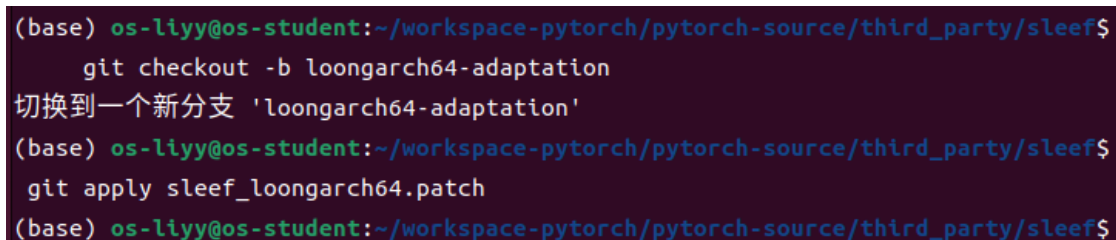
20.3.7 源码适配

现有的 PyTorch 源码和所依赖的第三方库，基本均未对 LoongArch64 架构进行专门适配。在编译时需要根据 LoongArch64 架构和处理器的支持情况，修改源代码中不兼容的部分。经测试，使用 rc1.6 版本的 gcc 交叉编译工具链需要修改的内容相比 rc1.5 版本更少，但仍需进行一些适配修改。

20.3.7.1 应用 SLEEF 库补丁

将项目仓库中的 `sleef_loongarch64.patch` 补丁 (https://gitee.com/sdu-aes-lab/libtorch-for-loongarch64/blob/master/patches/sleef_loongarch64.patch) 文件拷贝到 `pytorch-source/third_party/sleef` 目录下，并应用该补丁，执行如下操作：

```
cd ~/workspace-pytorch/pytorch-source/third_party/sleef
git checkout -b loongarch64-adaptation
git apply sleef_loongarch64.patch
# 若遇到问题，可尝试：
patch -p1 < sleef_loongarch64.patch
```



```
(base) os-liyy@os-student:~/workspace-pytorch/pytorch-source/third_party/sleef$
git checkout -b loongarch64-adaptation
切换到一个新分支 'loongarch64-adaptation'
(base) os-liyy@os-student:~/workspace-pytorch/pytorch-source/third_party/sleef$
git apply sleef_loongarch64.patch
(base) os-liyy@os-student:~/workspace-pytorch/pytorch-source/third_party/sleef$
```

图 20-6 应用 sleef 适配补丁

该补丁主要解决了 SLEEF 库在 LoongArch64 架构下的编译兼容性问题，包括对 SIMD 指令集的识别和相关数学函数的实现路径选择。

20.3.7.2 应用 cpuinfo 库补丁（非必做）

如果编译过程中遇到 `cpuinfo` 相关的错误，可以应用 `cpuinfo_ifneed.patch` 补丁。一般情况下不需要应用此补丁，仅在遇到特定编译错误时使用。将补丁文件拷贝到 `pytorch-source/third_party/cpuinfo` 目录下，执行如下操作：

```
cd ~/workspace-pytorch/pytorch-source/third_party/cpuinfo
git checkout -b loongarch64-adaptation
git apply cpuinfo_ifneed.patch
# 若遇到问题，可尝试：
patch -p1 < cpuinfo_ifneed.patch
```

20.3.7.3 gcc 13.3 源码适配（若使用 gcc 13.3 工具链）

若使用龙芯提供的 gcc 13.3 交叉编译工具链（可从龙芯资料包中获取 loongarch64-linux-gnu-gcc13.3.tar.gz，解压至/opt/目录），则除了上述 SLEEF 库补丁外，还需要对 PyTorch 源码中部分文件手动添加缺失的标准库头文件引用，以兼容 gcc 13.3 更严格的编译检查。执行如下操作：

```
cd ~/workspace-pytorch/pytorch-source

# vim 后添加 #include <stdint>:
vim third_party/kineto/libkineto/src/SampleListener.h
vim c10/util/BFloat16.h
vim aten/src/ATen/core/dynamic_type.h

# vim 后添加 #include <stdexcept>:
vim torch/csrc/jit/runtime/logging.cpp
vim torch/csrc/lazy/core/multi_wait.cpp
```

以上每个文件均需在已有的#include 区域中添加对应的头文件引用。

20.3.8 设置环境变量

在开始交叉编译之前，需要设置 LOONGARCH64_ROOT_PATH 和 CROSS_COMPILE 两个环境变量，分别指向交叉编译工具链的根路径和交叉编译器的前缀，执行如下操作：

```
conda activate pytorch-loongarch64-env
export LOONGARCH64_ROOT_PATH=~/workspace-pytorch/loongson-gnu-toolchain-8.3-x86_64-
loongarch64-linux-gnu-rc1.6
export CROSS_COMPILE=loongarch64-linux-gnu-
```

LOONGARCH64_ROOT_PATH 变量会被前面编写的 CMake 工具链文件读取，用于定位交叉编译器和目标架构的系统库路径。CROSS_COMPILE 变量用于指定交叉编译器名称的前缀。

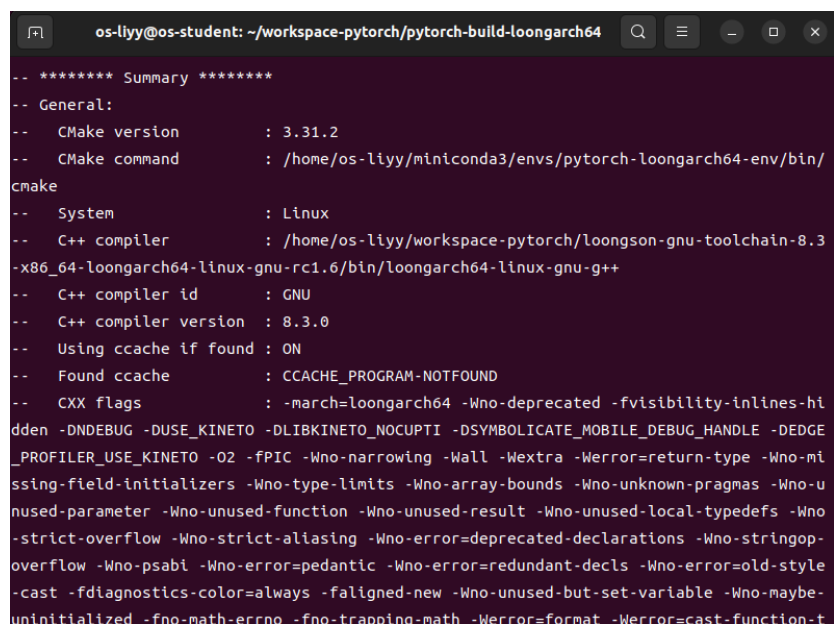
20.3.9 交叉编译 LibTorch

完成上述准备工作后，即可开始 LibTorch 的交叉编译。首先创建构建目录：

```
cd ~/workspace-pytorch
mkdir -p pytorch-build-loongarch64
cd pytorch-build-loongarch64
```

然后执行 cmake 配置命令，其中 CAFFE2_CUSTOM_PROTOC_EXECUTABLE 和 NATIVE_BUILD_DIR 需要使用绝对路径，指向前面预编译的 protoc 和 sleef 的产物路径。cmake 配置命令如下：

```
cmake -DCMAKE_TOOLCHAIN_FILE=../loongarch64_setup.cmake \
      -DCMAKE_INSTALL_PREFIX:PATH="$HOME/workspace-pytorch/pytorch-install-loongarch64-o3" \
      -DCMAKE_BUILD_TYPE=Release \
      -DPYTHON_EXECUTABLE:PATH=`which python3` \
      -DCMAKE_CXX_STANDARD=17 \
      -DCMAKE_CXX_FLAGS="-march=loongarch64 -O3 -Wno-error" \
      -DCAFFE2_CUSTOM_PROTOC_EXECUTABLE="$HOME/workspace-pytorch/pytorch-source/build_host_protoc/bin/protoc" \
      -DNATIVE_BUILD_DIR="$HOME/workspace-pytorch/pytorch-source/third_party/sleef/_host" \
      -DBUILD_SHARED_LIBS=ON \
      -DUSE_CUDA=OFF \
      -DUSE_NNPack=OFF \
      -DUSE_OPENMP=OFF \
      -DBUILD_PYTHON=OFF \
```



```
os-liyy@os-student: ~/workspace-pytorch/pytorch-build-loongarch64
-- ***** Summary *****
-- General:
-- CMake version      : 3.31.2
-- CMake command      : /home/os-liyy/miniconda3/envs/pytorch-loongarch64-env/bin/cmake
-- System             : Linux
-- C++ compiler        : /home/os-liyy/workspace-pytorch/loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.6/bin/loongarch64-linux-gnu-g++
-- C++ compiler id     : GNU
-- C++ compiler version : 8.3.0
-- Using ccache if found : ON
-- Found ccache        : CCACHE_PROGRAM-NOTFOUND
-- CXX flags           : -march=loongarch64 -Wno-deprecated -fvisibility-inlines-hidden -DDEBUG -DUSE_KINETO -DLIBKINETO_NOUPTI -DSYMBOLICATE_MOBILE_DEBUG_HANDLE -DEDGE_PROFILER_USE_KINETO -O2 -fPIC -Wno-narrowing -Wall -Wextra -Werror=return-type -Wno-missing-field-initializers -Wno-type-limits -Wno-array-bounds -Wno-unknown-pragmas -Wno-unused-parameter -Wno-unused-function -Wno-unused-result -Wno-unused-local-typedefs -Wno-strict-overflow -Wno-strict-aliasing -Wno-error=deprecated-declarations -Wno-stringop-overflow -Wno-psabi -Wno-error=pedantic -Wno-error=redundant-decls -Wno-error=old-style -cast -fdiagnostics-color=always -faligned-new -Wno-unused-but-set-variable -Wno-maybe-uninitialized -fno-math-errno -fno-trapping-math -Werror=format -Werror=cast-function-t
```

图 20-7 进行 cmake 配置 libtorch

```

-- USE_NNPack          : OFF
-- USE_NUMPY           : ON
-- USE_OBSERVERS       : OFF
-- USE_OPENCL          : OFF
-- USE_OPENCV          : OFF
-- USE_OPENMP          : OFF
-- USE_TBB             : OFF
-- USE_VULKAN          : OFF
-- USE_PROF            : OFF
-- USE_QNNPACK         : OFF
-- USE_PYTORCH_QNNPACK : OFF
-- USE_XNNPACK         : OFF
-- USE_REDIS           : OFF
-- USE_ROCKSDB         : OFF
-- USE_ZMQ             : OFF
-- USE_DISTRIBUTED     : ON
--   USE_MPI           : OFF
--   USE_GLOO          : ON
--   USE_GLOO_WITH_OPENSSL : OFF
--   USE_TENSORPIPE    : ON
-- USE_DEPLOY          : OFF
-- Public Dependencies : caffe2::Threads
-- Private Dependencies : cpuinfo;fp16;tensorpipe;gloo;foxi_loader;rt;fmt::fmt-header-only;kineto;gcc_s;gcc;dl
-- USE_COREML_DELEGATE : OFF
-- BUILD_LAZY_TS_BACKEND : ON
-- Configuring done (6.8s)
-- Generating done (0.3s)
-- Build files have been written to: /home/os-liyy/workspace-pytorch/pytorch-build-loongarch64

```

图 20-8 cmake 及 build libtorch

上述 cmake 参数的含义如下：

- (1) -DCMAKE_TOOLCHAIN_FILE 指定前面编写的 CMake 工具链文件，告知 cmake 使用 LoongArch64 交叉编译工具链进行编译。
- (2) -DCMAKE_INSTALL_PREFIX 指定 LibTorch 的安装路径。
- (3) -DCAFFE2_CUSTOM_PROTOC_EXECUTABLE 指定预编译的 protoc 可执行文件路径，避免在交叉编译过程中尝试运行目标架构的 protoc 导致执行失败。
- (4) -DNATIVE_BUILD_DIR 指定预编译的 sleef 库路径。
- (5) -DBUILD_SHARED_LIBS=ON 表示构建动态链接库。
- (6) -DUSE_CUDA=OFF 禁用 CUDA 支持，因为 LoongArch64 设备通常不配备 NVIDIA GPU。
- (7) -DUSE_NNPack=OFF、-DUSE_OPENMP=OFF、-DUSE_MKLDNN=OFF 禁用当前架构不支持的加速库。
- (8) -DBUILD_PYTHON=OFF 禁用 Python 绑定的构建，仅构建 C++ 库。

cmake 配置完成后，执行编译：

```
make -j$(nproc)
# 若遇到异常，可使用不带并行参数的 make 命令重试
make
```

```
(pytorch-loongarch64-env) os-lyyy@os-student:~/workspace-pytorch/pytorch-build-loongarch64$ make -j$(nproc) && make install
[ 1%] Building C object confu-deps/cpulinfo/deps/clog/CMakeFiles/clog.dir/src/clog.c.o
[ 1%] Building CXX object third_party/protobuf/cnake/CMakeFiles/libprotobuf-lite.dir/_/src/google/protobuf/any_lite.cc.o
[ 1%] Building C object third_party/tensorpipe/tensorpipe/CMakeFiles/tensorpipe_uv.dir/_/third_party/libuv/src/fs-poll.c.o
[ 1%] Building CXX object third_party/protobuf/cnake/CMakeFiles/libprotobuf.dir/_/src/google/protobuf/any_lite.cc.o
[ 1%] Linking C static library .././../lib/libclog.a
[ 1%] Built target clog
[ 1%] Building CXX object third_party/gloo/gloo/CMakeFiles/gloo.dir/algorithm.cc.o
[ 1%] Building C object third_party/tensorpipe/tensorpipe/CMakeFiles/tensorpipe_uv.dir/_/third_party/libuv/src/idna.c.o
[ 1%] Building C object third_party/tensorpipe/tensorpipe/CMakeFiles/tensorpipe_uv.dir/_/third_party/libuv/src/inet.c.o
[ 1%] Building C object third_party/tensorpipe/tensorpipe/CMakeFiles/tensorpipe_uv.dir/_/third_party/libuv/src/random.c.o
[ 1%] Building C object third_party/tensorpipe/tensorpipe/CMakeFiles/tensorpipe_uv.dir/_/third_party/libuv/src/strscpy.c.o
[ 1%] Building C object third_party/tensorpipe/tensorpipe/CMakeFiles/tensorpipe_uv.dir/_/third_party/libuv/src/threadpool.c.o
[ 1%] Building CXX object third_party/gloo/gloo/CMakeFiles/gloo.dir/allgather.cc.o
[ 1%] Building CXX object third_party/protobuf/cnake/CMakeFiles/libprotobuf-lite.dir/_/src/google/protobuf/arena.cc.o
[ 1%] Building CXX object third_party/protobuf/cnake/CMakeFiles/libprotobuf.dir/_/src/google/protobuf/arena.cc.o
[ 1%] Building C object third_party/tensorpipe/tensorpipe/CMakeFiles/tensorpipe_uv.dir/_/third_party/libuv/src/timer.c.o
[ 1%] Building C object third_party/tensorpipe/tensorpipe/CMakeFiles/tensorpipe_uv.dir/_/third_party/libuv/src/uv-common.c.o
[ 1%] Building C object third_party/tensorpipe/tensorpipe/CMakeFiles/tensorpipe_uv.dir/_/third_party/libuv/src/uv-data-getter-setters.c.o
[ 1%] Building CXX object third_party/protobuf/cnake/CMakeFiles/libprotobuf-lite.dir/_/src/google/protobuf/extension_set.cc.o
[ 1%] Building CXX object third_party/protobuf/cnake/CMakeFiles/libprotobuf.dir/_/src/google/protobuf/extension_set.cc.o
[ 1%] Building CXX object third_party/tensorpipe/tensorpipe/CMakeFiles/tensorpipe_uv.dir/_/third_party/libuv/src/version.c.o
[ 1%] Building CXX object third_party/gloo/gloo/CMakeFiles/gloo.dir/allgather.cc.o
[ 1%] Building C object third_party/tensorpipe/tensorpipe/CMakeFiles/tensorpipe_uv.dir/_/third_party/libuv/src/unix/async.c.o
[ 1%] Building C object third_party/tensorpipe/tensorpipe/CMakeFiles/tensorpipe_uv.dir/_/third_party/libuv/src/unix/core.c.o
```

图 20-10 make 及 make install 执行 1

```
os-lyyy@os-student: ~/workspace-pytorch/pytorch-build-loongarch64
[ 98%] Building CXX object caffe2/CMakeFiles/torch_cpu.dir/_/torch/csrc/api/src/nn/options/linear.cpp.o
[ 98%] Building CXX object caffe2/CMakeFiles/torch_cpu.dir/_/torch/csrc/api/src/nn/options/padding.cpp.o
[ 98%] Building CXX object caffe2/CMakeFiles/torch_cpu.dir/_/torch/csrc/api/src/nn/options/pooling.cpp.o
[ 98%] Building CXX object caffe2/CMakeFiles/torch_cpu.dir/_/torch/csrc/api/src/nn/options/rnn.cpp.o
[ 98%] Building CXX object caffe2/CMakeFiles/torch_cpu.dir/_/torch/csrc/api/src/nn/options/vision.cpp.o
[ 98%] Building CXX object caffe2/CMakeFiles/torch_cpu.dir/_/torch/csrc/api/src/nn/options/transformer.cpp.o
[ 98%] Building CXX object caffe2/CMakeFiles/torch_cpu.dir/_/torch/csrc/api/src/optin/adagrad.cpp.o
[ 98%] Building CXX object caffe2/CMakeFiles/torch_cpu.dir/_/torch/csrc/api/src/optin/adam.cpp.o
[ 98%] Building CXX object caffe2/CMakeFiles/torch_cpu.dir/_/torch/csrc/api/src/optin/adamw.cpp.o
[ 98%] Building CXX object caffe2/CMakeFiles/torch_cpu.dir/_/torch/csrc/api/src/optin/lbfgs.cpp.o
[ 98%] Building CXX object caffe2/CMakeFiles/torch_cpu.dir/_/torch/csrc/api/src/optin/optimizer.cpp.o
[ 98%] Building CXX object caffe2/CMakeFiles/torch_cpu.dir/_/torch/csrc/api/src/optin/rmsprop.cpp.o
[ 98%] Building CXX object caffe2/CMakeFiles/torch_cpu.dir/_/torch/csrc/api/src/optin/serialize.cpp.o
[ 98%] Building CXX object caffe2/CMakeFiles/torch_cpu.dir/_/torch/csrc/api/src/optin/sgd.cpp.o
[ 98%] Building CXX object caffe2/CMakeFiles/torch_cpu.dir/_/torch/csrc/api/src/optin/schedulers/lr_scheduler.cpp.o
[ 98%] Building CXX object caffe2/CMakeFiles/torch_cpu.dir/_/torch/csrc/api/src/optin/schedulers/step_lr.cpp.o
[100%] Building CXX object caffe2/CMakeFiles/torch_cpu.dir/_/torch/csrc/api/src/serialize/input-archive.cpp.o
[100%] Building CXX object caffe2/CMakeFiles/torch_cpu.dir/_/torch/csrc/api/src/serialize/output-archive.cpp.o
[100%] Linking CXX shared library ../lib/libtorch_cpu.so
[100%] Built target torch_cpu
[100%] Building CXX object caffe2/CMakeFiles/torch.dir/_/empty.cpp.o
[100%] Linking CXX shared library ../lib/libtorch.so
[100%] Built target torch
```

图 20-11 make 执行完成

编译过程可能需要较长时间，取决于主机的硬件配置。编译完成后，执行安装命令：

```
make install
```

```
os-liyy@os-student: ~/workspace-pytorch/pytorch-build-loongarch64
[ 34%] Built target renamePUREC_SCALAR.h_generated
[ 34%] Built target sleefpurec_scalar
[ 34%] Built target sleefdetpurec_scalar
[ 34%] Built target renamePURECFMA_SCALAR.h_generated
[ 34%] Built target sleefpurecfma_scalar
[ 35%] Built target sleefdetpurecfma_scalar
[ 35%] Built target sleef
[ 35%] Building CXX object caffe2/CMakeFiles/torch_cpu.dir/__/torch/csrc/distributed/c10d/GlooDeviceFactory.cpp.o
[ 35%] Building CXX object caffe2/CMakeFiles/torch_cpu.dir/__/torch/csrc/distributed/c10d/ProcessGroupGloo.cpp.o
[ 35%] Linking CXX shared library ../lib/libtorch_cpu.so
[100%] Built target torch_cpu
[100%] Building CXX object caffe2/CMakeFiles/torch.dir/__/empty.cpp.o
[100%] Linking CXX shared library ../lib/libtorch.so
[100%] Built target torch
[100%] Built target torch_global_deps
[100%] Built target arraymap
Install the project...
-- Install configuration: "Release"
-- Set non-toolchain portion of runtime path of "/home/os-liyy/workspace-pytorch/pytorch-install-loongarch64-o3/bin/protoc-3.13.0" to "$ORIGIN/../lib"
-- Set non-toolchain portion of runtime path of "/home/os-liyy/workspace-pytorch/pytorch-install-loongarch64-o3/lib/libc10.so" to "$ORIGIN"
-- Set non-toolchain portion of runtime path of "/home/os-liyy/workspace-pytorch/pytorch-install-loongarch64-o3/lib/libtorch_cpu.so" to "$ORIGIN"
-- Set non-toolchain portion of runtime path of "/home/os-liyy/workspace-pytorch/pytorch-install-loongarch64-o3/lib/libtorch.so" to "$ORIGIN"
-- Set non-toolchain portion of runtime path of "/home/os-liyy/workspace-pytorch/pytorch-install-loongarch64-o3/lib/libtorch_global_deps.so" to "$ORIGIN"
```

图 20-12 make install 执行

安装成功后，在~/workspace-pytorch/pytorch-install-loongarch64-o3/目录下将包含完整的 LibTorch 库文件，包括头文件（include 目录）、动态链接库（lib 目录）和 CMake 配置文件（share 目录）。

20.3.10 替换 TorchConfig.cmake

编译安装完成后，需要使用项目仓库中提供的 TorchConfig.cmake 文件替换安装目录中的默认配置文件，以确保 LibTorch 在 LoongArch64 架构下的链接路径正确，执行如下操作：

```
cd ~/workspace-pytorch
wget https://gitee.com/sdu-aes-lab/libtorch-for-loongarch64/raw/master/cmake/TorchConfig.cmake
mv ./TorchConfig.cmake ~/workspace-pytorch/pytorch-install-loongarch64-o3/share/cmake/Torch/TorchConfig.cmake
```

至此，LibTorch for LoongArch64 的适配构建完成。在~/workspace-pytorch/pytorch-install-loongarch64-o3/目录下即可获得为 LoongArch64 架构设备构建的 LibTorch 库。该目录即为 LibTorch 的安装前缀，后续使用 LibTorch 进行项目开发时，需要在 CMakeLists.txt 中将 Torch_DIR 指向该目录下的 share/cmake/Torch 子目录。

参考资料：本实验详细文档及源码见 <https://gitee.com/sdu-aes-lab/libtorch-for-loongarch64>，龙芯官方全流程验证见龙芯工控公众号《案例分享——龙芯 AI 生态之 libtorch》。

第21章 龙芯神经网络训练实验

21.1 实验目的

掌握在实验平台上部署已移植好的 LibTorch 运行环境的方法，学会编写、交叉编译并运行基于 LibTorch 的 C++ 训练/推理程序。

根据 LibTorch 的使用方法，在实验平台上实现信号处理的卷积神经网络训练示例，并通过调整训练过程、优化代码结构以及合理配置运行环境来加速模型训练过程。

21.2 实验步骤

21.2.1 准备代码环境

在龙芯板卡上运行一个完整的 LibTorch 训练过程，本实验以一维信号分类为例。

文件/目录	作用
main.cpp	设置超参数、选择设备、构造模型/优化器/数据集，循环训练和测试，最终保存 model.pt。
dataset.h / dataset.cpp	读取 CSV 索引和 TXT 序列文件，生成 LibTorch Dataset。
model.h / model.cpp	定义一维 CNN：5 组 Conv1d+ReLU+BatchNorm，接自适应池化和两层全连接输出 2 类。
trainer.h / trainer.cpp	封装 train/test，负责 DataLoader、loss、反向传播、推理评估和耗时输出。
data/indice	存放 label 与样本文件名索引。
data/training_dataset	存放训练和测试样本的 TXT 序列。

表 21-1 文件说明

从资料中下载实验代码和数据，代码结构和组成如表 21-1 所示。这里需要注意在代码中修改 data 的相对路径为 your path to data 。

```
// main.cpp
// 构造训练和测试数据集
Dataset train_dataset("./data/indice/test_indice.csv", "./data/training_dataset", Dataset::Phase::Train);
Dataset test_dataset("./data/indice/test_indice.csv", "./data/training_dataset", Dataset::Phase::Test);
```

修改完成后，查看代码中对于 Torch 的调用，使用方法与 PyTorch 大致一致将代码下载到 Ubuntu 主机中，方便后续完成代码的交叉编译得到龙芯架构的可执行文件。

此处展示 main.cpp 的构建过程：

```
// main.cpp
#include <string>
#include <torch/torch.h>
#include "dataset.h"
#include "model.h"
#include "trainer.h"

int main() {
    // 超参数设置
    std::string data_root = "./data";
    int train_batch_size = 32;
    int test_batch_size = 32;
    int total_epoch_num = 30;
    int log_interval = 10;
    int num_workers = 1;
    torch::set_num_threads(1);
    torch::set_num_interop_threads(1);

    // 设置随机数种子
    torch::manual_seed(1);

    // 获取设备类型
    torch::DeviceType device_type = torch::kCPU;
    if (torch::cuda::is_available()) {
        device_type = torch::kCUDA;
    }
    torch::Device device(device_type);
    // 构造网络
    Model model;
    model.to(device);

    // 设置优化器
    // torch::optim::SGD optimizer(
    //     model.parameters(), torch::optim::SGDOptions(0.01).momentum(0.5));
    torch::optim::SGD optimizer(
        model.parameters(), torch::optim::SGDOptions(0.01).momentum(0.9).weight_decay(1e-4));
```

```

// 构造训练和测试数据集
Dataset train_dataset("./data/indice/test_indice.csv", "./data/training_dataset", Dataset::Phase::Train);
Dataset test_dataset("./data/indice/test_indice.csv", "./data/training_dataset", Dataset::Phase::Test);

// Trainer 初始化
Trainer trainer(log_interval);
for (size_t epoch = 1; epoch <= total_epoch_num; ++epoch) {
    // 运行训练
    trainer.train(
        epoch,
        model,
        optimizer,
        device,
        train_dataset,
        train_batch_size,
        num_workers);

    // 运行测试
    trainer.test(model, device, test_dataset, test_batch_size, num_workers);
}

torch::serialize::OutputArchive archive;
model.save(archive);
archive.save_to("model.pt");
printf("Save the training result to model.pt.\n");

return 0;
}

```

本次实验的评测指标包括 trainer 中记录的 accuracy、F1 score 与推理延迟，观察不同的优化操作对精度和推理以及训练延迟的影响。

21.2.2 交叉编译

在 Ubuntu 主机上解压龙芯 LoongArch64 GNU 交叉编译工具链，并将工具链 bin 目录加入 PATH。配置完成后用 gcc -v 验证。

（更多版本可以在龙芯 GNU 编译工具链 | 龙芯开源社区获得，推荐使用 1.5 版本）。

```
export          PATH=$PATH:/your_path/loongson-gnu-toolchain-8.3-x86_64-loongarch64-linux-gnu-rc1.5/bin/
export ARCH=loongarch64
export CROSS_COMPILE=loongarch64-linux-gnu
loongarch64-linux-gnu-gcc -v
```

本机编译和板卡运行都需要保持 CMakeLists.txt 中的 LibTorch 路径一致，由于平台存储有限，将 Libtorch 的运行库放在临时路径/tmp 下。

Cmake 配置如下：

```
cmake_minimum_required(VERSION 3.0 FATAL_ERROR)
project(LibTorchDemo)
# (!!! 注意开发板-上位机路径一致) 设置 LibTorch 的安装路径

set(TORCH_INSTALL_PREFIX "/tmp/loongTensor")
# 设置 LibTorch 的配置文件路径
set(Torch_DIR "${TORCH_INSTALL_PREFIX}/share/cmake/Torch")
# 设置 Caffe2 的配置文件路径
set(Caffe2_DIR "${TORCH_INSTALL_PREFIX}/share/cmake/Caffe2")
# 编译选项
set(CMAKE_CXX_FLAGS_RELEASE "-O3")
if(NOT CMAKE_BUILD_TYPE)
    set(CMAKE_BUILD_TYPE Release CACHE STRING "Build type" FORCE)
endif()
# Release 编译优化
set(CMAKE_CXX_FLAGS_RELEASE "${CMAKE_CXX_FLAGS_RELEASE} -O3 -DNDEBUG -flto")
set(CMAKE_C_FLAGS_RELEASE    "${CMAKE_C_FLAGS_RELEASE}    -O3 -DNDEBUG -flto")

# Release 链接优化
set(CMAKE_EXE_LINKER_FLAGS_RELEASE
    "${CMAKE_EXE_LINKER_FLAGS_RELEASE} -Wl,-O1 -flto"
)

set(CMAKE_CXX_STANDARD 14)
# 查找 LibTorch 包
find_package(Torch REQUIRED)
```

```
# 添加可执行文件
add_executable(train main.cpp model.cpp dataset.cpp trainer.cpp)

# 链接 LibTorch 库
target_link_libraries(train PRIVATE ${TORCH_LIBRARIES} pthread)

# 使用 find_package 设置的头文件路径
target_include_directories(train PRIVATE ${TORCH_INCLUDE_DIRS})

target_compile_options(train PRIVATE -O3 -DNDEBUG -fno)

target_link_options(train PRIVATE -Wl,-O1 -fno)
```

准备好了 CMake 配置之后，需要执行本机交叉编译，需要在 Ubuntu 环境下使用龙芯交叉编译工具链。首先在工程根目录创建 build 目录，使用 loongarch64-linux-gnu-gcc/g++ 配置 CMake，并执行 make -j4。

```
mkdir build
cd build
cmake -DCMAKE_C_COMPILER=/your_path/bin/loongarch64-linux-gnu-gcc \
      -DCMAKE_CXX_COMPILER=/your_path/bin/loongarch64-linux-gnu-g++ ..
make -j4
```

编译后使用 file build/train 检查是否为 LoongArch64 可执行文件。

```
CMake Warning at /tmp/loongTensor/share/cmake/Torch/TorchConfig.cmake:22 (message):
  static library eigen_blas_LIBRARY-NOTFOUND not found.
Call Stack (most recent call first):
  /tmp/loongTensor/share/cmake/Torch/TorchConfig.cmake:113 (append_torchlib_if_found)
  CMakeLists.txt:25 (find_package)

CMake Warning at /tmp/loongTensor/share/cmake/Torch/TorchConfig.cmake:22 (message):
  static library asmjit_LIBRARY-NOTFOUND not found.
Call Stack (most recent call first):
  /tmp/loongTensor/share/cmake/Torch/TorchConfig.cmake:123 (append_torchlib_if_found)
  CMakeLists.txt:25 (find_package)

-- Found Torch: /tmp/loongTensor/lib/libtorch.so
-- Configuring done
-- Generating done
```

```
[ 60%] Building CXX object CMakeFiles/train.dir/dataset.cpp.o
[ 60%] Building CXX object CMakeFiles/train.dir/model.cpp.o
[ 80%] Building CXX object CMakeFiles/train.dir/trainer.cpp.o
[ 80%] Building CXX object CMakeFiles/train.dir/main.cpp.o
[100%] Linking CXX executable train
[100%] Built target train
```

图 21-1 交叉编译结果

```
file train
train: ELF 64-bit LSB executable, LoongArch, version 1 (SYSV), dynamically linked, interpreter
/lib64/ld.so.1, for GNU/Linux 4.15.0, not stripped
```

21.2.3 部署到实验平台

将 data 和编译之后的 train 可执行文件保持上位机的相对目录的条件下上传到龙芯平台。同时将 loongTensor 运行库上传到指定的目录 /tmp/loongTensor 下，保证编译时的配置使得可执行文件能成功链接。

推荐使用 U 盘上传，使用 lsblk 确认设备名，执行 mount /dev/sdX /mnt/usb 后，将 /mnt/usb 中的对应文件复制到平台上。注意，开发板 /tmp 目录重启后会刷新，loongTensor、train 和 data 若放在 /tmp，下次上电需要重新部署。

```
lsblk
// 设备名: /dev/sda1
mount /dev/sda1 /mnt/usb

// 复制数据
cp /mnt/usb/data ./
// 复制可执行文件
cp /mnt/usb/train ./
// 复制 loongTensor 到 tmp
cp /mnt/usb/loongTensor /tmp

// 开始训练
./train
```

```
-bash-5.0# ./train
Error in cpuinfo: processor architecture is not supported in cpuinfo

Dataloader time : 0.0132437s
Train Epoch: 1 [ 992/ 1280] Loss: 0.7077 F1: 0.4103
Epoch 1 Average F1 Score: 0.603616
Epoch batch_sum 40 time : 171.919s

Inference : Average loss: 0.0217 | Accuracy: 0.487
Inference batch_sum: 10 time : 38.711s

Dataloader time : 0.0132777s
Train Epoch: 2 [ 992/ 1280] Loss: 0.6953 F1: 0.4516
Epoch 2 Average F1 Score: 0.489747
Epoch batch_sum 40 time : 171.048s

Inference : Average loss: 0.0216 | Accuracy: 0.541
Inference batch_sum: 10 time : 39.0746s
```

图 21-2 部署运行示例

可以看到在 cpuinfo 中存在一个报错，这是因为 libtorch 内置 cpuinfo 未适配龙芯 LoongArch，这可能导致其无法通过 cpuinfo 获取 CPU 拓扑、缓

存、SIMD 能力。但在本实验中，不影响训练的稳定进行，感兴趣的同学可以参考 [pytorch/cpuinfo README](#) 和 [cpuinfo CMakeLists.txt](#), 了解如何使 libtorch 支持 LoongArch 的 cpuinfo 初始化。

21.3 练习：优化训练过程

epoch 固定为 30, batch size 固定为 32; 评分关注 F1 score 提升, 以及训练/推理执行延迟降低。

先运行默认版本的训练, 获得 baselin, 之后可以根据不同的方向进行对应的优化:

1) 人工智能方向:

- 在 F1 不明显下降的前提下减少卷积通道数、kernel size 或 Dropout 开销, 并记录改动动机和参数量变化。
- 模型结构优化
- 冻结部分参数, 减少不必要计算
-

2) OS/内核方向

- 使用 taskset 或进程优先级控制减少外部干扰。
- 减少与实验无关的驱动、文件系统或调试模块, 降低系统噪声。
- 观察不同优先级、时间片或 CPU 亲和性设置对训练进程的影响。
-

第22章 龙芯教学平台的综合实验

22.1 实验目的

通过设计和实现一个嵌入式硬件互动控制系统,旨在深化学生对嵌入式系统原理和应用的理解。将之前学习的电机、温湿度传感器、蜂鸣器、点阵、LED 等硬件模块融合在一个系统中,使学生能够更全面地理解各种硬件的工作原理和相互配合关系。实验要求学生能够合理整合来自不同传感器的数据,实现对环境的全面感知,从而为系统的智能控制打下基础。通过本实验,学生将能够全面理解和应用嵌入式系统的关键概念,培养其在实际工程中设计和开发嵌入式应用的能力。

在综合实验中,需要创建两个根目录,一个目录用于存放内核驱动文件和对应的 Makefile 文件,另一个目录用于存放用户空间的工程文件,这两个目录在实验中不相互依赖。

但要注意如果是实际应用时,因为驱动会存放在内核源码的某个部分编译进内核,所以可以放置一些用户空间可以访问的头文件在内核的 include 目录,如某个模块 ioctl 的命令码等,本实验中并不涉及这些操作,所以会存在某些完全相同的代码在用户空间和内核空间分别声明的情况。

22.2 驱动模块的统一编译

首先创建一个目录,将需要使用的之前实验中的驱动代码文件复制到该目录,注意把同目录下的头文件也一并复制。

22.2.1 I2C 模块

因为实验中使用了 CH422G、AHT20 和 STK8BA 三个 I2C 设备,并且 AHT20 和 STK8BA 都挂载在 I2C 总线上(GPIO68、69),所以设备之间存在争用,即对 GPIO68、69 的操作存在临界区,并且这三个设备使用的是相同的底层 API,这时就可以考虑将 I2C 的 API 抽象为一个模块,该模块不对应任何设备,仅用于提供 API。

首先通过之前的实验内容确认,根据设备存在区别的位置通常在通信协议方面,所以可以认为 I2C 模块向外提供的 API 为 I2c_Start()、I2c_Stop()、I2c_WrByte()、I2c_RdByte(),有了这四个方法,便可以实现对任何 I2C 设备的操作。

在目录下创建 softi2c.c、softi2c.h 和 I2c_Pins.h 三个文件,I2c_Pins.h 文件的作用会在稍后解释。

I2c_Pins.h 下声明内容如下,其中包括了 I2c3 总线和 CH422g 的 GPIO 端口,并定义了一个结构体 I2c_Pins 用于表示 i2c 通信中的 sda 和 scl 线。

```
#ifndef __I2C_PINS__
#define __I2C_PINS__

#define SDA3_GPIO    68
#define SCL3_GPIO    69
```

```

#define CH422G_SDA_GPIO      142
#define CH422G_SCL_GPIO      141

typedef struct softi2c_pins {
    int sda;
    int scl;
} I2c_Pins;

#endif

```

在 softi2c.h 中声明内容如下，这里使用自旋锁仅为保险起见（死循环等待锁），如果可以确定该锁的获取和释放不会出现在任何中断处理函数中，可以选用互斥锁（休眠等待锁），可提高程序运行效率。其余内容和之前实验中声明相比，增加了一个 I2c_Pins 指针参数，用来指定 I2c 通信使用的 sda 和 scl。

```

#ifndef __SOFTI2C_H__
#define __SOFTI2C_H__
#include <linux/gpio.h>
#include <linux/spinlock.h>
//引入 I2c_Pins 结构体及 GPIO
#include "I2c_Pins.h"

#define I2C_WRITE_ADDR(addr) ((addr << 1) | 0)
#define I2C_READ_ADDR(addr) ((addr << 1) | 1)

//i2c 锁--自旋锁
extern void get_i2c_lock(I2c_Pins *pins);
extern void free_i2c_lock(I2c_Pins *pins);

#define ACK    1
#define NACK   0

extern void I2c_Start(I2c_Pins *pins);
extern void I2c_Stop(I2c_Pins *pins);
extern void I2c_WrByte(I2c_Pins *pins, unsigned char dat);
extern unsigned char I2c_RdByte(I2c_Pins *pins, int ack);
#endif

```

然后在 softi2c.c 中实现 API，注意提供 API 和模块和其他模块的不同，本模块不需要引用 softi2c.h 文件，该头文件是使用 i2c 的设备模块所需的，在本

模块中，仅需要实现这些接口，并通过 EXPORT_SYMBOL 让内核将这些方法放在内核的全局区便可，然后其他模块便可以先通过 softi2c.h 通过编译，运行时再在内核的全局区找到这些方法的实现位置。

```
#include <linux/delay.h>
#include <linux/init.h>
#include <linux/module.h>
#include <linux/gpio.h>
#include <linux/platform_device.h>
#include "I2c_Pins.h"

#define GPIO_INDEX_32(gpio) (u32)(1 << (gpio % 32))
#define I2C_DELAY_TIME      10

#define GPCFG_3              0x1fe104b0    //I2C3 0x00770000
#define GPCFG_CH422         0x1fe104d4    //CH422G 0x07700000
static void __iomem *GPCFG_V;
```

首先编写模块的初始化和退出方法，因为同一条总线同时可能运行多种设备，所以 GPIO 的申请/释放和复用配置要放在本模块中，设备驱动中便可以不进行相关操作。

```
static int __init softi2c_init(void)
{
    int ret = 0;
    u32 GPCFG_read, GPCFG_write;

    printk("softi2c api init.\n");

    GPCFG_V = ioremap(GPCFG_3, sizeof(u32));
    GPCFG_read = ioread32(GPCFG_V);
    GPCFG_write = (GPCFG_read & 0x77007777);
    iowrite32(GPCFG_write, GPCFG_V);
    iounmap(GPCFG_V);

    GPCFG_V = ioremap(GPCFG_CH422, sizeof(u32));
    GPCFG_read = ioread32(GPCFG_V);
    GPCFG_write = (GPCFG_read & 0x70077777);
    iowrite32(GPCFG_write, GPCFG_V);
    iounmap(GPCFG_V);

    gpio_request(SDA3_GPIO, "sda3");
    gpio_request(SCL3_GPIO, "scl3");
```

```

ret = gpio_direction_output(SDA3_GPIO, 1);
if(ret < 0) {
    printk("can't set sda3 gpio!\r\n");
}
ret = gpio_direction_output(SCL3_GPIO, 1);
if(ret < 0) {
    printk("can't set scl3 gpio!\r\n");
}

gpio_request(CH422G_SDA_GPIO, "ch422g_sda");
gpio_request(CH422G_SCL_GPIO, "ch422g_scl");
ret = gpio_direction_output(CH422G_SDA_GPIO, 0);
if(ret < 0) {
    printk("can't set ch422g sda gpio!\r\n");
}
ret = gpio_direction_output(CH422G_SCL_GPIO, 0);
if(ret < 0) {
    printk("can't set ch422g scl gpio!\r\n");
}

return 0;
}

```

```

static void __exit softi2c_exit(void)
{
    printk("softi2c api exit.\n");
    gpio_free(SDA3_GPIO);
    gpio_free(SCL3_GPIO);
    gpio_free(CH422G_SDA_GPIO);
    gpio_free(CH422G_SCL_GPIO);
}

module_init(softi2c_init);
module_exit(softi2c_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR("yangjinrun");
MODULE_ALIAS("SoftI2c_API");

```

然后声明一个自旋锁 `softi2c3_lock`，并编写获取、释放锁的方法，其中根据不同的总线获取不同的锁，这里保留了可扩展性，可以增加其他的锁。

```

DEFINE_SPINLOCK(softi2c3_lock);
void get_i2c_lock(I2c_Pins *pins)
{
    if (pins->sda == SDA3_GPIO && pins->scl == SCL3_GPIO)
    {
        spin_lock(&softi2c3_lock);
    }
    else
    {
        printk("i2c spinlock for %d,%d doesn't exist\n"
            , pins->sda, pins->scl);
    }
}
EXPORT_SYMBOL(get_i2c_lock);

void free_i2c_lock(I2c_Pins *pins)
{
    if (pins->sda == SDA3_GPIO && pins->scl == SCL3_GPIO)
    {
        spin_unlock(&softi2c3_lock);
    }
    else
    {
        printk("i2c spinlock for %d,%d doesn't exist\n"
            , pins->sda, pins->scl);
    }
}
EXPORT_SYMBOL(free_i2c_lock);

```

通用通信 API 在之前的实验中介绍过，这里仅对修改作介绍。
首先是 GPIO 操作，增加 pins 参数，并使用 inline 修饰符提高执行效率。

```

static inline void SDA_D_OUT(I2c_Pins *pins) {
    gpio_direction_output(pins->sda, 1);
}
static inline void SDA_D_IN(I2c_Pins *pins) {
    gpio_direction_input(pins->sda);
}
static inline void SCL_SET(I2c_Pins *pins) {
    gpio_set_value(pins->scl, 1);
}

```

```

static inline void SCL_CLR(I2c_Pins *pins) {
    gpio_set_value(pins->scl, 0);
}
static inline void SDA_SET(I2c_Pins *pins) {
    gpio_set_value(pins->sda, 1);
}
static inline void SDA_CLR(I2c_Pins *pins) {
    gpio_set_value(pins->sda, 0);
}
static inline int SDA_READ(I2c_Pins *pins) {
    return gpio_get_value(pins->sda);
}

```

start 和 stop 方法，增加了 pins 参数，并使用 EXPORT_SYMBOL 让其他模块可以访问这个方法。

```

void I2c_Start( I2c_Pins *pins )    // 操作起始
{
    SDA_D_OUT(pins);    // 设置 SDA 为输出方向
    SDA_SET(pins);    // 发送起始条件的数据信号
    SCL_SET(pins);
    udelay(I2C_DELAY_TIME);
    SDA_CLR(pins);    //发送起始信号
    udelay(I2C_DELAY_TIME);
    SCL_CLR(pins);    //钳住 I2C 总线，准备发送或接收数据
}
EXPORT_SYMBOL(I2c_Start);

void I2c_Stop( I2c_Pins *pins )    // 操作结束
{
    SDA_D_OUT(pins);    // 设置 SDA 为输出方向
    SDA_CLR(pins);
    SCL_CLR(pins);
    udelay(I2C_DELAY_TIME);
    SCL_SET(pins);
    SDA_SET(pins);    // 发送 I2C 总线结束信号
    udelay(I2C_DELAY_TIME);
}
EXPORT_SYMBOL(I2c_Stop);

```

接收函数中使用的 Ack 和 NAck 方法，增加 pins 参数。

```

static void I2c_Ack( I2c_Pins *pins ) {
    SCL_CLR(pins);
    SDA_D_OUT(pins);
    SDA_CLR(pins);
    udelay(I2C_DELAY_TIME);
    SCL_SET(pins);
    udelay(I2C_DELAY_TIME);
    SCL_CLR(pins);
}

static void I2c_NAck( I2c_Pins *pins ) {
    SCL_CLR(pins);
    SDA_D_OUT(pins);
    SDA_SET(pins);
    udelay(I2C_DELAY_TIME);
    SCL_SET(pins);
    udelay(I2C_DELAY_TIME);
    SCL_CLR(pins);
}

```

读写方法和 Start、Stop 方法相同，增加 pins 参数，并使用 EXPORT_SYMBOL 声明该函数名，不要落下参数 pins 的传递。

```

void I2c_WrByte( I2c_Pins *pins, unsigned char dat ) { // 写一个字节数据
    unsigned char i;
    SDA_D_OUT(pins);
    SCL_CLR(pins);
    for( i = 0; i != 8; i++ ) { // 输出 8 位数据
        if( dat & 0x80 ) SDA_SET(pins);
        else SDA_CLR(pins);
        dat <<= 1;
        udelay(I2C_DELAY_TIME);
        SCL_SET(pins);
        udelay(I2C_DELAY_TIME); // 可选延时
        SCL_CLR(pins);
    }
    SDA_SET(pins);
    udelay(I2C_DELAY_TIME);
    SCL_SET(pins); // 接收应答
    udelay(I2C_DELAY_TIME);
    SCL_CLR(pins);
}

EXPORT_SYMBOL(I2c_WrByte);

```

```

unsigned char I2c_RdByte( I2c_Pins *pins, int ack ) {
    unsigned char i, ret = 0;
    SDA_D_IN(pins);
    for (i = 0; i < 8; i++) {
        SCL_CLR(pins);
        udelay(I2C_DELAY_TIME);
        SCL_SET(pins);
        ret <<= 1;
        if (SDA_READ(pins)) ret++;
        udelay(I2C_DELAY_TIME);
    }
    if (ack) I2c_Ack(pins);
    else I2c_NAck(pins);
    return ret;
}

EXPORT_SYMBOL(I2c_RdByte);

```

至此，I2c 模块编写完毕，其他模块使用该模块 API 时，内核会自动识别到其他模块依赖 I2c 模块，要先插入 I2c 模块，再插入其他模块。

22.2.2 I2C 设备驱动修改

在上一节中，我们将 GPIO 口的初始化放在了 I2c 模块中，并在通用 API 上增加了 I2c_Pins 参数，所以这一节我们来修改 aht20.c，stk8ba.c 和 ch422g.c。因为对于这三个模块来讲，修改几乎是相同的（除了 ch422g 读写时不需要获取锁之外），所以本节只介绍 aht20.c 的详细修改和 ch422g.c 修改与其他两者的不同之处。

首先，全局声明仅需要保留设备名、子设备号和设备地址，并在设备结构体中将两个引脚声明换为 I2c_Pins 结构体变量 pins。

```

#define AHT20_NAME      "aht20"          /* 设备名 */
#define AHT20_MINOR     147              /* 子设备号 */
#define AHT20_ADDR      0x38

struct aht20_struct {
    dev_t devid;                /* 设备号 */
    void *private_data;        /* 私有数据 */
    I2c_Pins pins;
    unsigned char write_addr;
    unsigned char read_addr;
};

static struct aht20_struct aht20_dev;

```

除了发送命令和接收命令之外的所有 I2c 函数均可删除，并在这两个函数上增加设备结构体指针参数，用于获取 pins，函数内增加获取锁和释放锁的函数，并给所有 API 传递 pins 指针参数。请注意锁的使用，自旋锁无法重复获取，一个 get_i2c_lock() 必须在之后对应一个 free_i2c_lock()，如果在同一函数中连续执行 get_i2c_lock() 或 free_i2c_lock() 会导致模块崩溃，严重时可导致系统崩溃！

```
void AHT20_SendCmd( struct aht20_struct *dev, unsigned char *cmd_buf, size_t cnt ) {
    int i;
    get_i2c_lock(&dev->pins);
    I2c_Start(&dev->pins);    // 启动总线
    I2c_WrByte(&dev->pins, dev->write_addr);
    for (i = 0; i < cnt; i++) I2c_WrByte(&dev->pins, (unsigned char)(cmd_buf[i]));
    I2c_Stop(&dev->pins);    // 结束总线
    free_i2c_lock(&dev->pins);
}

void AHT20_Recv( struct aht20_struct *dev, unsigned char *buf, size_t cnt ) {
    int i;
    get_i2c_lock(&dev->pins);
    I2c_Start(&dev->pins);
    I2c_WrByte(&dev->pins, dev->read_addr);
    for (i = 0; i < cnt; i++) buf[i] = I2c_RdByte(&dev->pins, cnt - i - 1);
    I2c_Stop(&dev->pins);
    free_i2c_lock(&dev->pins);
}
```

然后对于 read/write 等调用以上通信函数的方法，需要先获取私有数据指针 dev，并将该参数传入通信函数。

```
static ssize_t aht20_read(struct file *filp, char __user *buf, size_t cnt, loff_t *offt)
{
    unsigned char databuf[7], cmd[3];
    int ret;
    struct aht20_struct *dev = (struct aht20_struct *)filp->private_data;

    mdelay(50);

    cmd[0] = 0xac;
    cmd[1] = 0x33;
    cmd[2] = 0x00;
    AHT20_SendCmd(dev, cmd, 3);
```

其余方法不变，请注意在模块初始化函数和卸载函数中，删除了已经在 I2c 模块中执行的大量初始化。

```

static int __init aht20_init(void) {
    int ret = 0;
    printk("aht20 module init\n");
    aht20_dev.pins.sda = SDA3_GPIO;
    aht20_dev.pins.scl = SCL3_GPIO;
    aht20_dev.write_addr = I2C_WRITE_ADDR(AHT20_ADDR);
    aht20_dev.read_addr = I2C_READ_ADDR(AHT20_ADDR);

    ret = misc_register(&aht20_miscdev);
    if(ret < 0){
        printk("misc device register failed!\n");
        return -EFAULT;
    }
    return ret;
}

```

对于 ch422g.c，第一点不同就是其无需加锁，如果想要统一代码格式，也可以在 I2c 模块中增加一个锁（如 DEFINE_SPINLOCK(i2c_ch422_lock)），再在获取/释放锁方法中增加对 ch422g 对应引脚的判断和加锁。

```

static void CH422_SendCmd(struct ch422g_struct *dev, unsigned short cmd) {
    //因为 ch422g 使用单独的 GPIO，可以不用加锁
    //get_i2c_lock(&dev->pins);
    I2c_Start(&dev->pins);    // 启动总线
    I2c_WrByte(&dev->pins, (unsigned char)(cmd>>8));
    I2c_WrByte(&dev->pins, (unsigned char)cmd);    // 发送数据
    I2c_Stop(&dev->pins);    // 结束总线
    //free_i2c_lock(&dev->pins);
}

```

还有一点不同便是引脚编号，使用在 I2c_Pins.h 中定义的另一对 GPIO 引脚，其他内容改动方式与另外两个文件相同。

```

ch422g_dev.pins.sda = CH422G_SDA_GPIO;
ch422g_dev.pins.scl = CH422G_SCL_GPIO;

```

22. 2. 3 其余驱动修改

其余驱动的操作方式均不相同，可以直接复制到目录下，但需要对代码进行一定优化。

之前的驱动中配置复用寄存器时，映射得到的虚拟地址在模块卸载时才释放，如果两个模块使用的复用寄存器地址重合，可能会导致问题，所以配置完复用配置寄存器后，直接调用 ioummap 释放虚拟地址到物理地址的映射。

```

GPCFG_V = ioremap(GPCFG, sizeof(u32));
GPCFG_read = ioread32(GPCFG_V);
GPCFG_write = GPCFG_read & 0x 77777707;
iowrite32(GPCFG_write, GPCFG_V);
iounmap(GPCFG_V);

```

为了用户空间使用方便和安全，将一些接口删除或转换得更合理，如删除 stk8ba.c 中的 write 方法，将 stk8ba 作为只读设备。或将蜂鸣器 beep.c 中的 write 换成 ioctl，使用 BEEP_ON 和 BEEP_OFF 控制蜂鸣器的开关：

```

static long beep_ioctl(struct file *filp, unsigned int cmd, unsigned long arg) {
    struct beep_struct *dev = filp->private_data;
    if (_IOC_TYPE(cmd) != BEEP_MAGIC)
        return -EINVAL;

    switch (cmd)
    {
    case BEEP_ON:
        gpio_set_value(dev->beep_gpio, 0);    /* 打开蜂鸣器 */
        break;
    case BEEP_OFF:
        gpio_set_value(dev->beep_gpio, 1);    /* 关闭蜂鸣器 */
        break;
    default:
        break;
    }
    return 0;
}

```

```

/* 设备操作函数 */
static struct file_operations beep_fops = {
    .owner = THIS_MODULE,
    .open = beep_open,
    .unlocked_ioctl = beep_ioctl,
    .release = beep_release,
};

```

将设备结构体的结构体名和变量名特殊化，如上一节中的 aht20，可以增加可维护性。

```

struct aht20_struct {
    dev_t devid;          /* 设备号      */
    void *private_data;    /* 私有数据 */
    I2c_Pins pins;
    unsigned char write_addr;
    unsigned char read_addr;
};
static struct aht20_struct aht20_dev;

```

之后便可以编写 Makefile 文件, 统一编译所有模块, 将 KDIR 赋值为 MY_KDIR 的值, MY_KDIR 是使用 setenv.sh 或直接编辑 ~/.bashrc (不推荐) 配置的路径, 为目标内核所在目录, MY_KDIR 的值就是之前实验中 KDIR 的值, 将所有 Makefile 中的 KDIR 换为 MY_KDIR, 方便随时切换内核源码进行编译。

```

obj-m += softi2c.o
obj-m += aht20.o
obj-m += beep.o
obj-m += ch422g.o
obj-m += motor.o
obj-m += rfid.o
rfid-objs := rc522.o rc522_api.o
obj-m += stk8ba.o
obj-m += aip1944.o
obj-m += led.o
obj-m += key.o
obj-m += irdrv.o
KDIR := $(MY_KDIR)
all:
    make -C $(KDIR) M=$(PWD) modules
clean:
    make -C $(KDIR) M=$(PWD) clean

```

22.3 创建工程项目

与之前的项目不同, 这次的用户空间程序使用工程项目格式创建。注意工程项目格式存在多种模板, 本实验演示中仅介绍其中一种。

首先在工程项目目录下创建 src、inc、obj 三个目录, 并创建一个 Makefile 文件, 如图 20-1。



图 20-1 创建文件

Makefile 文件编写内容如下，将最终生成的可执行文件（app）所需的文件名.o 格式使用+=添加到 objs 中，并且对应的.c 文件均存在于 src 目录中，即可通过 make 命令自动编译生成可执行文件文件。编译选项“-lpthread”可选，如果需要进行多线程编程则使用该选项，有能力的同学可以自行研究并修改/编写 Makefile 文件。

```
CC =loongarch64-linux-gnu-gcc
INC =-I ./inc
SRC =./src
OBJ =./obj
objs +=main.o
objs +=aht20.o
objs +=beep.o
objs +=motor.o
objs +=stk8ba.o
objs +=aip1944.o
objs +=rc522.o
objs +=ch422g.o
objs +=key16.o
objs +=led.o
objs +=ir.o

opath=$(patsubst %,$(OBJ)/%,$(objs))
target =app

all: $(target)
$(target):$(opath)
    $(CC) -O2 -lpthread $^ -o $@ $(INC)
$(opath):$(OBJ)/%.o:$(SRC)/%.c
    $(CC) -O2 $^ -c -o $@ $(INC)

clean:
    rm $(target) $(opath)
```

在 src 目录中创建源文件，如 main.c，aht20.c，beep.c，注意 src 目录中所有文件只能有一个 main() 函数，其余文件用于提供 API。声明的 API 在对应的 aht20.h、beep.h 等头文件中，头文件同一放在 inc 目录下。obj 目录用于暂存编译生成的.o 文件。以上述的三个文件来演示文件结构，如图 20-2，左图为编译前，右图为编译后，编译后产生了.c 文件对应的.o 文件（中间文件），最后由所有.o 文件编译生成 app 文件。

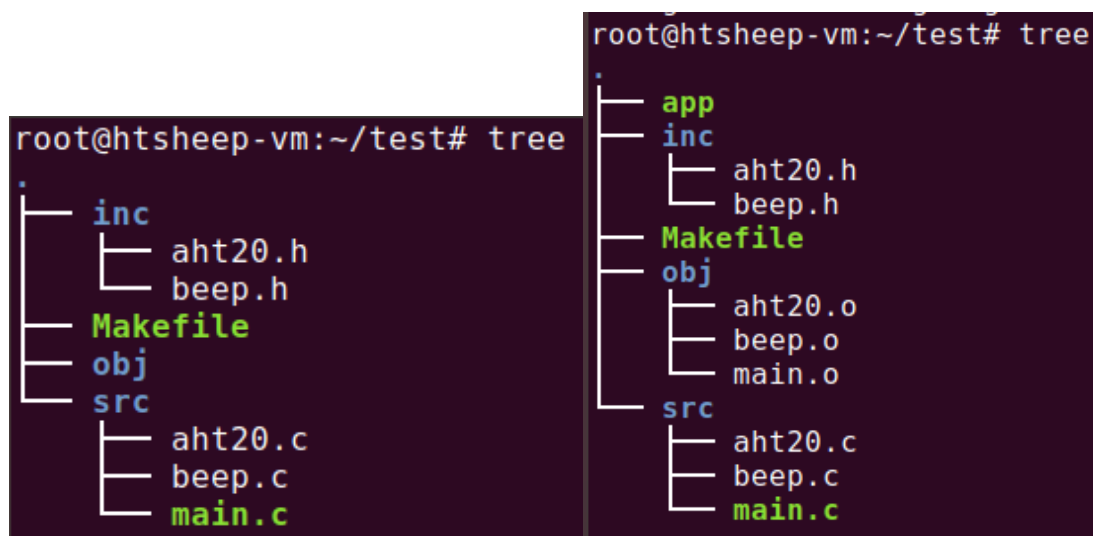


图 20-2 项目目录树结构

```
root@htsheep-vm:~/test# make
loongarch64-linux-gnu-gcc -O2 -lpthread src/main.c -c -o obj/main.o -I ./inc
loongarch64-linux-gnu-gcc -O2 -lpthread src/aht20.c -c -o obj/aht20.o -I ./inc
loongarch64-linux-gnu-gcc -O2 -lpthread src/beep.c -c -o obj/beep.o -I ./inc
loongarch64-linux-gnu-gcc -O2 obj/main.o obj/aht20.o obj/beep.o -o app -I ./inc
```

图 20-3 编译项目

src 中 main.c 用于声明 main() 函数，执行程序功能，其他源文件用于实现对应设备的 API，对应的头文件用于声明 API 函数，供 main.c 使用，当然执行程序功能函数可以放在特殊的文件中，也可以直接放在对应设备的源文件中，可根据自己习惯进行编写，下一节将介绍设备的 API 编写。

22.4 设备 API

考虑到设备较多，因此仅对两个简单的设备进行详细介绍，一个主要用于输入，一个主要用于输出，其余设备将只介绍头文件提供的 API，具体实现请自行查阅实验提供的源代码。

22.4.1 温湿度传感器

温湿度传感器 AHT20 在实验中属于输入设备，系统使用其传入的数据，仅需对其进行打开、关闭和读取数据操作，考虑到用户在读取数据时大部分情况下只需要读取实际的温湿度，所以提供了三个读取 API：读取原数据、读取温湿度数据（1 位小数）、解码温湿度（1 位小数）。如果需要精确到 2 位小数的温度数据可自行编写，之前的实验指导书有介绍解码原理。

aht20.h 文件声明如下，其告知用户原数据大小为 7 字节，并提供了 5 个方法，操作时并不需要用户提供文件描述符 fd，也就是使系统调用层透明化，让用户更直观地使用。

由于需要获取的数据为数组或 2 个数据温、湿度，所以使用指针回传，并通过返回值提示操作结果。

```

#ifndef __AHT20_H__
#define __AHT20_H__

#define AHT20_ORI_SIZE      7    //Byte

extern int open_aht20(void);
extern int close_aht20(void);
extern int read_aht20_origin(unsigned char *databuf);
extern void aht20_encode(unsigned char *databuf, int *ret_tem, int *ret_hum);
extern int read_aht20(int *ret_tem, int *ret_hum);

#endif

```

aht20.c 文件全局预处理和定义如下，在全局静态声明文件名和文件描述符 fd，使其作用域限制在本文件，避免用户访问，而且不用考虑变量名特殊化。

```

#include "aht20.h"
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/stat.h>
static char filename[] = "/dev/aht20";
static int fd = -1;

```

开启和关闭设备函数，为保证 API 自由度，在 open 函数返回文件描述符，如果 API 能提供足够丰富的功能，可以考虑不返回 fd（返回 0 作为成功标志）。

在 close 方法中关闭文件后记得将 fd 重新置为-1，用于其他方法判断文件是否被打开。

```

int open_aht20(void) {
    fd = open(filename, O_RDWR);
    if (fd < 0) {
        printf("can't open file %s\n", filename);
        return -1;
    }
    return fd;
}

```

```

int close_aht20(void) {
    int ret;
    ret = close(fd);
    if (ret < 0) {
        printf("can't close file %s\n", filename);
        return -1;
    }
    fd = -1;
    return ret;
}

```

通常可以提供较为底层的 API，其可以达到和系统调用相同的自由度，获取原数据的 API 对于输出设备就是这样的一个底层方法，在 API 中做好严谨的判断可以减轻调试压力。

注意 read 和 write 的返回值，它们的返回值就是驱动中覆写的 read 和 write 的返回值，通常代表读取/写入的字节数，如果这一步返回有问题，可以自行到对应驱动中修改，正常读取时 read 返回用户传入的欲读取大小，正常写入时返回内核写入的数据大小。

```

int read_aht20_origin(unsigned char *databuf) {
    int ret;
    if (fd < 0) {
        printf("没有打开文件:%s\n", filename);
        return -1;
    }
    ret = read(fd, databuf, AHT20_ORI_SIZE);
    if (ret != AHT20_ORI_SIZE || (databuf[0] & 0x68) != 0x08) {
        printf("数据读取失败\n");
        return -1;
    }
    return 0;
}

```

aht20 的解码函数，之前的指导书中讲到过解码方法和为什么选择用整数表示，这里返回的值保留了 1 位小数，即将返回值/10.0 可以得到实际温湿度。

```

void aht20_encode(unsigned char *databuf, int *ret_tem, int *ret_hum) {
    int tem, hum;
    hum = databuf[1];
    hum = (hum<<8) | databuf[2];
    hum = (hum<<8) | databuf[3];
    hum >>= 4;
    hum = (hum*1000) >> 20;
}

```

```

        tem = databuf[3];
        tem &= 0x0000000F;
        tem = (tem<<8) | databuf[4];
        tem = (tem<<8) | databuf[5];
        tem = ((tem*2000) >> 20) - 500;
        *ret_hum = hum;
        *ret_tem = tem;
    }

```

最后是常用的顶层 API，通常顶层 API 建立在严谨的底层 API 之上，所以一些常见的判断（如是否打开设备、返回是否正常等）可以不再做判断和打印，当然想要根据不同方法进行错误信息的打印是更好的，可以逐层排除问题。

```

int read_aht20(int *ret_tem, int *ret_hum) {
    unsigned char ori[AHT20_ORI_SIZE];
    int ret, hum, tem;
    if (read_aht20_origin(ori) < 0) return -1;
    aht20_encode(ori, ret_tem, ret_hum);
    return 0;
}

```

22. 4. 2 数码管

数码管是一个典型的输出设备，其解码传入的数据然后将其显示在数码管上。在编写 API 时，首先要保证其底层 API 能实现尽量精细粒度的操作，然后再对常用操作进行抽象。

试验箱中的数码管有 4 个，每个数码管有 8 个 LED 灯，因此最精细的粒度就是实现对每个灯的操作，这就需要设置一个缓冲区，在用户对每个数码管做完操作后，可以将其一并写入，提高操作和显示效率。

例程中编写了如下的 API，其中提供了多种底层 API 可供选用，带后缀_flush 的方法是执行完相同前缀方法后执行 ch422g_flush() 马上写入让其显示在数码管上，篇幅起见之后仅会介绍一个。

通过简单的宏定义，让用户可以通过或操作制作掩码，再通过 set_mask 或 clear_mask 实现掩码对应灯的亮/灭。

```

#ifndef __CH422G_H__
#define __CH422G_H__
#define CH422G_M1 0x01 //中上
#define CH422G_R1 0x02 //右上
#define CH422G_R2 0x04 //右下
#define CH422G_M3 0x08 //中下
#define CH422G_L2 0x10 //左下
#define CH422G_L1 0x20 //左上
#define CH422G_M2 0x40 //中间
#define CH422G_PT 0x80 //点

extern int open_ch422g(void);
extern int close_ch422g(void);
extern void ch422g_flush(void);
extern void ch422g_set_char(int idx, char ch);
extern void ch422g_set_char_flush(int idx, char ch);
extern void ch422g_set_num(int idx, int num);
extern void ch422g_set_num_flush(int idx, int num);
extern void ch422g_set_tube(int idx, unsigned char ch);
extern void ch422g_set_tube_flush(int idx, unsigned char ch);
extern void ch422g_set_mask(int idx, unsigned char mask);
extern void ch422g_set_mask_flush(int idx, unsigned char mask);
extern void ch422g_clear_mask(int idx, unsigned char mask);
extern void ch422g_clear_mask_flush(int idx, unsigned char mask);
extern void ch422g_reset(void);
extern const unsigned char BCD_decode_tab[];
#endif

```

头文件和全局声明，这里增加了一个缓冲区 `tubes` 和解码数组 `BCD_decode_tab`。`open` 和 `close` 函数和 `aht20` 完全相同（如果想要关闭文件后数码管清空，可以在 `close` 方法里调用 `close()` 前调用 `ch422g_reset()`，具体会在之后介绍）。

```

#include "ch422g.h"
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/types.h>

```

```

#include <sys/stat.h>
#include <sys/types.h>
#include <sys/ioctl.h>

static char filename[] = "/dev/ch422g";
static int fd = -1;
static unsigned char tubes[5];
const unsigned char BCD_decode_tab[ 29 ] = { 0X3F, 0X06, 0X5B, 0X4F, 0X66, 0X6D, 0X7D, 0X07,
0X7F, 0X6F, 0X77, 0X7C, 0X58, 0X5E, 0X79, 0X71, 0x00, 0x46, 0x40, 0x41, 0x39, 0x0F, 0x08, 0x76,
0x38, 0x73, 0x80, 0xFF, 0x00 };

```

底层方法 `ch422g_flush()` 是唯一一个直接使用系统调用 `write()` 的方法，其将缓冲区 `tubes` 发送给内核驱动。

```

void ch422g_flush(void) {
    if (fd < 0) {
        printf("没有打开文件: %s\n", filename);
        return;
    }
    if (write(fd, tubes, sizeof(tubes)) < 0) {
        printf("数码管写入失败\n");
    }
}

```

数码管支持显示一些简单字符，可以定义一个 `set_char` 方法用于使某个数码管显示字符，如下给出了十六进制数和空格的显示操作，想要显示更多字符可以自行研究。

```

void ch422g_set_char(int idx, char ch) {
    if (ch <= '9' && ch >= '0') tubes[idx] = BCD_decode_tab[ch - '0'];
    else if (ch <= 'F' && ch >= 'A') tubes[idx] = BCD_decode_tab[10 + ch - 'A'];
    else if (ch <= 'f' && ch >= 'a') tubes[idx] = BCD_decode_tab[10 + ch - 'a'];
    else if (ch == ' ') tubes[idx] = '\0';
    else tubes[idx] = ch;
}

```

`_flush` 后缀仅为多调用一个 `ch422g_flush()` 自动刷新，之后不再介绍。

```

void ch422g_set_char_flush(int idx, char ch) {
    ch422g_set_char(idx, ch);
    ch422g_flush();
}

```

数码管较为常用的是设置数字，因此编写 `set_num` 方法用于显示一位数字。

```
void ch422g_set_num(int idx, int num) {  
    tubes[idx] = BCD_decode_tab[num % 10];  
}
```

如果读者通过头文件的宏定义和或操作，自己创建了符号，可以通过 `set_tube` 写入。

```
void ch422g_set_tube(int idx, unsigned char ch) {  
    tubes[idx] = ch;  
}
```

如果通过宏定义创建了掩码，可以通过下面的两个方法置位/清零。

```
void ch422g_set_mask(int idx, unsigned char mask) {  
    tubes[idx] |= mask;  
}
```

```
void ch422g_clear_mask(int idx, unsigned char mask) {  
    tubes[idx] &= ~mask;  
}
```

如果想要清空数码管，可以使用 `reset` 方法。

```
void ch422g_reset(void) {  
    ch422g_set_tube(1, 0x00);  
    ch422g_set_tube(2, 0x00);  
    ch422g_set_tube(3, 0x00);  
    ch422g_set_tube(4, 0x00);  
    ch422g_flush();  
}
```

使用上面介绍的 API 可以实现任何显示，如显示 AHT20 传来的温度数据，可以先设置为数字，再使用 `set_mask` 添加小数点。

```
ch422g_set_num(2, tem / 100);  
ch422g_set_num(3, tem / 10);  
ch422g_set_mask(3, CH422G_PT);  
ch422g_set_num(4, tem);  
ch422g_flush();
```

22. 4. 3 蜂鸣器

蜂鸣器 API 如下，对之前的代码进行了改动，使用了更为方便的 `ioctl`，当然对于 API 的使用者来说是透明的，完全可以使用之前的 `write` 实现。

```
#ifndef __BEEP_H__
#define __BEEP_H__

#define BEEP_MAGIC    'b'
#define BEEP_OFF      _IO(BEEP_MAGIC, 1)    /* 关蜂鸣器 */
#define BEEP_ON       _IO(BEEP_MAGIC, 2)     /* 开蜂鸣器 */

extern int open_beep(void);
extern int close_beep(void);
extern void beep_on(void);
extern void beep_off(void);

#endif
```

22. 4. 4 电机

电机 API 如下，提供了底层间接调用 `ioctl` 的 `control` 方法，和直接控制电机模式的四个方法。

```
#ifndef __MOTOR_H__
#define __MOTOR_H__

#define MOTOR_MAGIC    'k'
#define CMD_standby_IO(MOTOR_MAGIC, 1)
#define CMD_forward_IO(MOTOR_MAGIC, 2)
#define CMD_backward   _IO(MOTOR_MAGIC, 3)
#define CMD_brake      _IO(MOTOR_MAGIC, 4)

extern int open_motor(void);
extern int close_motor(void);
extern int motor_control(unsigned int cmd);
extern void motor_standby(void);
extern void motor_forward(void);
extern void motor_backward(void);
extern void motor_brake(void);

#endif
```

22. 4. 5 重力传感器

重力传感器提供了原数据读取和解码后 xyz 数据读取方法。

```
#ifndef __STK8BA_H__
#define __STK8BA_H__

#define STK8BA_ORI_SIZE      6
#define STK8BA_XYZ_SIZE     3

extern int read_stk8ba_xyz(int *databuf);
extern int read_stk8ba_origin(unsigned char *databuf);
extern int open_stk8ba(void);
extern int close_stk8ba(void);

#endif
```

22. 4. 6 智能卡

智能卡 API 提供了底层的 ioctl 间接调用，和几个常用功能的调用 API，如需其他功能可自行实现

```
#ifndef __RC522_H__
#define __RC522_H__

#include <stddef.h>

#define RC522_PASSWD_SIZE  6
#define RC522_ID_SIZE      4
#define RC522_BLOCK_SIZE   16

#define RC522_MAGIC         'r'
#define RC522_CHANGE_PASSWD _IOW(RC522_MAGIC, 1, unsigned char [RC522_PASSWD_SIZE])
#define RC522_CHANGE_BLOCK  _IOW(RC522_MAGIC, 2, unsigned char)
#define RC522_READ_CARD     _IO(RC522_MAGIC, 3)
#define RC522_WRITE_CARD    _IO(RC522_MAGIC, 4)
#define RC522_CHANGE_KEY    _IO(RC522_MAGIC, 5)
#define RC522_GET_ID        _IOR(RC522_MAGIC, 6, char [RC522_ID_SIZE])
```

```

extern int open_rc522(void);
extern int close_rc522(void);
extern int rc522_ioctl(unsigned int cmd, unsigned long arg);
extern int rc522_getid(char *id);
extern int rc522_change_passwd(unsigned char *passwd);
extern int rc522_change_block(unsigned char block);
extern int rc522_write(char *writebuf, size_t cnt);
extern int rc522_read(char *readbuf);
#endif

```

22. 4. 7 16*16 点阵

点阵 API 实现了设置数据、配置掩码等功能，实现了一个播放连续字模（数据）的 API，需要传入 32*n 字节的规定格式的数据，可以实现文字的滚动/幻灯片播放，可以根据需求更改为二级指针参数以实现更灵活的控制，或者增加延时参数用于控制播放速度等。

```

#ifndef __AIP1944_H__
#define __AIP1944_H__

#define AIP1944_SLIDE_MODE 1
#define AIP1944_ROLL_MODE 2

extern unsigned char aip1944_demo[384];

extern int open_aip1944(void);
extern int close_aip1944(void);
extern void aip1944_flush(void);
extern void aip1944_set_data(unsigned char *buf);
extern void aip1944_set_mask(unsigned char *buf);
extern void aip1944_clear_mask(unsigned char *buf);
extern void aip1944_display(unsigned char *buf, int byte_cnt, int mode);
extern void aip1944_display_clear(void);

#endif

```

22. 4. 8 4*4 键盘

键盘 API 实现了简单的扫描函数。

```

#ifndef __KEY16_H__
#define __KEY16_H__

#include <stdint.h>

#define KEY16(idx)  (uint16_t)(1 << idx)

extern int open_key16(void);
extern int close_key16(void);
extern uint16_t key_scan(void);

#endif

```

22. 4. 9 LED

LED 实现了灯的单独和统一控制，提供反转函数 `led_neg`，并且需要修改 LED 数目时仅修改 `LED_COUNT` 即可。

```

#ifndef __LED_H__
#define __LED_H__

#define LED_COUNT 3

extern int open_led(void);
extern int close_led(void);
extern void led_control(unsigned char *buf);
extern void led_on(int idx);
extern void led_off(int idx);
extern void led_neg(int idx);

#endif

```

22. 4. 10 红外遥控

红外部分定义了读取和解码 API，将 `input_event` 结构体解码为 `key_table_t` 结构体并回传。

```

#ifndef __KEY_CODE_H
#define __KEY_CODE_H

#include <linux/input.h>

```

```

struct key_table_t {
    unsigned char key_val;
    unsigned char key_code;
    unsigned char key_name[20];
};

#define KEY_NUM sizeof(key_table) / sizeof(key_table[0])

extern int open_ir(void);
extern int close_ir(void);
extern int ir_read(struct input_event *iebuf);
extern int ir_encode(struct input_event *ie, struct key_table_t *ktbuf);

#endif

```

22.5 测试 API 功能

在工程中 main 函数的结构通常为初始化-主循环-释放内存，通常主循环中每次循环可以进行键盘扫描、执行操作、数据更新、配置设备等。本节中将编写一个 main.c 文件模板，编写完模板后，可根据自己需要的功能添加 API，并在主循环中添加对应判断和操作等。

22.5.1 依赖头文件

在 main.c 中声明头文件，同时包含 API 头文件。

```

#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <pthread.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include "aht20.h"
#include "stk8ba.h"
#include "motor.h"
#include "beep.h"
#include "ch422g.h"
#include "rc522.h"
#include "aip1944.h"
#include "key16.h"
#include "led.h"
#include "ir.h"

```

22.5.2 全局声明

定义全局变量和命令码等，首先声明了 `msleep` 用于实现毫秒级的休眠（用户空间只有秒和微秒的休眠函数），然后声明了系统中红外遥控使用的结构体，因为红外遥控的 `read` 方法在读取不到时会阻塞，所以需要有一个线程用于读取红外遥控，然后在主线程中处理红外遥控发来的消息，这就需要标志位 `ir_flag` 和互斥锁 `ir_lock`。

然后声明了键盘命令码 `System_OP_Type` 类型，用于处理键盘信息。

此外定义了 `logo_pth`，用于运行自动播放 `logo` 的线程。

声明了系统的初始化、退出函数，并定义了键盘解码和键盘事件函数，用于处理用户按下按键后的操作。此外还定义了 `ch422g_showdata`，可以通过键盘控制 `ch422g_mode`，以在 `ch422g` 上显示温湿度。

```
#define msleep(ms)    usleep(1000 * (ms))

struct ir_info_struct {
    struct input_event ir_event;
    int ir_flag;
    pthread_mutex_t ir_lock;
};

typedef enum key_op {
    OP_LED1,
    OP_LED2,
    OP_LED3,
    OP_SHOW_TEM,
    OP_SHOW_HUM,
    OP_BEEP,
    OP_MOTOR_STANDBY,
    OP_MOTOR_FORWARD,
    OP_MOTOR_BACKWARD,
    OP_MOTOR_BRAKE,
    OP_ERROR
} System_OP_Type;
```

声明全局变量，三个 `pthread_t` 变量对应 `logo`、红外遥控、智能卡三个线程，这些都是可能阻塞或需要独自执行的代码，注意播放 `logo` 的是点阵设备，如果需要用点阵设备做别的工作，那么可以根据需求选择是否使用线程。

`logo_mode` 用于确定 `logo` 的显示方式。

全局变量 `xyz`、`tem`、`hum` 用于存储重力数据和温湿度数据，例程中是在主线程（主函数）中获取并处理数据，所以没有加锁，如果想要通过线程获取数据，那么可以分别进行加锁（比如定义 `xyz_lock` 和 `th_lock` 分别对应重力数据和温湿度数据）。

ch422g_mode 变量用于确定数码管显示什么内容。

```
pthread_t logo_pth, ir_pth, rc_pth;
int logo_mode = AIP1944_SLIDE_MODE;
int xyz[3], tem, hum;
struct ir_info_struct ir_info;

enum {
    CH422G_NOTHTING,
    CH422G_TEM,
    CH422G_HUM
} ch422g_mode;
```

最后声明系统中的方法，分别定义了系统初始化、系统退出、键盘解码、键盘处理、红外处理和刷新数码管函数。

```
void system_init(void);
void system_exit(void);
System_OP_Type key_decode(uint16_t key);
void key_handler(uint16_t key);
void ir_handler(struct input_event *ir_event);
void ch422g_showdata(void);
```

22.5.3 线程函数

按线程标准格式 `void (*)(function)(void *)` 声明线程各个主函数，首先编写 logo 线程。主循环中不断调用 API 显示 logo，并设置主线程发送 cancel 信号可以立即取消此线程。

```
void *logo_display(void *arg) {
    //收到 cancel 信号，取消此线程
    pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL);
    //立即取消此线程
    pthread_setcanceltype(PTHREAD_CANCEL_ASYNCHRONOUS, NULL);
    while (1) {
        aip1944_display(aip1944_demo, sizeof(aip1944_demo), logo_mode);
        aip1944_display_clear();
    }
}
```

红外遥控线程因为涉及临界区（不同线程可以访问同一变量，如定义的 ir_info 变量），所以需要加锁，这里使用的是线程互斥锁 pthread_mutex_t，加锁和释放锁之间的代码应尽可能少，尽量将需要加锁的代码放在同一部分（如

果两段需要加锁的代码相隔的代码过多，可以考虑对两段分别加锁、释放锁，效率可能更高）。

这里的 `ir_flag` 是用于确定是否接到消息，如果接到消息则置为 1，之后该线程则仅接收而不写入，而主线程中会在一轮循环中查看到 `ir_flag` 被置 1，读取之后再将其置 0，即完成一轮信号输入。

```
void *ir_receive(void *arg) {
    struct input_event eventbuf;
    //收到 cancel 信号，取消此线程
    pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL);
    //立即取消此线程
    pthread_setcanceltype(PTHREAD_CANCEL_ASYNCHRONOUS, NULL);
    while (1) {
        ir_read(&eventbuf);
        pthread_mutex_lock(&ir_info.ir_lock);
        if (!ir_info.ir_flag) {
            memcpy(&ir_info.ir_event, &eventbuf, sizeof(eventbuf));
            ir_info.ir_flag = 1;
        }
        pthread_mutex_unlock(&ir_info.ir_lock);
    }
}
```

智能卡因为涉及功能较复杂，所以智能卡线程仅判断是否读取到卡，如果读取到卡则 led 灯亮起，否则熄灭。

```
void *rc522_handler(void *arg) {
    char readbuf[RC522_BLOCK_SIZE];
    int ret;
    //收到 cancel 信号，取消此线程
    pthread_setcancelstate(PTHREAD_CANCEL_ENABLE, NULL);
    //立即取消此线程
    pthread_setcanceltype(PTHREAD_CANCEL_ASYNCHRONOUS, NULL);
    while (1) {
        ret = rc522_read(readbuf);
        if (ret == sizeof(readbuf)) led_on(0);
        else led_off(0);
    }
}
```

22.5.4 主线程

主线程即主函数，`main` 函数较简单，其进行系统初始化后进入主循环，每轮

循环获取按键输入、判断是否退出、按键操作、红外操作、读取传感器、显示数码管，并在确定退出时调用系统退出函数。

```
int main(int argc, char *argv[])
{
    uint16_t key;
    struct input_event ir_event;

    system_init();

    while (1)
    {
        key = key_scan();
        if (key & KEY16(15))
            break;

        key_handler(key);
        ir_handler(&ir_event);
        read_aht20(&tem, &hum);
        read_stk8ba_xyz(xyz);
        //printf("x:%d, y:%d, z:%d\n", xyz[0], xyz[1], xyz[2]);
        ch422g_showdata();

    }

    system_exit();
    return 0;
}
```

系统初始化函数用于开启设备和运行线程。

```
void system_init(void) {
    int ret = 0;
    open_stk8ba();
    open_aht20();
    open_beep();
    open_motor();
    open_aip1944();
    open_rc522();
    open_ch422g();
    open_key16();
    open_led();
    open_ir();
}
```

```

ret = pthread_create(&logo_pth, NULL, logo_display, NULL);
if (ret) {
    printf("logo 初始化失败! \n");
    return;
}
ret = pthread_create(&ir_pth, NULL, ir_receive, NULL);
if (ret) {
    printf("红外处理初始化失败! \n");
    return;
}
ret = pthread_create(&rc_pth, NULL, rc522_handler, NULL);
if (ret) {
    printf("智能卡初始化失败! \n");
    return;
}
printf("系统初始化成功\n");
}

```

系统退出函数会发送 cancel 消息给线程，等待线程结束后关闭所有设备。

```

void system_exit(void) {
    pthread_cancel(logo_pth);
    pthread_cancel(ir_pth);
    pthread_cancel(rc_pth);
    pthread_join(logo_pth, NULL);
    pthread_join(ir_pth, NULL);
    pthread_join(rc_pth, NULL);

    close_stk8ba();
    close_aht20();
    close_beep();
    close_motor();
    close_aip1944();
    close_rc522();
    close_ch422g();
    close_key16();
    close_led();
    close_ir();
    printf("系统退出\n");
}

```

按键解码函数根据按下的按键返回命令码，这里对按键 0~9 配置了功能。

```

System_OP_Type key_decode(uint16_t key) {
    if (key & KEY16(0))
        return OP_LED1;
    if (key & KEY16(1))
        return OP_LED2;
    if (key & KEY16(2))
        return OP_LED3;
    if (key & KEY16(3))
        return OP_SHOW_TEM;
    if (key & KEY16(4))
        return OP_SHOW_HUM;
    if (key & KEY16(5))
        return OP_BEEP;
    if (key & KEY16(6))
        return OP_MOTOR_STANDBY;
    if (key & KEY16(7))
        return OP_MOTOR_FORWARD;
    if (key & KEY16(8))
        return OP_MOTOR_BACKWARD;
    if (key & KEY16(9))
        return OP_MOTOR_BRAKE;
    return OP_ERROR;
}

```

键盘事件函数用于处理不同的命令。

```

void key_handler(uint16_t key)
{
    static int beep_is_on = 0;

    System_OP_Type op = key_decode(key);
    switch (op)
    {
    case OP_LED1:
        led_neg(0);
        break;
    case OP_LED2:
        led_neg(1);
        break;
    case OP_LED3:
        led_neg(2);
        break;
    }
}

```

```

case OP_SHOW_TEM:
    ch422g_mode = CH422G_TEM;
    break;
case OP_SHOW_HUM:
    ch422g_mode = CH422G_HUM;
    break;
case OP_BEEP:
    if (beep_is_on)
        beep_off();
    else
        beep_on();
    beep_is_on = beep_is_on ? 0 : 1;
    break;
case OP_MOTOR_STANDBY:
    motor_standby();
    break;
case OP_MOTOR_FORWARD:
    motor_forward();
    break;
case OP_MOTOR_BACKWARD:
    motor_backward();
    break;
case OP_MOTOR_BRAKE:
    motor_brake();
    break;
case OP_ERROR:
    break;
default:
    break;
}
}

```

ch422g_showdata 则会根据 ch422g 模式进行温度或湿度的显示,因为例程中获取数据和显示数据是同线程调用,所以不用加锁。

```

void ch422g_showdata(void) {
    switch (ch422g_mode) {
    case CH422G_TEM:
        ch422g_set_num(2, tem / 100);
        ch422g_set_num(3, tem / 10);
        ch422g_set_mask(3, CH422G_PT);
        ch422g_set_num(4, tem);

```

```

        ch422g_flush();
        break;
    case CH422G_HUM:
        ch422g_set_num(2, hum / 100);
        ch422g_set_num(3, hum / 10);
        ch422g_set_mask(3, CH422G_PT);
        ch422g_set_num(4, hum);
        ch422g_flush();
        break;
    default:
        break;
}
}

```

对于红外遥控的处理，这里仅在查询红外遥控是否发来消息时进行加锁，而没有对后续进行加锁，这样可以提高整体效率，但因为红外遥控按下一次按键可能发送多次消息，如果 `ir_flag` 被线程置 1 后在极短的时间内被本函数置 0，可能导致这次按键的消息被二次读取，导致操作执行两次，因此可以考虑将锁的释放放在最后或者在线程中加一点延迟等方法。

```

void ir_handler(struct input_event *ir_event)
{
    struct key_table_t ir_key;
    pthread_mutex_lock(&ir_info.ir_lock);
    if (ir_info.ir_flag)
    {
        ir_encode(&ir_info.ir_event, &ir_key);
        ir_info.ir_flag = 0;
    }
    else
    {
        pthread_mutex_unlock(&ir_info.ir_lock);
        return;
    }
    pthread_mutex_unlock(&ir_info.ir_lock);
    switch (ir_key.key_code)
    {
    case KEY_1:
        led_neg(0);
        led_neg(1);
        led_neg(2);
        break;
    }
}

```

```

        case KEY_2:
            printf("key2\n");
            if (logo_mode == AIP1944_SLIDE_MODE)
                logo_mode = AIP1944_ROLL_MODE;
            else
                logo_mode = AIP1944_SLIDE_MODE;
            pthread_cancel(logo_pth);
            pthread_join(logo_pth, NULL);
            pthread_create(&logo_pth, NULL, logo_display, NULL);
            break;
        default:
            break;
    }
}

```

至此，基本工程项目模板与各设备 API 编写完毕，可根据个人兴趣继续完善 API 和功能。

22.6 练习：综合应用

本章综合了之前的所有实验，通过实际项目设计，帮助学生将所学知识和技能应用于实际场景。学生需要结合上一节中的基础 API 进行编程，实现一个功能完整的应用系统。

任务要求：

系统整合与功能实现：根据之前的实验任务和 API，整合不同的模块和设备，实现一个完整的应用系统。例如设计一个智能打卡中控设备，可以选择时间、温湿度、打卡、摄像头、拍照、数码管、LED 灯、点阵显示、蜂鸣器等设备，并通过编写程序实现各设备的协同工作。